

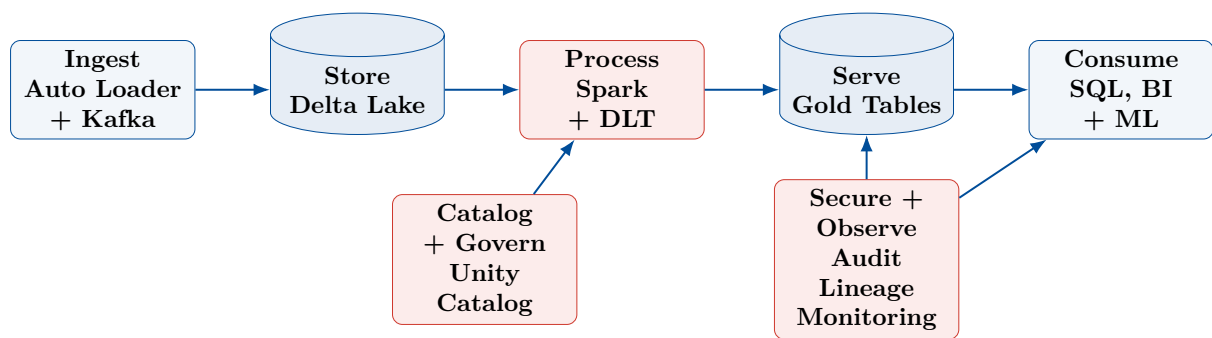
---

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

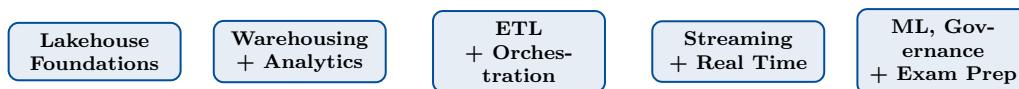
---

# Databricks Master Data Engineer

## Data Engineering on the Databricks Data Intelligence Platform



*Inside this book*



*From lakehouse foundations to governed pipelines, streaming, analytics, and machine-learning data platforms.*

---

**PhD Aned Esquerra Arguelles**

A 24-week, milestone-driven academic and hands-on Databricks data engineering program

August 31, 2026

---

# Preface

Welcome to *Databricks Master Data Engineer*. This text is designed as an academic, hands-on, progressive program for learning how modern data platforms are built on the Databricks Data Intelligence Platform. By working through the architecture discussions, service trade-offs, and weekly labs, you will move from foundational lakehouse storage to secure, governed, production-grade pipelines, streaming systems, and machine-learning operations.

This book is the theory-and-architecture companion to a 24-week Databricks Data Engineering training program. Where the weekly practice workspace asks you to *deploy* pipelines and build projects, this book explains *why* each capability exists, what operational failure mode it addresses, and how a data engineer reasons about scale, security, governance, cost, and reliability.

## Who This Book Is For

This book is written for aspiring and practicing data engineers, analytics engineers, platform engineers, database developers, and technical leads who want a rigorous path into Databricks data engineering. It assumes basic SQL, Python, command-line comfort, and familiarity with data formats such as CSV, JSON, and Parquet. Prior Spark or cloud experience helps, but the first part establishes the workspace, Delta Lake, medallion, and Spark foundations used throughout the rest of the program.

## How This Book Is Organized

The book follows six Parts, matching six curriculum milestones, and 24 chapters, matching 24 training weeks:

- **Part I: Lakehouse Foundations** builds the base: the Databricks platform and workspace, Delta Lake internals, the medallion architecture, and Apache Spark on Databricks.
- **Part II: Warehousing and Analytics** develops Delta Lake optimization, Databricks SQL warehouses, AI/BI dashboards and Genie, and Unity Catalog governance.
- **Part III: Batch ETL and Orchestration** covers PySpark performance tuning, Delta Live Tables, Auto Loader and change data capture, and Databricks Workflows orchestration.
- **Part IV: Streaming and Real-Time** introduces Structured Streaming, state and watermarks, Kafka/Event Hubs/Kinesis integration, and real-time lakehouse architectures.
- **Part V: Machine Learning and MLOps** connects data engineering to MLflow, the Feature Store, Mosaic AI and Vector Search, and model serving with monitoring.

- **Part VI: Security, Production, and Certification** consolidates identity and network security, Unity Catalog lineage and Delta Sharing, CI/CD with Asset Bundles and Terraform, capstone work, and certification-oriented review.

Appendices provide the standing environment setup, a CLI/REST/SQL quick reference, worked solutions for the weekly labs and capstones, and a consolidated cross-cloud equivalence reference.

## Reading Paths

- **New to Databricks:** read Parts I–III in order before attempting streaming, MLOps, or governance-heavy architectures.
- **Already using Spark or a cloud warehouse:** skim Chapters 1–4 for terminology, then focus on the Databricks-specific layers: DLT, Auto Loader, Unity Catalog, and Mosaic AI.
- **Building a portfolio:** complete each hands-on project and keep architecture diagrams, CLI commands, failure notes, and DBU cost estimates as evidence.
- **Preparing for interviews or certification:** work every Key Takeaway, Interview Q&A, and Exercise section; practice explaining trade-offs rather than memorizing feature names.
- **Operating a regulated platform:** read the security and governance material alongside every technical chapter, not only at the end.

## Conventions

Code identifiers, Databricks CLI commands, table names, SQL statements, and file names appear in **monospace**. Python and PySpark listings use the shared style defined in the preamble; shell commands, SQL, and YAML bundle definitions use listing captions to state their purpose. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Every architecture discussion treats cost (in DBUs and cloud infrastructure), reliability, security, and governance as first-class design dimensions.

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>0</b> | <b>Getting Started: Your Databricks Workspace and Environment</b> | <b>1</b>  |
| 0.1      | Why Chapter 0 Exists . . . . .                                    | 1         |
| 0.2      | Creating a Workspace . . . . .                                    | 2         |
| 0.2.1    | What You Need Before Sign-up . . . . .                            | 2         |
| 0.2.2    | Understanding the Cost Model . . . . .                            | 2         |
| 0.3      | Securing the Workspace . . . . .                                  | 2         |
| 0.3.1    | Identity Basics . . . . .                                         | 2         |
| 0.3.2    | Personal Access Tokens versus OAuth . . . . .                     | 3         |
| 0.4      | Installing and Configuring the Tooling . . . . .                  | 3         |
| 0.4.1    | The Databricks CLI . . . . .                                      | 3         |
| 0.4.2    | The Python SDK . . . . .                                          | 3         |
| 0.4.3    | Environment Variables and Profiles . . . . .                      | 4         |
| 0.5      | Cost Guardrails to Stay Within Credits . . . . .                  | 4         |
| 0.5.1    | Auto-Termination and Cluster Policies . . . . .                   | 4         |
| 0.5.2    | Serverless First Where Possible . . . . .                         | 4         |
| 0.5.3    | Budget Alerts and the Billing Tables . . . . .                    | 4         |
| 0.6      | Naming Conventions . . . . .                                      | 4         |
| 0.7      | Verification: Hello World . . . . .                               | 5         |
| 0.8      | Design Decision Guide . . . . .                                   | 5         |
| 0.9      | Operational Guidance for the Whole Program . . . . .              | 6         |
| 0.9.1    | Performance and Reliability . . . . .                             | 6         |
| 0.9.2    | Security and Governance Checklist . . . . .                       | 6         |
| 0.10     | Further Reading . . . . .                                         | 6         |
| 0.11     | Exercises . . . . .                                               | 7         |
| 0.12     | Curated Community Resources . . . . .                             | 7         |
| <b>I</b> | <b>Lakehouse Foundations</b>                                      | <b>10</b> |
| <b>1</b> | <b>The Databricks Platform and Workspace Fundamentals</b>         | <b>11</b> |
| 1.1      | Week 1 Roadmap and Mental Model . . . . .                         | 12        |
| 1.2      | Monday: Platform Architecture and Workspace Objects . . . . .     | 12        |
| 1.2.1    | The Control Plane and the Data Plane . . . . .                    | 12        |
| 1.2.2    | Workspace Objects . . . . .                                       | 12        |
| 1.3      | Tuesday: Notebooks, Git Folders, and dbutils . . . . .            | 13        |
| 1.3.1    | Notebooks and Magic Commands . . . . .                            | 13        |
| 1.3.2    | Git Folders . . . . .                                             | 13        |
| 1.4      | Wednesday: Compute, the CLI, and the SDK . . . . .                | 13        |
| 1.4.1    | Choosing Compute . . . . .                                        | 13        |
| 1.4.2    | Cluster Policies . . . . .                                        | 14        |

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| 1.4.3    | The CLI and SDK in Daily Work . . . . .                                        | 14        |
| 1.5      | Thursday Hands-on Project: Provision, Authenticate, Smoke-Test . . . . .       | 14        |
| 1.6      | Friday Use Case: Workspace Layout for a 200-Person Data Organization . . . . . | 16        |
| 1.6.1    | Requirements and Assumptions . . . . .                                         | 16        |
| 1.6.2    | Architecture Decisions . . . . .                                               | 17        |
| 1.6.3    | Risks and Mitigations . . . . .                                                | 17        |
| 1.7      | Design Decision Guide . . . . .                                                | 17        |
| 1.8      | Operational Guidance for Week 1 . . . . .                                      | 18        |
| 1.8.1    | Performance and Reliability . . . . .                                          | 18        |
| 1.8.2    | Security and Governance Checklist . . . . .                                    | 18        |
| 1.8.3    | Troubleshooting Playbook . . . . .                                             | 18        |
| 1.9      | Interview Q&A . . . . .                                                        | 19        |
| 1.9.1    | Concepts and Trade-offs . . . . .                                              | 19        |
| 1.10     | Further Reading . . . . .                                                      | 19        |
| 1.11     | Exercises . . . . .                                                            | 20        |
| <b>2</b> | <b>Delta Lake Foundations</b>                                                  | <b>21</b> |
| 2.1      | Week 2 Roadmap and Mental Model . . . . .                                      | 22        |
| 2.2      | Monday: Data Files and the Transaction Log . . . . .                           | 22        |
| 2.2.1    | Parquet Data Files . . . . .                                                   | 22        |
| 2.2.2    | The JSON Transaction Log . . . . .                                             | 22        |
| 2.2.3    | Atomic Commits . . . . .                                                       | 23        |
| 2.3      | Tuesday: ACID, Concurrency, and MERGE . . . . .                                | 23        |
| 2.3.1    | Optimistic Concurrency . . . . .                                               | 23        |
| 2.3.2    | MERGE INTO . . . . .                                                           | 23        |
| 2.4      | Wednesday: Schema Discipline, Time Travel, and Recovery . . . . .              | 23        |
| 2.4.1    | Schema Enforcement and Evolution . . . . .                                     | 23        |
| 2.4.2    | Constraints and Generated Columns . . . . .                                    | 24        |
| 2.4.3    | Time Travel and RESTORE . . . . .                                              | 24        |
| 2.4.4    | VACUUM and Retention . . . . .                                                 | 24        |
| 2.5      | Thursday Hands-on Project: Enforcement, Evolution, and Recovery . . . . .      | 24        |
| 2.6      | Friday Use Case: Auditable 90-Day History for a Financial Lake . . . . .       | 26        |
| 2.6.1    | Requirements and Assumptions . . . . .                                         | 26        |
| 2.6.2    | Architecture Decisions . . . . .                                               | 26        |
| 2.6.3    | Risks and Mitigations . . . . .                                                | 27        |
| 2.7      | Design Decision Guide . . . . .                                                | 27        |
| 2.8      | Operational Guidance for Week 2 . . . . .                                      | 27        |
| 2.8.1    | Performance and Reliability . . . . .                                          | 27        |
| 2.8.2    | Security and Governance Checklist . . . . .                                    | 28        |
| 2.8.3    | Troubleshooting Playbook . . . . .                                             | 28        |
| 2.9      | Interview Q&A . . . . .                                                        | 28        |
| 2.9.1    | Concepts and Trade-offs . . . . .                                              | 28        |
| 2.10     | Further Reading . . . . .                                                      | 29        |
| 2.11     | Exercises . . . . .                                                            | 29        |
| <b>3</b> | <b>The Medallion Architecture: Bronze, Silver, and Gold</b>                    | <b>30</b> |
| 3.1      | Week 3 Roadmap and Mental Model . . . . .                                      | 30        |
| 3.2      | Monday: Why Layers, and the Cost of Skipping Them . . . . .                    | 31        |
| 3.2.1    | Progressive Refinement . . . . .                                               | 31        |
| 3.2.2    | Idempotent Reprocessing . . . . .                                              | 31        |
| 3.3      | Tuesday: Bronze—The Evidence Layer . . . . .                                   | 31        |
| 3.3.1    | Append-Only Raw Capture . . . . .                                              | 31        |

|           |                                                                          |           |
|-----------|--------------------------------------------------------------------------|-----------|
| 3.3.2     | Schema-on-Read Trade-offs                                                | 32        |
| 3.4       | Wednesday: Silver Conformance and the Gold Star Schema                   | 32        |
| 3.4.1     | Silver: Validation, Dedup, Quarantine                                    | 32        |
| 3.4.2     | Reconciliation                                                           | 32        |
| 3.4.3     | Gold: The Dimensional Contract                                           | 32        |
| 3.5       | Thursday Hands-on Project: The Olist Medallion                           | 33        |
| 3.6       | Friday Use Case: Two Years of Reprocessing Without Double-Counting       | 34        |
| 3.6.1     | Requirements and Assumptions                                             | 34        |
| 3.6.2     | Architecture Decisions                                                   | 34        |
| 3.6.3     | Risks and Mitigations                                                    | 35        |
| 3.7       | Design Decision Guide                                                    | 35        |
| 3.8       | Operational Guidance for Week 3                                          | 35        |
| 3.8.1     | Performance and Reliability                                              | 35        |
| 3.8.2     | Security and Governance Checklist                                        | 36        |
| 3.8.3     | Troubleshooting Playbook                                                 | 36        |
| 3.9       | Interview Q&A                                                            | 36        |
| 3.9.1     | Concepts and Trade-offs                                                  | 36        |
| 3.10      | Further Reading                                                          | 37        |
| 3.11      | Exercises                                                                | 37        |
| <b>4</b>  | <b>Apache Spark on Databricks</b>                                        | <b>39</b> |
| 4.1       | Week 4 Roadmap and Mental Model                                          | 39        |
| 4.2       | Monday: Architecture and Lazy Evaluation                                 | 40        |
| 4.2.1     | Driver, Executors, DAG, Stages, Tasks                                    | 40        |
| 4.2.2     | Lazy Evaluation and Catalyst                                             | 40        |
| 4.3       | Tuesday: Transformations, Actions, and the Shuffle                       | 40        |
| 4.3.1     | Narrow versus Wide                                                       | 40        |
| 4.3.2     | Partitions and the 200 Default                                           | 41        |
| 4.4       | Wednesday: The Spark UI, Caching, and Partitioning                       | 41        |
| 4.4.1     | Reading the Spark UI                                                     | 41        |
| 4.4.2     | Caching Deliberately                                                     | 41        |
| 4.4.3     | Repartition versus Coalesce                                              | 41        |
| 4.5       | Thursday Hands-on Project: Find the Shuffle, Fix the Skew                | 41        |
| 4.6       | Friday Use Case: The Nightly Job That Grew from 40 Minutes to 6 Hours    | 43        |
| 4.6.1     | Requirements and Assumptions                                             | 43        |
| 4.6.2     | Architecture Decisions                                                   | 43        |
| 4.6.3     | Risks and Mitigations                                                    | 44        |
| 4.7       | Design Decision Guide                                                    | 44        |
| 4.8       | Operational Guidance for Week 4                                          | 44        |
| 4.8.1     | Performance and Reliability                                              | 44        |
| 4.8.2     | Security and Governance Checklist                                        | 45        |
| 4.8.3     | Troubleshooting Playbook                                                 | 45        |
| 4.9       | Interview Q&A                                                            | 45        |
| 4.9.1     | Concepts and Trade-offs                                                  | 45        |
| 4.10      | Further Reading                                                          | 46        |
| 4.11      | Exercises                                                                | 46        |
| 4.12      | Milestone 1 Capstone: The Retail Medallion Lakehouse                     | 47        |
| <b>II</b> | <b>Warehousing and Analytics</b>                                         | <b>50</b> |
| <b>5</b>  | <b>Delta Lake Optimization: OPTIMIZE, Z-ORDER, and Liquid Clustering</b> | <b>51</b> |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 5.1      | Week 5 Roadmap and Mental Model . . . . .                                  | 52        |
| 5.2      | Monday: File Layout, Statistics, and the Small-File Problem . . . . .      | 52        |
| 5.2.1    | How a Delta Scan Reads . . . . .                                           | 52        |
| 5.2.2    | The Small-File Problem . . . . .                                           | 52        |
| 5.3      | Tuesday: OPTIMIZE and Z-ORDER . . . . .                                    | 53        |
| 5.3.1    | Compaction . . . . .                                                       | 53        |
| 5.3.2    | Z-ORDER . . . . .                                                          | 53        |
| 5.4      | Wednesday: Liquid Clustering, Deletion Vectors, VACUUM . . . . .           | 53        |
| 5.4.1    | Liquid Clustering . . . . .                                                | 53        |
| 5.4.2    | Deletion Vectors . . . . .                                                 | 54        |
| 5.4.3    | VACUUM and Predictive Optimization . . . . .                               | 54        |
| 5.5      | Thursday Hands-on Project: The Small-File Benchmark . . . . .              | 54        |
| 5.6      | Friday Use Case: Remediating Two Million Tiny Files . . . . .              | 55        |
| 5.6.1    | Symptoms and Diagnosis . . . . .                                           | 55        |
| 5.6.2    | Remediation Strategy . . . . .                                             | 56        |
| 5.6.3    | Risks and Mitigations . . . . .                                            | 56        |
| 5.7      | Design Decision Guide . . . . .                                            | 56        |
| 5.8      | Operational Guidance for Week 5 . . . . .                                  | 57        |
| 5.8.1    | Performance and Reliability . . . . .                                      | 57        |
| 5.8.2    | Security and Governance Checklist . . . . .                                | 57        |
| 5.8.3    | Troubleshooting Playbook . . . . .                                         | 58        |
| 5.9      | Interview Q&A . . . . .                                                    | 58        |
| 5.9.1    | Concepts and Trade-offs . . . . .                                          | 58        |
| 5.10     | Further Reading . . . . .                                                  | 59        |
| 5.11     | Exercises . . . . .                                                        | 59        |
| <b>6</b> | <b>Databricks SQL and SQL Warehouses</b>                                   | <b>60</b> |
| 6.1      | Week 6 Roadmap and Mental Model . . . . .                                  | 61        |
| 6.2      | Monday: Warehouse Types and Photon . . . . .                               | 61        |
| 6.2.1    | Serverless, Pro, Classic . . . . .                                         | 61        |
| 6.2.2    | Photon Under the Hood . . . . .                                            | 61        |
| 6.3      | Tuesday: Warehouse SQL and the Query Profile . . . . .                     | 62        |
| 6.3.1    | Analytical SQL on the Lakehouse . . . . .                                  | 62        |
| 6.3.2    | Reading the Query Profile . . . . .                                        | 62        |
| 6.4      | Wednesday: Materialized Views, Caching, and Cost Attribution . . . . .     | 62        |
| 6.4.1    | Materialized Views versus Scheduled Gold . . . . .                         | 62        |
| 6.4.2    | Scaling and Cost . . . . .                                                 | 62        |
| 6.5      | Thursday Hands-on Project: Serve the Gold Star Schema . . . . .            | 63        |
| 6.6      | Friday Use Case: 500 Analysts, One 9 A.M. Burst, Zero Contention . . . . . | 64        |
| 6.6.1    | Requirements and Assumptions . . . . .                                     | 64        |
| 6.6.2    | Architecture Decisions . . . . .                                           | 64        |
| 6.6.3    | Risks and Mitigations . . . . .                                            | 65        |
| 6.7      | Design Decision Guide . . . . .                                            | 65        |
| 6.8      | Operational Guidance for Week 6 . . . . .                                  | 65        |
| 6.8.1    | Performance and Reliability . . . . .                                      | 65        |
| 6.8.2    | Security and Governance Checklist . . . . .                                | 66        |
| 6.8.3    | Troubleshooting Playbook . . . . .                                         | 66        |
| 6.9      | Interview Q&A . . . . .                                                    | 66        |
| 6.9.1    | Concepts and Trade-offs . . . . .                                          | 66        |
| 6.10     | Further Reading . . . . .                                                  | 67        |
| 6.11     | Exercises . . . . .                                                        | 67        |

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| <b>7</b> | <b>AI/BI Dashboards and Genie</b>                                            | <b>68</b> |
| 7.1      | Week 7 Roadmap and Mental Model . . . . .                                    | 69        |
| 7.2      | Monday: AI/BI Dashboards . . . . .                                           | 69        |
| 7.2.1    | Datasets and Visualizations . . . . .                                        | 69        |
| 7.2.2    | Parameters, Cross-Filters, and Publishing . . . . .                          | 69        |
| 7.3      | Tuesday: Genie Spaces . . . . .                                              | 70        |
| 7.3.1    | What Genie Needs to Succeed . . . . .                                        | 70        |
| 7.3.2    | Debugging a Wrong Answer . . . . .                                           | 70        |
| 7.4      | Wednesday: The Semantic Layer . . . . .                                      | 70        |
| 7.4.1    | Certified Views and Named Measures . . . . .                                 | 70        |
| 7.5      | Thursday Hands-on Project: Executive Dashboard and Genie Space . . . . .     | 71        |
| 7.6      | Friday Use Case: Certified Numbers and Curious Users . . . . .               | 71        |
| 7.6.1    | Requirements and Assumptions . . . . .                                       | 72        |
| 7.6.2    | Architecture Decisions . . . . .                                             | 72        |
| 7.6.3    | Risks and Mitigations . . . . .                                              | 72        |
| 7.7      | Design Decision Guide . . . . .                                              | 73        |
| 7.8      | Operational Guidance for Week 7 . . . . .                                    | 73        |
| 7.8.1    | Performance and Reliability . . . . .                                        | 73        |
| 7.8.2    | Security and Governance Checklist . . . . .                                  | 73        |
| 7.8.3    | Troubleshooting Playbook . . . . .                                           | 74        |
| 7.9      | Interview Q&A . . . . .                                                      | 74        |
| 7.9.1    | Concepts and Trade-offs . . . . .                                            | 74        |
| 7.10     | Further Reading . . . . .                                                    | 75        |
| 7.11     | Exercises . . . . .                                                          | 75        |
| <b>8</b> | <b>Unity Catalog Foundations and Data Governance</b>                         | <b>76</b> |
| 8.1      | Week 8 Roadmap and Mental Model . . . . .                                    | 77        |
| 8.2      | Monday: The Hierarchy and the Namespace . . . . .                            | 77        |
| 8.2.1    | Metastore to Table . . . . .                                                 | 77        |
| 8.2.2    | Managed versus External . . . . .                                            | 77        |
| 8.3      | Tuesday: The Privilege Model . . . . .                                       | 78        |
| 8.3.1    | Groups, Owners, and GRANT . . . . .                                          | 78        |
| 8.3.2    | Row Filters and Column Masks . . . . .                                       | 78        |
| 8.4      | Wednesday: Volumes, Dynamic Views, and Audit . . . . .                       | 78        |
| 8.4.1    | Volumes for File Data . . . . .                                              | 78        |
| 8.4.2    | Dynamic Views . . . . .                                                      | 78        |
| 8.4.3    | Audit . . . . .                                                              | 79        |
| 8.5      | Thursday Hands-on Project: Govern the Medallion Estate . . . . .             | 79        |
| 8.6      | Friday Use Case: HIPAA-Grade Governance for a Healthcare Lakehouse . . . . . | 80        |
| 8.6.1    | Requirements and Assumptions . . . . .                                       | 80        |
| 8.6.2    | Architecture Decisions . . . . .                                             | 80        |
| 8.6.3    | Risks and Mitigations . . . . .                                              | 81        |
| 8.7      | Design Decision Guide . . . . .                                              | 81        |
| 8.8      | Operational Guidance for Week 8 . . . . .                                    | 81        |
| 8.8.1    | Performance and Reliability . . . . .                                        | 81        |
| 8.8.2    | Security and Governance Checklist . . . . .                                  | 82        |
| 8.8.3    | Troubleshooting Playbook . . . . .                                           | 82        |
| 8.9      | Interview Q&A . . . . .                                                      | 82        |
| 8.9.1    | Concepts and Trade-offs . . . . .                                            | 82        |
| 8.10     | Further Reading . . . . .                                                    | 83        |
| 8.11     | Exercises . . . . .                                                          | 83        |



|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| 8.12 Milestone 2 Capstone: The Governed Analytics Lakehouse . . . . .   | 84        |
| <b>III Batch ETL and Orchestration</b>                                  | <b>86</b> |
| <b>9 PySpark Deep Dive and Performance Tuning</b>                       | <b>87</b> |
| 9.1 Week 9 Roadmap and Mental Model . . . . .                           | 88        |
| 9.2 Monday: Catalyst, Tungsten, and When Not to RDD . . . . .           | 88        |
| 9.2.1 The Optimization Stack . . . . .                                  | 88        |
| 9.2.2 When (Not) to Use RDDs . . . . .                                  | 88        |
| 9.3 Tuesday: Joins and AQE . . . . .                                    | 89        |
| 9.3.1 Broadcast versus Sort-Merge . . . . .                             | 89        |
| 9.3.2 Skew and Salting . . . . .                                        | 89        |
| 9.4 Wednesday: Shuffle Tuning, UDFs, and Memory . . . . .               | 89        |
| 9.4.1 Shuffle Partitions and I/O . . . . .                              | 89        |
| 9.4.2 UDF Performance . . . . .                                         | 89        |
| 9.5 Thursday Hands-on Project: The Attributable Tuning Report . . . . . | 90        |
| 9.6 Friday Use Case: The 2 TB Join That Misses Its SLA . . . . .        | 91        |
| 9.6.1 Requirements and Assumptions . . . . .                            | 91        |
| 9.6.2 Architecture Decisions . . . . .                                  | 91        |
| 9.6.3 Risks and Mitigations . . . . .                                   | 92        |
| 9.7 Design Decision Guide . . . . .                                     | 92        |
| 9.8 Operational Guidance for Week 9 . . . . .                           | 93        |
| 9.8.1 Performance and Reliability . . . . .                             | 93        |
| 9.8.2 Security and Governance Checklist . . . . .                       | 93        |
| 9.8.3 Troubleshooting Playbook . . . . .                                | 93        |
| 9.9 Interview Q&A . . . . .                                             | 93        |
| 9.9.1 Concepts and Trade-offs . . . . .                                 | 93        |
| 9.10 Further Reading . . . . .                                          | 94        |
| 9.11 Exercises . . . . .                                                | 94        |
| <b>10 Delta Live Tables: Declarative Pipelines</b>                      | <b>96</b> |
| 10.1 Week 10 Roadmap and Mental Model . . . . .                         | 96        |
| 10.2 Monday: Datasets and the Declarative DAG . . . . .                 | 97        |
| 10.2.1 Streaming Tables and Materialized Views . . . . .                | 97        |
| 10.2.2 Development versus Production Mode . . . . .                     | 98        |
| 10.3 Tuesday: Expectations—Quality as Code . . . . .                    | 98        |
| 10.3.1 Warn, Drop, Fail . . . . .                                       | 98        |
| 10.4 Wednesday: Operating Pipelines . . . . .                           | 98        |
| 10.4.1 Triggered versus Continuous . . . . .                            | 98        |
| 10.4.2 Full Refresh and Schema Evolution . . . . .                      | 98        |
| 10.4.3 The Event Log . . . . .                                          | 98        |
| 10.5 Thursday Hands-on Project: The Medallion as DLT . . . . .          | 99        |
| 10.6 Friday Use Case: DLT or Hand-Written—Drawing the Line . . . . .    | 100       |
| 10.6.1 Requirements and Assumptions . . . . .                           | 100       |
| 10.6.2 Architecture Decisions . . . . .                                 | 100       |
| 10.6.3 Risks and Mitigations . . . . .                                  | 101       |
| 10.7 Design Decision Guide . . . . .                                    | 101       |
| 10.8 Operational Guidance for Week 10 . . . . .                         | 101       |
| 10.8.1 Performance and Reliability . . . . .                            | 101       |
| 10.8.2 Security and Governance Checklist . . . . .                      | 101       |
| 10.8.3 Troubleshooting Playbook . . . . .                               | 102       |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| 10.9 Interview Q&A . . . . .                                                    | 102        |
| 10.9.1 Concepts and Trade-offs . . . . .                                        | 102        |
| 10.10 Further Reading . . . . .                                                 | 103        |
| 10.11 Exercises . . . . .                                                       | 103        |
| <b>11 Auto Loader, Incremental Ingestion, and Change Data Capture</b>           | <b>104</b> |
| 11.1 Week 11 Roadmap and Mental Model . . . . .                                 | 105        |
| 11.2 Monday: Auto Loader Architecture . . . . .                                 | 105        |
| 11.2.1 Discovery: Listing versus Notification . . . . .                         | 105        |
| 11.2.2 Exactly-Once and the Checkpoint . . . . .                                | 105        |
| 11.3 Tuesday: Schemas and the Rescued Column . . . . .                          | 106        |
| 11.3.1 Inference, Hints, and Evolution . . . . .                                | 106        |
| 11.3.2 The Rescued Data Column . . . . .                                        | 106        |
| 11.4 Wednesday: CDC with MERGE and AUTO CDC . . . . .                           | 106        |
| 11.4.1 The MERGE Pattern . . . . .                                              | 106        |
| 11.4.2 AUTO CDC for SCD . . . . .                                               | 106        |
| 11.5 Thursday Hands-on Project: Incremental Files plus SCD2 Customers . . . . . | 107        |
| 11.6 Friday Use Case: 500 Files per Minute plus a Nightly CDC . . . . .         | 108        |
| 11.6.1 Requirements and Assumptions . . . . .                                   | 108        |
| 11.6.2 Architecture Decisions . . . . .                                         | 108        |
| 11.6.3 Risks and Mitigations . . . . .                                          | 109        |
| 11.7 Design Decision Guide . . . . .                                            | 109        |
| 11.8 Operational Guidance for Week 11 . . . . .                                 | 109        |
| 11.8.1 Performance and Reliability . . . . .                                    | 109        |
| 11.8.2 Security and Governance Checklist . . . . .                              | 110        |
| 11.8.3 Troubleshooting Playbook . . . . .                                       | 110        |
| 11.9 Interview Q&A . . . . .                                                    | 110        |
| 11.9.1 Concepts and Trade-offs . . . . .                                        | 110        |
| 11.10 Further Reading . . . . .                                                 | 111        |
| 11.11 Exercises . . . . .                                                       | 111        |
| <b>12 Workflows, Jobs, and Pipeline Orchestration</b>                           | <b>112</b> |
| 12.1 Week 12 Roadmap and Mental Model . . . . .                                 | 112        |
| 12.2 Monday: Jobs, Tasks, and Compute Topology . . . . .                        | 113        |
| 12.2.1 Anatomy of a Job . . . . .                                               | 113        |
| 12.2.2 Scheduling . . . . .                                                     | 113        |
| 12.3 Tuesday: Parameters, Task Values, and Control Flow . . . . .               | 113        |
| 12.3.1 Static versus Runtime Values . . . . .                                   | 113        |
| 12.3.2 Conditionals and For-Each . . . . .                                      | 114        |
| 12.4 Wednesday: Reliability Engineering . . . . .                               | 114        |
| 12.4.1 Retries, Timeouts, Notifications . . . . .                               | 114        |
| 12.4.2 Repair Runs . . . . .                                                    | 114        |
| 12.4.3 Monitoring the Estate . . . . .                                          | 114        |
| 12.5 Thursday Hands-on Project: The Orchestrated Medallion . . . . .            | 114        |
| 12.6 Friday Use Case: Sixty Pipelines, Three Teams, One 6 A.M. SLA . . . . .    | 116        |
| 12.6.1 Requirements and Assumptions . . . . .                                   | 116        |
| 12.6.2 Architecture Decisions . . . . .                                         | 116        |
| 12.6.3 Risks and Mitigations . . . . .                                          | 117        |
| 12.7 Design Decision Guide . . . . .                                            | 117        |
| 12.8 Operational Guidance for Week 12 . . . . .                                 | 117        |
| 12.8.1 Performance and Reliability . . . . .                                    | 117        |
| 12.8.2 Security and Governance Checklist . . . . .                              | 117        |

|           |                                                                                    |            |
|-----------|------------------------------------------------------------------------------------|------------|
| 12.8.3    | Troubleshooting Playbook . . . . .                                                 | 118        |
| 12.9      | Interview Q&A . . . . .                                                            | 118        |
| 12.9.1    | Concepts and Trade-offs . . . . .                                                  | 118        |
| 12.10     | Further Reading . . . . .                                                          | 119        |
| 12.11     | Exercises . . . . .                                                                | 119        |
| 12.12     | Milestone 3 Capstone: The Orchestrated Batch ELT Platform . . . . .                | 120        |
| <b>IV</b> | <b>Streaming and Real-Time</b>                                                     | <b>122</b> |
| <b>13</b> | <b>Structured Streaming Foundations</b>                                            | <b>123</b> |
| 13.1      | Week 13 Roadmap and Mental Model . . . . .                                         | 124        |
| 13.2      | Monday: The Micro-Batch Model . . . . .                                            | 124        |
| 13.2.1    | Sources, Incremental Execution, Sinks . . . . .                                    | 124        |
| 13.2.2    | Micro-Batch versus Continuous . . . . .                                            | 124        |
| 13.3      | Tuesday: Output Modes and Triggers . . . . .                                       | 125        |
| 13.3.1    | Choosing the Output Mode . . . . .                                                 | 125        |
| 13.3.2    | Choosing the Trigger . . . . .                                                     | 125        |
| 13.4      | Wednesday: Checkpoints, Recovery, and Monitoring . . . . .                         | 125        |
| 13.4.1    | What the Checkpoint Holds . . . . .                                                | 125        |
| 13.4.2    | Monitoring a Stream . . . . .                                                      | 125        |
| 13.5      | Thursday Hands-on Project: Kill It, Restart It, Prove It . . . . .                 | 126        |
| 13.6      | Friday Use Case: Sub-Minute Ingestion for Ad-Tech Events . . . . .                 | 127        |
| 13.6.1    | Requirements and Assumptions . . . . .                                             | 127        |
| 13.6.2    | Architecture Decisions . . . . .                                                   | 127        |
| 13.6.3    | Risks and Mitigations . . . . .                                                    | 128        |
| 13.7      | Design Decision Guide . . . . .                                                    | 128        |
| 13.8      | Operational Guidance for Week 13 . . . . .                                         | 128        |
| 13.8.1    | Performance and Reliability . . . . .                                              | 128        |
| 13.8.2    | Security and Governance Checklist . . . . .                                        | 129        |
| 13.8.3    | Troubleshooting Playbook . . . . .                                                 | 129        |
| 13.9      | Interview Q&A . . . . .                                                            | 129        |
| 13.9.1    | Concepts and Trade-offs . . . . .                                                  | 129        |
| 13.10     | Further Reading . . . . .                                                          | 130        |
| 13.11     | Exercises . . . . .                                                                | 130        |
| <b>14</b> | <b>State Management, Watermarks, and Exactly-Once Semantics</b>                    | <b>131</b> |
| 14.1      | Week 14 Roadmap and Mental Model . . . . .                                         | 132        |
| 14.2      | Monday: Event Time and Windows . . . . .                                           | 132        |
| 14.2.1    | Why Event Time . . . . .                                                           | 132        |
| 14.2.2    | Window Types . . . . .                                                             | 132        |
| 14.3      | Tuesday: Watermarks . . . . .                                                      | 133        |
| 14.3.1    | The Contract . . . . .                                                             | 133        |
| 14.3.2    | Choosing the Delay . . . . .                                                       | 133        |
| 14.4      | Wednesday: Stateful Operators and the State Store . . . . .                        | 133        |
| 14.4.1    | Deduplication and Aggregations . . . . .                                           | 133        |
| 14.4.2    | Arbitrary State and RocksDB . . . . .                                              | 133        |
| 14.5      | Thursday Hands-on Project: Late, Duplicated, and Out of Order . . . . .            | 134        |
| 14.6      | Friday Use Case: Fifteen Minutes of Lateness, Thirty Seconds of Patience . . . . . | 135        |
| 14.6.1    | Requirements and Assumptions . . . . .                                             | 135        |
| 14.6.2    | Architecture Decisions . . . . .                                                   | 135        |
| 14.6.3    | Risks and Mitigations . . . . .                                                    | 136        |

|           |                                                                         |            |
|-----------|-------------------------------------------------------------------------|------------|
| 14.7      | Design Decision Guide . . . . .                                         | 136        |
| 14.8      | Operational Guidance for Week 14 . . . . .                              | 136        |
| 14.8.1    | Performance and Reliability . . . . .                                   | 136        |
| 14.8.2    | Security and Governance Checklist . . . . .                             | 136        |
| 14.8.3    | Troubleshooting Playbook . . . . .                                      | 137        |
| 14.9      | Interview Q&A . . . . .                                                 | 137        |
| 14.9.1    | Concepts and Trade-offs . . . . .                                       | 137        |
| 14.10     | Further Reading . . . . .                                               | 138        |
| 14.11     | Exercises . . . . .                                                     | 138        |
| <b>15</b> | <b>Kafka, Event Hubs, and Kinesis Integration</b>                       | <b>139</b> |
| 15.1      | Week 15 Roadmap and Mental Model . . . . .                              | 140        |
| 15.2      | Monday: The Log Model for Data Engineers . . . . .                      | 140        |
| 15.2.1    | Topics, Partitions, Groups . . . . .                                    | 140        |
| 15.2.2    | Offset Semantics . . . . .                                              | 140        |
| 15.3      | Tuesday: The Spark Kafka Source, Securely . . . . .                     | 140        |
| 15.3.1    | Connection and Deserialization . . . . .                                | 140        |
| 15.3.2    | Event Hubs via the Kafka Endpoint . . . . .                             | 141        |
| 15.4      | Wednesday: Kinesis, Throughput, and Replay . . . . .                    | 141        |
| 15.4.1    | The Kinesis Source . . . . .                                            | 141        |
| 15.4.2    | Replay and Reconciliation . . . . .                                     | 141        |
| 15.5      | Thursday Hands-on Project: Local Kafka to Bronze, with Replay . . . . . | 142        |
| 15.6      | Friday Use Case: On-Prem Kafka to Event Hubs, Zero Downtime . . . . .   | 143        |
| 15.6.1    | Requirements and Assumptions . . . . .                                  | 143        |
| 15.6.2    | Architecture Decisions . . . . .                                        | 143        |
| 15.6.3    | Risks and Mitigations . . . . .                                         | 144        |
| 15.7      | Design Decision Guide . . . . .                                         | 144        |
| 15.8      | Operational Guidance for Week 15 . . . . .                              | 145        |
| 15.8.1    | Performance and Reliability . . . . .                                   | 145        |
| 15.8.2    | Security and Governance Checklist . . . . .                             | 145        |
| 15.8.3    | Troubleshooting Playbook . . . . .                                      | 145        |
| 15.9      | Interview Q&A . . . . .                                                 | 145        |
| 15.9.1    | Concepts and Trade-offs . . . . .                                       | 145        |
| 15.10     | Further Reading . . . . .                                               | 146        |
| 15.11     | Exercises . . . . .                                                     | 146        |
| <b>16</b> | <b>Real-Time Lakehouse Architectures</b>                                | <b>148</b> |
| 16.1      | Week 16 Roadmap and Mental Model . . . . .                              | 149        |
| 16.2      | Monday: The Streaming Medallion . . . . .                               | 149        |
| 16.2.1    | Layer by Layer . . . . .                                                | 149        |
| 16.2.2    | One Table, Two Writers . . . . .                                        | 149        |
| 16.3      | Tuesday: foreachBatch and Streaming Joins . . . . .                     | 150        |
| 16.3.1    | foreachBatch: The Correct Escape Hatch . . . . .                        | 150        |
| 16.3.2    | Streaming Joins . . . . .                                               | 150        |
| 16.4      | Wednesday: Serving and the Latency/Cost Model . . . . .                 | 150        |
| 16.4.1    | Real-Time Gold to Dashboards . . . . .                                  | 150        |
| 16.4.2    | The Latency/Cost Dial . . . . .                                         | 151        |
| 16.5      | Thursday Hands-on Project: The Real-Time Orders Pipeline . . . . .      | 151        |
| 16.6      | Friday Use Case: Ten Thousand Vehicles, One Dispatch Screen . . . . .   | 152        |
| 16.6.1    | Requirements and Assumptions . . . . .                                  | 152        |
| 16.6.2    | Architecture Decisions . . . . .                                        | 152        |
| 16.6.3    | Risks and Mitigations . . . . .                                         | 153        |

|           |                                                              |            |
|-----------|--------------------------------------------------------------|------------|
| 16.7      | Design Decision Guide                                        | 153        |
| 16.8      | Operational Guidance for Week 16                             | 154        |
| 16.8.1    | Performance and Reliability                                  | 154        |
| 16.8.2    | Security and Governance Checklist                            | 154        |
| 16.8.3    | Troubleshooting Playbook                                     | 154        |
| 16.9      | Interview Q&A                                                | 154        |
| 16.9.1    | Concepts and Trade-offs                                      | 154        |
| 16.10     | Further Reading                                              | 155        |
| 16.11     | Exercises                                                    | 155        |
| 16.12     | Milestone 4 Capstone: The Real-Time Fraud Telemetry Platform | 156        |
| <b>V</b>  | <b>Machine Learning and MLOps</b>                            | <b>158</b> |
| <b>17</b> | <b>MLflow and Experiment Tracking</b>                        | <b>159</b> |
| 17.1      | Week 17 Roadmap and Mental Model                             | 159        |
| 17.2      | Monday: Tracking, Runs, and Artifacts                        | 160        |
| 17.2.1    | What a Run Records                                           | 160        |
| 17.2.2    | Comparing Runs Honestly                                      | 161        |
| 17.3      | Tuesday: Signatures, Inputs, and Reproducibility             | 161        |
| 17.3.1    | Model Signatures and Input Examples                          | 161        |
| 17.3.2    | Reproducing a Run                                            | 161        |
| 17.4      | Wednesday: The Unity Catalog Model Registry                  | 161        |
| 17.4.1    | Versions, Aliases, Governance                                | 161        |
| 17.5      | Thursday Hands-on Project: Six Runs, One Champion            | 162        |
| 17.6      | Friday Use Case: Experiment Governance for a Bank            | 163        |
| 17.6.1    | Requirements and Assumptions                                 | 163        |
| 17.6.2    | Architecture Decisions                                       | 163        |
| 17.6.3    | Risks and Mitigations                                        | 164        |
| 17.7      | Design Decision Guide                                        | 164        |
| 17.8      | Operational Guidance for Week 17                             | 164        |
| 17.8.1    | Performance and Reliability                                  | 164        |
| 17.8.2    | Security and Governance Checklist                            | 164        |
| 17.8.3    | Troubleshooting Playbook                                     | 165        |
| 17.9      | Interview Q&A                                                | 165        |
| 17.9.1    | Concepts and Trade-offs                                      | 165        |
| 17.10     | Further Reading                                              | 166        |
| 17.11     | Exercises                                                    | 166        |
| <b>18</b> | <b>Feature Engineering and the Feature Store</b>             | <b>167</b> |
| 18.1      | Week 18 Roadmap and Mental Model                             | 167        |
| 18.2      | Monday: Leakage and Point-in-Time Discipline                 | 168        |
| 18.2.1    | Recognizing Leakage                                          | 168        |
| 18.2.2    | As-of Computation                                            | 168        |
| 18.3      | Tuesday: Feature Tables and Training Sets                    | 168        |
| 18.3.1    | Feature Engineering in Unity Catalog                         | 168        |
| 18.3.2    | Training Sets with Timestamp Lookups                         | 169        |
| 18.4      | Wednesday: Online Serving and Skew Prevention                | 169        |
| 18.4.1    | The Same Features, Online                                    | 169        |
| 18.4.2    | Freshness and Monitoring                                     | 169        |
| 18.5      | Thursday Hands-on Project: Point-in-Time Churn Features      | 170        |
| 18.6      | Friday Use Case: The Churn Model That Lied                   | 171        |

|           |                                                            |            |
|-----------|------------------------------------------------------------|------------|
| 18.6.1    | Requirements and Assumptions                               | 171        |
| 18.6.2    | Architecture Decisions                                     | 171        |
| 18.6.3    | Risks and Mitigations                                      | 172        |
| 18.7      | Design Decision Guide                                      | 172        |
| 18.8      | Operational Guidance for Week 18                           | 172        |
| 18.8.1    | Performance and Reliability                                | 172        |
| 18.8.2    | Security and Governance Checklist                          | 172        |
| 18.8.3    | Troubleshooting Playbook                                   | 173        |
| 18.9      | Interview Q&A                                              | 173        |
| 18.9.1    | Concepts and Trade-offs                                    | 173        |
| 18.10     | Further Reading                                            | 174        |
| 18.11     | Exercises                                                  | 174        |
| <b>19</b> | <b>Mosaic AI, Vector Search, and Generative AI (RAG)</b>   | <b>175</b> |
| 19.1      | Week 19 Roadmap and Mental Model                           | 176        |
| 19.2      | Monday: Foundation Models and the Serving Surface          | 176        |
| 19.2.1    | The Building Blocks                                        | 176        |
| 19.2.2    | Cost and Latency Arithmetic                                | 176        |
| 19.3      | Tuesday: Vector Search                                     | 177        |
| 19.3.1    | From Documents to an Index                                 | 177        |
| 19.3.2    | Querying with Filters                                      | 177        |
| 19.4      | Wednesday: The RAG Chain and Evaluation                    | 177        |
| 19.4.1    | Assembling the Chain                                       | 177        |
| 19.4.2    | Evaluation as a Pipeline Stage                             | 178        |
| 19.5      | Thursday Hands-on Project: The Program's Own RAG Assistant | 178        |
| 19.6      | Friday Use Case: The Support Copilot with a Cost Ceiling   | 179        |
| 19.6.1    | Requirements and Assumptions                               | 179        |
| 19.6.2    | Architecture Decisions                                     | 180        |
| 19.6.3    | Risks and Mitigations                                      | 180        |
| 19.7      | Design Decision Guide                                      | 180        |
| 19.8      | Operational Guidance for Week 19                           | 181        |
| 19.8.1    | Performance and Reliability                                | 181        |
| 19.8.2    | Security and Governance Checklist                          | 181        |
| 19.8.3    | Troubleshooting Playbook                                   | 181        |
| 19.9      | Interview Q&A                                              | 181        |
| 19.9.1    | Concepts and Trade-offs                                    | 181        |
| 19.10     | Further Reading                                            | 182        |
| 19.11     | Exercises                                                  | 182        |
| <b>20</b> | <b>Model Serving, Monitoring, and MLOps</b>                | <b>184</b> |
| 20.1      | Week 20 Roadmap and Mental Model                           | 184        |
| 20.2      | Monday: Model Serving                                      | 185        |
| 20.2.1    | Serverless Endpoints                                       | 185        |
| 20.2.2    | Endpoint Hygiene                                           | 185        |
| 20.3      | Tuesday: Batch Scoring at Scale                            | 186        |
| 20.3.1    | The Alias-Loaded UDF                                       | 186        |
| 20.3.2    | Scoring Discipline                                         | 186        |
| 20.4      | Wednesday: Monitoring and the Retraining Loop              | 186        |
| 20.4.1    | Lakehouse Monitoring                                       | 186        |
| 20.4.2    | The Gated Loop                                             | 186        |
| 20.5      | Thursday Hands-on Project: Serve, Score, Drift, Gate       | 187        |
| 20.6      | Friday Use Case: The Lender's MLOps Control Plane          | 188        |

|           |                                                                          |            |
|-----------|--------------------------------------------------------------------------|------------|
| 20.6.1    | Requirements and Assumptions                                             | 188        |
| 20.6.2    | Architecture Decisions                                                   | 188        |
| 20.6.3    | Risks and Mitigations                                                    | 188        |
| 20.7      | Design Decision Guide                                                    | 188        |
| 20.8      | Operational Guidance for Week 20                                         | 189        |
| 20.8.1    | Performance and Reliability                                              | 189        |
| 20.8.2    | Security and Governance Checklist                                        | 189        |
| 20.8.3    | Troubleshooting Playbook                                                 | 189        |
| 20.9      | Interview Q&A                                                            | 190        |
| 20.9.1    | Concepts and Trade-offs                                                  | 190        |
| 20.10     | Further Reading                                                          | 190        |
| 20.11     | Exercises                                                                | 191        |
| 20.12     | Milestone 5 Capstone: The End-to-End MLOps Pipeline                      | 191        |
| <b>VI</b> | <b>Security, Production, and Certification</b>                           | <b>194</b> |
| <b>21</b> | <b>Security, IAM, SCIM, and Network Isolation</b>                        | <b>195</b> |
| 21.1      | Week 21 Roadmap and Mental Model                                         | 196        |
| 21.2      | Monday: Identity Architecture                                            | 196        |
| 21.2.1    | Account versus Workspace                                                 | 196        |
| 21.2.2    | SCIM Provisioning                                                        | 196        |
| 21.3      | Tuesday: Service Principals, OAuth, and Secrets                          | 197        |
| 21.3.1    | Machine Identity                                                         | 197        |
| 21.3.2    | Secret Scopes                                                            | 197        |
| 21.4      | Wednesday: Network Isolation                                             | 197        |
| 21.4.1    | Private Connectivity and Perimeter Controls                              | 197        |
| 21.4.2    | What IP Lists Do Not Do                                                  | 197        |
| 21.5      | Thursday Hands-on Project: Harden a Lab Workspace                        | 198        |
| 21.6      | Friday Use Case: Contractors, Service Principals, and No Public Internet | 198        |
| 21.6.1    | Requirements and Assumptions                                             | 199        |
| 21.6.2    | Architecture Decisions                                                   | 199        |
| 21.6.3    | Risks and Mitigations                                                    | 199        |
| 21.7      | Design Decision Guide                                                    | 200        |
| 21.8      | Operational Guidance for Week 21                                         | 200        |
| 21.8.1    | Performance and Reliability                                              | 200        |
| 21.8.2    | Security and Governance Checklist                                        | 200        |
| 21.8.3    | Troubleshooting Playbook                                                 | 201        |
| 21.9      | Interview Q&A                                                            | 201        |
| 21.9.1    | Concepts and Trade-offs                                                  | 201        |
| 21.10     | Further Reading                                                          | 201        |
| 21.11     | Exercises                                                                | 202        |
| <b>22</b> | <b>Unity Catalog Deep Dive: Lineage, Audit, and Delta Sharing</b>        | <b>203</b> |
| 22.1      | Week 22 Roadmap and Mental Model                                         | 204        |
| 22.2      | Monday: Lineage                                                          | 204        |
| 22.2.1    | What Gets Captured                                                       | 204        |
| 22.3      | Tuesday: The Audit Practice                                              | 205        |
| 22.3.1    | system.access.audit as an Alerting Source                                | 205        |
| 22.3.2    | Classification-Driven Policy                                             | 205        |
| 22.4      | Wednesday: Delta Sharing and Clean Rooms                                 | 205        |
| 22.4.1    | Sharing Without Copies                                                   | 205        |



|           |                                                                               |            |
|-----------|-------------------------------------------------------------------------------|------------|
| 22.4.2    | Clean Rooms                                                                   | 205        |
| 22.5      | Thursday Hands-on Project: Trace, Alert, Share, Revoke                        | 206        |
| 22.6      | Friday Use Case: The Supply-Chain Consortium                                  | 207        |
| 22.6.1    | Requirements and Assumptions                                                  | 207        |
| 22.6.2    | Architecture Decisions                                                        | 207        |
| 22.6.3    | Risks and Mitigations                                                         | 208        |
| 22.7      | Design Decision Guide                                                         | 208        |
| 22.8      | Operational Guidance for Week 22                                              | 208        |
| 22.8.1    | Performance and Reliability                                                   | 208        |
| 22.8.2    | Security and Governance Checklist                                             | 208        |
| 22.8.3    | Troubleshooting Playbook                                                      | 209        |
| 22.9      | Interview Q&A                                                                 | 209        |
| 22.9.1    | Concepts and Trade-offs                                                       | 209        |
| 22.10     | Further Reading                                                               | 210        |
| 22.11     | Exercises                                                                     | 210        |
| <b>23</b> | <b>CI/CD with Databricks Asset Bundles, Terraform, and Cost Optimization</b>  | <b>211</b> |
| 23.1      | Week 23 Roadmap and Mental Model                                              | 212        |
| 23.2      | Monday: Asset Bundles                                                         | 212        |
| 23.2.1    | Anatomy                                                                       | 212        |
| 23.2.2    | Targets and Modes                                                             | 213        |
| 23.3      | Tuesday: Terraform for the Workspace Perimeter                                | 213        |
| 23.3.1    | What Terraform Owns versus What Bundles Own                                   | 213        |
| 23.3.2    | State Discipline                                                              | 214        |
| 23.4      | Wednesday: The Pipeline and the FinOps Loop                                   | 214        |
| 23.4.1    | GitHub Actions Shape                                                          | 214        |
| 23.4.2    | FinOps from System Tables                                                     | 215        |
| 23.5      | Thursday Hands-on Project: Bundle, Pipeline, and the Cost Report              | 215        |
| 23.6      | Friday Use Case: Thirty Pipelines, Three Environments, Monthly Accountability | 216        |
| 23.6.1    | Requirements and Assumptions                                                  | 216        |
| 23.6.2    | Architecture Decisions                                                        | 216        |
| 23.6.3    | Risks and Mitigations                                                         | 217        |
| 23.7      | Design Decision Guide                                                         | 217        |
| 23.8      | Operational Guidance for Week 23                                              | 217        |
| 23.8.1    | Performance and Reliability                                                   | 217        |
| 23.8.2    | Security and Governance Checklist                                             | 217        |
| 23.8.3    | Troubleshooting Playbook                                                      | 218        |
| 23.9      | Interview Q&A                                                                 | 218        |
| 23.9.1    | Concepts and Trade-offs                                                       | 218        |
| 23.10     | Further Reading                                                               | 219        |
| 23.11     | Exercises                                                                     | 219        |
| <b>24</b> | <b>Final Capstone and Certification Preparation</b>                           | <b>220</b> |
| 24.1      | Week 24 Roadmap and Mental Model                                              | 220        |
| 24.2      | Monday: The Integrated Architecture Review                                    | 221        |
| 24.2.1    | One Map                                                                       | 221        |
| 24.2.2    | The Weak-Link Audit                                                           | 221        |
| 24.3      | Tuesday: Certification Domains                                                | 221        |
| 24.3.1    | Mapping Domains to Chapters                                                   | 221        |
| 24.4      | Wednesday: Mock Exam Discipline                                               | 222        |
| 24.4.1    | How to Take the Mocks                                                         | 222        |
| 24.4.2    | The Two-Pass Standard                                                         | 222        |



|                                                                    |            |
|--------------------------------------------------------------------|------------|
| 24.5 Thursday: Final Capstone Build . . . . .                      | 222        |
| 24.6 Friday Use Case: The Architecture Defense . . . . .           | 222        |
| 24.6.1 The Structure That Scores . . . . .                         | 223        |
| 24.6.2 Risks and Mitigations . . . . .                             | 223        |
| 24.7 Design Decision Guide . . . . .                               | 223        |
| 24.8 Operational Guidance for Week 24 . . . . .                    | 224        |
| 24.8.1 Performance and Reliability . . . . .                       | 224        |
| 24.8.2 Security and Governance Checklist . . . . .                 | 224        |
| 24.8.3 Troubleshooting Playbook . . . . .                          | 224        |
| 24.9 Interview Q&A . . . . .                                       | 224        |
| 24.9.1 Capstone-Grade Questions . . . . .                          | 224        |
| 24.10 Further Reading . . . . .                                    | 225        |
| 24.11 Exercises . . . . .                                          | 225        |
| 24.12 Milestone 6 Capstone: The Governed Data Platform . . . . .   | 226        |
| <b>A Environment Setup and Sample Data</b>                         | <b>228</b> |
| A.1 Accounts and Workspace . . . . .                               | 228        |
| A.2 Local Tooling . . . . .                                        | 228        |
| A.3 Repository and Credential Conventions . . . . .                | 229        |
| A.4 Workspace Naming Conventions . . . . .                         | 229        |
| A.5 Sample Datasets . . . . .                                      | 229        |
| A.6 Troubleshooting Checklist . . . . .                            | 230        |
| <b>B CLI, SDK, and API Quick Reference</b>                         | <b>231</b> |
| B.1 Databricks CLI Essentials . . . . .                            | 231        |
| B.2 Databricks SDK for Python . . . . .                            | 232        |
| B.3 REST API . . . . .                                             | 232        |
| B.4 Spark SQL and Delta Quick Reference . . . . .                  | 233        |
| B.5 PySpark and Streaming Quick Reference . . . . .                | 233        |
| B.6 Asset Bundle Skeleton . . . . .                                | 234        |
| <b>C Lab Solutions and Acceptance Guidance</b>                     | <b>235</b> |
| C.1 Part I: Lakehouse Foundations (Weeks 1–4) . . . . .            | 235        |
| C.1.1 Week 1: Workspace, CLI, and Smoke Test . . . . .             | 235        |
| C.1.2 Week 2: Schema Enforcement, Evolution, and RESTORE . . . . . | 235        |
| C.1.3 Week 3: Medallion Transformations . . . . .                  | 235        |
| C.1.4 Week 4: Spark Skew Remediation . . . . .                     | 236        |
| C.1.5 Milestone 1 Capstone . . . . .                               | 236        |
| C.2 Part II: Warehousing and Analytics (Weeks 5–8) . . . . .       | 236        |
| C.2.1 Week 5: OPTIMIZE versus Liquid Clustering . . . . .          | 236        |
| C.2.2 Week 6: Warehouse and Materialized View . . . . .            | 236        |
| C.2.3 Week 7: Dashboard and Genie Space . . . . .                  | 236        |
| C.2.4 Week 8: Grants and Masking View . . . . .                    | 236        |
| C.2.5 Milestone 2 Capstone . . . . .                               | 236        |
| C.3 Part III: Batch ETL and Orchestration (Weeks 9–12) . . . . .   | 237        |
| C.3.1 Week 9: PySpark Tuning Evidence Table . . . . .              | 237        |
| C.3.2 Week 10: DLT Expectations . . . . .                          | 237        |
| C.3.3 Week 11: Auto Loader and CDC . . . . .                       | 237        |
| C.3.4 Week 12: Orchestrated Workflow . . . . .                     | 237        |
| C.3.5 Milestone 3 Capstone . . . . .                               | 237        |
| C.4 Part IV: Streaming and Real-Time (Weeks 13–16) . . . . .       | 237        |
| C.4.1 Week 13: Checkpointed Stream Restart . . . . .               | 237        |

|          |                                                                          |            |
|----------|--------------------------------------------------------------------------|------------|
| C.4.2    | Week 14: Watermark and Dedup . . . . .                                   | 237        |
| C.4.3    | Week 15: Kafka Replay Reconciliation . . . . .                           | 238        |
| C.4.4    | Week 16: Real-Time Medallion . . . . .                                   | 238        |
| C.4.5    | Milestone 4 Capstone . . . . .                                           | 238        |
| C.5      | Part V: ML and MLOps (Weeks 17–20) . . . . .                             | 238        |
| C.5.1    | Week 17: Autologged Champion . . . . .                                   | 238        |
| C.5.2    | Week 18: Point-in-Time Training Set . . . . .                            | 238        |
| C.5.3    | Week 19: RAG with Evaluation . . . . .                                   | 238        |
| C.5.4    | Week 20: Serving and Drift Gate . . . . .                                | 238        |
| C.5.5    | Milestone 5 Capstone . . . . .                                           | 238        |
| C.6      | Part VI: Security, Production, and Certification (Weeks 21–24) . . . . . | 239        |
| C.6.1    | Week 21: Service Principal and Secret Scope . . . . .                    | 239        |
| C.6.2    | Week 22: Lineage, Audit, and Sharing . . . . .                           | 239        |
| C.6.3    | Week 23: Bundle and Terraform Deployment . . . . .                       | 239        |
| C.6.4    | Week 24: Mock Exams and Remediation . . . . .                            | 239        |
| C.6.5    | Milestone 6 Capstone . . . . .                                           | 239        |
| <b>D</b> | <b>Production-Readiness and Governance Checklists</b>                    | <b>240</b> |
| D.1      | Workspace and Unity Catalog (Chapters 0, 1, 8) . . . . .                 | 240        |
| D.2      | Delta Lake and Storage (Chapters 2, 3, 5) . . . . .                      | 241        |
| D.3      | SQL Warehouses and BI (Chapters 6, 7) . . . . .                          | 241        |
| D.4      | DLT and Batch Pipelines (Chapters 9–12) . . . . .                        | 241        |
| D.5      | Streaming and Real-Time (Chapters 13–16) . . . . .                       | 242        |
| D.6      | ML and Mosaic AI (Chapters 17–20) . . . . .                              | 242        |
| D.7      | Security and Networking (Chapters 21, 22) . . . . .                      | 243        |
| D.8      | CI/CD and FinOps (Chapters 23, 24) . . . . .                             | 243        |
| <b>E</b> | <b>Certification Exam Mapping and Study Guide</b>                        | <b>245</b> |
| E.1      | The Two Exams at a Glance . . . . .                                      | 245        |
| E.2      | Domain-to-Chapter Mapping . . . . .                                      | 245        |
| E.2.1    | Where the Exams Probe . . . . .                                          | 246        |
| E.3      | Study Strategy . . . . .                                                 | 246        |
| E.4      | Self-Assessment Checklist . . . . .                                      | 247        |
| <b>F</b> | <b>Interview Question Bank</b>                                           | <b>249</b> |
| F.1      | Platform and Workspace (Chapter 1) . . . . .                             | 249        |
| F.2      | Delta Lake and Medallion Design (Chapters 2, 3, 5) . . . . .             | 249        |
| F.3      | Spark Execution and Performance (Chapters 4, 9) . . . . .                | 250        |
| F.4      | SQL Warehousing and BI (Chapters 6, 7) . . . . .                         | 251        |
| F.5      | Governance and Security (Chapters 8, 21, 22) . . . . .                   | 251        |
| F.6      | Ingestion, Pipelines, and Orchestration (Chapters 10–12) . . . . .       | 252        |
| F.7      | Streaming and Real-Time (Chapters 13–16) . . . . .                       | 253        |
| F.8      | ML and MLOps (Chapters 17–20) . . . . .                                  | 253        |
| F.9      | Delivery and CI/CD (Chapter 23) . . . . .                                | 254        |
| F.10     | Capstone and Architecture Defense (Chapter 24) . . . . .                 | 255        |
| <b>G</b> | <b>Cross-Cloud Service Equivalence</b>                                   | <b>256</b> |
| G.1      | Platform Foundations and Compute . . . . .                               | 256        |
| G.2      | Table Storage and the Lakehouse . . . . .                                | 256        |
| G.3      | The Medallion Pattern . . . . .                                          | 257        |
| G.4      | Managed Spark and Execution Engines . . . . .                            | 257        |
| G.5      | SQL Warehousing and BI . . . . .                                         | 258        |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| G.6 Governance and Catalogs . . . . .                             | 258        |
| G.7 Declarative Pipelines, Ingestion, and Orchestration . . . . . | 258        |
| G.8 Streaming . . . . .                                           | 259        |
| G.9 Machine Learning and MLOps . . . . .                          | 259        |
| G.10 Security, Identity, and Networking . . . . .                 | 260        |
| G.11 Infrastructure as Code and CI/CD . . . . .                   | 260        |
| G.12 Certification and Readiness . . . . .                        | 261        |
| <b>Glossary</b>                                                   | <b>262</b> |

# List of Figures

|      |                                                                                                                                                                |     |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1.1  | A multi-team workspace layout: one account, environment-split workspaces, one regional metastore, policy-bound compute, and service-principal production runs. | 16  |
| 2.1  | An auditable Delta table: versioned log with 90-day retention, key-based corrections, auditor time-travel queries, and history as change evidence.             | 26  |
| 3.1  | Zero-double-count restatement: rebuild into parallel tables, then swap via view or catalog rename in one auditable step.                                       | 34  |
| 4.1  | The evidence-driven tuning loop: locate, attribute, hypothesize, measure.                                                                                      | 43  |
| 4.2  | Milestone 1 reference architecture: medallion lakehouse with quality gates and recovery drill.                                                                 | 48  |
| 5.1  | Small-file remediation: pause, snapshot, batch compaction, then prevent recurrence with clustering and write-cadence changes.                                  | 56  |
| 6.1  | Workload-class warehouse topology over one governed Gold layer: isolation without data copies.                                                                 | 64  |
| 7.1  | One semantic layer feeding both certified dashboards and exploratory Genie, with lineage into the audit review.                                                | 72  |
| 8.1  | Healthcare governance: group-scoped layer access, default masking, break-glass with review, and audit everywhere.                                              | 80  |
| 8.2  | Milestone 2 reference architecture: governed serving from optimized Gold through one semantic layer.                                                           | 85  |
| 9.1  | The one-week performance engagement: every phase produces an artifact.                                                                                         | 91  |
| 10.1 | The adoption boundary: DLT where declaration wins, hand-written Spark where control wins.                                                                      | 100 |
| 11.1 | Dual-path ingestion: high-rate files via notification-mode Auto Loader, nightly CDC via SCD2, reconciled by a gated job.                                       | 108 |
| 12.1 | Federated orchestration: team-owned jobs, contract tables between teams, one platform-owned ledger and SLA monitor.                                            | 116 |
| 13.1 | Sub-minute ingestion: partition-scaled Kafka, capped micro-batches, one Bronze table serving both latencies.                                                   | 127 |
| 14.1 | Split-brain lateness design: fast alerts in update mode, settled windows in append mode, one watermark over both.                                              | 135 |

|      |                                                                                                                                                 |     |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 15.1 | Migration topology: mirror the old bus into the new, consume both through one deduplicating stream, retire the bridge. . . . .                  | 143 |
| 16.1 | Fleet lakehouse: one Bronze log feeding a real-time dispatch tier and a reconciled nightly tier. . . . .                                        | 152 |
| 16.2 | Milestone 4 reference architecture: dedup-before-rules fraud telemetry. . . . .                                                                 | 157 |
| 17.1 | Regulated model lifecycle: every link from data to production prediction is recorded and queryable. . . . .                                     | 163 |
| 18.1 | The audit: find the leak, find the skew, replace both with one governed feature definition. . . . .                                             | 171 |
| 19.1 | Governed copilot: redaction before indexing, evaluation gating changes, token budgets bounding cost. . . . .                                    | 179 |
| 20.1 | The MLOps control loop: sense, decide (with a human gate), act by alias, stay reversible. . . . .                                               | 187 |
| 20.2 | Milestone 5 reference architecture: governed MLOps loop with the RAG branch. . . . .                                                            | 192 |
| 21.1 | Zero-public-internet posture: IdP-driven identity, group-scoped grants, private data paths, continuous audit. . . . .                           | 199 |
| 22.1 | Consortium sharing: open-protocol shares serve every member platform; the provider keeps credentials, audit, and the revocation switch. . . . . | 207 |
| 23.1 | The release train: every change rides the same validated, gated track; cost closes the loop monthly. . . . .                                    | 216 |
| 24.1 | Milestone 6: the complete platform—one governed estate, deployed as code, measured continuously. . . . .                                        | 227 |

# List of Tables

|      |                                                                |     |
|------|----------------------------------------------------------------|-----|
| 1    | Naming conventions used across all labs and capstones. . . . . | 5   |
| 2    | Chapter 0 decision guide. . . . .                              | 6   |
| 1.1  | Compute options and their engineering trade-offs. . . . .      | 14  |
| 1.2  | Week 1 scenario risks and mitigations. . . . .                 | 17  |
| 1.3  | Week 1 decision guide. . . . .                                 | 18  |
| 1.4  | Week 1 troubleshooting quick reference. . . . .                | 18  |
| 2.1  | Week 2 scenario risks and mitigations. . . . .                 | 27  |
| 2.2  | Week 2 decision guide. . . . .                                 | 27  |
| 2.3  | Week 2 troubleshooting quick reference. . . . .                | 28  |
| 3.1  | One job, one audience, one write pattern per layer. . . . .    | 33  |
| 3.2  | Week 3 scenario risks and mitigations. . . . .                 | 35  |
| 3.3  | Week 3 decision guide. . . . .                                 | 35  |
| 3.4  | Week 3 troubleshooting quick reference. . . . .                | 36  |
| 4.1  | Week 4 scenario risks and mitigations. . . . .                 | 44  |
| 4.2  | Week 4 decision guide. . . . .                                 | 44  |
| 4.3  | Week 4 troubleshooting quick reference. . . . .                | 45  |
| 5.1  | Week 5 scenario risks and mitigations. . . . .                 | 56  |
| 5.2  | Week 5 decision guide. . . . .                                 | 57  |
| 5.3  | Week 5 troubleshooting quick reference. . . . .                | 58  |
| 6.1  | Week 6 scenario risks and mitigations. . . . .                 | 65  |
| 6.2  | Week 6 decision guide. . . . .                                 | 65  |
| 6.3  | Week 6 troubleshooting quick reference. . . . .                | 66  |
| 7.1  | Week 7 scenario risks and mitigations. . . . .                 | 72  |
| 7.2  | Week 7 decision guide. . . . .                                 | 73  |
| 7.3  | Week 7 troubleshooting quick reference. . . . .                | 74  |
| 8.1  | Week 8 scenario risks and mitigations. . . . .                 | 81  |
| 8.2  | Week 8 decision guide. . . . .                                 | 81  |
| 8.3  | Week 8 troubleshooting quick reference. . . . .                | 82  |
| 9.1  | Week 9 scenario risks and mitigations. . . . .                 | 92  |
| 9.2  | Week 9 decision guide. . . . .                                 | 92  |
| 9.3  | Week 9 troubleshooting quick reference. . . . .                | 93  |
| 10.1 | Week 10 scenario risks and mitigations. . . . .                | 101 |

|      |                                                                              |     |
|------|------------------------------------------------------------------------------|-----|
| 10.2 | Week 10 decision guide. . . . .                                              | 101 |
| 10.3 | Week 10 troubleshooting quick reference. . . . .                             | 102 |
| 11.1 | Week 11 scenario risks and mitigations. . . . .                              | 109 |
| 11.2 | Week 11 decision guide. . . . .                                              | 109 |
| 11.3 | Week 11 troubleshooting quick reference. . . . .                             | 110 |
| 12.1 | Week 12 scenario risks and mitigations. . . . .                              | 117 |
| 12.2 | Week 12 decision guide. . . . .                                              | 117 |
| 12.3 | Week 12 troubleshooting quick reference. . . . .                             | 118 |
| 13.1 | Week 13 scenario risks and mitigations. . . . .                              | 128 |
| 13.2 | Week 13 decision guide. . . . .                                              | 128 |
| 13.3 | Week 13 troubleshooting quick reference. . . . .                             | 129 |
| 14.1 | Week 14 scenario risks and mitigations. . . . .                              | 136 |
| 14.2 | Week 14 decision guide. . . . .                                              | 136 |
| 14.3 | Week 14 troubleshooting quick reference. . . . .                             | 137 |
| 15.1 | Week 15 scenario risks and mitigations. . . . .                              | 144 |
| 15.2 | Week 15 decision guide. . . . .                                              | 144 |
| 15.3 | Week 15 troubleshooting quick reference. . . . .                             | 145 |
| 16.1 | The latency/cost dial: four settings, four price points. . . . .             | 151 |
| 16.2 | Week 16 scenario risks and mitigations. . . . .                              | 153 |
| 16.3 | Week 16 decision guide. . . . .                                              | 153 |
| 16.4 | Week 16 troubleshooting quick reference. . . . .                             | 154 |
| 17.1 | Week 17 scenario risks and mitigations. . . . .                              | 164 |
| 17.2 | Week 17 decision guide. . . . .                                              | 164 |
| 17.3 | Week 17 troubleshooting quick reference. . . . .                             | 165 |
| 18.1 | Week 18 scenario risks and mitigations. . . . .                              | 172 |
| 18.2 | Week 18 decision guide. . . . .                                              | 172 |
| 18.3 | Week 18 troubleshooting quick reference. . . . .                             | 173 |
| 19.1 | Week 19 scenario risks and mitigations. . . . .                              | 180 |
| 19.2 | Week 19 decision guide. . . . .                                              | 180 |
| 19.3 | Week 19 troubleshooting quick reference. . . . .                             | 181 |
| 20.1 | Week 20 scenario risks and mitigations. . . . .                              | 188 |
| 20.2 | Week 20 decision guide. . . . .                                              | 189 |
| 20.3 | Week 20 troubleshooting quick reference. . . . .                             | 189 |
| 21.1 | Each control has a known blind spot; the posture is the combination. . . . . | 197 |
| 21.2 | Week 21 scenario risks and mitigations. . . . .                              | 199 |
| 21.3 | Week 21 decision guide. . . . .                                              | 200 |
| 21.4 | Week 21 troubleshooting quick reference. . . . .                             | 201 |
| 22.1 | Week 22 scenario risks and mitigations. . . . .                              | 208 |
| 22.2 | Week 22 decision guide. . . . .                                              | 208 |
| 22.3 | Week 22 troubleshooting quick reference. . . . .                             | 209 |
| 23.1 | Week 23 scenario risks and mitigations. . . . .                              | 217 |
| 23.2 | Week 23 decision guide. . . . .                                              | 217 |

|      |                                                                                           |     |
|------|-------------------------------------------------------------------------------------------|-----|
| 23.3 | Week 23 troubleshooting quick reference. . . . .                                          | 218 |
| 24.1 | Exam domains mapped to chapters and their remedial labs. . . . .                          | 222 |
| 24.2 | Week 24 scenario risks and mitigations. . . . .                                           | 223 |
| 24.3 | Week 24 decision guide. . . . .                                                           | 223 |
| 24.4 | Week 24 troubleshooting quick reference. . . . .                                          | 224 |
| A.1  | Standard datasets and where they are used. . . . .                                        | 229 |
| E.1  | Exam domains mapped to chapters and remedial labs (extends Chapter 24's mapping). . . . . | 246 |



# Getting Started: Your Databricks Workspace and Environment

## Learning Objectives

- Create a Databricks workspace, distinguish the free trial from Community Edition and an organizational workspace, and identify the inputs required during sign-up.
- Secure the workspace by avoiding shared admin use, understanding personal access token (PAT) policy, and preferring OAuth for daily work.
- Install and validate the Databricks CLI and `databricks-sdk-py` on Windows PowerShell, and compare token configuration with OAuth profiles.
- Configure CLI profiles, the workspace host, and VS Code so examples in later chapters can run reproducibly.
- Establish cost guardrails with cluster policies, auto-termination, serverless compute, and billing alerts.
- Apply the catalog, schema, and naming conventions used throughout this book.
- Verify the environment with the CLI and a short notebook create, run, and cleanup workflow.

## Key Terms

**Workspace:** the top-level Databricks environment where notebooks, clusters, jobs, and data objects live. **Control plane:** Databricks-managed services (web UI, jobs scheduler, cluster manager). **Data plane:** compute and storage in your cloud account. **DBU:** Databricks Unit, the per-second compute billing unit. **CLI profile:** a named set of host and credential configuration for the Databricks CLI.

## 0.1 Why Chapter 0 Exists

Before creating Delta tables, pipelines, streams, or serving endpoints, you need a safe sandbox. This chapter builds that sandbox deliberately: workspace, identity, command-line tooling, compute guardrails, naming rules, and a tiny verification exercise. The goal is not to become a workspace administrator in one sitting. The goal is to remove the avoidable problems that derail hands-on learning: working with a personal admin identity every day, forgetting auto-termination, losing track of CLI profiles, creating objects without naming conventions, or discovering charges after a lab was supposed to be cleaned up.

This book assumes a Windows workstation and PowerShell. Most Databricks commands are the

same on macOS, Linux, and Windows Terminal, but file paths, environment variables, quoting, and here-documents differ. Where the shell matters, listings use native PowerShell conventions.

### Note

The examples in this chapter use a personal trial workspace or an organization-approved sandbox. Do not run training commands in a production workspace unless your employer has explicitly approved the workspace, compute policy, budget, and cleanup process.

## 0.2 Creating a Workspace

### 0.2.1 What You Need Before Sign-up

To start a Databricks trial, prepare a work or personal email address and—for a full trial on your own cloud account—access to an AWS, Azure, or GCP subscription where the workspace’s data plane will run. Databricks also offers express and Community Edition paths that require no cloud account at all [20]. Choose deliberately:

- **Full trial (own cloud account):** complete feature set including classic clusters, Unity Catalog with your own storage, and VNet/VPC options. Compute VMs bill to your cloud account; Databricks bills DBUs against trial credits.
- **Express / serverless trial:** no cloud account required; serverless compute only. Covers most of this book’s SQL, pipeline, and ML labs.
- **Community Edition:** a permanently free, limited workspace. Sufficient for Delta Lake, Spark, and MLflow fundamentals; not sufficient for Unity Catalog, SQL warehouses, or workflows at production scale.

### Definition

**The Databricks free trial** is a credit grant, not an unlimited free account. Trial credits cover DBU consumption; on the full trial, the underlying cloud infrastructure (VMs, storage, egress) bills to your own cloud account from the first minute a classic cluster runs.

### 0.2.2 Understanding the Cost Model

Databricks cost has two components: **DBUs** (the platform’s compute unit, billed per second, with different rates per workload type—all-purpose, jobs, SQL, serverless) and **cloud infrastructure** (the VMs, disks, and egress in your cloud account, billed by the cloud provider). A stopped cluster bills neither; an idle all-purpose cluster bills both until auto-termination fires.

### Common Pitfall

**Assuming the trial means no bill.** Trial credits absorb DBUs, but cloud VM hours from forgotten classic clusters, storage of intermediate tables, and cross-region egress still bill to your cloud account. Chapter 23 builds the habit of reading `system.billing.usage` weekly.

## 0.3 Securing the Workspace

### 0.3.1 Identity Basics

The account you create with is an administrator. For daily learning work this is acceptable in a trial, but build the production habit early: administrators administer; engineers engineer.

When your trial allows, create a second user or a service principal for pipeline work, and keep admin actions (user management, metastore binding, network configuration) distinct from data work.

### 0.3.2 Personal Access Tokens versus OAuth

A personal access token (PAT) is a long-lived credential tied to your user. OAuth issues short-lived tokens negotiated by the CLI. The rules of thumb:

- **Interactive local work:** use OAuth via `databricks auth login`; the CLI refreshes tokens for you.
- **Automation:** use a service principal with OAuth machine-to-machine credentials, not a person's PAT.
- **If a PAT is unavoidable** (some Community Edition flows): set the shortest expiry allowed, store it in an environment variable or secret manager, and never commit it.

#### Tip

Check your workspace's PAT policy early. Many organizations disable PATs entirely or cap their lifetime; learning the OAuth path first saves a painful re-authentication surprise in Week 23's CI/CD lab.

## 0.4 Installing and Configuring the Tooling

### 0.4.1 The Databricks CLI

The Databricks CLI (v0.200 and later) is the command plane for workspace objects, clusters, jobs, pipelines, secrets, and Asset Bundles [18]. Install it with the native installer or via package managers, then authenticate:

```

1 # Verify Python and Git first
2 python --version
3 git --version
4
5 # CLI check (install from https://docs.databricks.com/dev-tools/cli/ if missing
6 # )
7 databricks --version
8
9 # OAuth login; stores a profile in ~/.databrickscfg
10 databricks auth login --host https://<your-workspace>.azuredatabricks.net
11
12 # Who am I? The first real verification
13 databricks current-user me
14
15 # Can I see the workspace?
16 databricks workspace list /

```

Listing 1: Installing and authenticating the Databricks CLI on Windows.

### 0.4.2 The Python SDK

The `databricks-sdk-py` package mirrors the REST API and powers the automation scripts in Chapters 1, 12, and 23 [22]:

```

1 # pip install databricks-sdk
2 from databricks.sdk import WorkspaceClient
3

```

```

4 w = WorkspaceClient() # reads the DEFAULT CLI profile or env vars
5 me = w.current_user.me()
6 print(f"Authenticated as {me.user_name} on {w.config.host}")

```

Listing 2: SDK installation check and identity call.

### 0.4.3 Environment Variables and Profiles

The CLI and SDK resolve credentials in a documented order: explicit flags, environment variables (DATABRICKS\_HOST, DATABRICKS\_TOKEN, or OAuth client fields), then the ~/.databrickscfg profile. On PowerShell, set session variables like this:

```

1 $env:DATABRICKS_HOST = 'https://<your-workspace>.azuredatabricks.net'
2 # Only if you must use a token; prefer OAuth profiles:
3 $env:DATABRICKS_TOKEN = '<from-your-secret-manager>'

```

Listing 3: Session-scoped credential environment variables.

## 0.5 Cost Guardrails to Stay Within Credits

### 0.5.1 Auto-Termination and Cluster Policies

Every classic cluster you create should have auto-termination set to 30 minutes or less. Better: enforce it. A **cluster policy** can fix autotermination\_minutes, cap worker counts, and restrict node types, so a forgotten cluster cannot bill beyond the policy's bounds [14].

### 0.5.2 Serverless First Where Possible

Serverless compute (notebooks, DLT, SQL warehouses) starts fast, bills per second, and idles to zero. For most labs in this book, serverless or the smallest job cluster is the economical choice; large all-purpose clusters are for the tuning chapters where you need to observe the behavior.

### 0.5.3 Budget Alerts and the Billing Tables

Set a budget alert in your cloud account on day one. Once Unity Catalog system tables are enabled, system.billing.usage gives you per-SKU DBU consumption to correlate with cloud VM cost [34]:

```

1 SELECT sku_name, ROUND(SUM(usage_quantity), 2) AS dbus
2 FROM system.billing.usage
3 WHERE usage_date >= current_date() - 7
4 GROUP BY sku_name
5 ORDER BY dbus DESC;

```

Listing 4: Weekly DBU consumption by SKU—run it every Friday.

#### Common Pitfall

**The weekend cluster.** The single most common first-month bill shock is an all-purpose cluster left running from Friday to Monday. Auto-termination plus a policy cap makes this a non-event; without them it is a three-figure lesson.

## 0.6 Naming Conventions

Labs reference objects by predictable names. The course catalog is **de\_training** with schemas **bronze**, **silver**, and **gold**; raw files land in the volume **de\_training.bronze.landing**. Clusters are **de-weekNN-cluster**, jobs **de-weekNN-<purpose>**, and bundle targets **dev**, **staging**,

prod. Consistency matters because runbooks, alerts, and acceptance checklists all reference these names.

| Object   | Convention                                  | Example                    |
|----------|---------------------------------------------|----------------------------|
| Catalog  | de_training (labs),<br>de_prod (capstone 6) | de_training                |
| Schemas  | medallion layers                            | bronze, silver, gold       |
| Volume   | purpose-named under bronze                  | de_training.bronze.landing |
| Tables   | layer-qualified business name               | de_training.silver.orders  |
| Clusters | week-scoped                                 | de-week04-cluster          |
| Jobs     | week-scoped with purpose                    | de-week12-nightly-elt      |

Table 1: Naming conventions used across all labs and capstones.

## 0.7 Verification: Hello World

Close the chapter by proving the loop end to end: CLI identity, a notebook run, a Delta table, and cleanup.

```

1 # 1. Identity
2 databricks current-user me
3
4 # 2. Create the course catalog and schemas (requires UC create rights)
5 databricks schemas create bronze --catalog de_training
6 databricks schemas create silver --catalog de_training
7 databricks schemas create gold --catalog de_training
8
9 # 3. In a notebook (or the SQL editor), create and read back a table:
10 # CREATE OR REPLACE TABLE de_training.bronze.hello AS
11 # SELECT 'hello lakehouse' AS message, current_timestamp() AS created_at;
12 # SELECT * FROM de_training.bronze.hello;

```

Listing 5: End-to-end smoke test: CLI identity plus a first table.

```

1 from databricks.sdk import WorkspaceClient
2 from databricks.sdk.service.jobs import SubmitTask, NotebookTask
3
4 w = WorkspaceClient()
5 run = w.jobs.submit(
6     run_name="ch0-smoke-test",
7     tasks=[SubmitTask(
8         task_key="smoke",
9         notebook_task=NotebookTask(
10             notebook_path="/Workspace/Users/<you>/ch0-smoke"),
11     )],
12 )
13 print(run.result().state.state_message) # expect: success

```

Listing 6: SDK smoke test: submit a notebook run programmatically.

If all three steps succeed, your identity, CLI, SDK, catalog, and compute path all work, and every subsequent chapter’s labs can run as written.

## 0.8 Design Decision Guide

Setup decisions echo for six months; the defaults below are the ones this book assumes.

| Decision                               | Choose the first when                | Choose the second when              |
|----------------------------------------|--------------------------------------|-------------------------------------|
| Trial vs. Community Edition            | You want the full platform surface   | Zero-cost fundamentals only         |
| Own cloud account vs. serverless trial | Classic clusters, VNet, your storage | No cloud account available          |
| OAuth profile vs. PAT                  | Always for local work                | Only where OAuth is unavailable     |
| Policy-bound vs. free clusters         | Any shared or lasting workspace      | Disposable personal sandboxes       |
| Serverless-first vs. classic-first     | Most labs                            | Tuning chapters that need the knobs |

Table 2: Chapter 0 decision guide.

If a later chapter’s lab behaves differently than expected, the cause is usually one of these five choices made differently. Check them first.

## 0.9 Operational Guidance for the Whole Program

### 0.9.1 Performance and Reliability

From Chapter 0 onward, treat the lab estate like a small production platform: clusters auto-terminate, budgets alert, and the Friday cost review runs weekly. The smoke test (identity, notebook run, first table) is your regression test after any environment change—a CLI upgrade, a new profile, a new workspace. Keep the setup reproducible: everything in this chapter should be re-executable from the repository README by a stranger.

### 0.9.2 Security and Governance Checklist

- Credentials live in OAuth profiles, environment variables, or secret scopes—never in files.
- The training catalog (`de_training`) is separate from any real data.
- Secret scanning is enabled on the course repository before the first push.
- Cloud budget alerts exist before the first classic cluster is created.

## 0.10 Further Reading

Start with the Databricks documentation portal for workspace and compute concepts [19, 52, 14]. Review the CLI [18] and Python SDK [22] references, and the trial/Community Edition options [20]. For the architecture this book builds toward, read the lakehouse paper [9] early; it frames every storage and governance decision in Parts I and II.

## Key Takeaways

- A Databricks workspace plus deliberate tooling (CLI profile, SDK, VS Code) is the sandbox every later lab assumes.
- Cost on Databricks is DBUs plus cloud infrastructure; auto-termination, cluster policies, and serverless-first habits keep trial credits intact.
- OAuth profiles beat long-lived tokens; production automation belongs to service principals, not people.

- Consistent catalog/schema/naming conventions turn six months of labs into a navigable, reviewable portfolio.
- The smoke test (identity, notebook run, first table) proves the full loop before Week 1 begins.

## 0.11 Exercises

1. **(Conceptual)** Explain the control plane / data plane split and name one object that lives in each.
2. **(Conceptual)** Your trial expires in 14 days. Which three artifacts would you export or document first, and why?
3. **(Hands-on)** Run the full Verification section. Capture `databricks current-user me` output and the `hello` table read as evidence.
4. **(Hands-on)** Create a cluster policy that caps auto-termination at 30 minutes and workers at 2. Create a cluster from it and show the enforced settings.
5. **(Hands-on)** Run the billing query after your first week of labs and identify your largest SKU.
6. **(Security)** Convert a token-based CLI profile to an OAuth profile. Explain what changed about credential lifetime.
7. **(Design)** Write the naming conventions for a two-team organization (finance, logistics) sharing one workspace, extending table 1.
8. **(Operations)** Draft the Friday cost-hygiene checklist you would run weekly for the rest of this program.

## 0.12 Curated Community Resources

Beyond this book and the vendor documentation, the following community-curated resources are worth your time. Do not try to consume everything at once: pick one resource per category, practice it with real data, and build something small before moving on.

### Practical Case Studies (Video Walkthroughs)

Fifteen end-to-end builds that show how the tools work together:

1. SQL Data Warehouse from Scratch — SQL, ETL, data modeling, star schemas (<https://lnkd.in/gcrJ2qde>)
2. End-to-End Data Analytics Project — Python, Pandas, SQL Server, data cleaning (<https://lnkd.in/gHxTN-B2>)
3. Netflix Data Engineering Project — Python, SQL, ELT, exploratory analysis (<https://lnkd.in/gMkbpWmh>)
4. DuckDB and Python Data Engineering Project — API ingestion, pipeline development (<https://lnkd.in/gEPnubGA>)
5. Airflow Podcast Data Pipeline — Airflow, APIs, scheduling (<https://lnkd.in/gmy5rS4Z>)
6. Automated Python ETL Pipeline with Airflow — SQL Server, DAG development (<https://lnkd.in/ghrBc8BE>)



7. PySpark and dbt Data Engineering Project — incremental loading, dimensional modeling (<https://lnkd.in/gVZjvyR7>)
8. Apache Spark Data Engineering Project — PySpark, Databricks (<https://lnkd.in/gVNkqk5A>)
9. Airbnb Data Engineering Project — dbt, Snowflake, AWS, testing (<https://lnkd.in/g2f4TxPW>)
10. AWS ETL Pipeline Project — S3, Lambda, Glue, Athena, serverless ETL (<https://lnkd.in/gFC3NkNs>)
11. AWS Data Warehouse Project — PySpark, Glue, Redshift (<https://lnkd.in/gyZtHMEF>)
12. End-to-End Azure Data Engineering Project — Data Factory, Databricks, ADLS (<https://lnkd.in/gUpbRjzE>)
13. Spotify Azure Data Engineering Project — streaming, Delta Lake (<https://lnkd.in/gUGKuZ8N>)
14. Uber Real-Time Data Engineering Project — Event Hubs, structured streaming (<https://lnkd.in/gu7Tpg7D>)
15. End-to-End GCP Data Engineering Project — Cloud Storage, Dataproc, BigQuery (<https://lnkd.in/gv8Yedfu>)

## Free Video Courses

Twenty videos covering the essential skill path — recommended learning order: Python, SQL, Linux, Git, Docker, data modeling, data warehousing, Spark, Kafka, Airflow, dbt, cloud, portfolio projects.

- **Foundations:** Data Engineering Course for Beginners (<https://lnkd.in/gprfkqnp>); Big Data Engineer Masterclass (<https://lnkd.in/gqPkrg63>)
- **Core tools:** Python for Data Engineers (<https://lnkd.in/gZeKJEi4>); SQL for Big Data Engineering (<https://lnkd.in/gt5DZXwQ>); PostgreSQL Full Course (<https://lnkd.in/gAU5n7gM>); Linux Command Line (<https://lnkd.in/gA-7zFaQ>); Git and GitHub Tutorial (<https://lnkd.in/gs73kHau>); Docker Tutorial (<https://lnkd.in/g59TCWPD>)
- **Warehousing and modeling:** Data Warehousing Beginner's Guide ([https://lnkd.in/gS\\_-iJAb](https://lnkd.in/gS_-iJAb)); Data Modeling, Star Schema and Kimball (<https://lnkd.in/g4it9vpy>)
- **Batch processing:** PySpark Tutorial for Beginners (<https://lnkd.in/gre86dH6>); PySpark Full Course (<https://lnkd.in/gZXg38hk>)
- **Streaming:** Apache Kafka Full Course ([https://lnkd.in/gnnXc9\\_d](https://lnkd.in/gnnXc9_d)); Kafka Crash Course with Project (<https://lnkd.in/gP6DFCem>)
- **Pipelines:** Apache Airflow Tutorial (<https://lnkd.in/gbFzzReZ>); dbt Tutorial with Netflix Project (<https://lnkd.in/gHvmdjNX>); ELT with dbt, Snowflake and Airflow ([https://lnkd.in/g5ZP\\_xC3](https://lnkd.in/g5ZP_xC3))
- **Cloud:** AWS for Data Engineers (<https://lnkd.in/g8vRZmkp>)
- **End-to-end projects:** Airbnb Project with Snowflake, dbt and AWS (<https://lnkd.in/g2f4TxPW>); Real-Time E-Commerce with Kafka and Snowflake (<https://lnkd.in/g58V6A3P>)



## GitHub Repositories

Twenty free repositories to learn from, practice with, and build upon. Choose one roadmap, complete one project, study the code structure, then break it, fix it, and document what you learned.

- **Roadmaps and guides:** Data Engineering Zoomcamp (<https://lnkd.in/gxKmbbHc>); Data Engineer Handbook (<https://lnkd.in/gstev9z7>); Data Engineering Cookbook (<https://lnkd.in/gABzJ-vE>); Complete Data Engineer Roadmap (<https://lnkd.in/gmwQrmvm>)
- **Curated lists:** Awesome Data Engineering (<https://lnkd.in/gh4Ky273>); Awesome Big Data ([https://lnkd.in/g\\_y-n3zm](https://lnkd.in/g_y-n3zm)); Awesome Open-Source Data Engineering (<https://lnkd.in/ggDTw66U>)
- **Portfolio and analytics engineering:** Data Engineering Project Template (<https://lnkd.in/g-QKrMXV>); Jaffle Shop dbt Practice Project (<https://lnkd.in/gmXVhYKX>); dbt Core (<https://lnkd.in/gnCXxNbB>)
- **Ingestion and orchestration:** Apache Airflow (<https://lnkd.in/gqvG5F6N>); Airbyte (<https://lnkd.in/g2B9AVwM>); dlt (<https://lnkd.in/g3A8BMp3>)
- **Processing:** Apache Spark (<https://lnkd.in/g5CPE3hh>); Apache Kafka (<https://lnkd.in/gdH5T4xX>)
- **Storage and lakehouses:** DuckDB (<https://lnkd.in/gehQd42W>); Apache Iceberg (<https://lnkd.in/gVweVV8y>)
- **Quality, lineage and governance:** Great Expectations (<https://lnkd.in/g8TPyAy8>); OpenLineage ([https://lnkd.in/gz3\\_MiAP](https://lnkd.in/gz3_MiAP)); DataHub (<https://lnkd.in/gEcNMRkJ>)

## **Part I**

# **Lakehouse Foundations**

# The Databricks Platform and Workspace Fundamentals

## Learning Objectives

- Explain the Databricks control plane / data plane split and where notebooks, clusters, jobs, and data physically live.
- Navigate workspace objects—notebooks, Git folders, libraries, and the catalog explorer—and choose the right surface for a task.
- Compare all-purpose clusters, job clusters, pools, and serverless compute, and enforce standards with cluster policies.
- Authenticate and use the Databricks CLI and Python SDK against a workspace.
- Run a smoke-test notebook programmatically and verify compute, identity, and storage end to end.
- Design a workspace and compute layout for a multi-team organization with environment isolation.

## Cross-Cloud Equivalence

The Databricks **workspace** is the administrative home base; every cloud has an equivalent concept with a different blast radius.

| Concept             | Databricks                  | AWS                            | Azure / GCP                           |
|---------------------|-----------------------------|--------------------------------|---------------------------------------|
| Admin home          | Workspace + account console | AWS account + console          | Entra tenant / GCP project            |
| Notebook UX         | Databricks notebooks        | EMR Studio / SageMaker         | Synapse / Fabric notebooks; Vertex AI |
| Interactive compute | All-purpose cluster         | EMR cluster / Glue interactive | Synapse Spark pool / Dataproc         |
| Per-run compute     | Job cluster                 | Glue job / EMR Serverless      | Fabric job / Dataproc Serverless      |
| Compute guardrails  | Cluster policies            | IAM + service quotas           | Azure Policy / org policy             |
| CLI                 | <b>databricks</b>           | <b>aws</b>                     | <b>az</b> / <b>gcloud</b>             |
| Billing unit        | DBU + VM cost               | DPU / node-hours               | CU (Fabric) / slots (BigQuery)        |
| Auth for automation | Service principal + OAuth   | IAM role                       | Managed identity / service account    |

**Snowflake** collapses the compute decision into virtual warehouses (size, not topology); **Fabric** into a shared capacity pool. Databricks keeps explicit cluster control—more knobs,

more responsibility, which is why cluster policies matter from day one.

## 1.1 Week 1 Roadmap and Mental Model

Databricks is a data intelligence platform: one environment where data engineering, SQL analytics, data science, and governance share the same tables and the same catalog. Week 1 builds the mental model the rest of the book depends on: what the workspace is, how compute works, how code is organized, and how to drive everything from the CLI and SDK so that manual clicks can later become automation [19].

A useful starting model is the *plane split*. The **control plane**—managed by Databricks—hosts the web application, notebook storage, the jobs scheduler, and cluster management. The **data plane**—in your cloud account (classic) or Databricks’ account (serverless)—runs the compute that touches your data and holds the storage accounts where Delta tables live. Security reviews, network design, and cost analysis all start by asking which plane a component lives in [52].

### Definition

A **Databricks workspace** is the deployment boundary: one URL, one set of users and groups, one catalog binding, one network posture. An **account** (account console) can govern many workspaces, metastores, and budgets.

### Key Terms

**Cluster**: a set of VMs running the Databricks Runtime. **All-purpose cluster**: interactive compute shared by users. **Job cluster**: ephemeral compute created for one job run. **Pool**: idle instances kept warm to cut cluster start time. **Cluster policy**: an admin-defined template restricting cluster configuration. **Git folder (Repo)**: a workspace folder synced to a remote Git branch.

## 1.2 Monday: Platform Architecture and Workspace Objects

### 1.2.1 The Control Plane and the Data Plane

Every request you make—opening a notebook, starting a cluster, running a job—is orchestrated by the control plane, but data never has to leave your cloud account in a classic deployment. This separation is why Databricks can offer a SaaS experience while satisfying data-residency requirements: the control plane stores metadata and orchestration state; your storage accounts and cluster VMs hold the actual rows [39].

For a data engineer, the plane split drives three habits. First, cost: control-plane features are flat, but every running VM in the data plane bills cloud infrastructure plus DBUs. Second, networking: private connectivity, firewalls, and egress rules are data-plane concerns (Chapter 21). Third, availability: a control-plane incident blocks orchestration, while a data-plane incident blocks your running pipelines—different runbooks.

### 1.2.2 Workspace Objects

The workspace organizes work into **notebooks** (multi-language cells with results), **folders**, **Git folders**, **libraries**, and **compute**. The catalog explorer separates data assets (catalogs, schemas, tables, volumes) from code assets. A disciplined team keeps code in Git folders and uses plain workspace folders only for scratch work.

**Note**

Workspace notebooks are collaborative documents, not version-controlled source files. Anything you would want to review, roll back, or deploy belongs in a Git folder (or an Asset Bundle from Chapter 23 onward).

## 1.3 Tuesday: Notebooks, Git Folders, and dbutils

### 1.3.1 Notebooks and Magic Commands

A Databricks notebook mixes languages per cell using magic commands: `%python`, `%sql`, `%scala`, `%r`, `%sh`, and `%md` for markdown. Cells run against the attached cluster (or serverless compute) in order, and results render inline. Parameters enter through **widgets**, which are how a manual notebook becomes a reusable pipeline component.

**Definition**

**dbutils** is the notebook utilities library: `dbutils.widgets` for parameters, `dbutils.fs` for file operations, `dbutils.secrets` for scoped credentials, `dbutils.jobs.taskValues` for passing data between workflow tasks, and `dbutils.notebook.run` for notebook chaining.

```

1 dbutils.widgets.text("processing_date", "2026-08-31")
2 dbutils.widgets.text("source_system", "olist")
3 processing_date = dbutils.widgets.get("processing_date")
4 source_system = dbutils.widgets.get("source_system")
5
6 df = (spark.read
7     .option("header", True)
8     .csv(f"/Volumes/de_training/bronze/landing/{source_system}/"
9         f"dt={processing_date}/"))
10 (df.write.format("delta").mode("overwrite")
11     .option("replaceWhere", f"processing_date = '{processing_date}'")
12     .saveAsTable("de_training.bronze.orders_raw"))

```

Listing 1.1: Parameterized notebook pattern used throughout the book.

### 1.3.2 Git Folders

A Git folder clones a remote repository into the workspace, giving notebooks real version control: branches, commits, diffs, and pull requests [21]. The rule for this book: *labs may start in a scratch folder, but every capstone artifact lives in Git*. Week 23 upgrades Git folders into full CI/CD with Asset Bundles.

**Common Pitfall**

**Editing in two places.** A notebook edited both in the Git folder and through an external IDE without a pull will conflict. Pick one direction of truth—in this program, the remote repository is truth and the workspace is a working copy.

## 1.4 Wednesday: Compute, the CLI, and the SDK

### 1.4.1 Choosing Compute

Compute choice is the first cost and performance decision of every workload [14]:

| Compute type        | Best for                            | Caution                                          |
|---------------------|-------------------------------------|--------------------------------------------------|
| All-purpose cluster | Interactive notebooks, exploration  | Expensive per DBU; enforce auto-termination      |
| Job cluster         | Scheduled production runs           | Starts per run; cold-start latency               |
| Pool-backed cluster | Latency-sensitive interactive teams | Idle instances still bill cloud cost             |
| Serverless          | Notebooks, DLT, SQL with spiky load | Fewer configuration knobs; network features vary |
| SQL warehouse       | BI and SQL workloads (Chapter 6)    | Separate SKU family; right-size with auto-stop   |

Table 1.1: Compute options and their engineering trade-offs.

### 1.4.2 Cluster Policies

A cluster policy restricts what users can configure: fixed auto-termination, maximum workers, allowed node types, required tags. Policies are how a platform team makes the cheap choice the easy choice. For this program, create one policy—**de-training-policy**—that pins auto-termination to 30 minutes and caps workers at 4, and use it for every classic cluster you create.

### 1.4.3 The CLI and SDK in Daily Work

Chapter 0 installed the CLI and SDK; this week puts them to work. The pattern that matters: *everything you click in the UI, you can script*. Jobs, clusters, catalogs, permissions, secrets—all have CLI and SDK surfaces, which is what makes Chapter 23’s infrastructure-as-code possible [18, 22].

```

1 from databricks.sdk import WorkspaceClient
2
3 w = WorkspaceClient()
4
5 print("== Clusters ==")
6 for c in w.clusters.list():
7     print(f"{c.cluster_name:30s} state={c.state.value:<10s} "
8           f"creator={c.creator_user_name}")
9
10 print("== Jobs ==")
11 for j in w.jobs.list():
12     print(f"{j.settings.name:40s} id={j.job_id} "
13           f"creator={j.creator_user_name}")

```

Listing 1.2: SDK inventory script: clusters, jobs, and their owners.

## 1.5 Thursday Hands-on Project: Provision, Authenticate, Smoke-Test

This lab provisions the Week 1 environment end to end: OAuth CLI profile, a policy-guarded cluster, a Git folder, and a programmatically submitted smoke-test notebook. It is deliberately CLI-first so the same steps can become an Asset Bundle in Week 23.

#### Note

The commands below create billable resources. Use a trial or sandbox workspace, keep auto-termination tight, and delete the cluster after testing.

```

1 # 1. Authenticate (OAuth)
2 databricks auth login --host https://<your-workspace>.azuredatabricks.net
3 databricks current-user me
4
5 # 2. Create a cluster policy enforcing guardrails
6 '{
7   "name": "de-training-policy",
8   "definition": "{ \"autotermination_minutes\": {\"type\": \"fixed\", \"value\": 30}, \"num_workers\": {\"type\": \"range\", \"maxValue\": 4} }"
9 }' | Out-File -Encoding ascii policy.json
10 # (In the UI: Compute > Policies > Create, or paste policy.json via the API.)
11
12 # 3. Create a small all-purpose cluster from the policy
13 '{
14   "cluster_name": "de-week01-cluster",
15   "spark_version": "15.4.x-scala2.12",
16   "node_type_id": "Standard_DS3_v2",
17   "num_workers": 1,
18   "autotermination_minutes": 30,
19   "policy_id": "<policy-id>"
20 }' | Out-File -Encoding ascii cluster.json
21 databricks clusters create --json-file cluster.json
22
23 # 4. Clone the course repo as a Git folder
24 databricks repos create https://github.com/<you>/de-training gitHub

```

Listing 1.3: Week 1 lab: profile, policy-guarded cluster, Git folder.

```

1 from databricks.sdk import WorkspaceClient
2 from databricks.sdk.service.jobs import SubmitTask, NotebookTask
3
4 w = WorkspaceClient()
5 result = w.jobs.submit(
6     run_name="week1-smoke",
7     tasks=[SubmitTask(
8         task_key="smoke",
9         existing_cluster_id="<cluster-id>",
10        notebook_task=NotebookTask(
11            notebook_path="/Repos/<you>/de-training/src/week01_smoke"),
12        )],
13 ).result()
14 print(result.state.result_state) # SUCCESS

```

Listing 1.4: Submitting the smoke-test notebook via the SDK.

## Deliverables

- A working CLI OAuth profile and `current-user me` output.
- The cluster policy JSON and a running cluster created from it.
- The course Git folder cloned into the workspace.
- A successful SDK-submitted notebook run, visible in the Jobs UI.

## Acceptance Criteria

- Cluster shows auto-termination 30 minutes enforced by the policy (attempting 60 is rejected).
- The smoke notebook prints the current user, Spark version, and a one-row Delta write/read round trip.

- No credentials appear in any committed file; the repo passes a secret scan.
- The cluster is terminated (or auto-terminated) at lab end; a screenshot or `clusters list` output proves it.

### Stretch Goals

- Add a second CLI profile for a hypothetical “prod” workspace and demonstrate profile switching.
- Script the entire lab as one idempotent PowerShell script (create-if-missing for each resource).
- Write the lab’s DBU cost estimate from `system.billing.usage`.

## 1.6 Friday Use Case: Workspace Layout for a 200-Person Data Organization

A company with 200 data practitioners across three business units (finance, logistics, marketing) needs a Databricks estate. Requirements: environment isolation (dev/staging/prod), cost accountability per business unit, shared platform standards, and a path to audited production deployments.

### 1.6.1 Requirements and Assumptions

Assume a single cloud (Azure) with three subscriptions available. Business units vary in maturity: finance has strict change control, logistics is streaming-heavy, marketing is experimentation-heavy. The platform team owns policies, budgets, and the account console; domain teams own pipelines.

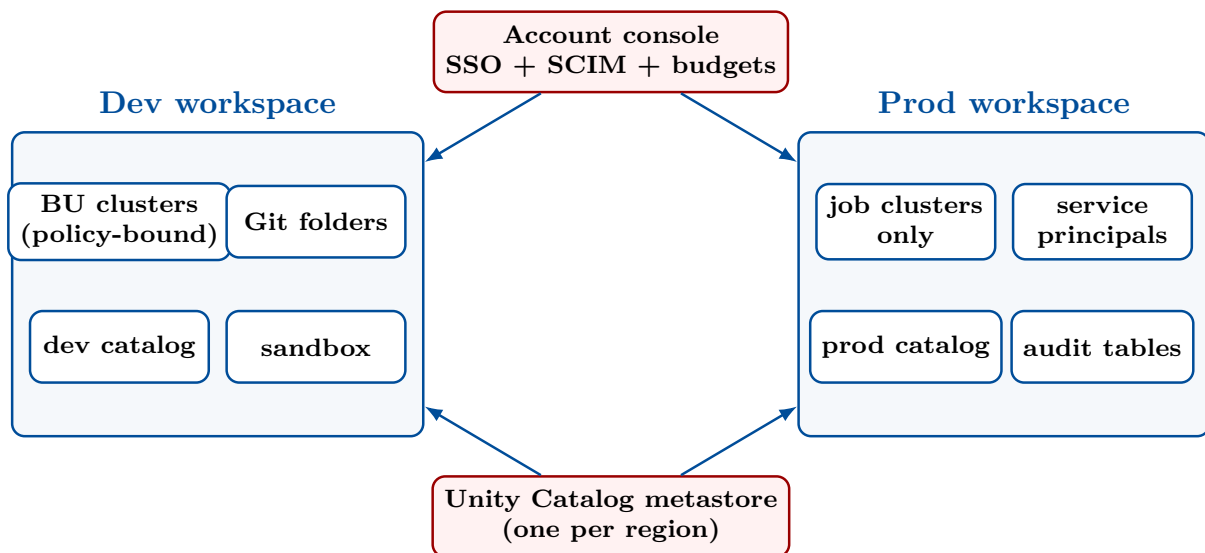


Figure 1.1: A multi-team workspace layout: one account, environment-split workspaces, one regional metastore, policy-bound compute, and service-principal production runs.



### 1.6.2 Architecture Decisions

**Workspace topology.** One dev and one prod workspace per environment pair rather than one workspace per business unit: workspace count drives administrative overhead, while catalogs and cluster policies provide the per-unit isolation actually needed. Finance’s change-control requirement is satisfied by prod being deploy-only via service principals—no human write access to prod catalogs.

**Compute governance.** Every BU gets a cluster policy (max workers, pinned auto-termination, required bu tag) and a pool if interactive latency matters. Production jobs run on job clusters created per run: cheaper, isolated, and disposable. Tags flow into `system.billing.usage`, giving per-unit chargeback without separate accounts.

**Identity.** SSO with SCIM provisioning from the IdP; BU groups synced; service principals for pipelines. Humans never run production jobs.

**Common Pitfall**

**One workspace per team.** It feels clean and becomes unmanageable: N metastores, N policy sets, N audit surfaces. Isolate with catalogs, groups, and policies inside fewer workspaces; split workspaces only across environment or trust boundaries.

### 1.6.3 Risks and Mitigations

| Risk                                         | Mitigation                                                |
|----------------------------------------------|-----------------------------------------------------------|
| BU cost overrun invisible until invoice      | Policy-enforced tags feeding weekly chargeback queries    |
| A workspace split later forced by compliance | Catalog-level isolation from day one keeps the exit cheap |
| Service-principal sprawl                     | One principal per pipeline domain, inventoried and owned  |
| Git folders forking from re-motes            | Remote is truth; weekly automation flags diverged folders |
| Policy exceptions becoming the norm          | Exceptions expire by default; renewal requires the owner  |

Table 1.2: Week 1 scenario risks and mitigations.

## 1.7 Design Decision Guide

Week 1’s choices are platform-topology choices; they are expensive to reverse, so make them deliberately.

| Decision                        | Choose the first when                  | Choose the second when                            |
|---------------------------------|----------------------------------------|---------------------------------------------------|
| All-purpose vs. job cluster     | Humans iterate interactively           | A scheduled pipeline runs unattended              |
| Serverless vs. classic compute  | Speed and zero-ops matter most         | Custom networking/init or lowest DBU rate         |
| One workspace vs. many          | Teams share governance and data        | Trust or environment boundaries require isolation |
| Git folder vs. workspace folder | Code must be reviewed or deployed      | Scratch exploration with no downstream            |
| Cluster policy vs. convention   | More than one person creates clusters  | Never—conventions are not enforcement             |
| Pool vs. cold start             | Interactive latency is worth idle cost | Schedules tolerate start-up minutes               |

Table 1.3: Week 1 decision guide.

The meta-lesson: topology decisions should be made once, written down, and enforced by policy. A workspace layout decided per-request is not a layout; it is an accumulation.

## 1.8 Operational Guidance for Week 1

### 1.8.1 Performance and Reliability

Cluster start time is the first reliability metric a new platform team feels: keep a small warm pool for interactive teams, use job clusters for schedules, and review cluster uptime versus actual usage weekly—an all-purpose cluster running 24/7 for a team working 8-hour days is a 3x cost with zero benefit. Track notebook job success rates from the jobs UI and promote any notebook run twice manually into a scheduled job with an owner.

### 1.8.2 Security and Governance Checklist

- PAT policy configured (or PATs disabled in favor of OAuth) before broad onboarding.
- Every classic cluster created from a policy with auto-termination and worker caps.
- No user-created local accounts beside SCIM-synced IdP groups.
- Course/team repos cloned as Git folders; scratch work quarantined in user folders.
- Weekly `system.billing.usage` review scheduled from day one.

### 1.8.3 Troubleshooting Playbook

| Symptom                    | Likely cause                                  | First action                                               |
|----------------------------|-----------------------------------------------|------------------------------------------------------------|
| Cluster fails to start     | Cloud vCPU quota exhausted                    | Check cloud quotas; request increase or shrink the cluster |
| CLI commands 401           | Expired OAuth session                         | Re-run <code>databricks auth login</code>                  |
| Notebook cannot find table | Missing <code>USE CATALOG/SELECT</code> grant | Verify grants as the affected principal, not as admin      |
| Job ran but wrote nothing  | Wrong catalog/schema default in job context   | Log <code>current_catalog()</code> at task start           |

Table 1.4: Week 1 troubleshooting quick reference.

## 1.9 Interview Q&A

### 1.9.1 Concepts and Trade-offs

1. **Q: Why does Databricks separate the control plane from the data plane?**  
A: So the SaaS control plane can orchestrate work while data and compute stay in the customer's cloud account—satisfying data residency, network isolation, and existing cloud commitments.
2. **Q: All-purpose or job cluster for a nightly pipeline?**  
A: Job cluster: created per run, priced lower, configured exactly for the workload, and destroyed after—no idle spend, no shared-state surprises.
3. **Q: What problem do cluster policies actually solve?**  
A: They make platform standards enforceable rather than advisory: budgets (auto-termination caps), security (node restrictions), and chargeback (required tags) become properties of every cluster, not documentation.
4. **Q: Git folder or workspace folder for production notebooks?**  
A: Git folder. Production code needs review, history, and deployability; workspace folders have none of these.
5. **Q: How would you give 200 users compute without 200 bill shocks?**  
A: Policy-bound all-purpose clusters per team, serverless where possible, job clusters for schedules, and weekly per-tag chargeback from `system.billing.usage`.
6. **Q: Serverless or classic compute for a team onboarding this week?**  
A: Serverless for speed and zero-ops; classic when labs need cluster policies, custom networking, or the tuning knobs Weeks 4 and 9 depend on.
7. **Q: What breaks first when a workspace has no naming conventions?**  
A: Cost attribution and access review—you cannot charge back or audit what you cannot name. Conventions are the cheapest governance you will ever buy.
8. **Q: What would make you split into a second workspace?**  
A: A hard trust or environment boundary—prod separation, a regulated enclave, or a business sale. Performance and organization are never reasons; catalogs and policies carry those.

## 1.10 Further Reading

Begin with the Databricks documentation portal [19] and the workspace [52] and compute [14] guides. Study Git folders [21], then the CLI [18] and Python SDK [22] references—they are the automation surface for the whole book. For the architectural motivation behind the platform, read the lakehouse paper [9] alongside the Delta Lake paper [8].

## Key Takeaways

- The control plane / data plane split explains Databricks' security, networking, and cost model in one picture.
- All-purpose clusters are for humans; job clusters are for pipelines; policies keep both honest.

- Notebooks are for exploration; Git folders are for anything you must review, roll back, or deploy.
- The CLI and SDK turn every UI action into automation—Week 23’s CI/CD is built on what you script this week.
- Workspace layout is an organizational decision: isolate with catalogs, groups, and policies before multiplying workspaces.

## 1.11 Exercises

1. **(Conceptual)** List three objects in the control plane and three in the data plane. Which plane does a Unity Catalog metastore’s metadata belong to?
2. **(Conceptual)** Why does a job cluster usually cost less than leaving an all-purpose cluster running for a nightly job, even if the job runs every night?
3. **(Hands-on)** Complete the Thursday lab, then extend it: create a second cluster without the policy and document what the policy prevented.
4. **(Hands-on)** Write an SDK script that lists all clusters older than 7 days with no recent activity and prints their estimated idle cost.
5. **(Design)** Extend fig. 1.1 with a staging workspace and show where bundle targets (Week 23) would point.
6. **(Design)** Write the cluster policy JSON for a streaming team that needs exactly 8 workers, spot instances, and a 60-minute auto-termination ceiling.
7. **(Governance)** Design the group structure (BU, role, environment) that SCIM would sync for the Friday scenario.
8. **(Operations)** Write the runbook entry for “prod workspace control plane unavailable”: what keeps running, what stops, who is paged?

# Delta Lake Foundations

**Learning Objectives**

- Explain Delta Lake’s architecture: Parquet data files plus the `_delta_log` transaction log, and how atomic commits work.
- Describe Delta’s ACID guarantees, optimistic concurrency control, and isolation levels for concurrent writers.
- Apply schema enforcement, deliberate schema evolution, check constraints, and generated columns.
- Use time travel (`VERSION AS OF`, `TIMESTAMP AS OF`), `DESCRIBE HISTORY`, and `RESTORE` for auditing and recovery.
- Write idempotent upserts with `MERGE INTO` and explain why it beats delete-then-insert.
- Reason about `VACUUM` retention and its interaction with recovery objectives.

**Cross-Cloud Equivalence**

Delta Lake is the **open table format** layer; the same idea—ACID tables over object storage—appears across every platform.

| Capability        | Databricks (Delta)                         | AWS / Azure / GCP                       | Snowflake / Fabric                            |
|-------------------|--------------------------------------------|-----------------------------------------|-----------------------------------------------|
| Table format      | Delta Lake                                 | Iceberg / Hudi / Delta on S3, ADLS, GCS | Micro-partitions / Delta on OneLake           |
| Transaction log   | <code>_delta_log</code> JSON + checkpoints | Iceberg metadata tree / Hudi timeline   | Internal / <code>_delta_log</code>            |
| Time travel       | <code>VERSION AS OF</code>                 | Iceberg snapshot IDs                    | Time Travel / Delta versions                  |
| Recovery          | <code>RESTORE</code>                       | Iceberg rollback                        | <code>UNDROP</code> / <code>RESTORE</code>    |
| Retention cleanup | <code>VACUUM</code>                        | Iceberg <code>expire_snapshots</code>   | Fail-safe / <code>VACUUM</code>               |
| Upsert            | <code>MERGE INTO</code>                    | Iceberg <code>MERGE</code> / Athena     | <code>MERGE</code> / T-SQL <code>MERGE</code> |
| Interop           | UniForm (Iceberg/Hudi reads)               | Glue Catalog sync                       | Iceberg catalog / OneLake Delta               |

**MongoDB** provides document-level ACID inside a database rather than table-level ACID over files—the contrast clarifies what problem the table format actually solves.

## 2.1 Week 2 Roadmap and Mental Model

Week 1 gave you the platform; Week 2 gives you the storage contract. Delta Lake is the layer that turns a folder of Parquet files into a *table*: transactional, versioned, schema-enforced, and queryable by time [8]. Nearly everything else in this program—medallion design, optimization, streaming exactly-once, governance, ML reproducibility—assumes these guarantees.

The mental model: **a Delta table is its log**. The data files are immutable Parquet; the `_delta_log` directory is an ordered sequence of JSON commit files describing which data files constitute each version. Readers open the log, learn the current file set, and scan only those files. Writers read the log, write new files, and attempt to commit a new log entry; if someone else committed first, they reconcile or retry. That one mechanism delivers atomicity, isolation, and time travel [53].

### Definition

**Delta Lake** is an open table format for cloud object storage: immutable Parquet data files, an append-only transaction log, and protocol guarantees (ACID commits, schema enforcement, versioning) layered on top.

### Key Terms

**Transaction log**: the `_delta_log` directory of commits. **Optimistic concurrency**: writers assume no conflict and retry on collision. **Schema enforcement**: writes must match the table schema unless evolution is requested. **Time travel**: querying any retained historical version. **Deletion vector**: a bitmap marking deleted rows without rewriting files (Chapter 5).

## 2.2 Monday: Data Files and the Transaction Log

### 2.2.1 Parquet Data Files

Delta stores rows in Apache Parquet: columnar, compressed, splittable, and self-describing [3]. Columnar layout means a query selecting three columns from a hundred reads only those columns' data—the foundation of analytical performance. Files are immutable: an update never modifies a Parquet file in place; it writes new files and flips the log pointer.

### 2.2.2 The JSON Transaction Log

Every commit appends a numbered JSON file (for example `00000000000000000007.json`) listing *actions*: add file, remove file, update metadata, change protocol. Every ten commits, Delta writes a Parquet **checkpoint** file summarizing the log so readers do not replay thousands of JSON files. `DESCRIBE HISTORY` is simply a readable view over the log [46].

```
1 DESCRIBE HISTORY de_training.silver.orders;
2 -- version, timestamp, operation, operationParameters, userName, ...
3 -- Every INSERT, DELETE, MERGE, OPTIMIZE, and RESTORE appears here.
```

Listing 2.1: Inspecting the transaction log through history.

### Common Pitfall

**Editing or deleting files directly in storage.** If you remove a Parquet file outside Delta (or worse, edit `_delta_log`), the log and the files disagree and the table corrupts. All changes go through Delta APIs—the log is the only writer of record.

### 2.2.3 Atomic Commits

A commit is a single file write into `_delta_log`; object stores make small file puts atomic. Either the new log entry exists (the version exists, fully) or it does not (nothing changed). A job that writes 10,000 files and fails before the commit leaves orphan files on storage—invisible to readers, cleanable later—but never a half-visible table.

## 2.3 Tuesday: ACID, Concurrency, and MERGE

### 2.3.1 Optimistic Concurrency

Concurrent writers read the log at a version, compute their changes, and race to write the next commit file. The loser re-reads the log and checks for semantic conflict: two jobs appending disjoint partitions can both succeed after reconciliation; two jobs updating the same rows conflict and one retries or fails. Delta’s default isolation for concurrent writes is snapshot isolation with conflict checking—strong enough for every workload in this book [8].

### 2.3.2 MERGE INTO

`MERGE INTO` is the lakehouse upsert: match target rows against a source, then update, insert, or delete in one atomic operation.

```

1 MERGE INTO de_training.silver.orders AS t
2 USING de_training.bronze.orders_deduped AS s
3 ON t.order_id = s.order_id
4 WHEN MATCHED AND s.updated_at > t.updated_at THEN
5   UPDATE SET *
6 WHEN NOT MATCHED THEN
7   INSERT *
8 WHEN NOT MATCHED BY SOURCE AND t.is_active = true THEN
9   UPDATE SET t.is_active = false; -- soft-delete pattern

```

Listing 2.2: The idempotent upsert pattern used across the book.

#### Definition

**Idempotent write:** a write whose effect is the same whether it runs once or many times. Key-based `MERGE` and partition-scoped `replaceWhere` overwrites are the two idempotent write patterns this book uses; blind `INSERT` is not idempotent.

#### Tip

Always give `MERGE` a deterministic match condition on a unique business key, and (for updates) a recency guard like `s.updated_at > t.updated_at`. That one clause makes replays and out-of-order CDC safe (Chapter 11 builds on this).

## 2.4 Wednesday: Schema Discipline, Time Travel, and Recovery

### 2.4.1 Schema Enforcement and Evolution

By default, a write whose schema does not match the table fails loudly—this is **schema enforcement**, and it is the first data-quality gate in every pipeline. Deliberate evolution opts in: `mergeSchema` adds new columns on append; `overwriteSchema` permits breaking rewrites. Additive changes evolve; breaking changes are a migration with a plan [53].

```

1 # New optional column appears in the source feed
2 (df.write.format("delta").mode("append")
3   .option("mergeSchema", "true")           # additive only
4   .saveAsTable("de_training.silver.orders"))
5
6 # A breaking change is a deliberate rewrite, never an accident:
7 (df.write.format("delta").mode("overwrite")
8   .option("overwriteSchema", "true")       # explicit breaking change
9   .saveAsTable("de_training.silver.orders"))

```

Listing 2.3: Deliberate, opt-in schema evolution.

### 2.4.2 Constraints and Generated Columns

CHECK constraints reject bad rows at write time; generated columns derive values (for example a date from a timestamp) so partitions and queries stay consistent. Both are cheap correctness at the table boundary.

```

1 ALTER TABLE de_training.silver.orders
2   ADD CONSTRAINT valid_amount CHECK (total_amount >= 0);
3
4 ALTER TABLE de_training.silver.orders
5   ADD COLUMN order_day DATE GENERATED ALWAYS AS (CAST(order_ts AS DATE));

```

Listing 2.4: Constraints and generated columns at the table boundary.

### 2.4.3 Time Travel and RESTORE

Because the log is versioned, any retained version is queryable: `VERSION AS OF n`, `TIMESTAMP AS OF '...'`. `RESTORE` makes a previous version the new head—a new commit, not a rewrite of history, so the incident itself remains auditable.

```

1 -- What did the table look like before the incident?
2 SELECT COUNT(*) FROM de_training.silver.orders VERSION AS OF 11;
3
4 -- Roll forward back to the good version (creates a new commit):
5 RESTORE TABLE de_training.silver.orders TO VERSION AS OF 11;

```

Listing 2.5: Recovery after a bad delete.

### 2.4.4 VACUUM and Retention

Time travel depends on old data files surviving. `VACUUM` deletes files no longer referenced beyond the retention horizon (default 168 hours = 7 days). Shorter retention ends time travel earlier; never vacuum a table you might still need to restore [46].

#### Common Pitfall

**VACUUM before the recovery drill.** The classic lab failure—and the classic production incident—is vacuuming away the versions you needed to restore. Retention is a recovery objective, not a disk-hygiene preference; change it through change control.

## 2.5 Thursday Hands-on Project: Enforcement, Evolution, and Recovery

This lab exercises the full Delta contract: create tables, watch enforcement reject a bad write, evolve a schema deliberately, “accidentally” delete a partition, and recover with time travel and `RESTORE`—documenting everything from the history.



```

1 from pyspark.sql import functions as F
2
3 # 1. Create a Silver table from seed CSVs
4 orders = (spark.read.option("header", True).option("inferSchema", True)
5           .csv("/Volumes/de_training/bronze/landing/olist/orders/"))
6 (orders.write.format("delta").mode("overwrite")
7  .saveAsTable("de_training.silver.orders"))
8 spark.sql("DESCRIBE HISTORY de_training.silver.orders").show(truncate=False)
9
10 # 2. Schema enforcement: this append MUST FAIL (column renamed)
11 bad = orders.withColumnRenamed("order_id", "id")
12 try:
13     bad.write.format("delta").mode("append") \
14         .saveAsTable("de_training.silver.orders")
15 except Exception as e:
16     print("Enforcement rejected the write:", type(e).__name__)
17
18 # 3. Deliberate evolution: add review_score as a new column
19 evolved = orders.withColumn("review_score", F.lit(None).cast("int"))
20 (evolved.write.format("delta").mode("append")
21  .option("mergeSchema", "true")
22  .saveAsTable("de_training.silver.orders"))
23
24 # 4. The accident: delete a partition
25 spark.sql("""DELETE FROM de_training.silver.orders
26            WHERE order_status = 'delivered'""")
27
28 # 5. Recover: find the last good version, then RESTORE
29 hist = spark.sql("DESCRIBE HISTORY de_training.silver.orders") \
30       .orderBy(F.desc("version")).limit(3)
31 hist.select("version", "operation", "timestamp").show(truncate=False)
32 last_good = 0 # replace with the version before the DELETE
33 spark.sql(f"""RESTORE TABLE de_training.silver.orders
34           TO VERSION AS OF {last_good}""")
35 assert spark.sql("""SELECT COUNT(*) c FROM de_training.silver.orders
36                   WHERE order_status = 'delivered'""").first().c > 0

```

Listing 2.6: Week 2 lab: enforcement, evolution, and recovery drills.

## Deliverables

- The history output showing create, rejected append, evolution, delete, and restore.
- A note explaining which step would have corrupted data without Delta's guarantees.
- The recovered row count matching the pre-delete baseline.

## Acceptance Criteria

- The mismatched append demonstrably fails with a schema error.
- Evolution adds only the new column; no existing data is rewritten.
- RESTORE returns the deleted partition; the post-restore count equals the baseline.
- VACUUM is *not* run before recovery; retention is left at 168 hours with a one-line justification.

## Stretch Goals

- Repeat recovery with `TIMESTAMP AS OF` instead of a version number.
- Add a `CHECK` constraint and prove it rejects a negative amount.

- Script the whole drill so a teammate can run it from a clean catalog.

## 2.6 Friday Use Case: Auditable 90-Day History for a Financial Lake

A financial-services firm must prove to auditors exactly what any table contained at any moment in the last 90 days, support same-day correction of erroneous loads, and never allow “the pipeline overwrote it” as an answer.

### 2.6.1 Requirements and Assumptions

Tables range from 10M to 2B rows. Corrected files arrive up to 5 days late. Auditors sample randomly. Retention policy requires 90 days of queryable history and 7 years of archived raw files (in Bronze, append-only).

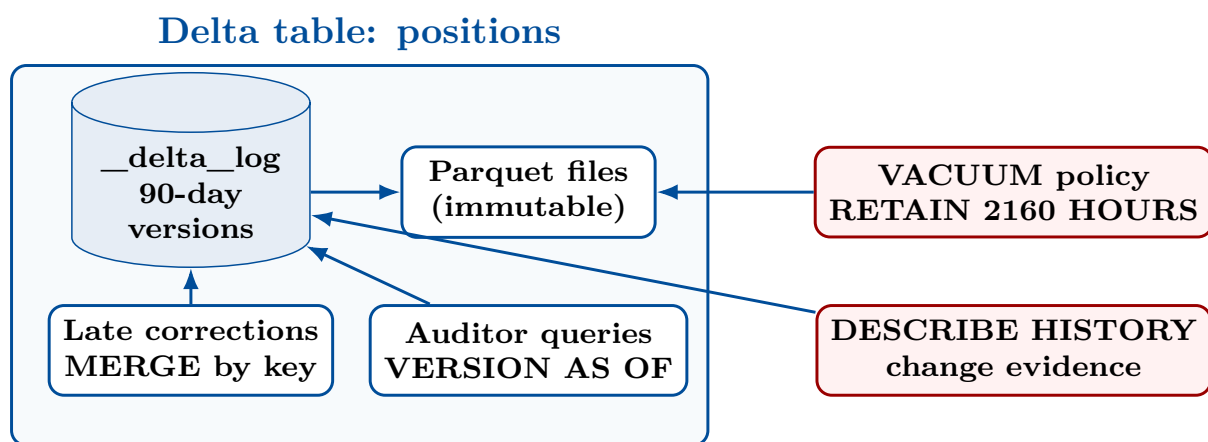


Figure 2.1: An auditable Delta table: versioned log with 90-day retention, key-based corrections, auditor time-travel queries, and history as change evidence.

### 2.6.2 Architecture Decisions

**Retention.** `VACUUM RETAIN 2160 HOURS` (90 days) on Silver and Gold; Bronze raw files archived by storage lifecycle beyond Delta retention. The vacuum job is scheduled, logged, and change-controlled.

**Corrections.** Late corrections arrive as replacement files keyed by business key; a keyed `MERGE` applies them atomically. Every correction is a visible commit with `userName` and operation parameters—the audit trail is the log itself, exportable via `DESCRIBE HISTORY` into a governance table.

**Evidence.** Monthly drill: pick a random table and timestamp, run `TIMESTAMP AS OF`, reconcile against the archived raw file, and file the result. Auditors trust rehearsed evidence, not asserted capability.

#### Common Pitfall

**Confusing log retention with data retention.** `VACUUM` controls time travel (files); the log itself is compacted by checkpoints and has its own retention. If auditors need 90 days of *history entries*, verify log retention matches—and export history to a governance table

for anything longer.

### 2.6.3 Risks and Mitigations

| Risk                                            | Mitigation                                                              |
|-------------------------------------------------|-------------------------------------------------------------------------|
| VACUUM run early under cost pressure            | Retention values are change-controlled config with sign-off             |
| A “temporary” schema override becomes permanent | <code>mergeSchema/overwriteSchema</code> allowed only via reviewed jobs |
| Auditor request exceeds log retention           | Monthly <code>DESCRIBE HISTORY</code> export to a governance table      |
| Late corrections overwrite newer state          | Recency guard ( <code>seq &gt; t.seq</code> ) in every CDC MERGE        |
| Recovery drill never practiced                  | Quarterly timed restore drill with measured RTO recorded                |

Table 2.1: Week 2 scenario risks and mitigations.

## 2.7 Design Decision Guide

Delta’s guarantees make the safe choice easy; the guide below names the defaults this program uses.

| Decision                                                  | Choose the first when              | Choose the second when                             |
|-----------------------------------------------------------|------------------------------------|----------------------------------------------------|
| Managed vs. external table                                | Databricks owns the data lifecycle | A non-Databricks writer owns the files             |
| <code>mergeSchema</code> vs. <code>overwriteSchema</code> | Evolution is additive              | The change is deliberately breaking                |
| <code>RESTORE</code> vs. <code>recompute</code>           | The table state was damaged        | The transformation logic was wrong                 |
| Long vs. short VACUUM horizon                             | Recovery/audit needs dominate      | Storage cost dominates and recovery is rebuildable |
| MERGE vs. append-then-dedup                               | Upserts arrive as changes          | Facts are immutable events                         |
| Constraint vs. downstream validation                      | The rule must never be violated    | The rule is advisory or source-dependent           |

Table 2.2: Week 2 decision guide.

When in doubt, pick the option that keeps history intact and the failure loud. Delta gives you both almost for free; spending them lightly is the only real mistake.

## 2.8 Operational Guidance for Week 2

### 2.8.1 Performance and Reliability

Small, frequent commits bloat the transaction log and the file count together—batch writers should commit per partition batch, not per file. Schedule `OPTIMIZE` (Chapter 5) and export `DESCRIBE HISTORY` monthly into a governance table so audit evidence survives log retention. Set `VACUUM` retention per table from the recovery objective, and record the value in the table comment where the next engineer will actually see it.

### 2.8.2 Security and Governance Checklist

- New tables default to managed; external tables carry a documented reason.
- VACUUM retention  $\geq$  the longest recovery objective; changes go through change control.
- No direct file manipulation in table storage paths—all writes through Delta APIs.
- PII-bearing tables tagged at creation so Chapter 22’s audit rules cover them.

### 2.8.3 Troubleshooting Playbook

| Symptom                   | Likely cause                            | First action                                               |
|---------------------------|-----------------------------------------|------------------------------------------------------------|
| ConcurrentAppendException | Too many writers racing one table       | Retry; then serialize writers or partition their keyspaces |
| Time travel query fails   | Version vacuumed away                   | Check last VACUUM time vs. requested version               |
| MERGE suddenly slow       | File layout degraded (many small files) | DESCRIBE DETAIL; run OPTIMIZE                              |
| Append rejected           | Schema enforcement working              | Diff writer schema against table; evolve deliberately      |

Table 2.3: Week 2 troubleshooting quick reference.

## 2.9 Interview Q&A

### 2.9.1 Concepts and Trade-offs

1. **Q: How can a file store be ACID?**

A: Files are immutable; the atomic unit is the single-file commit to the transaction log. Readers see a version only after its commit exists—so multi-file writes appear all at once or not at all.

2. **Q: Two jobs MERGE into the same table simultaneously—what happens?**

A: Optimistic concurrency: both read a snapshot, one commits first, the other re-reads and either reconciles (disjoint keys/partitions) or retries/fails on conflict.

3. **Q: When is mergeSchema unacceptable?**

A: When the change is breaking (type change, rename, drop) or when an upstream schema drift should halt the pipeline for review—enforcement is a feature, not friction.

4. **Q: RESTORE versus re-running the pipeline to fix a bad load?**

A: RESTORE is instant and auditable for table-level mistakes; re-running is correct when the bug is in the transformation logic itself. Both leave history intact.

5. **Q: Why not set VACUUM retention to 1 hour to save storage?**

A: You would delete the files time travel and long-running readers depend on. Retention must exceed your recovery objective and your longest query duration.

6. **Q: Why is the transaction log small-file-averse too?**

A: Thousands of tiny commits mean thousands of JSON log files to list and replay—checkpoints mitigate, but commit cadence is a design decision, like file size.

7. **Q: Can two Delta tables share one set of data files?**

A: Safely, no: each table’s log owns its file lifecycle, and one table’s VACUUM would delete files the other references. Shallow clones exist for exactly this need—reference without copy.

**8. Q: Does Delta’s ACID protect against a logically wrong write?**

A: No—ACID guarantees the commit is atomic and durable, not correct. Correctness comes from constraints, expectations, and reconciliation; Delta makes their failures recoverable via time travel and **RESTORE**.

## 2.10 Further Reading

Read the Delta Lake VLDB paper for the storage protocol [8] and the open-source documentation for command details [53]. The Databricks Delta guide covers time travel, constraints, and retention behavior on the platform [46], and the Parquet documentation explains the columnar format underneath [3]. The lakehouse paper frames why this layer displaced two-tier architectures [9].

## Key Takeaways

- A Delta table is its transaction log: immutable Parquet files plus atomic, versioned commits.
- Optimistic concurrency makes multi-writer pipelines safe without locks.
- Schema enforcement is a free data-quality gate; evolution is deliberate and opt-in.
- Time travel and **RESTORE** turn “we overwrote it” into a recoverable, auditable event.
- **VACUUM** retention is a recovery objective—treat it as change-controlled configuration.

## 2.11 Exercises

1. **(Conceptual)** Walk through a commit, step by step, from `df.write` to a reader seeing the new version.
2. **(Conceptual)** Why does immutability of data files make concurrent readers simpler?
3. **(Hands-on)** Complete the Thursday lab; capture the five history entries as a screenshot or exported table.
4. **(Hands-on)** Force a **MERGE** conflict between two sessions on the same keys; capture the conflict error and explain the resolution.
5. **(Hands-on)** Export **DESCRIBE HISTORY** for a table into `de_training.gold.table_history_audit` and query it for all **DELETE** operations.
6. **(Design)** Write the retention policy (**VACUUM** horizon, log retention, archive lifecycle) for the Friday scenario as a one-page decision record.
7. **(Governance)** Explain to an auditor how you would prove a specific row’s value on June 1st. List every artifact you would show.
8. **(Architecture)** When would you choose Iceberg interoperability (UniForm) for this lake, and what governance gap would it open?

# The Medallion Architecture: Bronze, Silver, and Gold

## Learning Objectives

- Justify the Bronze/Silver/Gold layering and articulate the cost of skipping a layer.
- Design Bronze tables as append-only, self-describing evidence with ingestion meta-data.
- Build Silver cleansing, conformance, deduplication, and quarantine patterns.
- Publish Gold dimensional models (facts, dimensions) for SQL, BI, and ML consumers.
- Make every layer transition idempotent and reconcilable by row count.
- Name catalogs, schemas, and tables for a multi-team Unity Catalog estate.

## Cross-Cloud Equivalence

The medallion is a **design pattern**, not a Databricks product—every platform expresses the same progressive refinement with its own primitives.

| Layer              | Databricks                                 | AWS / Azure / GCP<br>lake           | Snowflake / Fabric                      |
|--------------------|--------------------------------------------|-------------------------------------|-----------------------------------------|
| Bronze (raw)       | <b>bronze</b> schema, Delta                | <b>raw/</b> prefix<br>(S3/ADLS/GCS) | RAW schema /<br>Bronze Lakehouse        |
| Silver (validated) | <b>silver</b> schema, Delta                | <b>validated/</b> prefix            | STAGING schema /<br>Silver Lakehouse    |
| Gold (curated)     | <b>gold</b> star schema                    | <b>curated/</b> prefix              | ANALYTICS<br>schema / Gold<br>Lakehouse |
| Quality gate       | Expectations, con-<br>straints, quarantine | Glue DQ rules                       | Great Expectations /<br>pipeline gates  |
| Access boundary    | UC grants per schema                       | Lake Formation per<br>zone          | Role grants /<br>workspace roles        |

The pattern predates the name: staging/core/mart schemas in classical warehouses are the same idea with relational storage. What Delta adds is cheap reprocessing—Bronze can be kept forever and Silver rebuilt at will.

## 3.1 Week 3 Roadmap and Mental Model

Week 2 gave you a trustworthy table format; Week 3 organizes many tables into a trustworthy *system*. The medallion architecture is the standard answer to a recurring failure: pipelines that

transform raw data directly into reports, so that every bug fix requires either corrupting the report layer or re-pulling from the source [50].

The mental model: **each layer has one job and one audience**. Bronze answers “what exactly did we receive?” for engineers. Silver answers “what is the clean, current truth?” for pipelines. Gold answers “what does the business measure?” for analysts and models. A row may restate many times in Gold, but it lands in Bronze exactly once.

#### Definition

**The medallion architecture** is a lakehouse design pattern that progressively refines data through Bronze (raw, append-only), Silver (validated, conformed, deduplicated), and Gold (business-level, dimensional, consumable) layers, with quality gates and reconciliation at each transition.

#### Key Terms

**Quarantine:** a table holding rejected rows with rejection reasons. **Conformance:** aligning schemas, units, and reference data across sources. **Reconciliation:** the arithmetic proof that no rows were lost between layers. **Idempotent transition:** re-running a layer’s pipeline yields the same layer state. **Star schema:** Gold’s fact-plus-dimensions shape.

## 3.2 Monday: Why Layers, and the Cost of Skipping Them

### 3.2.1 Progressive Refinement

Each transition does exactly one class of work. Bronze → Silver: parsing, typing, validation, deduplication. Silver → Gold: business logic, conformance across domains, dimensional modeling. Mixing classes—business logic in Silver, typing in Gold—creates pipelines that cannot be rerun, debugged, or audited independently.

### 3.2.2 Idempotent Reprocessing

The layering pays off on the bad day: a Silver bug is fixed by reprocessing Silver from Bronze; a Gold business-rule change is fixed by rebuilding Gold from Silver. Neither touches the source. This only works if transitions are idempotent—key-based **MERGE** or partition-scoped **replaceAll**—and if Bronze retains everything needed to rebuild.

#### Common Pitfall

**Transforming in place.** Updating Bronze rows during cleansing destroys the evidence layer. Bronze is write-once; corrections and reinterpretations belong to downstream layers. If storage cost tempts you to mutate Bronze, remember what you are trading: the ability to ever rebuild.

## 3.3 Tuesday: Bronze—The Evidence Layer

### 3.3.1 Append-Only Raw Capture

Bronze tables store what arrived, as it arrived, plus metadata your future self will need: `_ingested_at`, `_source_file`, `_source_system`, and a pipeline run identifier. Types stay permissive (strings are common) because Bronze’s job is capture, not judgment.

```
1 from pyspark.sql import functions as F
2
```

```

3 raw = (spark.read.option("header", True)
4       .csv("/Volumes/de_training/bronze/landing/olist/orders/"))
5
6 bronze = (raw
7           .withColumn("_ingested_at", F.current_timestamp())
8           .withColumn("_source_file", F.input_file_name())
9           .withColumn("_source_system", F.lit("olist"))
10          .withColumn("_run_id", F.lit(spark.conf.get("spark.databricks.job.id", "
11                                     manual"))))
12
13 (bronze.write.format("delta").mode("append")
14   .saveAsTable("de_training.bronze.orders"))

```

Listing 3.1: Bronze landing with ingestion metadata.

### 3.3.2 Schema-on-Read Trade-offs

Keeping Bronze loosely typed means a malformed column never blocks ingestion—the failure surfaces in Silver where it can be quarantined and explained. The trade-off is discipline: Silver must parse defensively, and someone must own watching the quarantine.

## 3.4 Wednesday: Silver Conformance and the Gold Star Schema

### 3.4.1 Silver: Validation, Dedup, Quarantine

Silver is where data earns trust: typed columns, enforced constraints, deduplicated business keys, and a quarantine table catching what fails with a reason attached.

```

1 bronze = spark.read.table("de_training.bronze.orders")
2
3 typed = (bronze
4           .withColumn("order_ts", F.to_timestamp("order_purchase_timestamp"))
5           .withColumn("total_amount", F.col("payment_value").cast("decimal(12,2)")))
6
7 valid = typed.where("order_id IS NOT NULL AND total_amount >= 0")
8 rejected = (typed.where("order_id IS NULL OR total_amount < 0")
9             .withColumn("_reject_reason", F.lit("null key or negative
10          amount"))))
11
12 # Idempotent upsert into Silver; rejected rows are never dropped silently
13 (valid.write.format("delta").mode("append").saveAsTable("staging.orders_valid")
14   )
15
16 spark.sql("""MERGE INTO de_training.silver.orders t
17            USING staging.orders_valid s ON t.order_id = s.order_id
18            WHEN NOT MATCHED THEN INSERT *""")
19
20 (rejected.write.format("delta").mode("append")
21   .saveAsTable("de_training.silver.orders_quarantine"))

```

Listing 3.2: Silver cleansing with quarantine accounting.

### 3.4.2 Reconciliation

Every transition publishes its arithmetic: `bronze_in = silver_accepted + quarantined`. A one-line assertion in the pipeline (and later, a DLT expectation or a Workflow gate) turns silent row loss into a loud failure.

### 3.4.3 Gold: The Dimensional Contract

Gold is modeled for consumption: `fact_orders` (measures, foreign keys) surrounded by `dim_customer`, `dim_product`, `dim_date`. Facts are narrow and additive; dimensions are conformed and slowly



changing where history matters. Gold is the only layer analysts should query by default—Chapter 8 enforces that with grants.

| Layer      | Question answered    | Audience      | Write pattern              |
|------------|----------------------|---------------|----------------------------|
| Bronze     | What did we receive? | Engineers     | Append-only                |
| Silver     | What is clean truth? | Pipelines     | Keyed MERGE / replaceWhere |
| Gold       | What do we measure?  | Analysts, ML  | Scheduled rebuild / merge  |
| Quarantine | What failed and why? | Data stewards | Append-only                |

Table 3.1: One job, one audience, one write pattern per layer.

### 3.5 Thursday Hands-on Project: The Olist Medallion

Build the three-layer flow for the Olist e-commerce dataset [57]: land nine raw CSVs into Bronze, cleanse into Silver with quarantine, publish a Gold star schema, and reconcile every transition.

```

1 from pyspark.sql import functions as F
2
3 orders = spark.read.table("de_training.silver.orders")
4 items  = spark.read.table("de_training.silver.order_items")
5 cust   = spark.read.table("de_training.silver.customers")
6
7 dim_customer = cust.select("customer_id", "customer_unique_id",
8                             "customer_city", "customer_state")
9 dim_customer.write.format("delta").mode("overwrite") \
10    .saveAsTable("de_training.gold.dim_customer")
11
12 fact_orders = (orders.join(items, "order_id")
13    .groupBy("order_id", "customer_id", F.to_date("order_ts").alias("order_date"))
14    .agg(F.sum("price").alias("net_amount"),
15         F.count("order_item_id").alias("line_count")))
16 fact_orders.write.format("delta").mode("overwrite") \
17    .saveAsTable("de_training.gold.fact_orders")
18
19 # Reconciliation: Gold facts must equal Silver order count
20 silver_n = spark.table("de_training.silver.orders").count()
21 gold_n    = spark.table("de_training.gold.fact_orders").count()
22 assert silver_n == gold_n, f"reconciliation failed: {silver_n} != {gold_n}"

```

Listing 3.3: Gold star schema build with reconciliation.

#### Deliverables

- Bronze tables for all nine Olist CSVs with ingestion metadata columns.
- Silver tables with typed schemas, deduplicated keys, and populated quarantine tables.
- Gold `fact_orders`, `dim_customer`, `dim_product`, `dim_date`.
- A reconciliation notebook asserting accepted + rejected = input for every table.

#### Acceptance Criteria

- Full re-run from a clean clone changes zero Silver/Gold row counts (idempotence proof).

- At least one deliberately poisoned CSV lands in quarantine with a reason, and reconciliation still balances.
- Gold aggregates (total revenue) match a direct Bronze computation within rounding.
- Lineage columns (`_source_file`, `_ingested_at`) trace a Gold row to its raw file.

### Stretch Goals

- Add `dim_date` generated from the calendar range of the facts.
- Parameterize the Silver notebook by `processing_date` and prove a single-day re-run.
- Convert one Silver transition to `replaceWhere` partition overwrite and explain when it beats MERGE.

## 3.6 Friday Use Case: Two Years of Reprocessing Without Double-Counting

A retail chain changes its revenue recognition rule: cancelled-then-refunded orders must now net against the month of the original sale. Two years of Gold must be rebuilt. The CFO's constraint: at no point may the revenue number double-count, and the old numbers must remain queryable for the restatement filing.

### 3.6.1 Requirements and Assumptions

Bronze retains all raw extracts (append-only). Gold serves dashboards that finance reads daily. The restatement window is one weekend. Fifty downstream reports reference `gold.fact_orders` by name.

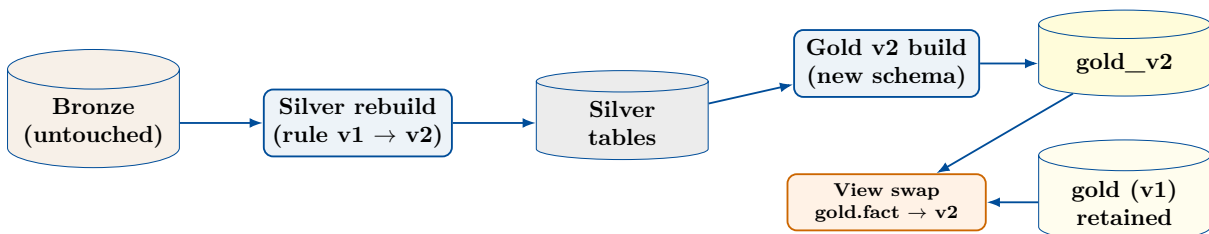


Figure 3.1: Zero-double-count restatement: rebuild into parallel tables, then swap via view or catalog rename in one auditable step.

### 3.6.2 Architecture Decisions

**Parallel build, atomic swap.** Silver and Gold v2 build alongside v1 from immutable Bronze. Reconciliation gates the cutover: v2 totals are compared against v1 with an expected-delta report derived from the rule change. Cutover is a view re-point (`CREATE OR REPLACE VIEW gold.fact_orders AS SELECT * FROM gold_v2.fact_orders`)—one commit, one history entry, instantly reversible.

**Evidence.** The old tables persist under `gold_v1_archive` for the filing; `DESCRIBE HISTORY` plus the reconciliation report form the restatement evidence pack.

**Common Pitfall**

**Backfilling in place during business hours.** Rewriting `gold.fact_orders` while dashboards read it produces mixed-rule numbers—the worst outcome: not wrong enough to notice, not right enough to file. Parallel build plus atomic swap exists precisely for this scenario.

**3.6.3 Risks and Mitigations**

| Risk                                      | Mitigation                                                        |
|-------------------------------------------|-------------------------------------------------------------------|
| Restatement weekend over-runs             | Parallel v2 build rehearsed; cutover is one reversible view swap  |
| Quarantine becomes a write-only graveyard | Weekly steward review with per-reason trends                      |
| Gold consumers query Silver directly      | Grants make Gold the only readable layer for analysts             |
| Bronze storage cost questioned            | Documented reprocessing value; lifecycle rules on raw volumes     |
| Reconciliation tolerances gamed           | Assertions are arithmetic on counts, not thresholds people choose |

Table 3.2: Week 3 scenario risks and mitigations.

**3.7 Design Decision Guide**

Medallion design is mostly discipline decisions made once per pipeline.

| Decision                                  | Choose the first when                     | Choose the second when                  |
|-------------------------------------------|-------------------------------------------|-----------------------------------------|
| Keyed MERGE vs. <code>replaceWhere</code> | Records arrive as changes                 | A partition is restated wholesale       |
| Quarantine vs. fail-fast                  | Bad rows are routine                      | Bad rows invalidate the batch           |
| Star schema vs. wide Gold table           | Many consumers and facts share dimensions | One consumer, one fixed question        |
| Three layers vs. two                      | Reprocessing or multiple consumers exist  | Truly single-use, disposable data       |
| View swap vs. in-place backfill           | Consumers must never see mixed state      | The table is internal and rebuildable   |
| Strict types in Bronze vs. strings        | The source contract is enforced upstream  | Sources drift and evidence matters more |

Table 3.3: Week 3 decision guide.

The pattern across the table: choose the option that keeps the transition idempotent and the evidence preserved. Both compound; both are invisible until the day they save you.

**3.8 Operational Guidance for Week 3****3.8.1 Performance and Reliability**

The reconciliation ledger is the layer’s heartbeat: every transition appends its arithmetic (input, accepted, quarantined) and a daily report trends it—a rising quarantine rate is a source-schema

change announcing itself before anyone calls. Reprocessing drills are scheduled, not aspirational: once a month, rebuild one Silver table from Bronze in a scratch schema and diff it. Gold rebuild cost should be measured and documented so restatement requests come with a known price.

### 3.8.2 Security and Governance Checklist

- Layer grants match the audience model: pipelines write Bronze, engineers write Silver, analysts read Gold.
- Quarantine tables inherit the source’s PII classification—rejected rows still contain PII.
- Bronze retention set from the reprocessing objective, not storage cost alone.
- Lineage metadata columns (`_source_file`, `_ingested_at`) populated on every Bronze write.

### 3.8.3 Troubleshooting Playbook

| Symptom                        | Likely cause                                | First action                                           |
|--------------------------------|---------------------------------------------|--------------------------------------------------------|
| Reconciliation mis-match       | Duplicate keys fanning out a join           | Dedup check on both sides; inspect the MERGE condition |
| MERGE “multiple matches” error | Duplicate business keys in the source batch | Dedup source on the key before merging                 |
| Rerun doubled Gold counts      | Non-idempotent write (blind append)         | Convert to keyed MERGE or <code>replaceWhere</code>    |
| Source file arrived late       | Upstream SLA slip                           | Re-run the partition-scoped transition for that date   |

Table 3.4: Week 3 troubleshooting quick reference.

## 3.9 Interview Q&A

### 3.9.1 Concepts and Trade-offs

1. **Q: Why keep Bronze forever when storage costs money?**  
A: Bronze is the only layer that can answer “what did the source actually say” and the only input for rebuilding after logic changes. Its cost is small next to re-acquisition or restatement failure.
2. **Q: When is two layers (skip Silver) acceptable?**  
A: Almost never for shared data. A single-source, single-consumer pipeline may collapse layers, but the moment a second consumer or a reprocessing need appears, you rebuild Silver under pressure.
3. **Q: How do you prove a transition lost no rows?**  
A: Reconciliation arithmetic per run: input count equals accepted plus quarantined, asserted in the pipeline and logged to a run ledger.
4. **Q: MERGE or `replaceWhere` for Silver?**  
A: MERGE for keyed upserts arriving as changes; `replaceWhere` for full partition restatements (daily snapshots). Both are idempotent; mixing them per table is not.
5. **Q: Why a star schema in Gold rather than one wide table?**  
A: Shared conformed dimensions let many facts join consistently, selective scans stay

cheap, and BI tools model relationships natively. Wide tables duplicate dimension logic into every fact.

6. **Q: Who owns the quarantine table?**

A: A named steward with a review cadence. Quarantine without an owner is where data goes to be forgotten; with one, it is your earliest upstream-quality signal.

7. **Q: Why does the medallion make streaming easier later?**

A: Each layer transition already has idempotent, keyed write semantics—the exact properties Structured Streaming sinks require. A well-built batch medallion becomes a streaming medallion by changing triggers, not contracts.

8. **Q: Where do source-system reference tables (country codes, calendars) live?**

A: As governed Gold dimensions with an owner and a refresh contract—they are conformed shared assets, and treating them as casual lookups is how three definitions of “fiscal month” happen.

### 3.10 Further Reading

The Databricks medallion guide is the canonical reference [50]; pair it with the Delta Lake documentation for the write patterns (`MERGE`, `replaceWhere`) that make layers idempotent [53, 46]. The lakehouse paper explains why this layering displaced the two-tier lake-plus-warehouse architecture [9]. The Olist dataset page documents the nine source tables used in the lab [57].

### Key Takeaways

- Bronze is evidence, Silver is truth, Gold is measurement—one job and one audience per layer.
- Every transition is idempotent (keyed `MERGE` or partition overwrite) and reconciled by arithmetic.
- Quarantine tables turn silent data loss into visible, explainable rejections.
- Reprocessing is a first-class design requirement: build v2 in parallel, gate on reconciliation, swap atomically.
- Catalog/schema naming (`catalog.bronze|silver|gold.table`) is governance infrastructure, not cosmetics.

### 3.11 Exercises

1. **(Conceptual)** A stakeholder proposes skipping Silver “for speed.” Write the three-question checklist you would ask before agreeing.
2. **(Conceptual)** Why is append-only Bronze incompatible with GDPR-style erasure, and what pattern resolves it (hint: Chapter 22 revisits this)?
3. **(Hands-on)** Complete the Thursday lab, including one poisoned file, and capture the reconciliation output.
4. **(Hands-on)** Add `dim_seller` to the Gold model and extend the reconciliation to cover it.
5. **(Design)** Design the medallion layout for a second source system (inventory) sharing the same Gold dimensions.

6. **(Design)** Write the cutover runbook for the Friday scenario, including the rollback step.
7. **(Governance)** Which layers should analysts be able to read by default, and how would you enforce it (preview of Chapter 8)?
8. **(Architecture)** At what data volume would you move the Silver transition from batch MERGE to a streaming design (preview of Part IV)? Justify with numbers.

# Apache Spark on Databricks

## Learning Objectives

- Explain Spark’s driver/executor architecture, DAG scheduler, and lazy evaluation.
- Distinguish narrow and wide transformations and predict when a shuffle occurs.
- Read the Spark UI and **explain** plans to locate expensive stages, skew, and spills.
- Control partitioning deliberately with **repartition**, **coalesce**, and shuffle settings.
- Use caching judiciously and recognize when it hurts.
- Diagnose a regressed nightly job and write an evidence-based tuning plan.

## Cross-Cloud Equivalence

Spark on Databricks is the **reference implementation** of managed Spark; the engine is the same everywhere, but the operational wrapper differs.

| Capability            | Databricks                         | AWS                          | Azure / GCP                            |
|-----------------------|------------------------------------|------------------------------|----------------------------------------|
| Managed Spark         | Databricks Runtime (Photon)        | Glue / EMR / EMR Serverless  | Synapse Spark / Fabric / Dataproc      |
| Cluster manager       | Built-in (no YARN to run)          | YARN / EMR managed           | Synapse / Dataproc managed             |
| GPU/vectorized engine | Photon                             | EMR runtime (partial)        | Fabric native engine                   |
| Autoscaling           | Autoscaling clusters + serverless  | EMR managed scaling          | Autoscale pools / Dataproc autoscaling |
| UI and diagnostics    | Spark UI + Ganglia + query profile | Spark UI on EMR              | Spark UI / Fabric monitoring           |
| Default table format  | Delta                              | Iceberg/Hudi/Delta by config | Delta (Fabric) / BigQuery tables       |

**Snowflake** plays this role with Snowpark (DataFrame API on its engine); **MongoDB** integrates via the Spark Connector. Skills from this chapter transfer to every managed-Spark offering almost unchanged.

## 4.1 Week 4 Roadmap and Mental Model

Databricks exists because of Spark, and Spark skills are the single highest-leverage engineering skill on the platform [59, 10]. Week 4 builds the execution mental model: driver, executors, DAG, stages, tasks—and the habits of reading plans and the UI before touching configuration.

The mental model: **Spark is a lazy, distributed compiler for dataflow**. You describe transformations on logical datasets; nothing runs until an action forces it; Catalyst compiles

your description into an optimized physical plan; the scheduler splits that plan into stages at shuffle boundaries; each stage runs as parallel tasks over partitions. Every performance conversation in this book is a conversation about one of those steps.

#### Definition

**Apache Spark** is a distributed data-processing engine: a driver program plans work as a DAG of transformations over partitioned data, and executors run that work as parallel tasks. **Lazy evaluation** means transformations build a plan; only actions (count, write, collect) execute it.

#### Key Terms

**Partition:** a chunk of data processed by one task. **Shuffle:** the all-to-all data movement between stages required by wide transformations. **Skew:** one partition holding disproportionately many records. **Spill:** writing intermediate data to disk when memory is exhausted. **Photon:** Databricks' vectorized execution engine for SQL/DataFrame workloads.

## 4.2 Monday: Architecture and Lazy Evaluation

### 4.2.1 Driver, Executors, DAG, Stages, Tasks

The **driver** hosts your `SparkSession`, builds the logical plan, optimizes it through Catalyst, and coordinates execution. **Executors** are JVM (plus Photon native) processes on cluster workers that run tasks and hold cached data. The physical plan splits into **stages** at shuffle boundaries; each stage launches one **task** per partition. Cluster sizing is arithmetic on these nouns: tasks per stage = partitions; parallel tasks  $\leq$  total executor cores.

### 4.2.2 Lazy Evaluation and Catalyst

Because transformations are lazy, Catalyst sees the whole pipeline before running anything: it pushes filters to the scan, prunes columns, constant-folds expressions, and reorders joins by statistics. This is why writing `filter` before `select` rarely matters—and why an early `collect()` or `count()` “for debugging” can silently destroy the optimization by forcing intermediate execution [4].

```
1 df = (spark.read.table("de_training.silver.orders")
2     .filter(F.col("order_status") == "delivered")
3     .groupBy("customer_state")
4     .agg(F.sum("total_amount").alias("revenue")))
5
6 df.explain(True)    # logical, analyzed, optimized, and physical plans
7 # Look for: Exchange hashpartitioning (the shuffle), and whether the
8 # filter appears in the scan (pushdown) or as a separate Filter node.
```

Listing 4.1: Reading the plan before running the job.

## 4.3 Tuesday: Transformations, Actions, and the Shuffle

### 4.3.1 Narrow versus Wide

**Narrow** transformations (`select`, `filter`, `withColumn`) map partition-in to partition-out with no data movement. **Wide** transformations (`groupBy`, `join`, `distinct`, `repartition`) require a **shuffle**: every executor sends slices of its data to every other executor. Shuffles dominate job cost: network, serialization, disk spill, and the barrier between stages.



**Common Pitfall**

**The accidental wide transformation.** `dropDuplicates()` with no columns, `orderBy` (global sort), and `join` on a non-partitioned key all shuffle. In a notebook it costs seconds; on 2 TB it costs hours. Run `explain` and count the `Exchange` nodes before you run the job.

### 4.3.2 Partitions and the 200 Default

After a shuffle, Spark produces `spark.sql.shuffle.partitions` partitions—default 200. For 2 GB of shuffled data that is 200 tiny tasks of overhead; for 2 TB it is 200 gigantic tasks that spill. Adaptive Query Execution (AQE, on by default in Databricks Runtime) coalesces small partitions and splits skewed ones at runtime from actual statistics [5]. Manual tuning still matters when AQE’s assumptions do not fit—Chapter 9 goes deep.

## 4.4 Wednesday: The Spark UI, Caching, and Partitioning

### 4.4.1 Reading the Spark UI

The Spark UI (Compute → cluster → Spark UI, or per-query in the notebook) is the evidence source: the **Jobs/Stages** tabs show task time distributions and shuffle read/write; the **SQL** tab shows the physical plan with per-node row counts and durations; the **Executors** tab shows memory pressure and GC. The diagnostic habit: find the slowest stage, look at its task summary—a long tail (max task time ≫ median) means skew; heavy shuffle read means a wide transformation to question.

### 4.4.2 Caching Deliberately

`cache()` pins a `DataFrame`’s computed partitions in memory (and disk) for reuse. Cache when a `DataFrame` is read repeatedly by downstream branches of your job. Do not cache data scanned once, data larger than executor memory (it spills or evicts, costing more than recompute), or data you then transform anyway—cache the *result* of the expensive transformation.

**Tip**

In notebooks, always `unpersist()` training caches at the end of a session on a shared cluster—your cached 50 GB is your teammate’s out-of-memory error.

### 4.4.3 Repartition versus Coalesce

`repartition(n)` shuffles into exactly  $n$  partitions (can increase or decrease); `coalesce(n)` only decreases, merging partitions without a full shuffle. Use `repartition` to fix skew or parallelism; `coalesce` before writing to reduce output file count—the small-file problem you will meet formally in Chapter 5.

## 4.5 Thursday Hands-on Project: Find the Shuffle, Fix the Skew

Using 10M NYC Taxi rows [58], build an aggregation with a deliberately skewed key, find the evidence in the Spark UI, and fix it.

```
1 from pyspark.sql import functions as F
2
3 trips = spark.read.table("samples.nyctaxi.trips")
4
```

```

5 # 1. Deliberate skew: 80% of rows share one key
6 skewed = trips.withColumn(
7     "hot_key",
8     F.when(F.rand(seed=42) < 0.8, F.lit("HOT")).otherwise(F.col("payment_type"))
9 )
10 agg = (skewed.groupBy("hot_key")
11     .agg(F.sum("fare_amount").alias("revenue"),
12         F.count("*").alias("trips")))
13 agg.write.mode("overwrite").saveAsTable("de_training.silver.week04_skewed")
14
15 # 2. Evidence: Spark UI > SQL tab > the Exchange + HashAggregate nodes;
16 #   Stages tab: one task processes ~80% of shuffle rows (max >> median).
17
18 # 3a. Fix with salting: spread the hot key across N buckets
19 salted = (skewed.withColumn("salt", (F.rand(seed=1) * 16).cast("int"))
20     .withColumn("salted_key", F.concat_ws("_", "hot_key", "salt")))
21 partial = (salted.groupBy("salted_key")
22     .agg(F.sum("fare_amount").alias("revenue"),
23         F.count("*").alias("trips")))
24 final = (partial.withColumn("hot_key", F.split("salted_key", "_").getItem(0))
25     .groupBy("hot_key")
26     .agg(F.sum("revenue").alias("revenue"), F.sum("trips").alias("trips"))
27 )
28 final.write.mode("overwrite").saveAsTable("de_training.silver.week04_saltred")
29
30 # 3b. Or let AQE try first:
31 spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

```

Listing 4.2: Week 4 lab: induce, observe, and fix skew.

## Deliverables

- Spark UI screenshots (or exported metrics) of the skewed stage: task summary with max >> median.
- The salted (or repartitioned) run's stage summary for comparison.
- A results table: job duration, shuffle read, max task time—before and after.

## Acceptance Criteria

- You identify the exact stage and node causing skew from UI evidence, not guesswork.
- The fix measurably reduces max task time while producing identical results (row-for-row reconciliation between skewed and salted outputs).
- One experiment deliberately disables AQE to show its effect, then re-enables it.
- The cluster used is terminated at lab end.

## Stretch Goals

- Repeat with a skewed *join* and compare salting vs. broadcast vs. AQE skew handling.
- Measure the effect of `spark.sql.shuffle.partitions` at 50, 200, and 2000 on the same job.
- Rewrite the aggregation in Spark SQL and compare physical plans with the DataFrame version.

## 4.6 Friday Use Case: The Nightly Job That Grew from 40 Minutes to 6 Hours

A partner’s nightly PySpark job processed 200 GB in 40 minutes last year; at 2 TB it now misses its 6 a.m. SLA by three hours. No code changed. You are asked for a tuning plan.

### 4.6.1 Requirements and Assumptions

The job reads partitioned Parquet, joins against a slowly growing dimension table, aggregates, and writes a daily Gold partition. Cluster size has not changed. You have Spark UI access to recent runs.

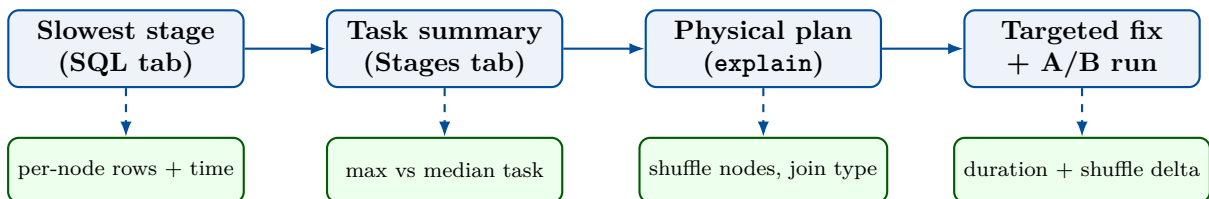


Figure 4.1: The evidence-driven tuning loop: locate, attribute, hypothesize, measure.

### 4.6.2 Architecture Decisions

**Diagnose before configuring.** The expected finding: the dimension table crossed the broadcast threshold as it grew, flipping a broadcast hash join to a sort-merge join with a full shuffle—or a new skewed key arrived with a data change. The evidence table: stage durations over the regression window, join type in the plan, task-time tails.

**The fix menu, in order.** (1) Re-enable broadcast for the dimension (`broadcast()` hint) if it still fits memory; (2) fix shuffle partition count for the new volume; (3) salt the skewed key; (4) right-size the cluster only after the plan is sane. Scaling hardware against a bad plan multiplies cost, not speed.

#### Common Pitfall

**“Just add workers.”** If the plan has a 200-partition shuffle over 2 TB or a skewed sort-merge join, more executors change nothing but the bill. Plans first, hardware second—and re-baseline after every fix so gains are attributable.

### 4.6.3 Risks and Mitigations

| Risk                                     | Mitigation                                             |
|------------------------------------------|--------------------------------------------------------|
| Tuning fix regresses next month silently | Weekly duration trend with a config-change log         |
| Skew returns with a new tenant           | Skew metric (max/median task time) alerted per run     |
| “Optimized” job breaks correctness       | Row-for-row reconciliation gate in every tuning change |
| Spark UI evidence lost with the cluster  | Metrics exported before job clusters terminate         |
| Dimension outgrows the broadcast hint    | Size assertion in the job; alert before the flip       |

Table 4.1: Week 4 scenario risks and mitigations.

## 4.7 Design Decision Guide

Spark decisions are measurement-backed by habit; the defaults below hold until the UI says otherwise.

| Decision                             |  | Choose the first when                         | Choose the second when                 |
|--------------------------------------|--|-----------------------------------------------|----------------------------------------|
| repartition vs. coalesce             |  | Increasing parallelism or fixing distribution | Only reducing output partitions        |
| Cache vs. recompute                  |  | A DataFrame feeds multiple actions            | Single-pass pipelines                  |
| Broadcast vs. sort-merge join        |  | One side provably fits in memory              | Both sides are large                   |
| Trust AQE vs. manual tuning          |  | Typical workloads                             | Known extreme skew or stale statistics |
| DataFrame API vs. RDD                |  | Everything in this book                       | Exotic partition-level control         |
| More partitions vs. bigger executors |  | Tasks spill or are too coarse                 | Scheduling overhead dominates          |

Table 4.2: Week 4 decision guide.

Every row has the same footnote: measure first. The decision guide narrows the hypothesis space; the Spark UI picks the row.

## 4.8 Operational Guidance for Week 4

### 4.8.1 Performance and Reliability

Adopt the evidence-retention habit early: for every production Spark job, the Spark UI metrics (stage durations, shuffle volumes, task summaries) of the last N runs should be exported or screenshotted before clusters terminate—job clusters delete their UI history with the VMs. Track p50 duration per job weekly; a 20% week-over-week regression with no config change is a data-shape change and deserves the same investigation as a failure.

### 4.8.2 Security and Governance Checklist

- Shared all-purpose clusters run with table access control via Unity Catalog only (no legacy workspace ACLs mixed in).
- Init scripts and uploaded libraries come from reviewed sources—they execute as the cluster’s identity.
- Notebook outputs that may contain PII rows are cleared before screenshots into tickets.
- Caches on shared clusters are unpersisted at session end.

### 4.8.3 Troubleshooting Playbook

| Symptom                   | Likely cause                               | First action                                                           |
|---------------------------|--------------------------------------------|------------------------------------------------------------------------|
| Driver out of memory      | <code>collect()/toPandas()</code> at scale | Find the driver-side pull; replace with sampled or aggregated read     |
| One task takes hours      | Skewed key in a wide transformation        | Task-summary tail in the UI; salt or AQE skew handling                 |
| Tasks spill to disk       | Partition too large / memory pressure      | Raise shuffle partitions or executor memory; check caching competition |
| First run slow, then fast | Cold metadata/cache                        | Warm-up is normal; benchmark with medians, not single runs             |

Table 4.3: Week 4 troubleshooting quick reference.

## 4.9 Interview Q&A

### 4.9.1 Concepts and Trade-offs

1. **Q: Walk me through what happens when I call `df.write.saveAsTable()`.**  
A: The driver runs Catalyst on the accumulated logical plan, produces a physical plan, splits it into stages at shuffles, schedules one task per partition per stage, executors run tasks and write files, and the final commit makes the table version visible.
2. **Q: Why is the shuffle the usual suspect?**  
A: It is the only operation requiring all-to-all data movement—network, serialization, disk spill—plus a barrier that prevents pipelining. Most big-job pathologies (skew, spill, tiny tasks) are shuffle pathologies.
3. **Q: What does AQE give you, and when do you override it?**  
A: Runtime re-optimization: coalescing small shuffle partitions, converting sort-merge to broadcast joins, splitting skewed partitions—from actual statistics. Override when its thresholds misfit: known-skewed keys, or a broadcast it declined that you can prove fits.
4. **Q: When do you cache?**  
A: When one computed DataFrame feeds multiple downstream actions and its recompute is expensive. Never for single-scan pipelines; always unpersist on shared clusters.
5. **Q: repartition or coalesce before writing 5,000 small files?**  
A: `coalesce` if reducing from an already-parallel stage (no full shuffle); `repartition` if you also need to change the distribution. Then let Chapter 5’s OPTIMIZE handle the rest.

6. **Q: What is the driver doing while executors run tasks?**

A: Tracking task completion, serving the UI, planning the next stage—and holding anything you `collect()`ed, which is why driver OOMs are usually self-inflicted.

7. **Q: Does Photon change anything about tuning method?**

A: The engine is faster per core, but the method is identical: read the plan and UI, fix joins/shuffles/skew first, and measure. Photon lowers the ceiling after the plan is sane, not before.

8. **Q: How do you size a cluster for a brand-new job?**

A: Estimate shuffle volume, pick partitions for ~100–200 MB each, give executors enough memory for the largest task plus spill headroom, then let the first three runs' UI metrics correct your estimate. Sizing is iterated, not derived.

## 4.10 Further Reading

Start with the RDD paper for the execution model's origins [59] and *Spark: The Definitive Guide* for a practitioner tour [10]. The Spark documentation covers the DataFrame API [4] and the performance-tuning guide covers AQE, join strategy, and shuffle configuration [5]. The Photon page explains Databricks' vectorized execution layer [49].

## Key Takeaways

- Spark compiles lazy transformations into stages of parallel tasks; shuffles split stages and dominate cost.
- `explain` and the Spark UI are the evidence sources—diagnose from them, not from intuition.
- Skew shows as a task-time tail; salting, repartitioning, or AQE skew handling are the fixes.
- Cache only reused, expensive-to-recompute DataFrames; unpersist on shared clusters.
- Tune the plan before the hardware: a bad join strategy scales costs, not throughput.

## 4.11 Exercises

1. **(Conceptual)** Define narrow and wide transformations and give two examples of each.
2. **(Conceptual)** Why does an early `count()` in a notebook break Catalyst's whole-pipeline optimization?
3. **(Hands-on)** Complete the Thursday lab; capture both stage summaries and the reconciliation check.
4. **(Hands-on)** Find the `Exchange` nodes in three different queries and explain what data movement each represents.
5. **(Hands-on)** Measure a cached vs. uncached two-branch pipeline; report when caching won and by how much.
6. **(Design)** Write the investigation report for the Friday scenario as if delivering it to the partner: evidence, hypothesis, fix, expected SLA.

7. **(Design)** Choose shuffle partition counts for 5 GB, 500 GB, and 5 TB shuffles; justify each.
8. **(Architecture)** When would you move this job from an all-purpose cluster to a job cluster or serverless? Quantify the trade-off.

## 4.12 Milestone 1 Capstone: The Retail Medallion Lakehouse

### Capstone Project

**Mission.** Close Part I by building the retail medallion lakehouse end to end: land the Olist dataset into Bronze via a Unity Catalog volume with ingestion metadata, cleanse into Silver with quarantine and reconciliation, publish a Gold star schema, and prove operability with a corruption-and-RESTORE recovery drill.

**Estimated time.** 6–8 hours, plus optional cost-modeling and security-review time.

**What you'll practice.**

- Volume-based raw landing with `_ingested_at/_source_file` metadata.
- Parameterized PySpark notebooks per layer (no code edits between runs).
- Idempotent Silver upserts and reconciled, quarantined rejects.
- Gold dimensional modeling and a measured Spark aggregation.
- Time-travel forensics and RESTORE recovery with history evidence.

### Scenario

Contoso Retail's analytics team receives nine CSV extracts nightly from the Olist-style order system. Today, analysts clean data in spreadsheets; every restatement is a manual crisis. You are building the lakehouse foundation that makes data trustworthy, reproducible, and recoverable.

### Requirements

- Catalog `de_training` with `bronze`, `silver`, `gold` schemas and volume `de_training.bronze.landing`.
- Bronze tables for all nine CSVs, append-only, with lineage metadata.
- Silver tables typed, deduplicated on business keys, with a quarantine table per source and a reconciliation assertion per transition.
- Gold `fact_orders`, `dim_customer`, `dim_product`, `dim_date`; a revenue-by-state aggregation measured in the Spark UI.
- A documented drill: corrupt a Silver table, find the last good version in history, RESTORE, verify counts.
- README with architecture diagram, setup commands, cost estimate (DBUs + storage), and lessons learned.

## Reference Architecture

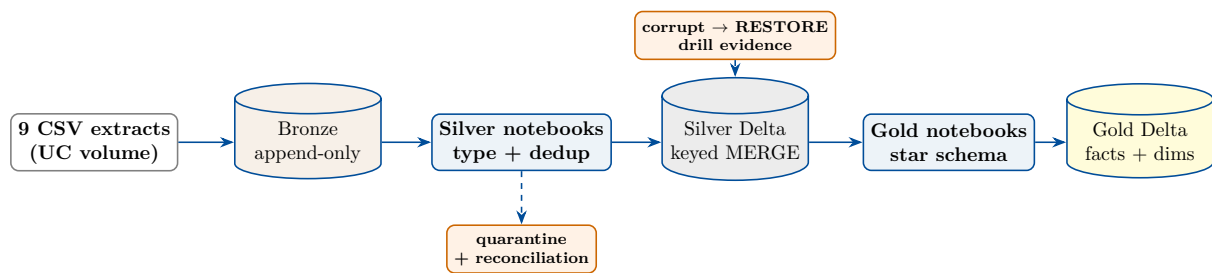


Figure 4.2: Milestone 1 reference architecture: medallion lakehouse with quality gates and recovery drill.

## Implementation Steps

1. Create the catalog, schemas, and volume; upload the nine Olist CSVs.
2. Write `bronze_ingest.py`: parameter-driven landing of any CSV with metadata columns.
3. Write `silver_transform.py`: typing, dedup, quarantine, keyed MERGE; assert reconciliation.
4. Write `gold_publish.py`: star schema build; measure the revenue aggregation in the Spark UI and record stage metrics.
5. Run the recovery drill and capture `DESCRIBE HISTORY` before/after `RESTORE`.
6. Write the README per the portfolio guide; estimate cost from `system.billing.usage`.

## Deliverables

- Git repository with the three notebooks, README, and architecture diagram.
- Reconciliation output proving accepted + rejected = input for every table.
- Recovery-drill evidence: history entries and matching pre/post row counts.
- Cost estimate with two documented optimization decisions.

## Acceptance Criteria

- Clean-clone re-run of all three layers completes with zero manual edits and zero row-count drift.
- A poisoned file demonstrably lands in quarantine while reconciliation still balances.
- The recovery drill restores the exact pre-corruption count, evidenced by history.
- No hardcoded credentials anywhere; the cluster used is policy-bound and terminated.

## Stretch Goals

- Wrap the three notebooks as a multi-task Workflow job (preview of Chapter 12).
- Add a Gold data-quality test suite (uniqueness, referential integrity) as a separate notebook.



- Rebuild Gold from Bronze in a parallel schema and cut over with a view swap, per Chapter 3's restatement pattern.

## Part II

# Warehousing and Analytics

# Delta Lake Optimization: OPTIMIZE, Z-ORDER, and Liquid Clustering

## Learning Objectives

- Explain how file-level statistics enable data skipping, and why the small-file problem defeats it.
- Compact tables with `OPTIMIZE` and cluster them with `ZORDER BY`.
- Choose liquid clustering (`CLUSTER BY`) for new tables and articulate when partitioning still applies.
- Use deletion vectors to make row-level deletes cheap, and auto-optimize versus scheduled compaction deliberately.
- Operate `VACUUM` and predictive optimization within a retention policy.
- Benchmark layout choices with controlled before/after measurements.

## Cross-Cloud Equivalence

Physical layout optimization exists on every analytical platform; the knobs differ, the physics does not.

| Capability             | Databricks (Delta)        | AWS                                     | Azure / GCP / Fabric                                          |
|------------------------|---------------------------|-----------------------------------------|---------------------------------------------------------------|
| Compaction             | <code>OPTIMIZE</code>     | Athena/Iceberg<br><code>OPTIMIZE</code> | <code>OPTIMIZE</code> (Fabric) /<br>BigQuery managed          |
| Multi-dim clustering   | <code>ZORDER BY</code>    | Redshift sort keys                      | Synapse column-store /<br>Fabric V-Order+ <code>ZORDER</code> |
| Rewritable-free layout | Liquid clustering         | Iceberg sort order evolution            | BigQuery clustering /<br>clustering keys (Snowflake)          |
| Row-level deletes      | Deletion vectors          | Iceberg position deletes                | Native / deletion vectors (Fabric)                            |
| Auto-maintenance       | Predictive optimization   | Redshift Auto WLM /<br>S3 Tables        | Automatic / BigQuery<br>autoclustering                        |
| Pruning metadata       | File stats, bloom filters | Zone maps (Redshift)                    | Columnstore segments /<br>block metadata                      |

**MongoDB** answers the same question with indexes and the working set rather than file layout—an operational-store solution to an analytical problem it was not designed for.

## 5.1 Week 5 Roadmap and Mental Model

Chapter 2 made tables correct; Chapter 5 makes them fast. Analytical query speed on a lakehouse is dominated by one question: *how many bytes must be read to answer this query?* Everything in this chapter—compaction, clustering, statistics, deletion vectors—exists to shrink that number [36].

The mental model: **a Delta scan is a funnel**. The query’s predicates are evaluated first against the transaction log’s per-file statistics (min/max per column, null counts), discarding files that cannot match; surviving files are opened, and column pruning plus row-group statistics discard more; only then are rows decoded. The optimizer’s job is to make the funnel narrow early. The engineer’s job is to arrange data so that statistics *can* discard—which is what clustering does.

### Definition

**Data skipping** is the elimination of data files (and row groups) from a scan based on metadata alone, without reading row content. It works only when the data layout correlates file contents with the query’s filter columns—the property that Z-ORDER and liquid clustering create.

### Key Terms

**Small-file problem:** thousands of tiny files whose metadata overhead and per-file cost dominate query time. **Z-ORDER:** multi-dimensional clustering that co-locates rows similar on several columns in the same files. **Liquid clustering:** Delta’s self-managing layout where clustering keys are metadata, not directory structure. **Deletion vector:** a bitmap marking deleted rows so deletes need not rewrite files. **Predictive optimization:** Databricks-managed automatic OPTIMIZE/ANALYZE/VACUUM for Unity Catalog tables.

## 5.2 Monday: File Layout, Statistics, and the Small-File Problem

### 5.2.1 How a Delta Scan Reads

Each commit records added files with per-column min/max statistics (for the first 32 columns by default). A query like `WHERE order_date = '2026-08-01'` consults the log and opens only files whose statistics admit that date. Bloom filters (opt-in) add point-lookup skipping for high-cardinality columns such as `order_id`.

### 5.2.2 The Small-File Problem

Every file costs: a listing/metadata entry, a footer read, a task slot, and open/close overhead. At thousands of 1 MB files, metadata work dwarfs data work; queries crawl, and OPTIMIZE itself becomes slow. Small files come from streaming micro-batches, over-partitioned writes, and un-coalesced outputs (Chapter 4). The fix is compaction; the prevention is write cadence [36].

### Common Pitfall

**Optimizing while streaming into the same table.** A compaction job rewriting files that a stream is actively appending creates commit conflicts and wasted work. Coordinate schedules—triggered streams followed by OPTIMIZE—or use predictive optimization, which is conflict-aware.

## 5.3 Tuesday: OPTIMIZE and Z-ORDER

### 5.3.1 Compaction

`OPTIMIZE` table rewrites small files into  $\sim 1$  GB targets. It is incremental (only unoptimized files are touched) and recorded in history like any commit. Run it on a schedule matched to write cadence: daily for batch-loaded tables, after each trigger group for streaming sinks.

### 5.3.2 Z-ORDER

`ZORDER BY (col1, col2)` adds multi-dimensional clustering: the compaction sorts data so rows similar on the given columns co-locate, making file-level statistics sharply effective for filters on those columns.

```

1 -- Baseline first: file count and a representative selective query
2 SELECT region, COUNT(*) c, SUM(net_amount) AS rev
3 FROM de_training.silver.events
4 WHERE event_date = DATE '2026-08-01' AND customer_id = 4217
5 GROUP BY region;
6
7 OPTIMIZE de_training.silver.events;                -- compaction only
8 OPTIMIZE de_training.silver.events
9   ZORDER BY (customer_id, event_date);              -- + clustering
10
11 DESCRIBE DETAIL de_training.silver.events;         -- numFiles,
12   sizeInBytes
13 DESCRIBE HISTORY de_training.silver.events;        -- OPTIMIZE
14   commits

```

Listing 5.1: Compaction and Z-ORDER, with evidence collection.

#### Tip

Choose Z-ORDER columns from *actual query patterns*: high-cardinality columns used in selective equality or range filters. Z-ORDER on low-cardinality columns (like `region` with 5 values) wastes the sort; the table's partition columns rarely need to be Z-ORDER columns too.

## 5.4 Wednesday: Liquid Clustering, Deletion Vectors, VACUUM

### 5.4.1 Liquid Clustering

Liquid clustering (`CLUSTER BY`) makes clustering a table *property* rather than a physical directory layout. Unlike Hive-style partitioning, clustering keys can change (`ALTER TABLE ... CLUSTER BY (...)`) without rewriting the table; unlike Z-ORDER, clustering is maintained incrementally on write and optimize, and it composes with automatic file sizing [41].

```

1 -- New table
2 CREATE TABLE de_training.gold.fact_orders_lc (
3   order_id BIGINT, customer_id BIGINT, order_date DATE,
4   region STRING, net_amount DECIMAL(12,2))
5 CLUSTER BY (order_date, customer_id);
6
7 -- Existing table: opt in without a rewrite
8 ALTER TABLE de_training.gold.fact_orders
9   CLUSTER BY (order_date, customer_id);
10
11 -- Change clustering keys later: metadata operation
12 ALTER TABLE de_training.gold.fact_orders

```

```
13 CLUSTER BY (region, order_date);
```

Listing 5.2: Liquid clustering: creation, change, and the partition migration.

### Definition

**Partitioning** encodes a column into directory paths—fast pruning, but immutable once written and dangerous at high cardinality. **Liquid clustering** records clustering in the log—similar pruning benefits, re-definable keys, no small-partition hazard. The default for new large Delta tables is liquid clustering; partitioning survives for genuinely coarse, stable dimensions like `country` or `business_date` at low cardinality.

## 5.4.2 Deletion Vectors

With deletion vectors enabled, `DELETE/UPDATE/MERGE` mark rows deleted via a bitmap stored alongside the file instead of rewriting it [43]. Reads apply the bitmap on the fly; a later `OPTIMIZE` physically purges. Enable deletion vectors on tables with frequent row-level mutations (CDC-heavy Silvers, GDPR deletes).

## 5.4.3 VACUUM and Predictive Optimization

`VACUUM table RETAIN 168 HOURS` removes unreferenced files past the horizon (Chapter 2 covered the recovery implications). With Unity Catalog managed tables, **predictive optimization** runs `OPTIMIZE`, `ANALYZE`, and `VACUUM` automatically where Databricks' telemetry shows benefit—on serverless, at no separate job cost. Enable it and spend your scheduling effort on the exceptions.

## 5.5 Thursday Hands-on Project: The Small-File Benchmark

Build a 50M-row events table degraded by tiny files, then measure the effect of compaction, `Z-ORDER`, and liquid clustering on the same selective query.

```
1 from pyspark.sql import functions as F
2 import time
3
4 # 1. Generate 50M rows as ~2000 tiny files (the streaming-aftermath simulation)
5 base = (spark.range(0, 50_000_000)
6         .withColumn("event_date", F.date_add(F.lit("2026-01-01"),
7         (F.rand(7) * 240).cast("int"))))
8         .withColumn("customer_id", (F.rand(7) * 500_000).cast("long"))
9         .withColumn("amount", F.rand(7) * 100))
10 base.repartition(2000).write.mode("overwrite") \
11     .saveAsTable("de_training.silver.events_raw")
12
13 def bench(table, where, label):
14     spark.sql(f"SELECT * FROM {table} LIMIT 1").count() # warm metadata
15     t0 = time.time()
16     spark.sql(f"SELECT COUNT(*) c, SUM(amount) s FROM {table} WHERE {where}") \
17         .collect()
18     return (label, round(time.time() - t0, 2))
19
20 sel = "event_date = '2026-05-01' AND customer_id = 4217"
21 results = [bench("de_training.silver.events_raw", sel, "baseline_small_files")]
22
23 # 2. Compaction + Z-ORDER copy
24 spark.sql("""CREATE TABLE de_training.silver.events_zo
25             AS SELECT * FROM de_training.silver.events_raw""")
26 spark.sql("""OPTIMIZE de_training.silver.events_zo
```

```

27         ZORDER BY (customer_id, event_date)"""
28 results.append(bench("de_training.silver.events_zo", sel, "optimize_zorder"))
29
30 # 3. Liquid-clustered copy
31 spark.sql("""CREATE TABLE de_training.silver.events_lc
32             CLUSTER BY (event_date, customer_id)
33             AS SELECT * FROM de_training.silver.events_raw""")
34 spark.sql("OPTIMIZE de_training.silver.events_lc")
35 results.append(bench("de_training.silver.events_lc", sel, "liquid_clustered"))
36
37 spark.createDataFrame(results, ["variant", "seconds"]) \
38     .write.mode("overwrite").saveAsTable("de_training.silver.week05_results")

```

Listing 5.3: Week 5 lab: degrade, then optimize, with controlled measurement.

## Deliverables

- The three variants (raw, Z-ORDER, liquid) with DESCRIBE DETAIL file counts.
- The `week05_results` timing table with cold-run and warm-run columns.
- A one-paragraph interpretation: which layout won, and why the statistics made it so.

## Acceptance Criteria

- Identical result values across all three variants (layout changes must not change data).
- File counts documented before and after compaction.
- Benchmarks repeated three times with the median reported (first-run cold effects excluded).
- VACUUM not run below 168-hour retention at any point.

## Stretch Goals

- Add a bloom filter on `customer_id` and measure the point-query delta.
- Enable deletion vectors, delete 1% of rows, and compare delete cost with/without.
- Repeat the benchmark on a partitioned variant and explain where partitioning beat clustering.

## 5.6 Friday Use Case: Remediating Two Million Tiny Files

A nightly streaming sink has appended micro-batches to `silver.events` for six months. Analysts report multi-minute scans; the metadata listing alone takes minutes; OPTIMIZE times out. The data is correct—the layout has collapsed.

### 5.6.1 Symptoms and Diagnosis

Evidence collected: DESCRIBE DETAIL shows 2.1M files averaging 0.8 MB; stage summaries show thousands of one-file tasks; the driver spends most of each query planning against an enormous file list. The streaming trigger fires every 30 seconds; no compaction has ever run.

5.6.2 Remediation Strategy

Two phases: **repair**, then **prevent**.

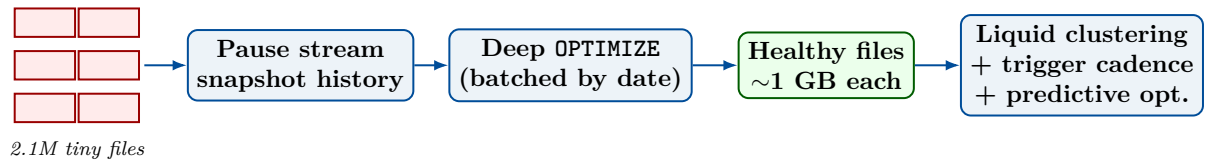


Figure 5.1: Small-file remediation: pause, snapshot, batch compaction, then prevent recurrence with clustering and write-cadence changes.

**Repair.** Pause or throttle the stream (triggered mode with a longer interval), snapshot the history, and run compaction in date-banded batches so each OPTIMIZE operates on a tractable file set. Verify counts against the pre-repair baseline; only then resume.

**Prevent.** Convert the table to liquid clustering on the dominant filter columns; move the stream from 30-second triggers to 5-minute micro-batches (or `availableNow` batches); enable predictive optimization; add a weekly file-count guardrail alert.

**Common Pitfall**

**VACUUM in the middle of repair.** During remediation you are one mistake away from needing history. Vacuum only after the repaired table has been validated and the stream has run cleanly against it for a full retention window.

5.6.3 Risks and Mitigations

| Risk                                       | Mitigation                                                     |
|--------------------------------------------|----------------------------------------------------------------|
| Compaction job becomes the new bottleneck  | Date-banded runs; predictive optimization absorbs routine work |
| Layout change regresses unmeasured queries | Benchmark harness covers the top five query patterns           |
| Small files return via a new producer      | File-count guardrail alert on every hot table                  |
| VACUUM mid-incident destroys evidence      | Freeze VACUUM during open incidents; runbook says so           |
| Clustering keys chosen by intuition        | Keys derived from query-history analysis, reviewed quarterly   |

Table 5.1: Week 5 scenario risks and mitigations.

5.7 Design Decision Guide

Layout decisions age with the table; choose for the query patterns you can verify, not the ones you imagine.



| Decision                                       | Choose the first when                        | Choose the second when                        |
|------------------------------------------------|----------------------------------------------|-----------------------------------------------|
| Liquid clustering vs. partition+Z-ORDER        | New large tables; evolving filter columns    | Legacy tables you cannot migrate yet          |
| Predictive optimization vs. scheduled OPTIMIZE | UC managed tables on serverless              | External tables or custom cadence needs       |
| Deletion vectors on vs. off                    | Row-level mutation is frequent               | Append-mostly tables with scan-sensitive SLAs |
| Bloom filter vs. statistics only               | High-cardinality point lookups               | Range-dominated queries                       |
| Daily vs. post-trigger compaction              | Batch-loaded tables                          | Streaming sinks with micro-batches            |
| Partition vs. cluster on date                  | Coarse day-grain filters and lifecycle drops | Selective queries on mixed columns            |

Table 5.2: Week 5 decision guide.

Benchmark before believing: the same clustering choice can win on one table and waste money on another. Chapter 5’s lab harness is the template for every layout decision you will ever make.

## 5.8 Operational Guidance for Week 5

### 5.8.1 Performance and Reliability

Optimization is a cadence, not an event: set `OPTIMIZE` frequency from write volume (daily for batch-loaded tables, after each trigger window for streaming sinks) and review file counts monthly against a growth guardrail. Prefer predictive optimization on Unity Catalog managed tables so maintenance follows measured benefit; keep scheduled jobs for external tables and exceptions. Track the benchmark trio—file count, selective-query latency, scan bytes—before and after every layout change.

### 5.8.2 Security and Governance Checklist

- `ALTER TABLE ... CLUSTER BY` and `OPTIMIZE` rights restricted to the platform group.
- Layout changes (clustering keys, partition retirement) documented in the change log with benchmark evidence.
- Deletion vectors enabled on tables with GDPR/row-delete obligations—and the purge cadence documented.
- `VACUUM` schedules reviewed against legal hold and recovery requirements quarterly.

### 5.8.3 Troubleshooting Playbook

| Symptom                         | Likely cause                                         | First action                                         |
|---------------------------------|------------------------------------------------------|------------------------------------------------------|
| OPTIMIZE conflicts with writers | Compaction racing the stream                         | Stagger schedules or enable predictive optimization  |
| Z-ORDER shows no gain           | Low-cardinality or rarely-filtered columns chosen    | Re-derive columns from actual query predicates       |
| File count not dropping         | Retention keeping old files / optimize not scheduled | Check history for OPTIMIZE commits; review VACUUM    |
| Reads slower after enabling DVs | Deletion bitmap merge overhead                       | Run OPTIMIZE to physically purge; review delete rate |

Table 5.3: Week 5 troubleshooting quick reference.

## 5.9 Interview Q&A

### 5.9.1 Concepts and Trade-offs

1. **Q: Why do small files hurt even when total bytes are the same?**

A: Per-file costs—listing, footer reads, task scheduling, open/close—scale with file count, not bytes. At millions of files, metadata work dominates and the driver becomes the bottleneck.

2. **Q: Z-ORDER versus liquid clustering?**

A: Both co-locate similar rows for data skipping. Liquid clustering keeps keys as metadata (changeable without rewrite, maintained incrementally, no partition-column commitment); Z-ORDER applies at compaction time on a fixed layout. New tables: liquid; legacy: Z-ORDER until migrated.

3. **Q: When does classic partitioning still win?**

A: Coarse, stable, low-cardinality dimensions that almost every query filters (e.g., `country`, business date at day grain), where directory pruning is exact and lifecycle operations drop whole directories.

4. **Q: What do deletion vectors change about a DELETE?**

A: The delete writes a bitmap instead of rewriting files—seconds instead of minutes—but every reader pays a small merge cost until the next OPTIMIZE purges physically.

5. **Q: Your OPTIMIZE job itself is slow. Why?**

A: Usually because it is fighting the same small-file mountain (schedule it more often, in bands) or competing with active writers (coordinate schedules or use predictive optimization).

6. **Q: Does OPTIMIZE change data or only layout?**

A: Layout only—it is a commit like any other, visible in history, and your reconciliation counts must be identical before and after. A row-count change after OPTIMIZE is a bug report, not a curiosity.

7. **Q: How would you convince a skeptic that liquid clustering helped?**

A: The lab’s evidence table: same query, same data, controlled medians across cold and warm runs, plus the file-level statistics story. Feelings are not benchmarks.

8. **Q: Does liquid clustering remove the need to think about layout?**

A: It removes the *irreversible* decision, not the decision: clustering keys still determine skipping quality, and they should still come from measured query patterns.

## 5.10 Further Reading

The Delta optimization guide covers compaction, Z-ORDER, bloom filters, and file sizing [36]; the liquid clustering documentation explains the layout model and migration [41]; deletion vectors are covered in [43]. For the storage fundamentals underneath, revisit the Delta paper [8] and the Parquet format docs [3]. Spark-level tuning complements this chapter [5].

## Key Takeaways

- Query speed on a lakehouse is a bytes-scanned problem; statistics-based skipping is the mechanism.
- The small-file problem is a write-cadence disease; compaction is the treatment, clustering the prevention.
- Liquid clustering is the default layout decision for new large tables; partitioning survives for coarse stable dimensions.
- Deletion vectors make row-level mutation cheap at the cost of reader-side merge until compaction.
- Benchmark layout changes with controlled medians, and never vacuum below your recovery horizon.

## 5.11 Exercises

1. **(Conceptual)** Explain data skipping to a PM in three sentences without using the word “statistics.”
2. **(Conceptual)** Why is Z-ORDER on a 5-value column wasted work?
3. **(Hands-on)** Complete the Thursday lab; capture the results table and one DESCRIBE DETAIL per variant.
4. **(Hands-on)** Write the date-banded compaction script for the Friday scenario, with per-band logging.
5. **(Hands-on)** Enable predictive optimization on one catalog and find the evidence of its runs in system tables.
6. **(Design)** Choose partition/cluster/Z-ORDER strategies for: (a) clickstream by day, (b) IoT by device, (c) orders by country and date. Justify each.
7. **(Operations)** Write the alert rule and runbook for “file count grows 20% week-over-week.”
8. **(Architecture)** When would you disable deletion vectors on a table? Give a workload where the reader-side merge cost matters.

# Databricks SQL and SQL Warehouses

**Learning Objectives**

- Explain SQL warehouse architecture: serverless, pro, and classic types, and how Photon accelerates queries.
- Write warehouse-grade SQL against Unity Catalog tables, including time travel and window functions.
- Choose between materialized views, streaming tables, and scheduled Gold tables for derived data.
- Size warehouses with min/max clusters, auto-stop, and channel selection; read the query profile.
- Attribute cost with `system.billing.usage` and warehouse monitoring.
- Design a warehouse topology that isolates BI bursts from engineering backfills.

**Cross-Cloud Equivalence**

Databricks SQL is the platform’s **warehouse-serving layer**; each cloud fields a serverless SQL engine with the same promises and different mechanics.

| Capability      | Databricks                             | AWS                             | Azure / GCP / Fabric                                |
|-----------------|----------------------------------------|---------------------------------|-----------------------------------------------------|
| SQL compute     | SQL warehouse (serverless/pro/classic) | Redshift Serverless / Athena    | Synapse serverless / BigQuery / Fabric Warehouse    |
| Engine          | Photon (vectorized)                    | Redshift engine / Trino         | Synapse / Dremel / Fabric engine                    |
| Autoscale       | Multi-cluster load balancing           | Concurrency scaling / per-query | Autoscale / slots / capacity smoothing              |
| Idle behavior   | Auto-stop, serverless scale-to-zero    | Pause / per-query billing       | Auto-pause / per-slot / capacity                    |
| Derived tables  | Materialized views, streaming tables   | Redshift MVs / Athena CTAS      | Synapse MVs / BigQuery MVs                          |
| Cost visibility | <code>system.billing.usage</code>      | CUR + CloudWatch                | Cost Management / billing export / Capacity Metrics |

**Snowflake** is the closest conceptual peer—virtual warehouses with per-second billing and result caching—but Databricks SQL queries the *same* Delta tables your pipelines write, with no copy into a proprietary store.

## 6.1 Week 6 Roadmap and Mental Model

The first five chapters built and optimized tables; Week 6 serves them. Databricks SQL (DB-SQL) is the warehouse surface: SQL editors, SQL warehouses, query federation, and the serving layer beneath AI/BI (Chapter 7). The engineering questions are operational: which warehouse type, what size, how does it scale, what does it cost, and how do you know a query is slow for a fixable reason [23, 40].

The mental model: **a SQL warehouse is a load-balanced fleet of query clusters over your Delta tables**. You submit SQL; the warehouse compiles it (Photon), pulls data from cloud storage through the cache, scales clusters between its min/max bounds under concurrency, and records everything in system tables. There is no data movement into the warehouse—the storage is the lake you already govern.

### Definition

**A SQL warehouse** is SQL-optimized compute for Databricks SQL: a managed set of clusters with a size (2X-Small to 4X-Large), a min/max cluster range, an auto-stop timer, and a type (serverless, pro, classic) controlling startup speed, features, and price.

### Key Terms

**Photon**: the vectorized execution engine. **Serverless warehouse**: instant-start, fast-autoscaling compute in Databricks’ account. **Query profile**: the per-operator execution breakdown with rows, bytes, and time. **Result cache**: reuse of a query’s results when query and data are unchanged. **Materialized view**: a managed, incrementally refreshed precomputed result.

## 6.2 Monday: Warehouse Types and Photon

### 6.2.1 Serverless, Pro, Classic

**Serverless** warehouses start in seconds, scale aggressively, run in Databricks’ cloud account, and support the newest features (including predictive optimization integration and AI functions). **Pro** adds classic-cloud control with fast start. **Classic** trades startup latency for the most configuration control and lowest DBU rate. Default to serverless for BI; drop to pro/classic when networking or custom initialization requires the data plane.

### 6.2.2 Photon Under the Hood

Photon re-implements Spark’s execution hot path in vectorized native code: columnar batches, SIMD instructions, cache-aware operators [49]. The practical consequences: scans, joins, and aggregations over Delta are dramatically faster than vanilla Spark SQL, and the speedup grows with data scanned—which is why Chapter 5’s layout work multiplies this chapter’s engine.

### Note

You do not “enable Photon” on a SQL warehouse—it is the engine. On classic compute you choose a Photon-enabled runtime; on serverless it is always on.

## 6.3 Tuesday: Warehouse SQL and the Query Profile

### 6.3.1 Analytical SQL on the Lakehouse

Databricks SQL speaks Spark SQL against the three-level namespace, with time travel, window functions, and QUALIFY:

```

1 -- Top products per region this quarter
2 SELECT region, product_id, SUM(net_amount) AS revenue
3 FROM de_training.gold.fact_orders
4 WHERE order_date >= DATE '2026-07-01'
5 GROUP BY region, product_id
6 QUALIFY ROW_NUMBER() OVER (PARTITION BY region ORDER BY revenue DESC) <= 10;
7
8 -- Point-in-time question against history (Chapter 2 pays off here)
9 SELECT COUNT(*) FROM de_training.gold.fact_orders
10 TIMESTAMP AS OF '2026-08-01T00:00:00Z';

```

Listing 6.1: Warehouse-grade analytical SQL patterns.

### 6.3.2 Reading the Query Profile

Every query has a profile (Query History → query → profile): a graph of operators with rows, bytes, peak memory, and duration. The diagnostic skill from Chapter 4 transfers directly: look for the operator holding the time—a scan reading more bytes than expected (layout problem, Chapter 5), a spill (memory), or an exchange skewing (join strategy).

## 6.4 Wednesday: Materialized Views, Caching, and Cost Attribution

### 6.4.1 Materialized Views versus Scheduled Gold

A DBSQL **materialized view** is a managed precomputed result with incremental refresh where possible; a **streaming table** maintains a continuously appended aggregation [23]. The decision rule: use a materialized view when consumers query a stable derived shape and freshness tolerance is minutes-to-hours; keep a scheduled Gold table when the derivation is complex business logic that belongs in your pipeline repo with tests (Chapter 3’s discipline).

```

1 CREATE MATERIALIZED VIEW de_training.gold.mv_daily_revenue
2 AS SELECT order_date, region, SUM(net_amount) AS revenue, COUNT(*) AS orders
3 FROM de_training.gold.fact_orders
4 GROUP BY order_date, region;

```

Listing 6.2: A materialized view over the Gold star schema.

### 6.4.2 Scaling and Cost

Concurrency scales by clusters: a 2X-Small warehouse with max 10 clusters absorbs a 9 a.m. dashboard burst by adding clusters, then scales back. Auto-stop kills idle spend. Attribution comes from system tables:

```

1 SELECT w.warehouse_name,
2        ROUND(SUM(u.usage_quantity), 2) AS dbus,
3        COUNT(DISTINCT u.usage_date) AS active_days
4 FROM system.billing.usage u
5 JOIN system.warehouses w USING (warehouse_id)
6 WHERE u.usage_date >= current_date() - 30
7 AND u.sku_name LIKE '%SQL%';

```

```

8 GROUP BY w.warehouse_name
9 ORDER BY dbus DESC;

```

Listing 6.3: Warehouse cost attribution from system tables.

### Common Pitfall

**One giant warehouse for everything.** A single maxed-out warehouse mixes the CEO’s dashboard with a backfill scan and makes both unpredictable. Split by workload class (executive BI, ad-hoc, engineering), set per-warehouse max clusters, and let each have its own auto-stop.

## 6.5 Thursday Hands-on Project: Serve the Gold Star Schema

Put the Month 1 Gold schema behind a serverless SQL warehouse, add a materialized view, capture query profiles, and attribute cost.

```

1 -- 1. Create warehouse (UI or API): type=serverless, size=2X-Small,
2 --   min=1, max=4, auto-stop=10 min, channel=Current.
3
4 -- 2. Baseline the daily-revenue query directly on the fact table
5 SELECT order_date, region, SUM(net_amount) AS revenue
6 FROM de_training.gold.fact_orders
7 GROUP BY order_date, region;           -- capture the profile
8
9 -- 3. Materialized view, then re-run the same question against it
10 CREATE MATERIALIZED VIEW de_training.gold.mv_daily_revenue AS
11 SELECT order_date, region, SUM(net_amount) AS revenue, COUNT(*) AS orders
12 FROM de_training.gold.fact_orders
13 GROUP BY order_date, region;
14
15 SELECT * FROM de_training.gold.mv_daily_revenue
16 WHERE order_date >= '2026-08-01';     -- capture the faster profile
17
18 -- 4. Cost: what did today's work cost?
19 SELECT sku_name, ROUND(SUM(usage_quantity),2) AS dbus
20 FROM system.billing.usage
21 WHERE usage_date = current_date()
22 GROUP BY sku_name;

```

Listing 6.4: Week 6 lab: warehouse setup, MV, and measurement.

### Deliverables

- A running serverless warehouse with documented size, scaling bounds, and auto-stop.
- Two captured query profiles (fact scan vs. MV) with an explanation of the difference.
- The day’s DBU attribution query output.

### Acceptance Criteria

- The MV returns results identical to the direct aggregate (reconciliation).
- The warehouse auto-stops within its timer; evidence from the warehouse events tab.
- Scaling bounds are justified in one paragraph (expected concurrency, SLA).
- The query-profile analysis names the dominant operator in the slow plan.

### Stretch Goals

- Create a second warehouse for “engineering” and prove workload isolation during a simulated burst.
- Compare classic vs. serverless startup latency with timestamps.
- Build the weekly per-warehouse cost report as a saved query with a schedule.

## 6.6 Friday Use Case: 500 Analysts, One 9 A.M. Burst, Zero Contention

A company’s 500 analysts open dashboards at 9 a.m.; simultaneously, data engineers run month-end backfills against the same Gold tables. Complaints: dashboards queue, backfills time out, and last month someone “fixed” it by doubling warehouse size—and the bill.

### 6.6.1 Requirements and Assumptions

Dashboards must load in under 10 seconds at p95; backfills may take hours but must finish by month-end close. Gold tables are liquid-clustered (Chapter 5). Budget grows slower than headcount.

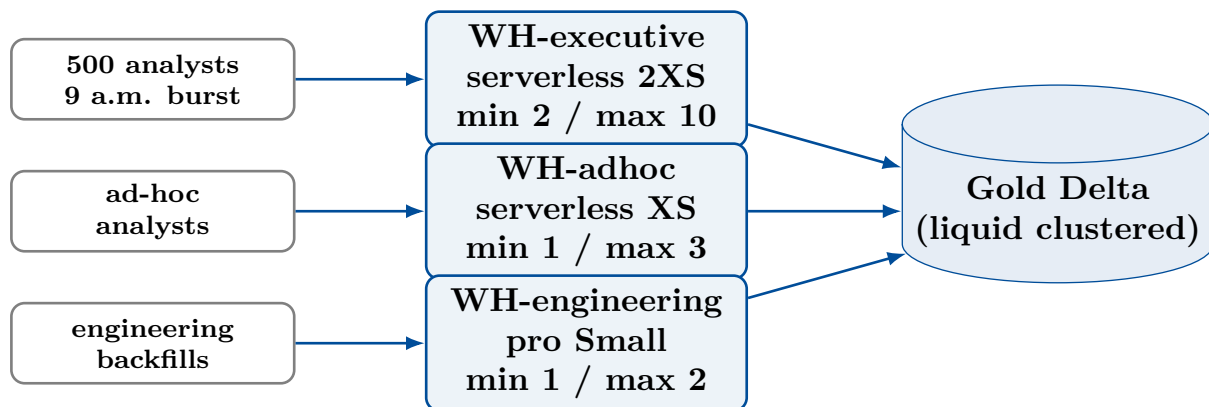


Figure 6.1: Workload-class warehouse topology over one governed Gold layer: isolation without data copies.

### 6.6.2 Architecture Decisions

**Three warehouses, three classes.** Executive BI (serverless, aggressive scaling, tight auto-stop), ad-hoc (small, cheap, forgiving), engineering (pro, sized for scans, auto-stop generous). The shared Delta storage means isolation costs no copies; the burst is absorbed by max-clusters scaling, not by an oversized idle floor.

**Query hygiene.** Dashboards query the materialized view, not the raw fact; the backfill writes at off-peak with its own warehouse; a saved monitoring query (profiles + billing) reviews p95 weekly. The doubling-the-size incident becomes a scaling-bounds conversation backed by queue-time metrics.



**Common Pitfall**

**Reading the queue as a sizing problem.** Queued queries at 9 a.m. with idle clusters mean the *max clusters* bound is the constraint, not size; a bigger single cluster would not have helped and would have billed more. Queue time, running time, and cluster count are three different metrics—read all three before changing anything.

**6.6.3 Risks and Mitigations**

| Risk                                          | Mitigation                                                   |
|-----------------------------------------------|--------------------------------------------------------------|
| Burst budget spikes with headcount            | Max-clusters caps per warehouse; monthly chargeback review   |
| A heavy query starves the executive warehouse | Workload classes isolated; per-user query limits             |
| Stale MV presented as fresh                   | Staleness alert; refresh timestamp rendered on the dashboard |
| Warehouse sprawl across teams                 | Inventory with owners; unused warehouses auto-flagged        |
| Classic warehouses forgotten running          | Auto-stop enforced by policy; weekly idle audit              |

Table 6.1: Week 6 scenario risks and mitigations.

**6.7 Design Decision Guide**

Warehouse decisions are workload-shape decisions; name the workload class first.

| Decision                               | Choose the first when                               | Choose the second when                       |
|----------------------------------------|-----------------------------------------------------|----------------------------------------------|
| Serverless vs. classic warehouse       | BI and bursty SQL; zero-ops valued                  | Custom networking or lowest-rate steady load |
| Materialized view vs. scheduled Gold   | Stable derived shape, tolerance for managed refresh | Complex tested business logic                |
| One warehouse vs. per-class warehouses | Tiny teams, light concurrency                       | Mixed latency classes or chargeback needs    |
| Size up vs. scale out                  | Individual queries are slow                         | Concurrency (queueing) is the problem        |
| Result cache reliance vs. MV           | Ad-hoc repeated questions                           | Shared, governed derived numbers             |
| Photon confidence vs. measurement      | Always measure                                      | —                                            |

Table 6.2: Week 6 decision guide.

The recurring trap is treating warehouse size as the only knob. Bounds, classes, caching, and derived tables usually matter more—and all four are cheaper than a bigger T-shirt size.

**6.8 Operational Guidance for Week 6****6.8.1 Performance and Reliability**

Review warehouse telemetry weekly: queue time (concurrency bound), median/p95 run time (query health), and cluster count over the day (scaling shape). Auto-stop settings are audited

against reality—a warehouse “stopped at night” that shows 2 a.m. activity is running something nobody scheduled. Materialized-view refresh status is monitored like any pipeline; a silently stale MV is a certified number quietly rotting.

### 6.8.2 Security and Governance Checklist

- Dashboards and scheduled queries publish under a least-privilege service identity.
- Warehouse access granted per workload class; engineers do not run backfills on the executive warehouse.
- Every warehouse has a documented owner, size rationale, and auto-stop value.
- Monthly cost attribution per warehouse is reviewed and tagged to a domain.

### 6.8.3 Troubleshooting Playbook

| Symptom                   | Likely cause                            | First action                                               |
|---------------------------|-----------------------------------------|------------------------------------------------------------|
| Queries queue at peak     | Max-clusters bound reached              | Read queue vs. run time; raise max for the burst window    |
| First dashboard load slow | Cold warehouse (auto-stop)              | Serverless type or a scheduled early-morning warm query    |
| MV returns stale rows     | Refresh not scheduled/failed            | Check refresh history; alert on staleness                  |
| Same query fast then slow | Result cache invalidated by new commits | Expected; cache hits only when data and text are unchanged |

Table 6.3: Week 6 troubleshooting quick reference.

## 6.9 Interview Q&A

### 6.9.1 Concepts and Trade-offs

1. **Q: Why is Databricks SQL different from connecting a BI tool to Spark?**  
A: Photon-vectorized execution, warehouse-level caching and scaling, governance integration, and per-query profiles—an analytical serving layer, not a general cluster with a SQL cell.
2. **Q: Serverless warehouse: what do you give up?**  
A: Data-plane networking options and some custom init control; you gain instant start, fast autoscaling, and no cluster management. For BI, the trade is almost always right.
3. **Q: Materialized view or a scheduled Gold table?**  
A: MV for stable derived shapes with freshness tolerance and simple logic; scheduled Gold when the derivation is versioned business logic that needs tests, review, and pipeline discipline.
4. **Q: How do you find who spent what on SQL last month?**  
A: `system.billing.usage` joined to `system.warehouses`, broken out by SKU and day; add `system.access.audit` for per-user query attribution.
5. **Q: A dashboard is slow only at 9 a.m. Diagnose.**  
A: Queue time at the concurrency bound: raise max clusters for the burst window, move the dashboard to the MV, and check that the warehouse is serverless so scale-up is seconds, not minutes.

6. **Q: When is a classic warehouse still the right answer?**

A: Custom networking/init requirements, the strictest per-DBU price on steady loads, or features gated to the classic tier—decided from requirements, not habit.

7. **Q: What does the query profile tell you that the UI duration does not?**

A: *Where* the time went: which operator scanned how many bytes and spilled what. Duration says slow; the profile says why.

8. **Q: How do you prove a warehouse right-sizing saved money?**

A: Same workload, measured weeks, `system.billing.usage` deltas per SKU—the cost chapter of the tuning report, with queue time held constant so the saving is not just slower service.

## 6.10 Further Reading

The Databricks SQL [23] and SQL warehouse [40] documentation cover types, sizing, and monitoring; Photon is explained in [49]. System-table billing and warehouse monitoring are documented in [34]. For the engine fundamentals, revisit Spark SQL tuning [5].

## Key Takeaways

- A SQL warehouse is a scalable query fleet over governed Delta tables—no copies, no separate store.
- Serverless is the BI default; pro/classic serve networking and control edge cases.
- Materialized views answer repeated derived questions; complex business logic still belongs in tested pipelines.
- Queue time, run time, and cluster count are different metrics—size nothing before reading all three.
- Cost attribution is a system-table query away; make it a weekly habit.

## 6.11 Exercises

1. **(Conceptual)** Contrast a SQL warehouse with an all-purpose cluster for a BI workload: three differences.
2. **(Conceptual)** Why does the result cache make some dashboard refreshes free, and when does it miss?
3. **(Hands-on)** Complete the Thursday lab, including both query profiles.
4. **(Hands-on)** Build the three-warehouse topology from the Friday case and measure a simulated burst (20 concurrent queries) against each.
5. **(Hands-on)** Write and schedule the weekly warehouse cost report; include a week-over-week delta column.
6. **(Design)** Choose warehouse type/size/bounds for: executive dashboards, a data-science team, and a dbt-style transformation workload. Justify each.
7. **(Operations)** Write the alert rule for “warehouse queue time > 60 s for 10 minutes.”
8. **(Architecture)** When would you put a materialized view in a DLT pipeline instead of DBSQL (preview of Chapter 10)?

# AI/BI Dashboards and Genie

## Learning Objectives

- Build AI/BI dashboards with curated datasets, parameters, and cross-filters over governed Gold data.
- Configure a Genie space with instructions, example queries, and trusted assets that yield correct SQL.
- Curate a semantic layer: certified views, named measures, and documented dimensions.
- Publish and schedule dashboards with governed credentials.
- Diagnose and fix wrong Genie answers with instructions rather than data hacks.
- Design a self-service analytics layer where certified and exploratory analytics coexist safely.

## Cross-Cloud Equivalence

AI/BI is Databricks' **native BI layer**; Genie is its natural-language front end. Every platform now pairs BI with an LLM assistant.

| Capability         | Databricks                            | AWS                      | Azure / GCP / Fabric                   |
|--------------------|---------------------------------------|--------------------------|----------------------------------------|
| BI surface         | AI/BI dashboards                      | QuickSight               | Power BI / Looker / Power BI (Fabric)  |
| NL analytics       | Genie spaces                          | Amazon Q in QuickSight   | Copilot (Power BI) / Gemini (Looker)   |
| Semantic steering  | Genie instructions + certified tables | Q topics                 | Semantic model + Copilot prep / LookML |
| Compute behind BI  | SQL warehouse                         | SPICE / direct query     | Import / DirectQuery / Direct Lake     |
| Scheduling/sharing | Dashboard subscriptions               | Scheduled emails         | Subscriptions / Looker schedules       |
| Governance link    | Unity Catalog lineage to tiles        | Q/QuickSight permissions | Purview / Looker permissions           |

The differentiator to articulate in interviews: AI/BI reads the governed Delta tables in place and inherits Unity Catalog lineage, so a dashboard tile traces back to the pipeline that produced it.

## 7.1 Week 7 Roadmap and Mental Model

Weeks 1–6 built governed, performant Gold data. Week 7 makes it *consumable*: dashboards for the known questions and Genie for the unknown ones. The engineering content is not chart placement—it is the semantic discipline that makes self-service safe: certified datasets, named measures, and instructions that encode business definitions so that “revenue” means one thing everywhere [11, 44].

The mental model: **a dashboard is a cached contract; Genie is a compiler**. Dashboards execute your SQL on a schedule and render the results—trust comes from the SQL being right and the source being certified. Genie compiles natural language into SQL at question time—trust comes from the space being scoped to clean Gold tables with instructions that teach it your vocabulary. Both fail the same way: confidently, when pointed at ungoverned data.

### Definition

**An AI/BI dashboard** is a Databricks-native BI artifact: a set of datasets (SQL against governed tables) feeding visualizations with parameters and cross-filters, published on a schedule. **A Genie space** is a curated natural-language analytics surface: selected tables plus instructions, example queries, and trusted assets that steer generated SQL.

### Key Terms

**Certified table/view**: a Gold asset marked as the governed source for a metric. **Instruction**: Genie steering text—definitions, join rules, filters. **Trusted asset**: a pre-verified query Genie can use as an authoritative answer. **Parameter**: a dashboard input bound across datasets. **Cross-filter**: a selection in one visualization filtering the others.

## 7.2 Monday: AI/BI Dashboards

### 7.2.1 Datasets and Visualizations

A dashboard’s **datasets** are SQL queries (or table references) executed on a SQL warehouse; **visualizations** render them; **parameters** thread user input through every dataset. Author datasets against Gold views—never Bronze—and prefer views that already encode the business grain so the dataset SQL stays thin and reviewable.

```

1 SELECT order_date, region, category,
2        SUM(net_amount) AS revenue,
3        COUNT(DISTINCT order_id) AS orders
4 FROM de_training.gold.v_sales_certified -- certified, governed
5 WHERE order_date BETWEEN :start_date AND :end_date
6      AND region IN (:regions)
7 GROUP BY order_date, region, category;
```

Listing 7.1: A parameterized dashboard dataset over a certified view.

### 7.2.2 Parameters, Cross-Filters, and Publishing

Parameters make one dashboard serve many slices (dates, regions, channels). Publishing snapshots the dashboard on a schedule with the publisher’s warehouse and credentials; subscribers receive rendered output. The governance consequence: *whoever publishes shares their access*—publish with a service identity whose Gold read rights are exactly what subscribers should see (Chapter 8 makes this enforceable).

## 7.3 Tuesday: Genie Spaces

### 7.3.1 What Genie Needs to Succeed

Genie answers questions by generating SQL over the tables you give it, steered by natural-language context [44]. The inputs, in order of leverage: (1) few, clean, well-named Gold tables with documented columns; (2) **instructions** defining business terms (“revenue excludes cancelled orders”), join paths, and default filters; (3) **example queries** pairing questions with verified SQL; (4) **trusted assets** for the exact certified answers.

#### Common Pitfall

**Genie over Bronze.** Pointed at raw tables, Genie produces fluent, plausible, wrong SQL: unparsed dates, pre-dedup counts, business terms guessed from column names. Genie amplifies your data quality—it does not create it. Scope spaces to certified Gold only.

### 7.3.2 Debugging a Wrong Answer

The Genie workflow when an answer is wrong: inspect the generated SQL, identify the semantic gap (wrong join? wrong filter? wrong measure?), then fix it *in the space*—an instruction, an example query, or a trusted asset—never by editing data to match the answer. Each fix generalizes: a good instruction repairs a whole class of future questions.

## 7.4 Wednesday: The Semantic Layer

### 7.4.1 Certified Views and Named Measures

Between Gold tables and consumers sits a thin, deliberate semantic layer: views that rename columns to business language, encode the certified metric definitions, and hide operational columns. The goal is that every tool—dashboards, Genie, notebooks, external BI—computes “revenue” identically because they all read the same view.

```

1 CREATE OR REPLACE VIEW de_training.gold.v_sales_certified AS
2 SELECT   o.order_date,
3          c.region,
4          p.category,
5          o.net_amount,           -- certified: net of cancels/
          refunds
6          o.order_id
7 FROM     de_training.gold.fact_orders o
8 JOIN     de_training.gold.dim_customer c USING (customer_id)
9 JOIN     de_training.gold.dim_product p USING (product_id)
10 WHERE    o.order_status NOT IN ('cancelled', 'refunded');
```

Listing 7.2: A certified semantic view with named measures.

#### Tip

Certification is a process, not a flag: a view becomes certified when finance reconciles it to the source system, an owner is named, and changes go through review. Then mark it—and grant broadly.

## 7.5 Thursday Hands-on Project: Executive Dashboard and Genie Space

Build a three-page executive dashboard on the certified Gold view, then a Genie space that correctly answers five business questions—including one it initially gets wrong.

```

1 -- 0. Certified semantic view (from Wednesday)
2 CREATE OR REPLACE VIEW de_training.gold.v_sales_certified AS
3 SELECT o.order_date, c.region, p.category, o.net_amount, o.order_id
4 FROM de_training.gold.fact_orders o
5 JOIN de_training.gold.dim_customer c USING (customer_id)
6 JOIN de_training.gold.dim_product p USING (product_id)
7 WHERE o.order_status NOT IN ('cancelled', 'refunded');
8
9 -- Dataset A: revenue trend          Dataset B: revenue by region
10 -- Dataset C: top categories        (all parameterized on :start_date/:end_date)
11
12 -- Genie space over de_training.gold (certified tables only) with instructions:
13 -- - "Revenue" = SUM(net_amount) from v_sales_certified, excluding cancels.
14 -- - "Last quarter" means the most recent completed calendar quarter.
15 -- - Join customer to orders via customer_id only.
16 -- Trusted asset: verified SQL for "revenue by state last quarter".

```

Listing 7.3: Week 7 lab assets: certified view plus the dashboard’s core datasets.

### Deliverables

- A published three-page dashboard (trend, geography, category) with date and region parameters.
- A Genie space over Gold with instructions, two example queries, and one trusted asset.
- A short log: five test questions, the one wrong answer, its generated SQL, and the instruction that fixed it.

### Acceptance Criteria

- Every dashboard dataset reads certified Gold views—no Bronze or ad-hoc tables.
- “Revenue by state last quarter” in Genie matches the dashboard’s certified number exactly.
- The wrong-answer fix is an instruction or trusted asset, not a data change, and generalizes (a second phrasing of the same question now works).
- The published dashboard runs on a service identity with Gold-only read access.

### Stretch Goals

- Add a cross-filter from the region map to the trend page.
- Measure Genie’s first-try accuracy on 10 questions before and after instructions.
- Schedule a Monday 7 a.m. subscription of the executive page to a distribution list.

## 7.6 Friday Use Case: Certified Numbers and Curious Users

A retailer’s finance team requires that any number leaving the analytics platform be certified and reconciled; meanwhile 300 business users want to ask Genie ad-hoc questions. Both must coexist without a shadow-Excel economy.

### 7.6.1 Requirements and Assumptions

Certified metrics change quarterly at most. Business questions are 80% recurring themes, 20% novel. Finance audits quarterly and currently distrusts anything not from their own extracts.

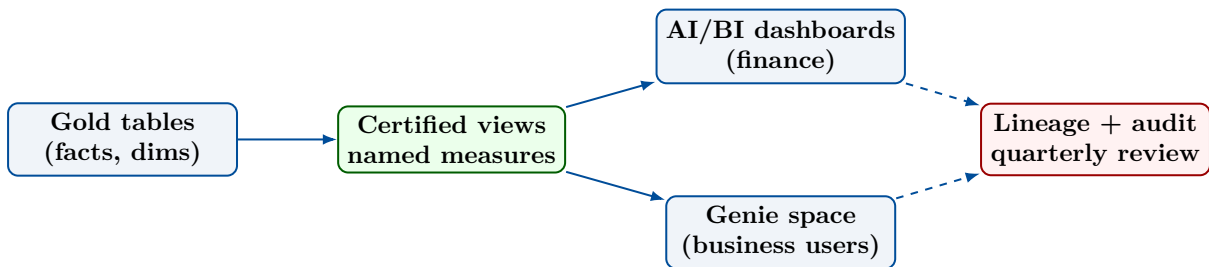


Figure 7.1: One semantic layer feeding both certified dashboards and exploratory Genie, with lineage into the audit review.

### 7.6.2 Architecture Decisions

**One semantic layer, two doors.** Both dashboards and Genie read the same certified views, so a number cannot diverge between the CFO’s dashboard and a manager’s Genie question. Genie’s space is scoped to certified tables; its instructions encode the finance-approved definitions; trusted assets hold the board-level answers verbatim.

**Trust mechanics.** Every certified view carries an owner and a reconciliation record (finance signs quarterly). Dashboard tiles inherit Unity Catalog lineage, so an auditor can click from a number to the view to the pipeline. Ad-hoc exploration happens in a separate “explore” Genie space over non-certified Gold, clearly labeled—curiosity is allowed, export to finance decks is not.

#### Common Pitfall

**Certifying the dashboard instead of the data.** A “certified dashboard” over uncertified tables certifies nothing durable—the next pipeline change silently alters it. Certify the view, let any tool read it, and audit at the view layer.

### 7.6.3 Risks and Mitigations

| Risk                                    | Mitigation                                               |
|-----------------------------------------|----------------------------------------------------------|
| Genie confidently wrong on a metric     | Golden question set re-run on every instruction change   |
| Two “revenue” definitions circulate     | One certified view; all tools read it; lineage proves it |
| Dashboard subscribers see too much      | Publish under a least-privilege service identity         |
| Explore space leaks into finance decks  | Explicit labeling, separate space, export guardrails     |
| Instruction debt accumulates unreviewed | Quarterly Genie space review with accuracy trend         |

Table 7.1: Week 7 scenario risks and mitigations.



## 7.7 Design Decision Guide

Self-service analytics succeeds or fails on semantic decisions, not chart choices.

| Decision                                | Choose the first when               | Choose the second when                            |
|-----------------------------------------|-------------------------------------|---------------------------------------------------|
| Dashboard vs. Genie                     | Known recurring questions           | Exploratory, phrased-in-the-moment questions      |
| Certified view vs. ad-hoc dataset       | Any number that leaves the room     | Analyst's own scratch exploration                 |
| Instruction vs. trusted asset           | A class of questions needs steering | One exact recurring question needs a fixed answer |
| Service identity vs. publisher identity | Anything with subscribers           | Personal draft dashboards only                    |
| AI/BI vs. external BI tool              | Governance colocation wins          | Organizational BI standard mandates it            |
| Refresh cadence: decision vs. maximum   | Always decision cadence             | Never the platform maximum by default             |

Table 7.2: Week 7 decision guide.

The unifying principle: one semantic layer, many doors. Every row above is a way of keeping the certified definition between the data and the decision-maker.

## 7.8 Operational Guidance for Week 7

### 7.8.1 Performance and Reliability

Track dashboard refresh duration and failure rate per dashboard; a tile that silently serves yesterday's data erodes trust faster than an honest error. For Genie, maintain the golden question set and re-run it after every instruction change—instruction edits are code changes with regression risk. Version the certified views (change control with reconciliation sign-off) because every tool downstream inherits their definitions.

### 7.8.2 Security and Governance Checklist

- Genie spaces scoped to certified Gold only; explore spaces clearly labeled.
- Dashboard publication identity has exactly the access subscribers should inherit.
- Subscriptions reviewed quarterly—recipient lists drift as roles change.
- Certified-view changes require reconciliation evidence before merge.

### 7.8.3 Troubleshooting Playbook

| Symptom                         | Likely cause                              | First action                                             |
|---------------------------------|-------------------------------------------|----------------------------------------------------------|
| Genie answers with wrong metric | Missing/ambiguous instruction             | Inspect generated SQL; add instruction or trusted asset  |
| Dashboard serves stale data     | Refresh schedule or warehouse cold        | Check refresh log and warehouse event history            |
| Parameter not filtering         | Dataset SQL missing the parameter binding | Rebind the parameter in the dataset, not the visual      |
| Subscription not received       | Publish identity lost access              | Republish under the service identity; check delivery log |

Table 7.3: Week 7 troubleshooting quick reference.

## 7.9 Interview Q&A

### 7.9.1 Concepts and Trade-offs

1. **Q: Why not just give analysts Power BI against Gold?**

A: You can—the point of AI/BI is colocation with governance: lineage to the tile, no extract pipelines, and one semantic layer shared with Genie. External BI is a valid choice when the organization’s standard demands it; the certified views remain the contract either way.

2. **Q: How do you stop Genie from hallucinating metrics?**

A: Scope to certified tables, encode definitions as instructions, provide example queries and trusted assets, and measure first-try accuracy on a fixed question set before rolling out.

3. **Q: Dashboard performance is poor. Where do you look?**

A: Dataset SQL (is it scanning the fact table instead of an MV or the certified view?), warehouse sizing/queueing (Chapter 6), and table layout (Chapter 5)—in that order.

4. **Q: Who should a published dashboard run as?**

A: A service identity whose data access equals what subscribers may see; publishing as a privileged user silently escalates every subscriber’s effective access.

5. **Q: How do you handle a metric definition change?**

A: Change the certified view under review, re-reconcile with finance, bump the Genie instructions in the same change—one definition, one place, every tool updates together.

6. **Q: What is the first thing you check when a certified number changes unexpectedly?**

A: The view’s lineage and the pipeline history—certified numbers change because upstream changed, and lineage turns “who touched this” into a query.

7. **Q: Can Genie replace a semantic layer?**

A: No—instructions steer generation, they do not enforce definitions. The certified views are the semantics; Genie is one of several doors onto them.

8. **Q: How many Genie spaces should a domain have?**

A: Few: one certified space per domain plus one explore space. Every additional space forks the instructions and example set—and the definitions they encode.

## 7.10 Further Reading

The AI/BI dashboards [11] and Genie [44] documentation are the product references. Revisit SQL warehouses for the compute underneath [40] and Unity Catalog for the certification and lineage mechanics [51, 13].

## Key Takeaways

- Dashboards are cached contracts; Genie is a compiler—both inherit the quality of the data and semantics you give them.
- Certified views with named measures are the single source of metric truth for every tool.
- Fix wrong Genie answers with instructions and trusted assets—never by editing data.
- Published dashboards run as the publisher; publish with a least-privilege service identity.
- Self-service and certification coexist when both doors open onto the same semantic layer.

## 7.11 Exercises

1. **(Conceptual)** Explain to finance why a Genie answer and a dashboard tile can be trusted to agree.
2. **(Conceptual)** List three instruction types that steer Genie and give an example of each.
3. **(Hands-on)** Complete the Thursday lab, including the wrong-answer fix log.
4. **(Hands-on)** Add a second Genie space over uncertified Gold labeled “explore only” and document the guardrail difference.
5. **(Hands-on)** Trace one dashboard tile’s lineage to its source tables in the catalog explorer; capture it.
6. **(Design)** Write the certification checklist (owner, reconciliation, review cadence) for a new certified view.
7. **(Operations)** A certified view must change schema. Write the change plan covering dashboards, Genie, and downstream notebooks.
8. **(Architecture)** When would you move a heavy dashboard dataset into a materialized view (Chapter 6) rather than tuning the SQL? Quantify.

# Unity Catalog Foundations and Data Governance

## Learning Objectives

- Explain the Unity Catalog object hierarchy: metastore, catalog, schema, table, volume, function, model.
- Distinguish managed from external tables and volumes from external locations.
- Implement least-privilege access with groups, ownership, and **GRANT/REVOKE**.
- Enforce row- and column-level security with dynamic views, row filters, and column masks.
- Query the audit system tables to answer “who touched what.”
- Design a governance model for a regulated multi-team lakehouse.

## Cross-Cloud Equivalence

Unity Catalog is Databricks’ **unified governance plane**; the hyperscalers split the same job across a catalog plus a policy engine.

| Capability          | Databricks (UC)                           | AWS                                   | Azure / GCP / Fabric                           |
|---------------------|-------------------------------------------|---------------------------------------|------------------------------------------------|
| Catalog             | Metastore → catalog → schema              | Glue Data Catalog                     | Purview + OneLake / Dataplex + BigLake         |
| Fine-grained grants | GRANT on any object + row filters + masks | Lake Formation LF-Tags + cell filters | Entra + RLS / policy tags / OneLake security   |
| File governance     | Volumes + external locations              | S3 + bucket policies                  | ADLS ACLs / GCS IAM                            |
| Audit               | <code>system.access.audit</code>          | CloudTrail                            | Activity Log / Cloud Audit Logs                |
| Lineage             | Built-in table/column lineage             | Glue / SageMaker lineage              | Purview / Dataplex lineage                     |
| Sharing             | Delta Sharing (Ch. 22)                    | Redshift sharing / S3 Access Grants   | Data Share / Analytics Hub / OneLake shortcuts |

**Snowflake** governs through its account role hierarchy; **MongoDB** through Atlas roles at cluster scope. Unity Catalog’s distinctive move is one governance model spanning tables, files, notebooks, models, and functions.

## 8.1 Week 8 Roadmap and Mental Model

Seven chapters of building now get a control plane. Unity Catalog (UC) is the reason “lake-house” and “governed” can appear in the same sentence: every asset—table, file volume, model, function, dashboard—lives in one hierarchy with one privilege model, one audit log, and automatic lineage [51].

The mental model: **UC is DNS plus ACLs for data**. The three-level namespace (`catalog.schema.table`) names every object unambiguously across the account; privileges (`SELECT`, `MODIFY`, `CREATE`, ...) attach to any level and flow downward; the metastore binds it all to storage credentials and audit. Govern once at the right level and every engine—notebooks, SQL warehouses, jobs, Genie—enforces it identically.

### Definition

**Unity Catalog** is Databricks’ unified governance layer: a per-region metastore containing catalogs (data domains/environments), schemas (groupings), and objects (tables, views, volumes, functions, models), with a single privilege model, audit log, and lineage capture across all Databricks workloads.

### Key Terms

**Metastore**: the regional UC root. **Managed table**: UC owns the storage lifecycle. **External table**: UC registers data you store and lifecycle-manage. **Volume**: governed access to files. **Dynamic view**: a view whose output changes with the querying principal. **Owner**: the principal responsible for an object, holding grant authority.

## 8.2 Monday: The Hierarchy and the Namespace

### 8.2.1 Metastore to Table

A metastore is created per cloud region and attached to workspaces. Below it: **catalogs** (the isolation unit—commonly per environment or domain), **schemas** (groupings—this book’s bronze/silver/gold), and **objects**. The three-level name makes every reference unambiguous and every grant addressable: `GRANT SELECT ON SCHEMA de_training.gold TO analysts` [51].

### 8.2.2 Managed versus External

Managed tables and volumes live in metastore-controlled storage; `DROP TABLE` deletes the data, and lifecycle is automatic. External tables register data at an external location you control; drop removes only the registration. Default to managed for anything Databricks pipelines own; use external for data shared with non-Databricks writers or under an external retention regime.

### Common Pitfall

**External-table sprawl**. Registering every table as external “for flexibility” forfeits managed lifecycle, complicates predictive optimization, and scatters your storage estate. Make external the exception with a stated reason—usually a non-Databricks writer or a mandated location.

## 8.3 Tuesday: The Privilege Model

### 8.3.1 Groups, Owners, and GRANT

Privileges grant to **groups** (synced from your IdP—Chapter 21), never individuals. Every object has an **owner** who can grant. The habit that keeps estates sane: grant at the highest level that expresses the intent (`SELECT ON SCHEMA gold`), and only drop to table level for genuine exceptions.

```

1 -- Engineers build; analysts consume; pipelines write.
2 GRANT USE CATALOG ON CATALOG de_training TO 'data-engineers', 'analysts';
3 GRANT USE SCHEMA ON SCHEMA de_training.silver TO 'data-engineers';
4 GRANT MODIFY ON SCHEMA de_training.silver TO 'data-engineers';
5 GRANT USE SCHEMA ON SCHEMA de_training.gold TO 'analysts';
6 GRANT SELECT ON SCHEMA de_training.gold TO 'analysts';
7 -- No Bronze grant for analysts: raw data is not a consumer surface.
8
9 -- The pipeline identity writes where humans only read:
10 GRANT MODIFY ON SCHEMA de_training.bronze TO 'sp-etl-prod';
11 GRANT MODIFY ON SCHEMA de_training.gold TO 'sp-etl-prod';

```

Listing 8.1: Least-privilege grants for the medallion estate.

### 8.3.2 Row Filters and Column Masks

For fine-grained control without view proliferation, UC supports **row filters** and **column masks** as named functions applied to tables—the policy lives with the table and applies to every reader automatically.

```

1 CREATE OR REPLACE FUNCTION de_training.gold.region_filter(region STRING)
2 RETURN IF(is_member('us_analysts'), region = 'US', true);
3
4 ALTER TABLE de_training.gold.fact_orders
5 SET ROW FILTER de_training.gold.region_filter ON (region);
6
7 CREATE OR REPLACE FUNCTION de_training.gold.mask_email(email STRING)
8 RETURN IF(is_member('pii_readers'), email, sha2(email, 256));
9
10 ALTER TABLE de_training.gold.dim_customer
11 ALTER COLUMN email SET MASK de_training.gold.mask_email;

```

Listing 8.2: Row filter and column mask as table-level policy.

## 8.4 Wednesday: Volumes, Dynamic Views, and Audit

### 8.4.1 Volumes for File Data

Files (CSVs, images, checkpoints) need governance too. A **volume** is a UC object over a storage path with the same GRANT model; external locations define the storage credential boundary. Rule: pipelines read and write `/Volumes/...` paths, never raw cloud URLs—so access, audit, and lineage cover files exactly like tables.

### 8.4.2 Dynamic Views

Before row filters/column masks—and still, for logic too rich for them—the **dynamic view** is the workhorse: a view whose `is_member()` / `current_user()` calls change output per principal (Chapter 7’s masking pattern). Prefer table-level filters/masks for simple policy; use dynamic views when masking logic is compound or shared across tables.

### 8.4.3 Audit

Every governed action lands in `system.access.audit`: who ran what statement against which object, when, from where [34].

```

1 SELECT event_time, user_identity.email AS actor, action_name,
2        request_params.full_name_arg AS object
3 FROM system.access.audit
4 WHERE request_params.full_name_arg = 'de_training.gold.dim_customer'
5        AND action_name = 'getTable'      -- plus queryText via Query History joins
6        AND event_date >= current_date() - 7
7 ORDER BY event_time DESC;
```

Listing 8.3: Who read the PII table this week?

## 8.5 Thursday Hands-on Project: Govern the Medallion Estate

Implement the full governance model for the Month 1 lakehouse: groups, schema-scoped grants, a masked dynamic view over customer PII, and audit proof that the mask works.

```

1 -- 1. Groups (account console or SCIM; shown for clarity)
2 --    data-engineers, analysts, pii-readers
3
4 -- 2. Grants per the medallion access matrix
5 GRANT USE CATALOG ON CATALOG de_training TO 'analysts';
6 GRANT USE SCHEMA ON SCHEMA de_training.gold TO 'analysts';
7 GRANT SELECT ON SCHEMA de_training.gold TO 'analysts';
8
9 -- 3. The masked view analysts actually read
10 CREATE OR REPLACE VIEW de_training.gold.v_dim_customer AS
11 SELECT customer_id,
12        CASE WHEN is_member('pii_readers') THEN email
13              ELSE sha2(email, 256) END AS email,
14        CASE WHEN is_member('pii_readers') THEN full_name
15              ELSE 'REDACTED' END AS full_name,
16        region, created_at
17 FROM de_training.gold.dim_customer;
18
19 REVOKE SELECT ON TABLE de_training.gold.dim_customer FROM 'analysts';
20 GRANT SELECT ON VIEW de_training.gold.v_dim_customer TO 'analysts';
21
22 -- 4. Proof: run both SELECTs as the test analyst principal and as owner;
23 --    then confirm the access appeared in system.access.audit.
```

Listing 8.4: Week 8 lab: groups, grants, masking, and proof.

### Deliverables

- The implemented access matrix: groups, grants per schema, the masked view.
- Side-by-side query output: owner (unmasked) vs. test analyst (masked).
- An audit query result showing the analyst’s masked read.

### Acceptance Criteria

- The test analyst can read Gold views and *cannot* read Silver, Bronze, or the base PII table (failed-read evidence captured).
- Masking is provably effective as the restricted principal—not merely assumed.
- All grants target groups; no individual holds a direct grant.

- The pipeline service principal can write Bronze/Silver/Gold but cannot **MANAGE** the catalog.

### Stretch Goals

- Replace the dynamic view with a column mask + row filter pair and compare governance ergonomics.
- Add a per-region row filter and prove two analysts in different regions see disjoint rows.
- Write the audit alert query: any read of the base PII table by a non-`pii_readers` principal.

## 8.6 Friday Use Case: HIPAA-Grade Governance for a Healthcare Lakehouse

A healthcare analytics platform holds claims and clinical extracts. Analysts query Gold; engineers build in Silver; only pipelines write Bronze; every access to patient identifiers must be auditable, and PII must be masked by default for all but a named break-glass group.

### 8.6.1 Requirements and Assumptions

Three environments (dev/staging/prod catalogs). Access reviews are quarterly; auditors sample monthly. Break-glass access must be time-boxed, logged, and reviewed within 24 hours.

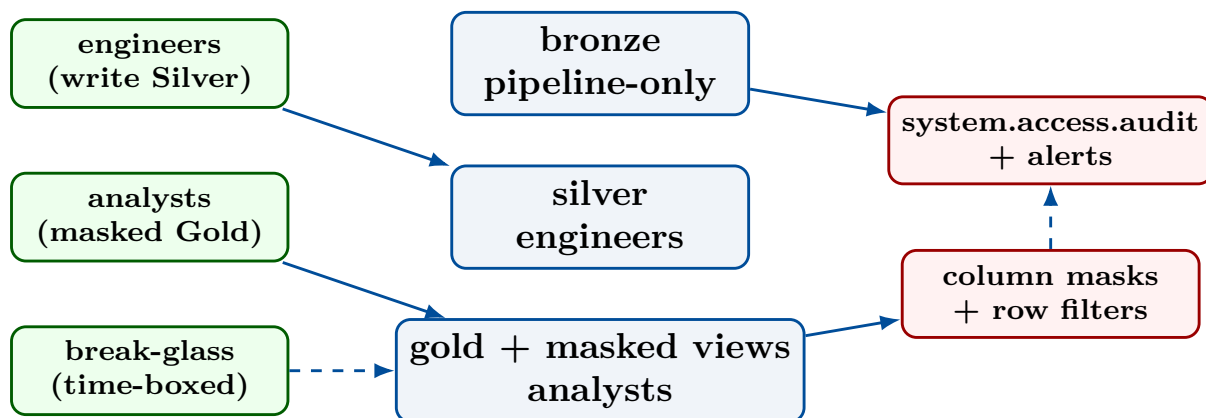


Figure 8.1: Healthcare governance: group-scoped layer access, default masking, break-glass with review, and audit everywhere.

### 8.6.2 Architecture Decisions

**Default-deny, group-grant.** Nobody has standing access to unmasked PII; `pii_readers` membership is provisioned by ticket, expires via IdP group automation, and every use lands in the audit alert the compliance dashboard reviews daily.

**Masking at the table, not the dashboard.** Column masks on `dim_patient` (and equivalents) make masking engine-independent: SQL warehouse, notebook, Genie, and export all enforce identically. Quarterly access reviews re-run the lab’s proof pattern against every PII-tagged object.



**Common Pitfall**

**A masked copy that drifts.** A nightly “sanitized extract” table is a second source of truth that misses corrections and multiplies the PII surface. Masks and dynamic views keep one governed table; copies exist only under a documented exception with its own retention and audit.

**8.6.3 Risks and Mitigations**

| Risk                                        | Mitigation                                          |
|---------------------------------------------|-----------------------------------------------------|
| Break-glass access becomes standing access  | Time-boxed IdP group with expiry and 24-hour review |
| Masked view drifts from base schema         | View change control with masked-output tests in CI  |
| Direct grants bypass the group model        | Monthly direct-grant diff; auto-revoke with notice  |
| External locations leak storage credentials | Credential inventory; scope per container; rotation |
| Audit lag during an incident                | Documented minutes-scale lag; queries tolerate it   |

Table 8.1: Week 8 scenario risks and mitigations.

**8.7 Design Decision Guide**

Governance decisions are cheapest at creation and most expensive at audit time.

| Decision                                | Choose the first when             | Choose the second when                 |
|-----------------------------------------|-----------------------------------|----------------------------------------|
| Schema-level vs. table-level grants     | The intent covers the layer       | True per-table exceptions              |
| Column mask/row filter vs. dynamic view | Declarative per-table policy      | Compound or cross-table masking logic  |
| Managed vs. external tables             | Pipelines own the data            | External writers or mandated locations |
| Volume vs. external location            | Governed file access for jobs     | Legacy tools need direct storage paths |
| Few catalogs vs. many                   | Environment/domain split suffices | Hard tenancy or blast-radius isolation |
| Owner as group vs. individual           | Always group                      | Never individual—people rotate         |

Table 8.2: Week 8 decision guide.

Design for the reviewer: a grant model you can print on one page is a grant model that survives an audit. Every exception should have a name, a reason, and an expiry.

**8.8 Operational Guidance for Week 8****8.8.1 Performance and Reliability**

Governance needs its own SLOs: grant changes propagate quickly, but audit and lineage tables lag minutes—build checks that tolerate the lag before paging. Review direct (non-group) grants monthly; they accumulate silently. Track object ownership hygiene: every object has a group owner, and departed-owner objects are reassigned before the account is deprovisioned.

### 8.8.2 Security and Governance Checklist

- Grants to groups only; direct grants expire by policy.
- External locations and storage credentials inventoried and least-privilege scoped.
- Quarterly access review runs the lab’s negative tests against production grants.
- PII classification tags applied at table creation (feeds Chapter 22’s alerts).

### 8.8.3 Troubleshooting Playbook

| Symptom                             | Likely cause                                      | First action                                               |
|-------------------------------------|---------------------------------------------------|------------------------------------------------------------|
| “Table not found” with correct name | Missing <code>USE CATALOG/USE SCHEMA</code>       | Check grants as the principal; namespace is not permission |
| Mask not applying in a notebook     | Reader is owner or has direct base-table grant    | Test as a non-owner restricted principal                   |
| Volume path access denied           | External location credential lacks storage rights | Verify the storage credential’s cloud IAM role             |
| Audit query returns nothing         | System tables not enabled for the metastore       | Ask the account admin to enable the schemas                |

Table 8.3: Week 8 troubleshooting quick reference.

## 8.9 Interview Q&A

### 8.9.1 Concepts and Trade-offs

1. **Q: Managed or external table—how do you decide?**  
A: Managed when Databricks owns the data lifecycle (default); external when a non-Databricks writer owns the files or storage location is mandated. External is the exception with a reason, not the default.
2. **Q: Where do you grant—catalog, schema, or table?**  
A: The highest level that matches intent: schema for “analysts read Gold,” catalog rarely (too broad), table only for true exceptions. High-level grants are reviewable; a thousand table grants are not.
3. **Q: Row filters/column masks versus dynamic views?**  
A: Filters and masks live on the table and apply to every consumer automatically—prefer them for declarative policy. Dynamic views remain for compound logic or when a shared semantic layer is itself the goal.
4. **Q: How do you prove least privilege to an auditor?**  
A: The grant matrix (group  $\times$  object), a negative test per principal class, and `system.access.audit` samples—implemented, tested, logged, in that order.
5. **Q: What does Unity Catalog buy you over bucket policies and a Hive metastore?**  
A: One namespace, one privilege model, one audit log, automatic lineage—across tables, files, models, and functions—enforced identically by every compute path. Bucket policies govern storage, not semantics.
6. **Q: What is the blast radius of a compromised storage credential?**  
A: Every external location and volume it backs—which is why credentials scope to single containers, rotate on schedule, and never carry wildcard storage rights.

7. **Q: Why test grants as a restricted principal rather than reviewing GRANTS?**

A: Grants compose (ownership, catalog inheritance, group nesting) in ways reviewers misread. The negative test is the only proof that survives composition.

8. **Q: What is the minimum viable governance for a new project?**

A: A catalog per environment, group-based grants at schema level, PII classification at table creation, and the audit alert on sensitive reads. Four moves, all cheap on day one, all painful retrofitted.

## 8.10 Further Reading

The Unity Catalog documentation is the reference [51]; system tables (audit, billing, lineage) are documented in [34]; lineage specifics are in [13]. The security overview places these controls in the platform's shared-responsibility model [39]. Chapter 22 goes deep on lineage, audit practice, and Delta Sharing [47].

## Key Takeaways

- Unity Catalog is one namespace, one privilege model, and one audit log for tables, files, models, and functions.
- Managed tables are the default; external tables need a stated reason.
- Grant to groups, at the highest level that expresses intent; individuals and table-level grants are exceptions.
- Prefer table-level row filters and column masks; dynamic views for compound or shared masking logic.
- Governance is proven by negative tests and audit queries, not by policy documents.

## 8.11 Exercises

1. **(Conceptual)** Draw the UC hierarchy for a two-environment, three-domain estate and place five objects in it.
2. **(Conceptual)** Why is REVOKE on the base table as important as GRANT on the masked view?
3. **(Hands-on)** Complete the Thursday lab, including the failed-read evidence as the analyst principal.
4. **(Hands-on)** Implement the PII audit alert as a scheduled query; plant a violating read and capture the alert.
5. **(Hands-on)** Register an external table over a cloud-storage path and document exactly what lifecycle responsibility you kept.
6. **(Design)** Write the access matrix for the Friday scenario (groups  $\times$  layers  $\times$  privileges), including break-glass expiry mechanics.
7. **(Operations)** Draft the quarterly access-review procedure: inputs, evidence, sign-off.
8. **(Architecture)** When would you split this estate into multiple metastores? What would it cost?

## 8.12 Milestone 2 Capstone: The Governed Analytics Lakehouse

### Capstone Project

**Mission.** Close Part II by turning the Month 1 lakehouse into a governed analytics product: liquid-clustered Gold storage, a serverless SQL warehouse, a published AI/BI dashboard, an instructed Genie space, and an enforced access matrix with masking and audit evidence.

**Estimated time.** 6–8 hours.

**What you'll practice.**

- Benchmarked storage optimization (liquid clustering vs. Z-ORDER).
- Warehouse topology with scaling bounds, auto-stop, and cost attribution.
- Certified semantic views feeding both dashboards and Genie.
- Group-based grants, column masking, and audit proof.

### Scenario

Contoso Retail's analytics platform is live but ungoverned: everyone reads everything, dashboards disagree, and nobody can say who saw customer emails last week. You are delivering the governed version—fast, certified, masked, auditable—without breaking the existing analysts' Monday morning.

### Requirements

- Gold fact table converted to liquid clustering with a before/after benchmark recorded.
- Serverless SQL warehouse (documented size, min/max, auto-stop) serving all BI.
- Certified view `gold.v_sales_certified` as the single metric source.
- Three-page AI/BI dashboard and a Genie space answering five questions correctly.
- Access matrix implemented: engineers/analysts/pipeline groups; customer PII masked via column mask or dynamic view; negative-test evidence.
- Audit evidence: a `system.access.audit` query showing a masked read by the analyst group.
- Cost report: warehouse + pipeline DBUs for the week, with two optimizations documented.

## Reference Architecture

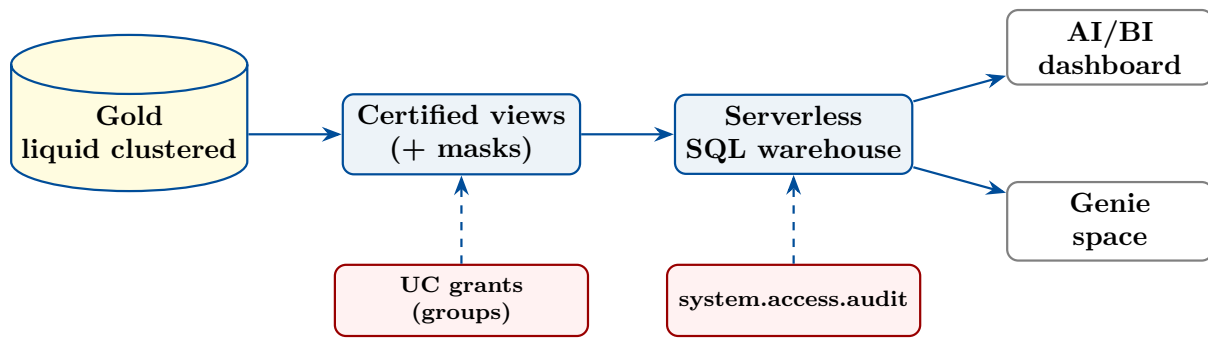


Figure 8.2: Milestone 2 reference architecture: governed serving from optimized Gold through one semantic layer.

## Implementation Steps

1. Run the Chapter 5 benchmark on the Gold fact; apply liquid clustering; record results.
2. Provision the warehouse; wire dashboards and Genie to it; capture query profiles.
3. Build the certified view; refactor the dashboard datasets to read it; write Genie instructions against it.
4. Implement groups, grants, and masking per the access matrix; run the negative tests.
5. Produce the audit and cost queries; file the evidence in the repo.

## Deliverables

- Benchmark table, query profiles, warehouse config, and cost report.
- Dashboard link/export and Genie test log (including one fixed wrong answer).
- Access matrix, negative-test evidence, and the masked-read audit output.

## Acceptance Criteria

- Analysts cannot read the base PII table; the masked view provably masks for the analyst principal.
- Genie’s “revenue by state last quarter” equals the dashboard’s certified number.
- Liquid clustering improved the benchmarked selective query measurably; results table committed.
- The warehouse auto-stops; the cost report identifies the top SKU and one implemented saving.

## Stretch Goals

- Replace the masking view with table-level column masks and row filters; document the trade-offs.
- Add a Genie “explore” space over non-certified Gold with a documented guardrail.
- Wire the PII audit alert to fire on any non-approved read and demonstrate it.

## **Part III**

# **Batch ETL and Orchestration**

# PySpark Deep Dive and Performance Tuning

## Learning Objectives

- Explain how Catalyst and Tungsten compile and execute DataFrame code, including whole-stage codegen.
- Choose join strategies—broadcast hash versus sort-merge—and direct them with hints and AQE.
- Tune shuffle partitions, memory, and parallelism for a given data volume.
- Remediate skew with salting and AQE skew handling, with measured evidence.
- Replace Python UDFs with built-ins or pandas UDFs and quantify the difference.
- Deliver an evidence-based performance report with attributable, isolated improvements.

## Cross-Cloud Equivalence

Spark tuning is **portable**—the same knobs exist on every managed Spark; what changes is the default and the telemetry surface.

| Capability         | Databricks                           | AWS (EMR/Glue)         | Azure / GCP                          |
|--------------------|--------------------------------------|------------------------|--------------------------------------|
| Adaptive execution | AQE on by default                    | AQE (EMR 6.x+)         | AQE<br>(Synapse/Fabric/Dataproc)     |
| Execution engine   | Photon (vectorized)                  | EMR runtime            | Fabric native / vanilla Spark        |
| Observability      | Spark UI + query profile + Ganglia   | Spark UI + Cloud-Watch | Spark UI + Synapse/Fabric monitoring |
| Join hints         | <code>broadcast()</code> , SQL hints | Same API               | Same API                             |
| Python UDF accel   | pandas UDFs, Arrow                   | pandas UDFs            | pandas UDFs                          |
| Skew handling      | AQE skew split + salting             | AQE skew (EMR)         | AQE skew                             |

Outside Spark: **Snowflake** pushes the same work into its engine via Snowpark with warehouse sizing as the main lever; **BigQuery** abstracts execution entirely—you trade the knobs for simplicity and lose the tuning surface this chapter teaches.

## 9.1 Week 9 Roadmap and Mental Model

Chapter 4 taught the execution model and the UI; Week 9 turns that literacy into a tuning method. The promise: most Spark performance problems are one of five—a bad join strategy, wrong shuffle partition count, skew, an expensive UDF, or a missing cache/predicate—and each has a signature you can read in the plan or the UI [5, 10].

The mental model: **tune in a loop with isolated variables**. Measure a baseline, change exactly one thing, measure again, attribute. A report that changes four configs at once proves nothing and teaches nobody; the acceptance criteria of this week’s lab enforce the discipline.

### Definition

**Adaptive Query Execution (AQE)** re-optimizes a query at stage boundaries using run-time statistics: coalescing small shuffle partitions, switching sort-merge joins to broadcast, and splitting skewed partitions. It is on by default in the Databricks Runtime; tuning means knowing when its defaults misfit.

### Key Terms

**Whole-stage codegen**: fusing operators into one generated Java/native function. **Broadcast join**: replicating the small side to every executor, avoiding the shuffle. **Salting**: artificially spreading a hot key across buckets. **pandas UDF**: a vectorized UDF exchanging Arrow columnar batches instead of per-row Python calls. **Spill**: intermediate data evicted to disk under memory pressure.

## 9.2 Monday: Catalyst, Tungsten, and When Not to RDD

### 9.2.1 The Optimization Stack

Your DataFrame code becomes an unresolved logical plan, then an analyzed plan (catalog-resolved), then an optimized logical plan (predicate pushdown, column pruning, constant folding, join reordering by statistics), then a physical plan (join strategy, aggregation mode), then generated code fused into whole stages. Reading `explain(True)` output top-to-bottom is reading this pipeline.

### 9.2.2 When (Not) to Use RDDs

RDDs trade the optimizer for control: no Catalyst, no codegen, no Photon, manual partitioning. The valid reasons in 2026 are rare—custom partitioner algorithms, legacy libraries. Every workload in this program stays on the DataFrame/SQL API; the moment you write `.rdd`, you have left the performance warranty.

### Common Pitfall

**Collecting to the driver “to inspect.”** `collect()`, `toPandas()`, and Python loops over rows move data to the driver; at scale they are the out-of-memory classic. Inspect with `show/limit/aggregates`; if you need local analysis, sample explicitly with `.sample()` first.



## 9.3 Tuesday: Joins and AQE

### 9.3.1 Broadcast versus Sort-Merge

A **sort-merge join** shuffles both sides by the join key, sorts, and merges—correct at any size, expensive by construction. A **broadcast hash join** sends the small side to every executor and probes a hash table—no shuffle of the big side. The threshold (`spark.sql.autoBroadcastJoinThreshold`, default 10 MB) and AQE’s runtime statistics decide; you direct with `broadcast(dim_df)` when you know better than the estimates [5].

```

1 from pyspark.sql import functions as F
2
3 facts = spark.read.table("de_training.gold.fact_orders")
4 prods = spark.read.table("de_training.gold.dim_product")
5
6 # If dim_product is small (check!), force broadcast:
7 facts.join(F.broadcast(prods), "product_id").explain(True)
8 # Physical plan should show BroadcastHashJoin, no Exchange on the fact side.
9
10 # AQE will convert SMJ to BHJ at runtime when stats prove it fits;
11 # the query profile shows "AdaptiveSparkPlan" with the final strategy.
```

Listing 9.1: Directing the join strategy with evidence.

### 9.3.2 Skew and Salting

Skew defeats parallelism: one task holds 80% of a shuffle. AQE’s skew-splitting helps by splitting the hot partition after the fact; **salting** prevents it by exploding the hot key across N buckets and replicating the small side per bucket. Chapter 4’s lab built the muscle; this week’s discipline is knowing when each applies.

## 9.4 Wednesday: Shuffle Tuning, UDFs, and Memory

### 9.4.1 Shuffle Partitions and I/O

`spark.sql.shuffle.partitions` (default 200) sets post-shuffle parallelism. Target ~100–200 MB per partition: 200 GB shuffled → ~1500 partitions. With AQE coalescing, overshooting is safer than undershooting. Watch *spill* in the UI—spilling tasks signal memory pressure that partition sizing or executor memory must absorb.

### 9.4.2 UDF Performance

A row-at-a-time Python UDF serializes every row between the JVM and Python—often 10–100x slower than a native expression. The ladder, best to worst: built-in SQL functions, **pandas (Arrow) UDFs** over columnar batches, plain Python UDFs, RDD `map`.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.functions import pandas_udf
3 import pandas as pd
4
5 # Best: built-in
6 df.withColumn("amount_band", F.when(F.col("amount") > 100, "high")
7                                     .otherwise("low"))
8
9 # Good: vectorized pandas UDF when the logic truly needs Python
10 @pandas_udf("string")
11 def risky_band(s: pd.Series) -> pd.Series:
```

```

12     return pd.cut(s, bins=[0, 50, 200, 1e9], labels=["low", "mid", "high"]).
       astype(str)
13
14 df.withColumn("risk_band", risky_band("amount"))
15
16 # Last resort: plain Python UDF (row-at-a-time serialization cost)

```

Listing 9.2: The UDF ladder: native, then vectorized, then—rarely—plain Python.

### Tip

Before writing any UDF, search the function list: `spark.sql.functions` covers dates, strings, arrays, maps, math, and windowing. Most “I need a UDF” moments are undiscovered built-ins.

## 9.5 Thursday Hands-on Project: The Attributable Tuning Report

Take a deliberately degraded 2 TB-scale job (or a 200 GB surrogate) and produce a tuning report where every improvement is isolated and measured.

```

1 from pyspark.sql import functions as F
2 import time
3
4 def timed(label, fn):
5     t0 = time.time(); fn(); return (label, round(time.time() - t0, 1))
6
7 facts = spark.read.table("de_training.silver.big_facts")
8 dim    = spark.read.table("de_training.silver.big_dim")
9
10 results = []
11 # Baseline: shuffle join, default partitions, Python UDF
12 spark.conf.set("spark.sql.adaptive.enabled", "false")
13 results.append(timed("baseline", lambda:
14     facts.join(dim, "k").groupBy("dim.v").agg(F.sum("amount")).count()))
15
16 # Opt 1: right-size shuffle partitions for the measured volume
17 spark.conf.set("spark.sql.shuffle.partitions", "1500")
18 results.append(timed("shuffle_partitions_1500", lambda:
19     facts.join(dim, "k").groupBy("dim.v").agg(F.sum("amount")).count()))
20
21 # Opt 2: broadcast the measured-small dimension
22 results.append(timed("broadcast_dim", lambda:
23     facts.join(F.broadcast(dim), "k")
24     .groupBy("dim.v").agg(F.sum("amount")).count()))
25
26 # Opt 3: re-enable AQE (skew split + coalesce) and compare
27 spark.conf.set("spark.sql.adaptive.enabled", "true")
28 results.append(timed("aqe_on", lambda:
29     facts.join(F.broadcast(dim), "k")
30     .groupBy("dim.v").agg(F.sum("amount")).count()))
31
32 spark.createDataFrame(results, ["experiment", "seconds"]) \
33     .write.mode("overwrite").saveAsTable("de_training.silver.week09_results")

```

Listing 9.3: Week 9 lab: four isolated optimizations, one evidence table.

### Deliverables

- The evidence table: one row per experiment, identical result values across all.

- Spark UI screenshots per experiment highlighting the stage that changed.
- A written report: symptom, hypothesis, change, measurement, conclusion—per optimization.

### Acceptance Criteria

- Each experiment changes exactly one variable; confounded results are redone.
- Result correctness is reconciled across all experiments.
- The report names the single largest win and explains its mechanism from the plan.
- One experiment deliberately regresses (e.g., broadcasting the large side) and is documented as a negative control.

### Stretch Goals

- Add a salted-join variant for an injected hot key and compare against AQE skew handling.
- Replace a Python UDF with a pandas UDF and quantify rows/second processed.
- Repeat the best configuration on Photon off vs. on (runtime toggle) and quantify the engine effect.

## 9.6 Friday Use Case: The 2 TB Join That Misses Its SLA

A logistics partner’s daily join job (2 TB facts against 40 GB of reference data) must finish by 06:00; it currently finishes at 09:15. You are engaged for a one-week performance engagement with a written deliverable.

### 9.6.1 Requirements and Assumptions

The job runs on a fixed-size job cluster; the cluster may be resized only with justification. Reference data grows 5% monthly. The business accepts logic-preserving rewrites but no result changes.

### 9.6.2 Architecture Decisions

**The engagement structure.** Day 1: capture three consecutive runs’ Spark UI exports and explain plans—the baseline dossier. Day 2: attribute time per stage; expect the classic finding (the “small” reference side grew past the broadcast threshold months ago, flipping the join to sort-merge). Days 3–4: isolated fixes with A/B runs—forced broadcast with a memory check, shuffle partition sizing, skew audit on the tenant key. Day 5: regression suite (row-for-row reconciliation against the slow baseline), the report, and the handover runbook.

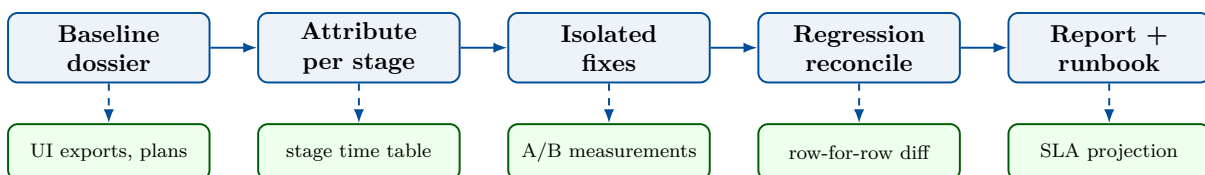


Figure 9.1: The one-week performance engagement: every phase produces an artifact.

**The expected fix.** `broadcast()` the reference side (40 GB compressed fits executor memory across the fleet—verified, not assumed), right-size shuffle partitions, and add a scheduled `ANALYZE TABLE` so AQE’s statistics stop drifting. Projected: 09:15 → 05:20 with the same cluster.

### Common Pitfall

**Promising the number before the dossier.** “We can get you under 6 a.m.” is a guess until stage attribution exists. Commit to the method and the evidence standard; let the measured bottleneck set the projection.

### 9.6.3 Risks and Mitigations

| Risk                                    | Mitigation                                                   |
|-----------------------------------------|--------------------------------------------------------------|
| Tuning report changes behavior subtly   | Regression suite with row-for-row reconciliation             |
| Salting key leaks into downstream logic | Salt columns dropped before publish; contract documented     |
| Config tuned for today’s volume only    | Quarterly re-baseline against volume growth                  |
| UDF logic forked from native behavior   | Unit tests pin UDF semantics against built-ins               |
| Hardware scale-up masks the real fix    | Plan-level fixes land first; sizing changes are separate PRs |

Table 9.1: Week 9 scenario risks and mitigations.

## 9.7 Design Decision Guide

Tuning decisions compound; isolate, measure, and record each one.

| Decision                           | Choose the first when      | Choose the second when          |
|------------------------------------|----------------------------|---------------------------------|
| Salting vs. AQE skew handling      | Extreme, known hot keys    | Moderate, emergent skew         |
| Built-in function vs. pandas UDF   | The function exists        | Genuinely custom row logic      |
| pandas UDF vs. Python UDF          | Always, if a UDF is needed | Never the plain Python UDF      |
| More partitions vs. more memory    | Spill with healthy GC      | GC pressure with small tasks    |
| Tune plan vs. scale hardware       | Always plan first          | Plan is sane and hardware-bound |
| <code>ANALYZE TABLE</code> cadence | Hot, changing tables       | Static reference data           |

Table 9.2: Week 9 decision guide.

The engagement rule from Friday applies to every row: baseline, one variable, re-measure, reconcile. A tuning change you cannot attribute is a superstition you now maintain.

## 9.8 Operational Guidance for Week 9

### 9.8.1 Performance and Reliability

Institutionalize the regression watch: p50/p95 duration per job trended weekly, with a config-change log so every performance delta has a suspect. Schedule `ANALYZE TABLE ... COMPUTE STATISTICS` on hot tables so Catalyst and AQE plan from current reality—stale statistics are the silent join-flipper. Keep the tuning experiments reproducible: the evidence table pattern from the lab becomes the team’s standard for any “make it faster” request.

### 9.8.2 Security and Governance Checklist

- Custom init scripts and libraries reviewed; they run with the cluster’s full data access.
- Shuffle partitions and memory configs changed through code review, not notebook conf cells.
- Tuning experiments never run against production tables with write modes enabled.
- Broadcast targets verified against actual sizes, not assumptions (a grown dimension is a bomb).

### 9.8.3 Troubleshooting Playbook

| Symptom                               | Likely cause                               | First action                                     |
|---------------------------------------|--------------------------------------------|--------------------------------------------------|
| Sudden job regression, no code change | Statistics drift flipped the join strategy | Diff the plan against last week’s; refresh stats |
| Broadcast join OOMs                   | “Small” side outgrew executor memory       | Measure it; drop the hint or raise memory        |
| Python UDF dominates stage time       | Row-at-a-time serialization                | Rewrite as built-in or pandas UDF                |
| Heavy GC in executor logs             | Memory pressure / oversized partitions     | Check spill; repartition; right-size executors   |

Table 9.3: Week 9 troubleshooting quick reference.

## 9.9 Interview Q&A

### 9.9.1 Concepts and Trade-offs

1. **Q: How does Catalyst decide broadcast versus sort-merge?**

A: From table statistics versus `autoBroadcastJoinThreshold`; AQE revises at runtime with measured sizes. Stale statistics are why a growing dimension silently flips a job to sort-merge.

2. **Q: When is salting better than AQE skew handling?**

A: When skew is extreme and known (one key with most rows)—salting spreads it by construction, while AQE splits only after the shuffle plan exists. Salting costs a replicated small side and a two-stage aggregation.

3. **Q: Why are pandas UDFs faster than Python UDFs?**

A: Arrow columnar batches cross the JVM/Python boundary once per batch instead of once per row, and the Python side runs vectorized pandas instead of per-row interpreter calls.

4. **Q: Your job spills heavily. Options?**

A: More shuffle partitions (smaller tasks), more executor memory, a different join strategy, or less cached competition for memory. Read which stage spills before choosing.

5. **Q: What do you check first when a job regresses with no code change?**

A: The data: volume, skew, and statistics freshness. Then the plan diff—a flipped join strategy from grown input is the usual culprit.

6. **Q: What is the fastest legitimate win in most Spark jobs?**

A: Fix the join: broadcast the provably small side and prune columns/filters into the scan. Shuffle avoidance beats micro-optimization every time.

7. **Q: How do you keep a tuned job tuned?**

A: Duration trend alerts, fresh statistics (`ANALYZE TABLE`), and the config-change log—regressions are caught as data-shape changes, not discovered as SLA breaches.

8. **Q: When is a job “tuned enough”?**

A: When the next measured improvement costs more engineering time than the compute it saves, given the SLA margin—tuning has a stopping rule, and it is economic.

## 9.10 Further Reading

The Spark performance-tuning guide is the core reference [5]; *Spark: The Definitive Guide* gives the practitioner’s tour of joins, partitions, and UDFs [10]. The Spark documentation covers AQE configuration [4], and the Photon page explains Databricks’ execution layer [49].

## Key Takeaways

- Tune in isolated experiments with reconciliation; confounded changes teach nothing.
- The five usual suspects: join strategy, shuffle partitions, skew, UDFs, and pushdown/caching.
- AQE handles the average case; your job is recognizing the misfit—known skew, provable broadcasts.
- Stay on the DataFrame API; RDDs and plain Python UDFs exit the optimizer’s warranty.
- A performance engagement is an evidence pipeline: dossier, attribution, isolated fixes, regression, report.

## 9.11 Exercises

1. **(Conceptual)** Explain why sort-merge joins exist at all, given broadcast joins are faster.
2. **(Conceptual)** What information do table statistics give Catalyst, and what goes wrong when they are stale?
3. **(Hands-on)** Complete the Thursday lab with all four experiments and the negative control.
4. **(Hands-on)** Find one UDF in an existing notebook and replace it with built-ins; measure the delta.
5. **(Hands-on)** Force a spill and identify it in the UI; then fix it two different ways.
6. **(Design)** Write the shuffle-partitions rule of thumb as a decision table for 1 GB / 100 GB / 10 TB shuffles.

7. **(Operations)** Add a scheduled `ANALYZE TABLE` to the Friday scenario's pipeline and justify its cadence.
8. **(Architecture)** When would you move this workload to DLT (Chapter 10) instead of hand-tuned Spark? What would you lose?

# Delta Live Tables: Declarative Pipelines

**Learning Objectives**

- Explain DLT’s declarative model: datasets, dependencies, and the automatically built DAG.
- Author streaming tables and materialized views in Python and SQL.
- Enforce data quality with expectations: warn, drop, and fail semantics.
- Operate pipelines: development versus production mode, triggered versus continuous, full refresh, and schema evolution.
- Query the DLT event log for quality metrics, lineage, and operational history.
- Decide deliberately between DLT and hand-written Spark plus Workflows.

**Cross-Cloud Equivalence**

DLT is Databricks’ **declarative pipeline** layer; the pattern—define datasets and quality, let the platform handle execution—has analogs everywhere.

| Capability            | Databricks (DLT)                     | AWS                           | Azure / GCP / Fabric                    |
|-----------------------|--------------------------------------|-------------------------------|-----------------------------------------|
| Declarative pipelines | DLT (Lakeflow Declarative Pipelines) | Glue workflows + DQ rules     | ADF/Fabric pipelines + gates / Dataform |
| Quality rules         | Expectations (warn/drop/fail)        | Glue Data Quality (Deequ)     | Great Expectations / dbt tests          |
| Streaming dataset     | Streaming table                      | Kinesis → Glue streaming      | Eventstream / Dataflow / Fabric streams |
| Batch dataset         | Materialized view                    | Redshift MV / Athena CTAS     | BigQuery MV / Fabric MV                 |
| Ops telemetry         | Event log (Delta table)              | CloudWatch + Glue job metrics | ADF monitoring / Dataflow metrics       |
| Lineage               | Automatic, into UC                   | Glue lineage (partial)        | Purview / Dataplex                      |

**Snowflake**’s analog is Dynamic Tables with **EXPECT** clauses emerging; dbt (cross-platform) shares the declarative-DAG philosophy with SQL models and tests.

## 10.1 Week 10 Roadmap and Mental Model

Chapter 9 squeezed performance out of hand-written Spark. Week 10 asks the architectural question: which pipelines should be hand-written at all? Delta Live Tables (DLT—being re-branded Lakeflow Declarative Pipelines) inverts control: you declare *datasets* (streaming tables, materialized views) and their *quality rules*; the framework builds the DAG, chooses batch or



streaming execution, retries, checkpoints, and records every quality metric [26].

The mental model: **DLT is a compiler from intent to pipeline**. Your code is a graph of dataset definitions; the runtime infers dependencies from table references, plans the execution order, and operates the result—including the parts hand-written pipelines get wrong under pressure: consistent checkpoints, quality telemetry, and lineage into Unity Catalog.

### Definition

**Delta Live Tables** is Databricks' declarative ETL framework. A **streaming table** ingests each source record exactly once (append processing); a **materialized view** maintains a query result, refreshed incrementally where possible. **Expectations** are per-dataset quality rules with warn, drop, or fail semantics.

### Key Terms

**Expectation:** `@dlt.expect`, `expect_or_drop`, `expect_or_fail`. **Event log:** the per-pipeline Delta table of runs, quality metrics, and lineage. **Development mode:** fast-iterating pipeline mode that reuses compute and skips retries. **Full refresh:** rebuilding datasets from scratch. **Trigger:** available-now (triggered) versus continuous processing.

## 10.2 Monday: Datasets and the Declarative DAG

### 10.2.1 Streaming Tables and Materialized Views

A streaming table wraps a streaming read—the append-only facts of your system (Auto Loader output, Kafka, CDC feeds). A materialized view wraps a query—DLT tracks its inputs and refreshes it incrementally when the plan allows. Dependencies are inferred: referencing LIVE names (or the table names) wires the DAG automatically.

```

1 import dlt
2 from pyspark.sql import functions as F
3
4 @dlt.table(comment="Raw orders, exactly-once from the landing volume")
5 def bronze_orders():
6     return (spark.readStream.format("cloudFiles")
7             .option("cloudFiles.format", "csv")
8             .option("header", True)
9             .load("/Volumes/de_training/bronze/landing/olist/orders/"))
10
11 @dlt.table(comment="Typed, deduplicated orders")
12 def silver_orders():
13     return (dlt.read_stream("bronze_orders")
14             .withColumn("order_ts", F.to_timestamp("order_purchase_timestamp"))
15             .withColumn("total_amount", F.col("payment_value").cast("decimal
16             (12,2)"))
17             .dropDuplicates(["order_id"]))
18
19 @dlt.table(comment="Daily revenue by region")
20 def gold_daily_revenue():
21     return (dlt.read("silver_orders")
22             .groupBy(F.to_date("order_ts").alias("order_date"), "region")
23             .agg(F.sum("total_amount").alias("revenue")))

```

Listing 10.1: A three-dataset DLT pipeline in Python.

## 10.2.2 Development versus Production Mode

Development mode attaches to a long-lived cluster, skips retries, and reuses compute between runs—fast iteration. Production mode isolates each run, retries on failure, and terminates compute—operational safety. Develop in dev mode; deploy in production mode; never let a demo in dev mode stand in for a production run.

## 10.3 Tuesday: Expectations—Quality as Code

### 10.3.1 Warn, Drop, Fail

Expectations attach quality rules to datasets [30]: `expect` records violations (warn), `expect_or_drop` removes offending rows, `expect_or_fail` stops the update. All three emit metrics to the event log, so “how much bad data did we see this week” is a SQL query, not an investigation.

```

1 @dlt.table
2 @dlt.expect("reasonable_amount", "total_amount < 1_000_000") -- warn: log it
3 @dlt.expect_or_drop("valid_key", "order_id IS NOT NULL")      -- drop:
   unusable
4 @dlt.expect_or_fail("pii_gate", "customer_email IS NOT NULL") -- fail: stop
   the line
5 def silver_orders_validated():
6     return dlt.read_stream("bronze_orders") # ... typed transform as above

```

Listing 10.2: The three expectation severities, deliberately assigned.

#### Common Pitfall

**Everything is expect.** Warn-only expectations produce metrics nobody acts on—aspirational quality control. The discipline: every rule gets a severity decision on purpose. Keys and PII fail; business-plausibility rules warn with an owner; everything in between drops with quarantine visibility.

## 10.4 Wednesday: Operating Pipelines

### 10.4.1 Triggered versus Continuous

A **triggered** pipeline processes available data and stops—batch economics for streaming sources. **Continuous** stays up for second-level latency at continuous cost. Most pipelines in this program are triggered on a schedule; Chapter 16 covers when continuous earns its DBUs.

### 10.4.2 Full Refresh and Schema Evolution

Incrementalization has limits: a logic change to a materialized view may require a **full refresh** (rebuild from source). Schema evolution of sources is handled per dataset (DLT tracks and adapts, or fails with a clear error depending on configuration). The operational habit: know per dataset whether tomorrow’s change deploys cleanly or needs a refresh—and budget the refresh window.

### 10.4.3 The Event Log

Every pipeline publishes an event log (a Delta table in the pipeline’s storage): run lifecycle, per-dataset quality metrics, lineage edges, and errors.

```

1 SELECT timestamp, details:flow_progress.data_quality.expectations
2 FROM event_log(TABLE(de_training.silver.silver_orders_validated))

```

```

3 WHERE event_type = 'flow_progress'
4 ORDER BY timestamp DESC
5 LIMIT 20;

```

Listing 10.3: Quality metrics from the event log.

## 10.5 Thursday Hands-on Project: The Medallion as DLT

Rebuild the Month 1 Bronze→Silver→Gold flow as one DLT pipeline with expectations, inject a poison record, and read the quality evidence from the event log.

```

1 # Pipeline notebook (selected datasets shown)
2 @dlt.table
3 @dlt.expect_or_drop("valid_order", "order_id IS NOT NULL")
4 @dlt.expect_or_fail("non_negative", "total_amount >= 0")
5 def silver_orders():
6     return (dlt.read_stream("bronze_orders")
7             .withColumn("total_amount", F.col("payment_value").cast("decimal
8             (12,2)"))
9             .dropDuplicates(["order_id"]))
10 # The drill: append one CSV with a NULL order_id and one negative amount.
11 # Expected: NULL-key row dropped (counted), negative amount fails the update.
12 # Then: fix the poison file, re-run, and confirm the pipeline recovers.

```

Listing 10.4: Week 10 lab core: pipeline with poison-record drill.

### Deliverables

- The DLT pipeline (Python or SQL) reproducing the medallion flow with expectations on Silver.
- Event-log queries showing the dropped NULL-key record and the failed negative-amount update.
- A recovery note: what the pipeline did, what you fixed, how the re-run converged.

### Acceptance Criteria

- The poisoned NULL-key row is dropped and *counted* in quality metrics—not silently absent.
- The `expect_or_fail` violation stops the update; Gold is untouched by the failed run.
- After fixing the source, one triggered run converges Silver and Gold to the correct state with no manual table surgery.
- The pipeline runs in production mode with a schedule, not only interactively.

### Stretch Goals

- Add a quarantine *dataset* capturing dropped rows with reasons, mirroring Chapter 3's pattern.
- Convert `gold_daily_revenue` to a materialized view and document when it refreshes incrementally vs. fully.
- Deploy the pipeline twice (dev/prod targets) from one definition—a preview of Chapter 23.

## 10.6 Friday Use Case: DLT or Hand-Written—Drawing the Line

A logistics company asks you to classify three pipeline candidates: (1) 200-table nightly CDC replication, (2) a fraud feature pipeline with exotic stateful logic, (3) a compliance reporting flow with strict audit requirements.

### 10.6.1 Requirements and Assumptions

The team knows Spark well and DLT not at all. The fraud pipeline uses `flatMapGroupsWithState` with custom timers. The compliance flow needs immutability evidence and sign-off gates.

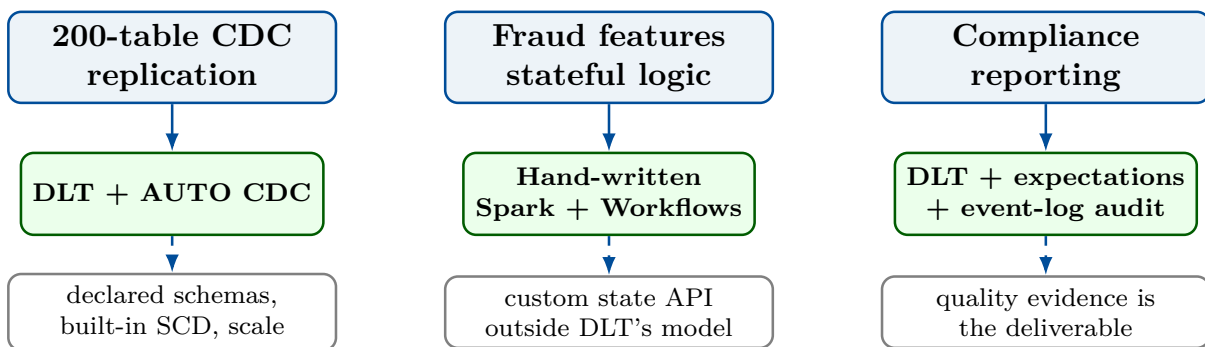


Figure 10.1: The adoption boundary: DLT where declaration wins, hand-written Spark where control wins.

### 10.6.2 Architecture Decisions

**CDC replication: DLT.** Two hundred tables of key-based upserts with SCD handling is exactly what AUTO CDC and expectations commoditize (Chapter 11 details the CDC mechanics). **Fraud features: hand-written.** Arbitrary stateful operators, custom timers, and side-effecting sinks live outside DLT's dataset model; forcing them in costs more than the DAG automation saves. **Compliance: DLT.** The event log plus expectations *is* the audit evidence the requirement asks for.

#### Common Pitfall

**All-or-nothing adoption.** The platforms compose: a Workflows job (Chapter 12) can run a DLT pipeline task *and* a hand-written Spark task. The boundary belongs inside the estate, per pipeline—not around it.

### 10.6.3 Risks and Mitigations

| Risk                                       | Mitigation                                               |
|--------------------------------------------|----------------------------------------------------------|
| Expectation drops quietly shrink a dataset | Drop-rate trend alert per dataset from the event log     |
| Dev-mode pipeline promoted untested        | Prod-mode deploy only via the bundle pipeline            |
| Full refresh blows the weekend window      | Refresh cost measured; calendar holds the slot           |
| Framework upgrade changes behavior         | Channel pinned; upgrade tested in dev target first       |
| DLT hides logic from reviewers             | Pipeline source is code in Git, reviewed like any change |

Table 10.1: Week 10 scenario risks and mitigations.

## 10.7 Design Decision Guide

DLT decisions are about who operates the pipeline: you, or the framework.

| Decision                                                    | Choose the first when                     | Choose the second when                |
|-------------------------------------------------------------|-------------------------------------------|---------------------------------------|
| DLT vs. hand-written Spark + Workflows                      | Standard ETL with quality rules           | Exotic stateful logic or side effects |
| Streaming table vs. materialized view                       | Append-only facts, exactly-once ingest    | Derived query results over inputs     |
| <code>expect_or_drop</code> vs. <code>expect_or_fail</code> | Row is expendable, pipeline must continue | Bad data invalidates the whole batch  |
| Triggered vs. continuous                                    | Minutes-freshness, cost-sensitive         | Measured sub-minute mandate           |
| Development vs. production mode                             | Iterating                                 | Anything scheduled or shared          |
| Full refresh vs. incremental                                | Logic change invalidates stored state     | Schema-compatible evolution           |

Table 10.2: Week 10 decision guide.

When torn, ask who will debug this at 03:00 and what evidence they will need. DLT’s event log is often the deciding vote—operability is a feature of the platform, not of your diligence.

## 10.8 Operational Guidance for Week 10

### 10.8.1 Performance and Reliability

Track per-dataset processing time and quality metrics from the event log as first-class telemetry—the pipeline that degrades announces itself there weeks before users notice. Budget and calendar the full refreshes: know which logic changes force them and how long they take at current volume. Pin the pipeline channel deliberately and test upgrades in dev mode; a channel auto-upgrade before a demo is a rite of passage you can skip.

### 10.8.2 Security and Governance Checklist

- Pipelines run as a service principal in production mode, never as a person.

- Expectation severities reviewed per rule: fail for keys/PII, drop with quarantine, warn with an owner.
- Event-log retention covers your audit window; export quality metrics for compliance.
- Pipeline target schemas are UC-governed like any table—declarative does not mean un-governed.

### 10.8.3 Troubleshooting Playbook

| Symptom                               | Likely cause                               | First action                                      |
|---------------------------------------|--------------------------------------------|---------------------------------------------------|
| Update fails on schema change         | Source evolution incompatible with dataset | Read the error’s dataset; evolve or full-refresh  |
| Expectation storm after source change | Upstream contract broke                    | Event log quality trend; pause downstream publish |
| Pipeline “stuck” starting             | Cluster quota or dev-mode reuse confusion  | Check compute events; verify mode and policy      |
| MV refreshed fully (slow)             | Change invalidated incremental plan        | Expected on logic change; schedule the window     |

Table 10.3: Week 10 troubleshooting quick reference.

## 10.9 Interview Q&A

### 10.9.1 Concepts and Trade-offs

1. **Q: What does DLT actually do that a notebook chain does not?**  
A: Dependency inference and DAG construction, execution-mode management (batch/streaming per dataset), checkpointing, retries, per-dataset quality metrics, an event log, and UC lineage—the operational layer you would otherwise hand-build.
2. **Q: Streaming table versus materialized view?**  
A: A streaming table appends each source record exactly once (facts); a materialized view maintains a query result over its inputs (derived state). Streaming tables answer “what arrived”; MVs answer “what is true.”
3. **Q: `expect_or_drop` versus `expect_or_fail`?**  
A: Drop when the row is expendable and the pipeline must continue (with quarantine visibility); fail when bad data invalidates the whole batch—keys, PII gates—and continuing would corrupt consumers.
4. **Q: When does a DLT change require a full refresh?**  
A: When the change invalidates incremental state—logic changes to a materialized view whose stored results encode the old logic. Know it per dataset before you deploy.
5. **Q: Why query the event log instead of trusting the UI?**  
A: The UI shows the last run; the event log is the durable, queryable record—trend analysis, audit evidence, and alerting all read it as a table.
6. **Q: What telemetry would you lose by leaving DLT?**  
A: Per-dataset quality metrics, the event log’s run history, and automatic lineage—you would rebuild all three before your first audit.
7. **Q: How do expectations interact with schema evolution?**  
A: Expectations evaluate the row as it arrives at the dataset; a schema change that renames

a column makes the expectation reference fail loudly—which is the point: quality rules pin the contract.

8. **Q: Can DLT and plain Spark coexist in one pipeline estate?**

A: That is the normal end state: DLT owns the standard flows, hand-written Spark owns the exotic ones, and Workflows orchestrates both as task types in the same jobs.

## 10.10 Further Reading

The DLT documentation is the reference [26]; expectations and the event log are detailed in [30]. Auto Loader (the streaming source used throughout) is covered in [45], and the medalion rationale in [50]. dbt practitioners should map the concepts across: datasets ~ models, expectations ~ tests, event log ~ run results.

## Key Takeaways

- DLT compiles declared datasets and quality rules into an operated pipeline—DAG, checkpoints, retries, metrics, lineage included.
- Streaming tables capture facts exactly once; materialized views maintain derived truth.
- Every expectation gets a deliberate severity: warn, drop (visible), or fail (protect downstream).
- The event log is your quality evidence base—query it, alert on it, audit with it.
- The DLT boundary is per pipeline: declaration wins for standard ETL; hand-written Spark wins for exotic control.

## 10.11 Exercises

1. **(Conceptual)** Explain the difference between DLT’s DAG inference and Airflow-style explicit dependencies; name one risk of each.
2. **(Conceptual)** Why does `expect_or_fail` on a PII column protect *downstream* tables, not just the dataset itself?
3. **(Hands-on)** Complete the Thursday lab, including the poison drill and event-log queries.
4. **(Hands-on)** Rewrite one dataset in SQL (DLT SQL) and compare readability with the Python version.
5. **(Hands-on)** Trend this week’s quality metrics from the event log and chart the drop rate per day.
6. **(Design)** Classify five of your current (or hypothetical) pipelines on the DLT boundary; defend each call in one sentence.
7. **(Operations)** Write the runbook for “pipeline failed on `expect_or_fail` at 03:00”: diagnosis, fix, re-run, evidence.
8. **(Architecture)** When would you graduate this pipeline’s scheduling from the DLT trigger to a Workflows job (Chapter 12)?

# Auto Loader, Incremental Ingestion, and Change Data Capture

**Learning Objectives**

- Explain Auto Loader’s architecture: cloudFiles source, directory listing versus notification mode, and checkpointed exactly-once semantics.
- Configure schema inference, schema hints, evolution, and the rescued-data column.
- Build triggered incremental ingestion pipelines from cloud storage volumes.
- Apply change data capture with `MERGE INTO` and `DLT AUTO CDC`, including SCD Type 1 and Type 2.
- Handle out-of-order and duplicate CDC events with `SEQUENCE BY`.
- Design ingestion for high file rates plus nightly CDC without double-processing.

**Cross-Cloud Equivalence**

Auto Loader is Databricks’ **managed file-ingestion** answer; CDC application is the second half of the pattern every platform implements.

| Capability     | Databricks                                     | AWS                          | Azure / GCP / Fabric                                        |
|----------------|------------------------------------------------|------------------------------|-------------------------------------------------------------|
| File ingestion | Auto Loader (cloud-Files)                      | S3 events + Glue bookmarks   | Event Grid + ADF / GCS transfer + Dataflow / OneLake events |
| Exactly-once   | Checkpoint + write-ahead offsets               | Bookmarks / idempotent sinks | Watermarks / manifests                                      |
| Schema drift   | Inference + hints + evolution                  | Glue schema detection        | ADF mapping drift / Dataflow                                |
| Malformed rows | <code>_rescued_data</code> column              | Dead-letter patterns         | Error outputs / DLQ                                         |
| CDC source     | Lakeflow Connect, partner tools                | DMS CDC to S3                | ADF CDC / Datas-tream                                       |
| CDC apply      | <code>MERGE</code> / <code>DLT AUTO CDC</code> | Glue/EMR <code>MERGE</code>  | T-SQL <code>MERGE</code> / Big-Query <code>MERGE</code>     |

**Snowflake** ingests with Snowpipe (notification-driven) and applies CDC with streams/tasks or dynamic tables; **MongoDB** sources CDC naturally via change streams.



## 11.1 Week 11 Roadmap and Mental Model

Most production data arrives as files and change feeds, not clean API calls. Week 11 covers the two ingestion workhorses: **Auto Loader**, which turns a growing directory of files into an exactly-once incremental stream, and **CDC application**, which turns a log of inserts/updates/deletes into a current (SCD1) or historical (SCD2) table [45, 12].

The mental model: **ingestion is a state machine, and the checkpoint is the state**. Auto Loader’s checkpoint records which files (or offsets) have been processed; the stream itself is conceptually stateless. Every exactly-once property, every recovery behavior, and every “why did it reprocess everything” mystery traces back to that checkpoint.

### Definition

**Auto Loader** (`cloudFiles`) is Databricks’ incremental file-ingestion source: it discovers new files in cloud storage (by directory listing or cloud notifications), tracks processed files in a checkpoint, infers and evolves schemas, and rescues malformed records—delivering exactly-once ingestion with a single `.load()` call.

### Key Terms

**Notification mode**: cloud-provider events (SQS/Event Grid/GCS) announce new files. **Directory listing**: periodic listing to detect files. **Schema hints**: user-supplied type overrides layered on inference. **\_rescued\_data**: the column catching unparseable records. **SEQUENCE BY**: the ordering key resolving out-of-order CDC events. **SCD**: slowly changing dimension—Type 1 overwrites, Type 2 versions history.

## 11.2 Monday: Auto Loader Architecture

### 11.2.1 Discovery: Listing versus Notification

In **directory-listing** mode (default), Auto Loader lists the source path and diffs against the checkpoint—simple, and fine to millions of files with partitioned directory layouts. In **notification mode**, the cloud provider pushes file events to a queue Auto Loader subscribes to—required at very high file rates or deeply nested layouts where listing becomes the bottleneck [45].

### 11.2.2 Exactly-Once and the Checkpoint

Each micro-batch commits file offsets to the checkpoint before writing; on restart, processing resumes from the checkpoint. Files already committed are never reprocessed; files never committed are always processed. Consequences to internalize: the checkpoint is per-stream state—two streams cannot share one; deleting it reprocesses everything; moving it without care skips or duplicates.

### Common Pitfall

**Deleting the checkpoint to “fix” a stuck stream.** The checkpoint is the memory of what has been ingested. Deleting it means a full reprocess—into a table that already has the data, unless you also reset the target. If reprocessing is truly the goal, plan it: snapshot or version the target, reset both sides, reconcile after.

## 11.3 Tuesday: Schemas and the Rescued Column

### 11.3.1 Inference, Hints, and Evolution

Auto Loader samples files to infer a schema, persists it at `cloudFiles.schemaLocation`, and applies it consistently. **Schema hints** override inference for known-trouble columns (`cloudFiles.schemaHints`), e.g. IDs that must stay strings. Evolution: with `cloudFiles.schemaEvolutionMode` (or `mergeSchema` patterns downstream), new columns are absorbed deliberately rather than silently.

### 11.3.2 The Rescued Data Column

With `cloudFiles.rescuedDataColumn` set (default `_rescued_data` in Databricks runtimes), any record that fails parsing lands in that column as raw JSON instead of failing the batch. The pipeline survives; the malformed evidence is queryable; a steward process can reprocess or report.

```

1 stream = (spark.readStream.format("cloudFiles")
2     .option("cloudFiles.format", "csv")
3     .option("header", True)
4     .option("cloudFiles.schemaHints", "order_id STRING, amount DECIMAL(12,2)")
5     .option("cloudFiles.schemaLocation",
6         "/Volumes/de_training/bronze/_schemas/orders")
7     .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
8     .option("rescuedDataColumn", "_rescued_data")
9     .load("/Volumes/de_training/bronze/landing/orders/"))
10
11 (stream.writeStream.format("delta")
12     .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/orders")
13     .trigger(availableNow=True)          # batch economics on a stream source
14     .toTable("de_training.bronze.orders_stream"))

```

Listing 11.1: Production-shaped Auto Loader configuration.

## 11.4 Wednesday: CDC with MERGE and AUTO CDC

### 11.4.1 The MERGE Pattern

A CDC feed is a stream of (key, operation, values, sequence). The hand-rolled application is the Chapter 2 MERGE with operation awareness and an ordering guard:

```

1 MERGE INTO de_training.silver.customers AS t
2 USING staging.cdc_batch AS s
3 ON t.customer_id = s.customer_id
4 WHEN MATCHED AND s.op = 'DELETE' THEN DELETE
5 WHEN MATCHED AND s.seq > t.seq THEN UPDATE SET *
6 WHEN NOT MATCHED AND s.op <> 'DELETE' THEN INSERT *;

```

Listing 11.2: CDC application with delete handling and recency guard.

### 11.4.2 AUTO CDC for SCD

DLT's AUTO CDC (formerly `apply_changes`) packages this: declare the key, the sequence column, and the SCD type; the framework builds the correct stateful application [12].

```

1 import dlt
2 from pyspark.sql import functions as F
3
4 dlt.create_streaming_table("silver_customers_scd2")
5

```

```

6 dlt.create_auto_cdc_flow(
7     target="silver_customers_scd2",
8     source="bronze_customer_cdc",
9     keys=["customer_id"],
10    sequence_by=F.col("updated_at"),      # out-of-order safety
11    stored_as_scd_type=2,                 # full history with __START_AT/
    __END_AT
12    except_column_list=["op", "updated_at"])

```

Listing 11.3: SCD Type 2 customer history via AUTO CDC.

### Definition

**SCD Type 1** overwrites: the table shows current truth only. **SCD Type 2** versions: each change closes the old row (`__END_AT`) and opens a new one, so “what did we believe on March 3rd?” is a query, not a restore. Choose Type 2 whenever downstream joins (features, finance allocations) depend on historical attribute values.

## 11.5 Thursday Hands-on Project: Incremental Files plus SCD2 Customers

Ingest daily CSV batches with triggered Auto Loader (including a mid-stream schema change and a corrupted row), then apply a CDC feed to a Silver customer table with full Type 2 history.

```

1 # Part A: Auto Loader drift drill
2 #   1. Run the listing-mode stream from lst:ch11-autoloader (availableNow).
3 #   2. Drop batch 2 WITH a new column (loyalty_tier) -> absorbed by evolution.
4 #   3. Drop batch 3 with one malformed row -> lands in _rescued_data.
5 #   4. Query: SELECT _rescued_data FROM de_training.bronze.orders_stream
6 #           WHERE _rescued_data IS NOT NULL;
7
8 # Part B: CDC into SCD2 (DLT pipeline as in lst:ch11-autocdc)
9 #   5. Apply day 1 CDC (inserts + one update).
10 #   6. Apply day 2 CDC (another update + one delete), INCLUDING an
11 #      out-of-order event with an older updated_at -> must be ignored
12 #      by SEQUENCE BY.
13 #   7. Verify: the changed customer has exactly two rows, contiguous
14 #      __START_AT/__END_AT; the out-of-order event changed nothing.

```

Listing 11.4: Week 11 lab outline: ingestion drift drill + SCD2 verification.

### Deliverables

- The Auto Loader stream definition with schema location, hints, evolution mode, and checkpoint—all under governed volumes.
- Evidence of the evolved column and the rescued row.
- The SCD2 table showing versioned customer history with contiguous validity windows.

### Acceptance Criteria

- Batch 2’s new column appears in Bronze without code edits; batch 3’s bad row is rescued, not fatal.
- A same-checkpoint re-run ingests zero files (exactly-once proof).
- The out-of-order CDC event demonstrably does not corrupt the SCD2 history.

- The delete produces a closed interval (or Type 1 removal per the chosen flow), documented.

### Stretch Goals

- Rebuild Part B with hand-written `MERGE` instead of `AUTO CDC` and compare the code you now maintain.
- Switch Auto Loader to notification mode and document the cloud wiring (queue, events, permissions).
- Write the steward query that reports rescued rows per day, and wire it as an expectation.

## 11.6 Friday Use Case: 500 Files per Minute plus a Nightly CDC

A payments platform receives partner settlement files at ~500 small files/minute around the clock, and a nightly CDC extract from the core banking database. Both feed the same Silver domain. Design the ingestion layer.

### 11.6.1 Requirements and Assumptions

Settlement files are CSV, ~2 KB each, arriving to a cloud container by partner/date hour. CDC is a nightly Debezium-exported feed. Downstream finance reconciliation runs at 06:00 and must reconcile to the penny against both sources.

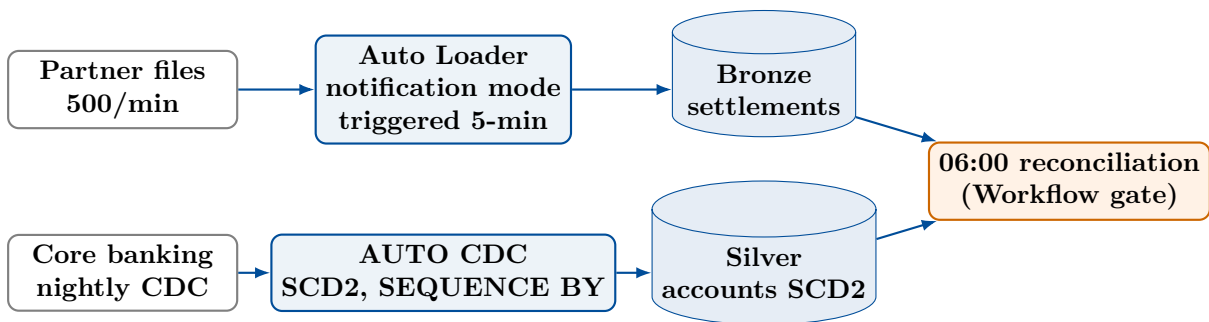


Figure 11.1: Dual-path ingestion: high-rate files via notification-mode Auto Loader, nightly CDC via SCD2, reconciled by a gated job.

### 11.6.2 Architecture Decisions

**File path.** At 500 files/minute (~700k/day), directory listing degrades—notification mode (cloud events to a queue) keeps discovery  $O(\text{new files})$ . Triggered 5-minute runs bound latency and compact naturally (Chapter 5’s small-file lesson applied at design time); the checkpoint and schema location live on governed volumes so the estate is auditable and redeployable.

**CDC path.** The nightly feed applies through `AUTO CDC` with `SEQUENCE BY updated_at`; the accounts table is SCD2 because settlement disputes replay history. The 06:00 reconciliation is a Workflow task (Chapter 12) comparing Bronze settlement sums against Silver account movements—a failing gate pages the on-call, not the CFO.

#### Common Pitfall

**One stream, two sources.** Do not multiplex the CDC feed and the file stream into one checkpointed query “for simplicity”: their rates, schemas, and recovery semantics differ.

Two streams, two checkpoints, one reconciliation gate.

11.6.3 Risks and Mitigations

| Risk                  |                  |  | Mitigation                                                             |
|-----------------------|------------------|--|------------------------------------------------------------------------|
| Checkpoint loss       | double-ingests   |  | Governed volumes, backup, and a documented reset-with-target procedure |
| Rescued data piles up | unreviewed       |  | Steward query weekly; spike alerts                                     |
| Notification queue    | expires silently |  | Queue-depth and age-of-oldest-event alerts                             |
| CDC sequence unique   | column not       |  | Uniqueness assertion on ( <b>key</b> , <b>seq</b> ) per batch          |
| Schema hints evolve   | rot as sources   |  | Hints versioned in the repo with the contract they encode              |

Table 11.1: Week 11 scenario risks and mitigations.

11.7 Design Decision Guide

Ingestion decisions set your recovery story; make them with the failure drill in mind.

| Decision                                | Choose the first when               | Choose the second when                    |
|-----------------------------------------|-------------------------------------|-------------------------------------------|
| Directory listing vs. notification mode | Moderate file counts, clean layouts | Millions of files or high arrival rates   |
| Rescued data vs. fail-on-parse          | Drifty external producers           | Strict contracted feeds                   |
| SCD Type 1 vs. Type 2                   | Current truth suffices              | History feeds features, finance, or audit |
| AUTO CDC vs. hand-written MERGE         | Standard upsert/SCD flows           | Unusual sequencing or side effects        |
| <b>availableNow</b> vs. processing-time | Batch economics on file arrival     | Continuous minute-level freshness         |
| Schema evolution vs. contract failure   | Additive, tolerated change          | Breaking change that must page someone    |

Table 11.2: Week 11 decision guide.

The two rules that override the table: one checkpoint per stream, and never delete state to fix a stall. Everything else is a judgment call you can revisit; those two are scars you cannot.

11.8 Operational Guidance for Week 11

11.8.1 Performance and Reliability

Checkpoints and schema locations are production assets: they live in governed volumes, appear in the runbook, and are covered by the same backup/DR thinking as tables. Steward the rescued-data column weekly—a spike means a producer changed shape. Trend files-ingested per run against files-arrived; divergence is a discovery lag or a stuck notification queue, not a data problem.

## 11.8.2 Security and Governance Checklist

- Landing volumes readable only by the ingestion identity and stewards.
- Notification-mode queues and their credentials scoped to the source container.
- CDC feeds carry operation semantics under contract (`op`, sequence column documented).
- Rescued data containing PII is governed to the source’s classification.

## 11.8.3 Troubleshooting Playbook

| Symptom                   | Likely cause                              | First action                                      |
|---------------------------|-------------------------------------------|---------------------------------------------------|
| Stream finds no new files | Checkpoint ahead of arrivals / wrong path | Verify source path listing; check trigger ran     |
| Everything reprocessed    | Checkpoint lost or reset                  | Confirm target reset plan before re-running       |
| Rescued-data spike        | Producer schema drift                     | Diff a rescued row against the inferred schema    |
| CDC applied out of order  | Missing/wrong <code>SEQUENCE BY</code>    | Verify the sequence column’s monotonicity per key |

Table 11.3: Week 11 troubleshooting quick reference.

## 11.9 Interview Q&A

### 11.9.1 Concepts and Trade-offs

1. **Q: How does Auto Loader achieve exactly-once?**  
A: The checkpoint durably records processed files/offsets per micro-batch commit; restarts resume from it. Exactly-once then depends on an idempotent sink—Delta commits—completing the contract.
2. **Q: Notification mode versus directory listing?**  
A: Listing is simple and robust up to large file counts with clean layouts; notification mode scales to millions of files and high arrival rates by replacing listing with cloud events—at the cost of queue setup and permissions.
3. **Q: What is the rescued-data column for?**  
A: Malformed records land there instead of failing the batch. The pipeline survives schema surprises; the evidence is queryable; reprocessing is a stewarded decision instead of a 3 a.m. incident.
4. **Q: SCD1 or SCD2 for customer dimension?**  
A: SCD2 when downstream logic depends on historical attributes (features, finance allocations, dispute replay); SCD1 when only current truth matters and history is noise. When in doubt in regulated domains: SCD2.
5. **Q: What does `SEQUENCE BY` protect against?**  
A: Out-of-order and replayed CDC events: without it, an older update can overwrite a newer one and silently regress your table state.
6. **Q: How does Auto Loader differ from a scheduled `COPY` of new files?**  
A: Discovery, exactly-once tracking, schema management, and rescued-data handling are built in—a copy script rebuilds all four badly, usually discovering it during an incident.

7. **Q: What is the first thing to check when a CDC table’s history looks wrong?**  
A: The sequence column’s uniqueness and monotonicity per key—nearly every SCD corruption is an ordering-contract violation upstream.
8. **Q: What belongs in the ingestion contract with an external producer?**  
A: Schema and compatibility rules, key and sequence semantics, arrival SLA, and the malformed-data policy. Auto Loader tolerates drift; the contract decides which drift is tolerated versus paged.

## 11.10 Further Reading

The Auto Loader documentation covers modes, schema management, and scaling [45]; the CDC guide covers AUTO CDC flows and SCD types [12]. The underlying streaming semantics are in the Structured Streaming guide [7]; Delta’s MERGE and idempotency patterns trace to Chapter 2’s foundations [53].

## Key Takeaways

- Auto Loader = discovery + checkpoint + schema management; the checkpoint is the pipeline’s memory—govern it.
- Notification mode is the scaling answer for high-rate file ingestion; listing is the simple default.
- Rescued data turns malformed records from incidents into stewarded backlog.
- CDC application is MERGE with operation awareness and a sequence guard; AUTO CDC packages it with SCD types.
- Separate sources get separate streams and checkpoints; reconciliation is an explicit, gated step.

## 11.11 Exercises

1. **(Conceptual)** Why can’t two streams share one checkpoint? What state would corrupt?
2. **(Conceptual)** Explain the failure sequence if schema inference runs without a schema location set.
3. **(Hands-on)** Complete the Thursday lab, including the out-of-order event test.
4. **(Hands-on)** Build the steward report for rescued rows and wire it as a DLT expectation.
5. **(Hands-on)** Replay the week’s files into a recovery table using a fresh checkpoint; reconcile counts against the primary.
6. **(Design)** Specify the notification-mode wiring (events, queue, permissions) for your cloud of choice.
7. **(Design)** Design the 06:00 reconciliation gate for the Friday scenario: inputs, tolerances, failure action.
8. **(Architecture)** At what arrival rate would you move the file path from triggered to continuous? Model the cost crossover.

# Workflows, Jobs, and Pipeline Orchestration

## Learning Objectives

- Compose multi-task jobs with dependency DAGs, job clusters, and schedules.
- Pass data between tasks with job parameters and task values.
- Control flow with conditional (**if/else**) and **for each** task types.
- Engineer reliability: retries, timeouts, notifications, and repair runs.
- Orchestrate DLT pipelines, notebooks, and SQL in one governed job.
- Monitor an estate of jobs with system tables and define an on-call standard.

## Cross-Cloud Equivalence

Databricks Workflows is the platform's **native orchestrator**; the concepts—DAGs, tasks, retries, SLAs—map onto every cloud's orchestrator.

| Capability       | Databricks                       | AWS                             | Azure / GCP / Fabric                  |
|------------------|----------------------------------|---------------------------------|---------------------------------------|
| Orchestrator     | Workflows (jobs + tasks)         | Step Functions / MWAA (Airflow) | ADF pipelines / Fabric Cloud Composer |
| Task compute     | Job cluster per task/task group  | Activity workers                | Activity IRs / Composer workers       |
| Parameters       | Job params + task values         | SFN input/output passing        | Pipeline params / Airflow XComs       |
| Conditional flow | if/else, for-each tasks          | Choice / Map states             | If Condition / ForEach activities     |
| Partial recovery | Repair run (failed + downstream) | Redrive / Airflow retries       | Rerun from activity / task retry      |
| API/IaC          | Jobs API, Asset Bundles          | SFN ASL / CDK / Terraform       | ARM / Bicep / Terraform               |

External orchestrators (Airflow, Dagster) still integrate via the Jobs API; the native advantage is colocation with clusters, catalogs, and lineage—one pane for run, retry, and data impact.

## 12.1 Week 12 Roadmap and Mental Model

Chapters 9–11 built the pipeline *components*; Week 12 assembles them into an *operated system*. A Workflow job is a DAG of tasks—notebooks, DLT pipelines, SQL, JARs, dbt, conditionals—



with scheduling, retries, notifications, and a repair model [25]. The engineering content is not drawing boxes; it is failure design: what happens at 03:00 when task 4 of 9 fails, and who finds out how.

The mental model: **a job is a small distributed system**. Tasks are processes with dependencies, identity (run-as principal), resources (job clusters), inputs (parameters, task values), and failure semantics (retry, timeout, downstream skip, repair). Design it like one and the 03:00 page is boring; design it like a script and the page is not.

#### Definition

**A Databricks job** is a scheduled or triggered DAG of **tasks** executed by Workflows. Each task has a type (notebook, pipeline, SQL, condition, for-each), compute (job cluster, serverless, warehouse), dependencies (**depends\_on**), and reliability settings (retries, timeout). A **repair run** re-executes only failed tasks and their dependents.

#### Key Terms

**Job parameter**: a value set at job level, shared by all tasks. **Task value**: a runtime value set by one task (`dbutils.jobs.taskValues.set`) and read by dependents. **Job cluster**: ephemeral compute created for a task or run. **Repair run**: partial re-execution of failed and downstream tasks. **Run-as**: the identity the job executes as—a service principal in production.

## 12.2 Monday: Jobs, Tasks, and Compute Topology

### 12.2.1 Anatomy of a Job

A job has: a schedule (cron or file-arrival/continuous trigger), tasks with **depends\_on** edges, per-task compute, notifications, and a run-as identity. The first design decision is compute topology: one shared job cluster for all tasks (cheap, coupled) versus a job cluster per task (isolated, slightly slower, failure-contained). Production standard in this book: *per-task job clusters for anything that can fail independently*.

### 12.2.2 Scheduling

Cron schedules handle time-based runs; **file-arrival triggers** start jobs when data lands (composing beautifully with Chapter 11’s partner files); continuous triggers keep jobs alive. Time zone and daylight-saving behavior are configuration, not folklore—set them explicitly.

#### Common Pitfall

**The shared-cluster cascade**. Six tasks on one job cluster: task 2’s memory leak kills tasks 3–6 as “failures” that were really collateral damage. Per-task clusters cost a few extra start minutes and buy you accurate failure attribution—the repair run then retries the task that actually failed.

## 12.3 Tuesday: Parameters, Task Values, and Control Flow

### 12.3.1 Static versus Runtime Values

**Job parameters** are declared at job level (e.g. `processing_date`, `environment`) and referenced by every task. **Task values** are computed at runtime and passed downstream—the row count

the ingestion task measured, the quality verdict the gate produced.

```

1 # In the quality-gate task:
2 bad = spark.sql("""
3     SELECT COUNT(*) c FROM de_training.silver.orders
4     WHERE order_id IS NULL OR total_amount < 0
5 """).first().c
6
7 if bad > 0:
8     dbutils.jobs.taskValues.set("quality", "failed")
9     raise ValueError(f"Quality gate failed: {bad} bad rows")
10 dbutils.jobs.taskValues.set("quality", "passed")
11 dbutils.jobs.taskValues.set("rows_silver",
12     spark.table("de_training.silver.orders").count())
13
14 # In a downstream task:
15 rows = dbutils.jobs.taskValues.get(task_key="quality_gate", key="rows_silver")

```

Listing 12.1: Task values: the runtime handoff between tasks.

### 12.3.2 Conditionals and For-Each

An `if/else` task evaluates a condition (often against a task value) and routes the DAG—the in-band alerting and branching mechanism. A `for each` task fans out over a list (e.g. one sub-run per partner feed) with concurrency controls—the answer to “we onboard 40 tables with one config table” without 40 hand-written DAGs.

## 12.4 Wednesday: Reliability Engineering

### 12.4.1 Retries, Timeouts, Notifications

Retries handle transient failure (with backoff); timeouts bound hung tasks; notifications (email, webhook to PagerDuty/Teams/Slack) route failure to humans. The policy question is not “retry or not” but *what is safe to retry*: idempotent tasks (Chapter 3’s discipline) retry freely; side-effecting tasks retry only after a state check.

### 12.4.2 Repair Runs

When a run fails at task 4 of 9, a **repair run** re-executes task 4 and its dependents with the same parameters—the other five tasks’ outputs are reused. Repair is the economic and operational alternative to full reruns, but it leans entirely on task idempotency: a non-idempotent task 2 “reused” by a repair is only safe because it succeeded the first time [28].

### 12.4.3 Monitoring the Estate

Job and run history live in `system.lakeflow` tables: success rates, duration trends, cost per job. The on-call standard this book adopts: every production job has an owner, a runbook link in its failure notification, and a weekly reliability review of the failure list.

## 12.5 Thursday Hands-on Project: The Orchestrated Medallion

Assemble Weeks 10–11 into one job: Auto Loader ingestion → DLT pipeline → quality-gate notebook → conditional alert, parameterized by date, with a forced failure and a repair run.

```

1 {
2     "name": "de-week12-nightly-elt",
3     "parameters": [{"name": "processing_date", "default": "{{ds}}"}],

```

```

4  "tasks": [
5    {"task_key": "ingest", "job_cluster_key": "jc",
6     "notebook_task": {"notebook_path": "src/ingest_autoloader",
7      "base_parameters": {"processing_date": "{{job.parameters
      .processing_date}}"}},
8    {"task_key": "pipeline", "depends_on": [{"task_key": "ingest"}],
9     "pipeline_task": {"pipeline_id": "<dlt-pipeline-id>"}},
10   {"task_key": "quality_gate", "depends_on": [{"task_key": "pipeline"}],
11    "job_cluster_key": "jc",
12    "notebook_task": {"notebook_path": "src/quality_gate"}},
13   {"task_key": "route", "depends_on": [{"task_key": "quality_gate"}],
14    "condition_task": {"op": "EQUAL_TO",
15     "left": "{{tasks.quality_gate.values.quality}}",
16     "right": "passed"}},
17   {"task_key": "alert", "depends_on": [{"task_key": "route", "outcome": "
18   false"}],
19   "notebook_task": {"notebook_path": "src/notify_oncall"}}
20 ],
21 "job_clusters": [{"job_cluster_key": "jc",
22  "new_cluster": {"spark_version": "15.4.x-scala2.12",
23   "node_type_id": "Standard_DS3_v2", "num_workers": 1}},
24 "schedule": {"quartz_cron_expression": "0 0 2 * * ?", "timezone_id": "UTC"}

```

Listing 12.2: Week 12 lab: job definition (JSON view of the DAG).

The drill: run the job; force the quality gate to fail (point it at a table with planted bad rows); observe the conditional alert fire; fix the data; then `databricks runs repair --run-id <id>` and capture that only failed and dependent tasks re-executed.

## Deliverables

- The job definition (UI export or JSON) with the five-task DAG.
- The forced-failure evidence: gate failure, alert task execution, notification.
- The repair-run page showing partial re-execution.

## Acceptance Criteria

- A same-day rerun (after success) changes zero Gold row counts—end-to-end idempotence.
- The forced failure pages via the conditional path, with the runbook link present.
- The repair run re-executes only the failed task and its dependents.
- The job runs as a service principal with exactly the grants from Chapter 8's matrix.

## Stretch Goals

- Add a `for each` task fanning ingestion over the nine Olist tables from a config list.
- Add a file-arrival trigger variant and compare its latency against the cron schedule.
- Emit per-task duration and row counts to a run-ledger Delta table and chart a week of runs.

## 12.6 Friday Use Case: Sixty Pipelines, Three Teams, One 6 A.M. SLA

A company runs 60 interdependent nightly pipelines across three teams with a shared 06:00 completion SLA. Today: cron-scheduled notebooks, failures discovered by customers, and monthly all-hands reruns. Design the orchestration layer.

### 12.6.1 Requirements and Assumptions

Cross-team dependencies exist (finance consumes logistics' Gold). Backfills happen weekly. Teams deploy independently. The platform team owns the on-call rotation and the SLA.

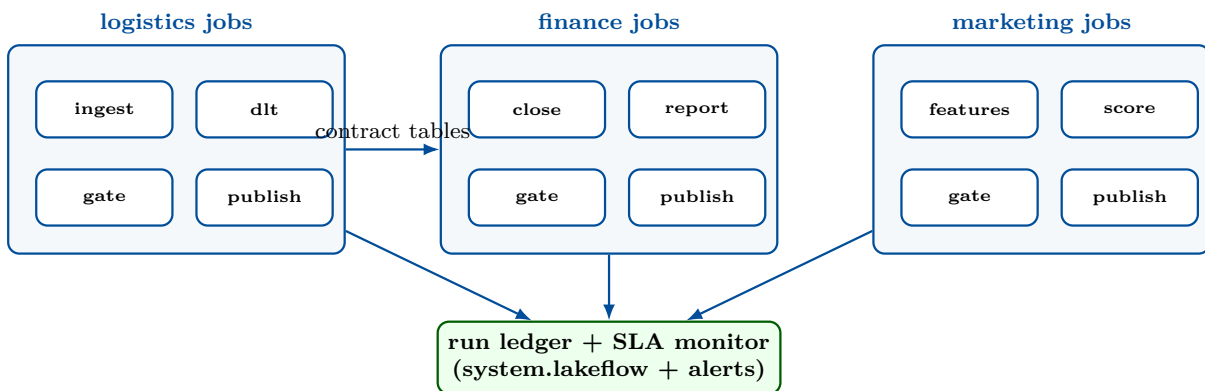


Figure 12.1: Federated orchestration: team-owned jobs, contract tables between teams, one platform-owned ledger and SLA monitor.

### 12.6.2 Architecture Decisions

**Federated jobs, contractual interfaces.** Each team owns its Workflow jobs; cross-team dependencies are *data contracts*, not task edges across team boundaries—finance’s job starts from a file-arrival trigger or a contract-table freshness check, not from a dependency on logistics’ internals. This is what lets teams deploy independently without 03:00 cross-team debugging.

**One ledger, one SLA monitor.** Every job writes run outcomes to a shared run-ledger table (task values make this cheap); the platform SLA monitor reads `system.lakeflow` plus the ledger, pages on trajectory-to-miss at 04:30 rather than on breach at 06:00, and drives the weekly reliability review. Backfills are bundle-parameterized runs (Chapter 23) against the same jobs, never hand-edited clones.

#### Common Pitfall

**Cross-team task dependencies.** Wiring team A’s task directly into team B’s job graph couples their deploys, retries, and failure pages. Depend on *published data with a freshness contract*; alert the producer when the contract breaks.

### 12.6.3 Risks and Mitigations

| Risk                                         | Mitigation                                                |
|----------------------------------------------|-----------------------------------------------------------|
| Cross-team dependency couples releases       | Contract tables with freshness SLAs, not task edges       |
| Repair run re-executes a side-effecting task | Idempotency review is a job-definition checklist item     |
| Alert fatigue from flaky tasks               | Failure budgets per job; flakiness is a reliability bug   |
| Run ledger becomes unqueryable lore          | Ledger schema versioned; weekly review uses saved queries |
| On-call pages without runbooks               | Notification template requires the runbook link to save   |

Table 12.1: Week 12 scenario risks and mitigations.

## 12.7 Design Decision Guide

Orchestration decisions are failure-shape decisions: choose based on what 03:00 looks like.

| Decision                               | Choose the first when                   | Choose the second when                      |
|----------------------------------------|-----------------------------------------|---------------------------------------------|
| Per-task vs. shared job cluster        | Tasks can fail independently            | Tiny chained tasks where start-up dominates |
| Task value vs. job parameter           | Value is computed at run-time           | Value is declared input                     |
| Conditional task vs. external alerting | The response is in-band and automatable | Humans must decide with context             |
| Workflows vs. external orchestrator    | The estate is Databricks-centric        | Orchestration spans many external systems   |
| Repair run vs. full re-run             | Tasks are idempotent                    | State is ambiguous—then fix idempotency     |
| Cron vs. file-arrival trigger          | Time-driven SLAs                        | Data-arrival-driven work                    |

Table 12.2: Week 12 decision guide.

Idempotency is the load-bearing assumption under repair runs, retries, and backfills. If you take one thing from Week 12: orchestration features are only as safe as the write patterns beneath them.

## 12.8 Operational Guidance for Week 12

### 12.8.1 Performance and Reliability

The run ledger is the estate’s health record: review failure rate, mean repair time, and p95 duration per job weekly. Trajectory alerting (elapsed + remaining-p50 vs. SLA) pages early; breach alerts write postmortems. Keep repair drills on the calendar—a repair path exercised monthly works at 03:00; one last used at launch is a rumor.

### 12.8.2 Security and Governance Checklist

- Production jobs run-as service principals with Chapter 8’s least-privilege matrix.

- Notification webhooks (PagerDuty/Teams) carry no payload data—keys and links only.
- Job parameters containing sensitive values come from secret scopes, not job config.
- Schedule time zones set explicitly and reviewed before daylight-saving transitions.

### 12.8.3 Troubleshooting Playbook

| Symptom                           | Likely cause                      | First action                                             |
|-----------------------------------|-----------------------------------|----------------------------------------------------------|
| Downstream task reads empty value | Task value key/task_key mismatch  | Log the set/get keys; check dependency edge              |
| Condition task always false       | Type mismatch in comparison       | Compare string forms; log the evaluated value            |
| Repair re-ran half the job        | Over-declared dependencies        | Tighten <code>depends_on</code> to true data edges       |
| Job fired an hour late/early      | Time zone or DST in cron schedule | Set <code>timezone_id</code> explicitly; re-check at DST |

Table 12.3: Week 12 troubleshooting quick reference.

## 12.9 Interview Q&A

### 12.9.1 Concepts and Trade-offs

1. **Q: Task values versus job parameters?**

A: Parameters are static inputs declared at job level; task values are runtime outputs of one task consumed by dependents. Confusing them is how “yesterday’s date” ends up hardcoded.

2. **Q: What makes a repair run safe?**

A: Task idempotency plus correct dependency modeling: repair reuses successful outputs and re-runs the failed subtree, so each task must produce the same effect when executed twice with the same inputs.

3. **Q: How do you alert on trajectory, not breach?**

A: Track per-task duration baselines in a run ledger; at 04:30 compare elapsed + remaining-p50 against the SLA and page early. Breach alerts tell you what you already lost.

4. **Q: Job cluster per task or one shared cluster?**

A: Per task for failure isolation and accurate attribution; shared only for tight-latency chains of small tasks where start-up dominates. State the trade-off explicitly in the job’s README.

5. **Q: When do you reach for Airflow instead of Workflows?**

A: When orchestration must span many non-Databricks systems with complex scheduling logic already expressed in Airflow—and even then, Workflows runs the Databricks tasks via the Jobs API while Airflow coordinates the perimeter.

6. **Q: How do you onboard a new pipeline into an existing job estate?**

A: As a new job with its own SLA and ledger entries, depending on contract tables—never as a task inside another team’s job.

7. **Q: What does the repair-run button assume about your code?**

A: That every task is idempotent and dependencies reflect true data edges. Repair is safe exactly to the extent your write discipline is real.

**8. Q: How would you decommission a job safely?**

A: Lineage first (who reads its outputs), then a deprecation notice in the run ledger, then schedule removal, then data-retention handling for its tables—the reverse of onboarding, with the same discipline.

## 12.10 Further Reading

The Workflows documentation covers jobs, task types, triggers, and repair [25]; the Jobs API is the automation surface for Chapter 23 [28]. System-table monitoring is documented in [34]. Revisit DLT [26] for how pipelines appear as tasks.

## Key Takeaways

- A job is a small distributed system: identity, compute, inputs, and failure semantics per task.
- Per-task job clusters buy failure isolation and honest attribution for a few start-up minutes.
- Task values carry runtime facts; parameters carry declared inputs—keep them separate.
- Repair runs are the economics of recovery; they rest entirely on idempotent tasks.
- Federate orchestration at team boundaries with data contracts and freshness checks, not cross-team task edges.

## 12.11 Exercises

1. **(Conceptual)** Diagram the difference between retry, rerun, and repair; give a scenario where each is the right response.
2. **(Conceptual)** Why does a shared job cluster corrupt failure attribution?
3. **(Hands-on)** Complete the Thursday lab, including the forced failure and repair evidence.
4. **(Hands-on)** Add the run-ledger emission to every task and produce the week-in-review query.
5. **(Hands-on)** Convert the cron schedule to a file-arrival trigger; measure arrival-to-start latency.
6. **(Design)** Write the data contract (schema, freshness SLA, quality gate) between logistics and finance from the Friday scenario.
7. **(Operations)** Draft the 04:30 trajectory alert query against the run ledger.
8. **(Architecture)** Which tasks of this job become bundle resources in Chapter 23, and which stay UI-managed? Defend the split.

## 12.12 Milestone 3 Capstone: The Orchestrated Batch ELT Platform

### Capstone Project

**Mission.** Close Part III by delivering the batch platform as one operated system: Auto Loader ingestion of daily Olist extracts, a DLT pipeline with expectations into Silver/Gold, a blocking quality gate, conditional failure alerting, and a demonstrated repair run—all parameterized and idempotent.

**Estimated time.** 6–8 hours.

**What you'll practice.**

- Triggered Auto Loader with schema evolution and rescued data.
- DLT expectations wired to real severity decisions.
- Multi-task Workflow design with task values, conditionals, and repair.
- Idempotence proof and DBU cost accounting per nightly run.

### Scenario

Contoso Retail's nightly load has grown from one CSV to nine partner feeds plus a CDC extract. The current cron-and-notebook setup double-loads on reruns and fails silently twice a month. You are replacing it with the orchestrated platform—and proving every failure mode is now loud, repairable, and reconciled.

### Requirements

- One Workflow job: ingest (Auto Loader) → DLT medallion pipeline → quality gate → publish → conditional alert path.
- Schema-evolving batch and one corrupted row handled without manual repair.
- Expectations with deliberate severities; quality metrics queryable from the event log.
- Forced mid-pipeline failure repaired (not fully rerun) with evidence.
- Same-day idempotence proof; per-run DBU cost from system tables.
- README with architecture diagram, runbook, and failure drill results.

### Reference Architecture

Reuse fig. 12.1's single-team slice: ingestion task → DLT task → gate → publish, with the conditional alert branch and the run ledger—deployed as one job with per-task job clusters.

### Implementation Steps

1. Stand up the Auto Loader volume ingestion with checkpoint and schema location under governance.
2. Convert the medallion flow to DLT with expectations (reuse Chapter 10).
3. Author the job JSON per listing 12.2; wire task values and the conditional alert.
4. Run the failure drill; repair; capture evidence.
5. Produce the cost report and the runbook; finalize the README.



**Deliverables**

- Job definition, DLT pipeline source, and the quality-gate notebook in Git.
- Failure-drill and repair-run evidence; idempotence proof.
- Cost-per-run report with the job-cluster decision documented.

**Acceptance Criteria**

- A poisoned feed demonstrably drops/fails per expectation severity, with event-log metrics.
- The repair run re-executes only the failed subtree; total rerun cost is documented.
- Gold row counts are identical across a forced same-day rerun.
- The job runs as a service principal; no credentials in any committed file.

**Stretch Goals**

- Add the `for each` fan-out over the nine feeds from a config table.
- Add the trajectory-based SLA alert from the Friday scenario.
- Wrap the job in an Asset Bundle with dev/prod targets (preview of Chapter 23).

## Part IV

# Streaming and Real-Time

# Structured Streaming Foundations

**Learning Objectives**

- Explain the infinite-table model and the micro-batch execution engine beneath it.
- Read from streaming sources (files, rate, Kafka preview) and write to Delta with checkpoints.
- Choose output modes (append, update, complete) and triggers (processing time, availableNow, continuous) deliberately.
- Operate streams: restart semantics, monitoring with listeners and metrics, and back-pressure.
- Prove a stream survives kill-and-restart without loss or duplication.
- Design a sub-minute ingestion layer for a high-rate event source.

**Cross-Cloud Equivalence**

Structured Streaming is Spark’s **streaming engine**; the same model—micro-batches or operators over an unbounded log—exists on every platform.

| Capability        | Databricks (Spark)                | AWS                               | Azure / GCP / Fabric                                       |
|-------------------|-----------------------------------|-----------------------------------|------------------------------------------------------------|
| Engine            | Structured Streaming              | Flink (Managed Service) / Lambda  | Stream Analytics / Dataflow (Beam) / Eventstream+Activator |
| Execution         | Micro-batch (continuous optional) | True streaming (Flink)            | Streaming / unified batch+stream (Beam)                    |
| Exactly-once      | Checkpoint + idempotent sink      | Flink checkpoints + 2PC sinks     | At-least-once + dedup / exactly-once (Dataflow)            |
| Replay            | Source offsets in checkpoint      | Kinesis iterators / Kafka offsets | Event Hubs checkpoints / Pub/Sub seek                      |
| Batch unification | availableNow trigger              | Flink batch mode                  | Dataflow’s core thesis                                     |
| Monitoring        | Stream metrics + listener         | CloudWatch / Flink metrics        | Azure Monitor / Dataflow UI / Fabric metrics               |

**Snowflake** approaches streaming with Snowpipe Streaming and dynamic tables (micro-batch by design); **MongoDB** with change streams feeding processors.

## 13.1 Week 13 Roadmap and Mental Model

Part III processed data that had already arrived; Part IV processes data *as* it arrives. Structured Streaming’s achievement is making that feel like batch: the same DataFrame API, the same Catalyst optimizer, the same Delta sinks—over an input that never ends [7]. Week 13 builds the execution model and the operational contract; Week 14 adds state and event time.

The mental model: **the stream is a table that grows; every trigger runs a small batch over the new rows**. Your query is planned once and executed incrementally; results land in the sink; the checkpoint remembers where the source was read to and what intermediate state exists. Understand that loop—trigger, plan, process new rows, commit offsets and state—and every streaming behavior becomes predictable.

### Definition

**Structured Streaming** treats a stream as an unbounded table. A query over it runs as a sequence of **micro-batches**: each trigger reads new source data since the last committed offset, applies the incrementally-executed query, writes the sink, and commits progress (offsets + state) to the **checkpoint**.

### Key Terms

**Output mode**: what the sink receives—**append** (new rows only), **update** (changed rows), **complete** (whole result). **Trigger**: when micro-batches run—processing-time interval, **availableNow** (run once over backlog, then stop), or continuous. **Checkpoint**: the fault-tolerance state directory. **Backpressure**: bounding batch size so processing keeps up with input.

## 13.2 Monday: The Micro-Batch Model

### 13.2.1 Sources, Incremental Execution, Sinks

A source (files via Auto Loader, Kafka, rate generator, Delta change feed) presents offsets; the engine reads a batch between the last committed offset and the current end; the query—planned once, including its stateful operators—processes incrementally; the sink (Delta in this program) commits transactionally. Because the sink write and the offset commit are coordinated through the checkpoint, a crash between them replays the batch—which is why sink idempotence (Chapter 2’s MERGE, Delta’s transactional replace) is the second half of exactly-once.

### 13.2.2 Micro-Batch versus Continuous

Micro-batch (the default) processes discrete chunks with seconds-to-minutes latency and full exactly-once support. **Continuous processing** runs long-lived operators for millisecond latency with at-least-once guarantees and a restricted operator set. The default answer in this program is micro-batch with a sane trigger; continuous is a deliberate, measured choice.

### Note

Real-time does not mean continuous. Most “real-time” business requirements—dashboards refreshing every minute, alerts within a minute—are micro-batch requirements. Reserve continuous processing for measured sub-second mandates.

## 13.3 Tuesday: Output Modes and Triggers

### 13.3.1 Choosing the Output Mode

The mode is dictated by the query’s semantics: a stateless filter *appends*; an aggregation that keeps updating a key’s value must *update* (or *complete*, rewriting the whole result—viable only for small results); aggregations over event time with watermarks append after the watermark closes the window (Chapter 14). Delta as a sink accepts append natively; update-style results use `foreachBatch + MERGE` (Chapter 16).

### 13.3.2 Choosing the Trigger

`processingTime="30 seconds"` for continuous low-latency flow; `availableNow=True` for scheduled, batch-priced streaming (process the backlog, stop, bill nothing until the next run—this is what makes Chapter 11’s triggered ingestion economical); `Trigger.Once` for one-shot backfills. The trigger is a cost-latency dial you set per pipeline, per environment.

```

1 stream = (spark.readStream.format("cloudFiles")
2     .option("cloudFiles.format", "json")
3     .option("cloudFiles.schemaLocation",
4         "/Volumes/de_training/bronze/_schemas/clicks")
5     .load("/Volumes/de_training/bronze/landing/clicks/"))
6
7 query = (stream.writeStream
8     .format("delta")
9     .outputMode("append")
10    .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/clicks")
11    .trigger(availableNow=True)           # or: processingTime="30 seconds"
12    .toTable("de_training.bronze.clicks"))
13
14 query.awaitTermination()
15 print(query.lastProgress)              # input vs processing rate, offsets

```

Listing 13.1: The streaming skeleton used across Part IV.

## 13.4 Wednesday: Checkpoints, Recovery, and Monitoring

### 13.4.1 What the Checkpoint Holds

The checkpoint directory stores: source offsets (where to resume), state-store snapshots (for stateful operators, Chapter 14), and the query’s metadata. Its rules, which this book treats as law: one checkpoint per query definition; never delete it casually (Chapter 11’s pitfall); put it on governed storage; treat incompatible query changes as requiring a new checkpoint.

### 13.4.2 Monitoring a Stream

`query.lastProgress` and the streaming UI expose input rate, processing rate, batch duration, and offsets. **Falling behind** shows as input rate durably above processing rate; **unhealthy batches** show as rising duration. The two levers: more parallelism (partitions, executors) or smaller batches (`maxFilesPerTrigger`, `maxOffsetsPerTrigger`).

#### Common Pitfall

**Changing a stateful query in place.** Adding an aggregation key or changing window logic invalidates checkpointed state; the restart fails loudly (by design) or, worse with forced config, silently mis-aggregates. Version the query, new checkpoint, and a planned backfill—the same discipline as Chapter 11’s checkpoint governance.

## 13.5 Thursday Hands-on Project: Kill It, Restart It, Prove It

Build a file-sourced streaming pipeline, run it under three triggers, then kill it mid-stream and prove—from counts, not hope—that nothing was lost or duplicated.

```

1 import uuid, time
2 from pyspark.sql import functions as F
3
4 # Producer: drop numbered JSON files into the landing path while streams run
5 def drop_batch(n, start):
6     rows = [{"event_id": str(uuid.uuid4()), "seq": i,
7              "ts": time.time()} for i in range(start, start + n)]
8     (spark.createDataFrame(rows).write.mode("append")
9      .json("/Volumes/de_training/bronze/landing/clicks/"))
10
11 drop_batch(500, 0)
12
13 q = (spark.readStream.format("cloudFiles")
14      .option("cloudFiles.format", "json")
15      .option("cloudFiles.schemaLocation", "/Volumes/de_training/bronze/_schemas/
16            /clicks")
17      .load("/Volumes/de_training/bronze/landing/clicks/")
18      .writeStream.format("delta")
19      .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/clicks")
20      .trigger(processingTime="5 seconds")
21      .toTable("de_training.bronze.clicks"))
22
23 drop_batch(500, 500)          # more data while running
24 q.stop()                     # the "crash" (mid-stream kill)
25 drop_batch(500, 1000)        # data arriving while the stream is DOWN
26
27 # Restart with the SAME checkpoint: must resume, not restart from zero
28 q2 = (spark.readStream.format("cloudFiles")
29       .option("cloudFiles.format", "json")
30       .option("cloudFiles.schemaLocation", "/Volumes/de_training/bronze/_
31             _schemas/clicks")
32       .load("/Volumes/de_training/bronze/landing/clicks/")
33       .writeStream.format("delta")
34       .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/clicks")
35       .trigger(availableNow=True)
36       .toTable("de_training.bronze.clicks"))
37
38 n = spark.table("de_training.bronze.clicks").count()
39 d = spark.table("de_training.bronze.clicks").dropDuplicates(["event_id"]).count()
40 assert n == 1500 == d, f"expected 1500 unique, got n={n} d={d}"

```

Listing 13.2: Week 13 lab: restart-safety proof.

### Deliverables

- The streaming pipeline with a governed checkpoint location.
- The kill-and-restart log: counts before stop, during downtime arrivals, after restart.
- The uniqueness assertion output proving 1500 rows, zero duplicates, zero loss.

### Acceptance Criteria

- Restart from the same checkpoint resumes—it does not reprocess committed files.
- Final table equals exactly the produced events; dedup count equals raw count.

- `lastProgress` metrics (input rate, batch duration) captured for at least 5 batches.
- Trigger comparison documented: processing-time vs. `availableNow` on the same backlog.

### Stretch Goals

- Add a `StreamingQueryListener` that logs batch metrics to a Delta table.
- Force a schema-incompatible query change; capture the restart failure and document the new-checkpoint procedure.
- Point the same source at a second sink with its own checkpoint; explain why that is safe while sharing a checkpoint is not.

## 13.6 Friday Use Case: Sub-Minute Ingestion for Ad-Tech Events

An ad-tech platform receives 50k events/second from bidder fleets. Downstream dashboards must see data within one minute; finance reconciles hourly; the events feed both real-time pacing and nightly billing.

### 13.6.1 Requirements and Assumptions

Events arrive as JSON through a managed Kafka (3 partitions per topic is the current—wrong—configuration). Peak is 3x average. One minute p95 end-to-end latency. Nightly billing runs from the same Bronze table.

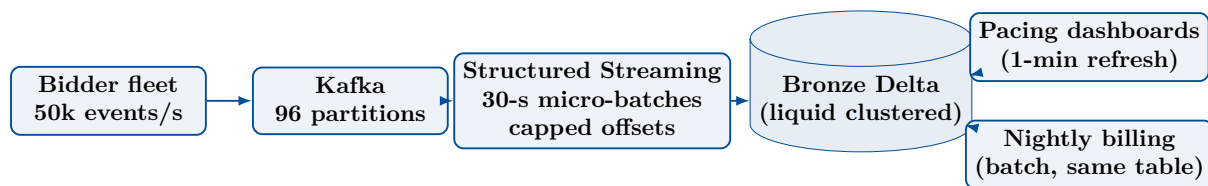


Figure 13.1: Sub-minute ingestion: partition-scaled Kafka, capped micro-batches, one Bronze table serving both latencies.

### 13.6.2 Architecture Decisions

**Parallelism first.** 50k/s over 3 Kafka partitions cannot feed enough Spark tasks; the design scales to 96 partitions (peak headroom 3x) so each micro-batch parallelizes. `maxOffsetsPerTrigger` caps batch size so a peak burst stretches latency slightly instead of OOMing executors—backpressure by configuration, monitored by the input-vs-processing-rate metric.

**One table, two consumers.** Bronze is liquid-clustered on `(event_hour, campaign_id)`; pacing dashboards read the last hour (a small, clustered slice); nightly billing scans by hour partition. Micro-batches at 30 seconds with coordinated predictive optimization keep file sizes healthy (Chapter 5’s lesson, streaming edition).

#### Common Pitfall

**Measuring latency at the dashboard only.** End-to-end latency is producer-to-Bronze plus dashboard refresh. Instrument both: `lastProgress` batch lag for the stream, refresh timestamp for the dashboard. A “slow pipeline” was, in the real incident this mirrors, a dashboard refreshing every 5 minutes against a healthy 30-second stream.

### 13.6.3 Risks and Mitigations

| Risk                                   | Mitigation                                                                      |
|----------------------------------------|---------------------------------------------------------------------------------|
| Peak burst triples normal rate         | Partition headroom and capped offsets; lag-based autoscaling review             |
| Retention gap truncates history        | Replay drills sized to retention; <code>failOnDataLoss</code> policy documented |
| Dashboard looks real-time but is stale | Freshness tile on every real-time dashboard                                     |
| Stream failure pages the wrong team    | Ownership tags on checkpoints and jobs; alert routing tested                    |
| Billing reads the same table mid-write | Delta snapshot isolation; documented writer slices                              |

Table 13.1: Week 13 scenario risks and mitigations.

## 13.7 Design Decision Guide

Streaming’s first decisions are semantic; the configuration follows from them.

| Decision                                      | Choose the first when         | Choose the second when                  |
|-----------------------------------------------|-------------------------------|-----------------------------------------|
| Append vs. update output mode                 | Event facts, new rows only    | Keyed results that evolve               |
| <code>availableNow</code> vs. processing-time | Minutes-freshness, cost-aware | Continuous low-latency flow             |
| Micro-batch vs. continuous                    | Nearly everything             | Measured sub-second mandates            |
| New checkpoint vs. reuse                      | Query semantics changed       | Never reuse across definitions          |
| Delta sink vs. external sink                  | Lakehouse consumers           | Operational systems needing push        |
| <code>failOnDataLoss</code> true vs. false    | Gaps must halt the pipeline   | Retention gaps are tolerable and logged |

Table 13.2: Week 13 decision guide.

Write the latency requirement down before choosing a trigger—“real-time” unquantified becomes the most expensive setting by default. The checkpoint governance rules from the chapter override every convenience.

## 13.8 Operational Guidance for Week 13

### 13.8.1 Performance and Reliability

Alert on the two rates—input versus processing—and on batch-duration trend; a stream that takes 25 seconds per 30-second trigger today falls over at the next traffic step. Back up checkpoint directories with the same care as tables (they are the recovery state), and document the new-checkpoint procedure (query version bump + backfill plan) before you need it. Upgrade runtimes against a copy of the stream first: state-store formats evolve.



### 13.8.2 Security and Governance Checklist

- Checkpoints and schema locations in governed volumes, one per query, named in the runbook.
- Source credentials (Kafka, cloud notifications) from secret scopes only.
- Sink table writes restricted to the streaming service principal.
- `failOnDataLoss` choice documented per stream with the retention window it assumes.

### 13.8.3 Troubleshooting Playbook

| Symptom                         | Likely cause                              | First action                                                                    |
|---------------------------------|-------------------------------------------|---------------------------------------------------------------------------------|
| Stream falls behind over hours  | Parallelism or batch-size bound           | Check rates in <code>lastProgress</code> ; partitions, then cap, then executors |
| Restart fails after code change | State schema incompatible with checkpoint | New checkpoint + planned backfill; never force the flag casually                |
| Duplicate rows after crash      | Sink not idempotent for replays           | MERGE/replace sink; verify checkpoint-sink pairing                              |
| Batch 0 huge, then normal       | Backlog on first start                    | Expected; cap with <code>maxFiles/OffsetsPerTrigger</code>                      |

Table 13.3: Week 13 troubleshooting quick reference.

## 13.9 Interview Q&A

### 13.9.1 Concepts and Trade-offs

1. **Q: Explain micro-batch execution to a backend engineer.**

A: The stream is an infinite table; each trigger runs a small incremental batch query over rows since the last committed offset, writes transactionally, and commits offsets to the checkpoint. Latency is the trigger interval plus batch time.

2. **Q: What exactly does the checkpoint make safe?**

A: Restart position (offsets), stateful-operator state, and query metadata. Combined with an idempotent/transactional sink, you get exactly-once; without the sink half you get at-least-once with duplicates.

3. **Q: Append versus update output mode?**

A: Append for event facts; update for keyed results that change over time (and then only to sinks that accept updates, or via `foreachBatch` MERGE into Delta); complete rewrites the whole result—small dimension-style outputs only.

4. **Q: When is `availableNow` the right trigger?**

A: Whenever the freshness requirement is minutes-to-hours: you get streaming sources with batch economics—process the backlog, stop, pay nothing until the next schedule.

5. **Q: A stream keeps up off-peak but falls behind at peak. Options?**

A: Check batch duration composition first: add source partitions (parallelism), cap batch size (backpressure), increase executors, or remove an accidental wide transformation. Scale the bottleneck, not the whole pipeline.

6. **Q: How does Structured Streaming differ from “running a batch job every minute”?**

A: The checkpointed offsets, incremental state, and exactly-once composition—a cron batch re-reads, re-processes, and re-writes; the stream processes each record once and remembers it did.

7. **Q: What metric would you graph on a stream’s dashboard first?**

A: Input rate versus processing rate per batch—it is the earliest, cleanest leading indicator of every streaming failure mode.

8. **Q: What is the smallest safe streaming change process?**

A: Copy the query, new checkpoint, replay into a recovery table, reconcile, then cut over. The query-under-test never shares state with the one in production.

## 13.10 Further Reading

The Structured Streaming Programming Guide is the reference [7]; the Kafka integration guide previews Chapter 15 [6]. Delta-as-sink semantics are covered in the Delta documentation [53, 46], and Auto Loader (Chapter 11) is the file source used throughout [45].

## Key Takeaways

- Structured Streaming is incremental batch over an unbounded table—same API, same optimizer, same sinks.
- Exactly-once = checkpoint (offsets + state) plus an idempotent sink; neither half suffices alone.
- Output mode follows query semantics; trigger choice is a cost-latency dial.
- One checkpoint per query; governed locations; new checkpoint for incompatible changes.
- Monitor input rate versus processing rate; fix parallelism and batch size before hardware.

## 13.11 Exercises

1. **(Conceptual)** Why does exactly-once require cooperation between checkpoint and sink?
2. **(Conceptual)** Which output mode can a grouped aggregation use with a Delta sink, and why is `complete` usually wrong at scale?
3. **(Hands-on)** Complete the Thursday lab, including the 1500-row uniqueness proof.
4. **(Hands-on)** Add `maxFilesPerTrigger` and observe the latency/throughput trade-off over a synthetic burst.
5. **(Hands-on)** Write the listener-based metrics table and chart batch duration over 20 batches.
6. **(Design)** Size Kafka partitions and trigger interval for the Friday scenario at 3x peak; show the arithmetic.
7. **(Operations)** Write the alert rule for “processing rate < 80% of input rate for 10 minutes.”
8. **(Architecture)** When would you choose DLT streaming tables (Chapter 10) over this hand-built stream? What changes operationally?

# State Management, Watermarks, and Exactly-Once Semantics

## Learning Objectives

- Distinguish event time from processing time and choose windows: tumbling, sliding, session.
- Configure watermarks to bound lateness and state size.
- Implement stateful deduplication and aggregations without unbounded state.
- Select and operate the RocksDB state store for large state.
- Compose checkpoint + idempotent sink into an end-to-end exactly-once argument.
- Design a late-tolerant, replay-safe fraud stream.

## Cross-Cloud Equivalence

Event-time state management is **the** hard part of streaming on every platform; the vocabulary transfers directly.

| Concept            | Databricks (Spark)                      | AWS (Flink)                      | Azure / GCP / Fabric                                |
|--------------------|-----------------------------------------|----------------------------------|-----------------------------------------------------|
| Event-time windows | <code>window()</code> , session windows | Tumbling/sliding/session windows | ASA windows / Beam windows / KQL <code>bin()</code> |
| Lateness control   | <code>withWatermark</code>              | Watermarks + allowed lateness    | ASA late-arrival policy / Beam watermarks           |
| Dedup              | <code>dropDuplicates</code> + watermark | Keyed state + TTL                | <code>arg_max</code> (KQL) / Beam state             |
| Custom state       | <code>flatMapGroupsWithState</code>     | Keyed process functions          | Beam stateful DoFns                                 |
| State backend      | HDFS-compatible / RocksDB               | RocksDB / heap                   | Managed / Beam state                                |
| State TTL          | Watermark-driven eviction               | State TTL config                 | TTL / watermark-driven                              |

The deepest contrast is philosophical: Spark and Flink make you *design* state (TTLs, backends, watermarks); managed engines (Dataflow, KQL) hide more of it—and give you less to tune when it breaks.

## 14.1 Week 14 Roadmap and Mental Model

Week 13’s streams were stateless filters and appends. The interesting questions—“revenue per 5-minute window,” “have we seen this event before,” “sessions per user”—are **stateful**: the answer for new data depends on remembered old data. Week 14 is about remembering the right things for the right amount of time, and no more [7].

The mental model: **watermark = state budget**. A watermark declares “events later than  $t - w$  are too late to matter.” That one declaration lets the engine close windows, emit results, and—critically—*delete* the state for closed keys. Without it, every stateful query is a slow-motion memory leak; with it, state size is bounded by the watermark horizon times the arrival rate.

### Definition

**Event time** is when the event happened (a timestamp in the data); **processing time** is when the engine sees it. A **watermark** is a moving threshold on event time: events older than  $\max(\text{event\_time}) - \text{delay}$  are dropped, and state older than that horizon is evicted.

### Key Terms

**Tumbling window**: fixed, non-overlapping intervals. **Sliding window**: fixed length, overlapping by slide. **Session window**: activity-gap-bounded, dynamic length. **State store**: where operator state lives—default in-memory + write-ahead, or RocksDB for large state. **Exactly-once**: checkpoint + idempotent sink + deterministic state.

## 14.2 Monday: Event Time and Windows

### 14.2.1 Why Event Time

Processing-time windows answer “what did we see between 10:00 and 10:05?” Event-time windows answer “what happened between 10:00 and 10:05?”—the question every business stakeholder is actually asking. The difference materializes the moment data is late, replayed, or reprocessed: processing-time answers change; event-time answers are stable [7].

### 14.2.2 Window Types

```

1 -- Tumbling: each event in exactly one 5-minute bucket
2 SELECT window(event_ts, '5 minutes') w, COUNT(*) FROM clicks GROUP BY w;
3
4 -- Sliding: 10-minute windows computed every 2 minutes (each event in 5 windows)
5 SELECT window(event_ts, '10 minutes', '2 minutes') w, COUNT(*) FROM clicks
6     GROUP BY w;
7
8 -- Session: window closes after 10 minutes of inactivity per user
9 SELECT session_window(event_ts, '10 minutes') s, user_id, COUNT(*)
10 FROM clicks GROUP BY user_id, s;
```

Listing 14.1: The three window shapes over the same stream.

### Common Pitfall

**Sliding-window multiplication surprise.** A 10-minute window with a 2-minute slide emits every event into five windows. Dashboards summing across windows then report 5x revenue. Slide deliberately, and document the overlap wherever the result is consumed.

## 14.3 Tuesday: Watermarks

### 14.3.1 The Contract

`withWatermark("event_ts", "10 minutes")` tells the engine: track the maximum event time seen; events arriving with `event_ts` older than  $max - 10$  minutes are dropped; window state older than the horizon may be evicted. In append mode, a window's result is emitted only when the watermark passes its end—latency is the watermark delay; in update mode, partial results emit continuously and finalize at the watermark.

### 14.3.2 Choosing the Delay

The watermark delay is a business decision wearing technical clothes: how late does real data arrive (measure the p99 lateness of your sources), and what is the cost of dropping later data versus waiting longer? Set it from measured lateness plus margin; monitor dropped-late-event counts; alert when they rise—your sources changed, not your logic.

#### Tip

Measure lateness empirically before setting the watermark: `SELECT percentile_approx(processing_ts - event_ts, 0.99)` over a representative day. The number you guess in a design doc is wrong; the p99 you measured is a spec.

## 14.4 Wednesday: Stateful Operators and the State Store

### 14.4.1 Deduplication and Aggregations

`dropDuplicates(["event_id"])` keeps per-key state of seen IDs—*with a watermark*, state for IDs older than the horizon is evicted; without one, it grows forever. Windowed aggregations behave the same way. The pattern that keeps every stateful pipeline in this book safe: **watermark first, then the stateful operator**.

### 14.4.2 Arbitrary State and RocksDB

`flatMapGroupsWithState` (Scala) and `applyInPandasWithState` (Python) give full control: per-key state with explicit timers—the tool for sessionization, pattern detection, and the fraud rule in the capstone. When state outgrows executor memory (high key cardinality  $\times$  long watermark), switch the state store to **RocksDB**: state spills to local disk/SSD with changelog checkpointing, at some latency cost per batch [38].

```

1 from pyspark.sql import functions as F
2
3 clean = (events
4     .withWatermark("event_ts", "10 minutes")           # state budget FIRST
5     .dropDuplicates(["event_id"]))                     # then the stateful op
6
7 windowed = (clean
8     .groupBy(F.window("event_ts", "5 minutes"), "card_id")
9     .agg(F.count("*").alias("txns"),
10         F.countDistinct("city").alias("cities"),
11         F.sum("amount").alias("total")))

```

Listing 14.2: Watermark-first dedup feeding a windowed aggregation.

## 14.5 Thursday Hands-on Project: Late, Duplicated, and Out of Order

Build a windowed aggregation over synthetic IoT events, inject late and duplicate records, and prove the watermark and dedup behave exactly as configured; then compare the default and RocksDB state stores.

```

1 from pyspark.sql import functions as F
2
3 events = (spark.readStream.format("rate")
4     .option("rowsPerSecond", 100).load()
5     .withColumn("event_id", F.expr("uuid()"))
6     .withColumn("device_id", (F.rand(1) * 50).cast("int"))
7     .withColumn("event_ts",
8         # inject jitter: some events arrive up to 15 min "late"
9         F.col("timestamp") -
10        F.expr("INTERVAL 1 SECOND") * (F.rand(2) * 900).cast("int")))
11
12 agg = (events
13     .withWatermark("event_ts", "10 minutes")
14     .dropDuplicates(["event_id"])
15     .groupBy(F.window("event_ts", "5 minutes"), "device_id")
16     .agg(F.count("*").alias("readings")))
17
18 q = (agg.writeStream.format("delta")
19     .outputMode("append")
20     .option("checkpointLocation", "/Volumes/de_training/silver/_ckpt/
    iot_windows")
21     .trigger(processingTime="30 seconds")
22     .toTable("de_training.silver.iot_windows"))
23
24 # Observe: rows with jitter <= 10 min land in their EVENT-TIME window;
25 # rows later than 10 min are dropped (count them via a listener metric).
26 # RocksDB variant:
27 spark.conf.set("spark.sql.streaming.stateStore.providerClass",
28     "com.databricks.sql.streaming.state.RocksDBStateStoreProvider")

```

Listing 14.3: Week 14 lab: lateness, duplication, and state backends.

### Deliverables

- The watermarked, deduplicated window pipeline writing to Delta.
- Evidence that in-watermark late events land in the correct event-time window and beyond-watermark events are dropped.
- A state-store comparison: operator state size and batch duration, default vs. RocksDB.

### Acceptance Criteria

- A replayed batch produces zero duplicate output rows.
- Dropped-late-event counts are measured (listener or metric), not assumed.
- State size is bounded and demonstrably stops growing after keys age past the watermark.
- The RocksDB trade-off (larger state headroom, slower per-batch) is quantified in one paragraph.

## Stretch Goals

- Implement a session-window variant and compare event-count semantics with tumbling.
- Build the same aggregation in update mode and MERGE results into a serving table via `foreachBatch`.
- Inject a source burst that doubles state size and chart the state-store behavior under load.

## 14.6 Friday Use Case: Fifteen Minutes of Lateness, Thirty Seconds of Patience

A card network’s fraud stream must tolerate up to 15 minutes of late events (intermittent regional links), deduplicate replays exactly, and still emit alerts within 30 seconds of event time for on-time data.

### 14.6.1 Requirements and Assumptions

95% of events arrive within 5 seconds; p99.9 lateness is 14 minutes. Replays occur on link flaps. Alerts drive automated card holds—a duplicate alert is a customer-facing incident; a missed 20-minute-late event is acceptable.

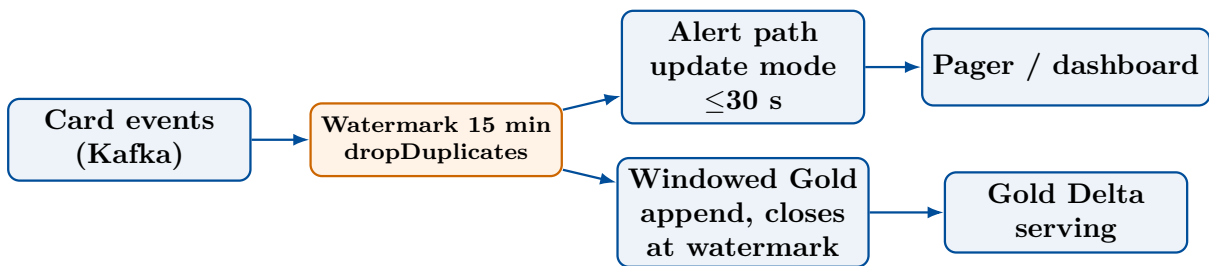


Figure 14.1: Split-brain lateness design: fast alerts in update mode, settled windows in append mode, one watermark over both.

### 14.6.2 Architecture Decisions

**One watermark, two consumers.** The 15-minute watermark (measured p99.9 + margin) bounds dedup and window state. The alert path runs in *update* mode off the deduplicated stream—on-time events alert in seconds, without waiting for windows to close. The Gold path runs in append mode: settled, exactly-once windowed aggregates for analysts and model features.

**Replay safety.** Deduplication is keyed on the network’s unique `event_id` *before* any rule evaluation—replay after a link flap re-sees the IDs, drops them, and emits zero duplicate alerts. The alert sink writes with the event ID as its key, making even the last hop idempotent.

#### Common Pitfall

**One mode for both paths.** Append mode delays alerts until windows close (15+ minutes late—useless for fraud); update mode into Gold forces you to manage changing aggregates by hand. The split—update for alerts, append for settled analytics—is the design, not a compromise.

### 14.6.3 Risks and Mitigations

| Risk                                    | Mitigation                                                     |
|-----------------------------------------|----------------------------------------------------------------|
| Fraud alert duplicates page customers   | Dedup before rules; keyed idempotent alert sink                |
| Watermark drops legitimate late regions | Lateness measured per region; per-source watermarks if needed  |
| State store grows past memory silently  | State-size alert; RocksDB migration runbook ready              |
| Replay storm after a link outage        | Capped triggers; replay into recovery table first              |
| Window definition changes mid-quarter   | Versioned query definitions; new checkpoint with backfill plan |

Table 14.1: Week 14 scenario risks and mitigations.

## 14.7 Design Decision Guide

Stateful-streaming decisions trade memory, lateness, and correctness; write the budget down.

| Decision                                       | Choose the first when                   | Choose the second when                |
|------------------------------------------------|-----------------------------------------|---------------------------------------|
| Tight vs. wide watermark                       | Freshness matters, sources are punctual | Late data is common and valuable      |
| <code>dropDuplicates</code> vs. windowed dedup | Global event-ID uniqueness              | Per-window duplicates suffice         |
| RocksDB vs. default state store                | State exceeds comfortable memory        | Small state; lowest latency per batch |
| Update vs. append mode                         | Alerts must not wait for window close   | Settled aggregates for analytics      |
| Session vs. tumbling windows                   | Activity bursts define the unit         | Calendar defines the unit             |
| Event time vs. processing time                 | Business truth, replay safety           | Pure operational telemetry            |

Table 14.2: Week 14 decision guide.

The watermark row is the one you will revisit most: set it from measured lateness, monitor the drops, and treat changes as business decisions with an owner, not config tweaks.

## 14.8 Operational Guidance for Week 14

### 14.8.1 Performance and Reliability

Monitor state size per operator (memory metrics in the streaming UI) with a growth alert; unbounded growth means a missing watermark, not a slow leak. Track dropped-late-event counts as a business metric—when it rises, a source got slower, and the watermark conversation belongs with the data owner. Review watermark values quarterly against measured p99 lateness; sources drift.

### 14.8.2 Security and Governance Checklist

- State and checkpoints (which contain actual row data) classified like the tables they feed.



- Watermark/dedup keys documented; they are the replay-safety contract.
- RocksDB changelog checkpoints land on governed storage.
- Late-drop policy (what lateness is acceptable) written down and owned by the business.

### 14.8.3 Troubleshooting Playbook

| Symptom                          | Likely cause                         | First action                                           |
|----------------------------------|--------------------------------------|--------------------------------------------------------|
| State grows without bound        | Stateful op without watermark        | Add watermark; plan a state-reset deployment           |
| Late events missing from results | Watermark tighter than real lateness | Measure p99 lateness; widen with a data-owner sign-off |
| Batches slow with large state    | Default state store spilling         | Move to RocksDB; verify changelog checkpointing        |
| Session windows never close      | Gap longer than activity bursts      | Check the gap value against measured inter-event times |

Table 14.3: Week 14 troubleshooting quick reference.

## 14.9 Interview Q&A

### 14.9.1 Concepts and Trade-offs

1. **Q: What does a watermark actually do?**

A: Two things: defines the lateness horizon after which events are dropped, and authorizes the engine to evict state older than the horizon—turning unbounded memory into a bounded budget.

2. **Q: Why is dropDuplicates dangerous without a watermark?**

A: Deduplication must remember every key it has ever seen; without eviction, state grows until the executors die. The watermark defines how long “seen” must be remembered.

3. **Q: Event time versus processing time—who cares?**

A: Anyone who replays data or has late arrivals. Event-time results are stable under reprocessing; processing-time results change with the clock. Billing, fraud, and SLAs are event-time questions.

4. **Q: When do you move to the RocksDB state store?**

A: When state size (keys  $\times$  per-key state  $\times$  watermark horizon) exceeds comfortable executor memory: RocksDB spills to disk with changelog checkpoints, trading per-batch latency for headroom.

5. **Q: Argue exactly-once for this pipeline, end to end.**

A: Source offsets checkpointed per batch; state deterministically derived from input and checkpointed; sink writes idempotent (keyed MERGE or Delta replace). A crash replays a batch into a sink that ignores replays—observable result: exactly-once.

6. **Q: A stakeholder wants “no late data dropped, ever.” Response?**

A: Then state is unbounded and windows never close—the honest design is a long watermark plus a batch reconciliation path for the long tail, which is what Chapter 16’s two-tier pattern provides.

7. **Q: How do you size the watermark for a source you have never measured?**

A: You do not—you measure first with a passive stream and a lateness histogram, then set the delay from p99 plus margin. Guessing is how silent data loss gets configured.

8. **Q: How do you test a watermark change before deploying it?**

A: Replay a representative day (including its lateness profile) into a scratch table with both watermark values and diff the window outputs—the comparison is the deployment review.

## 14.10 Further Reading

The Structured Streaming guide’s windowing and watermarking sections are the reference [7]; the RocksDB state store is covered in [38]. For the exactly-once composition with Delta sinks, revisit [46]; for the Kafka offset side, [6] (Chapter 15 expands it).

## Key Takeaways

- Event time is the business’s clock; processing time is the machine’s—build windows on the former.
- A watermark is a state budget: it bounds memory and defines “too late.”
- Watermark first, then the stateful operator—every time, no exceptions.
- RocksDB buys state headroom at per-batch latency; measure before and after.
- Exactly-once is a three-part composition: checkpointed offsets, deterministic state, idempotent sink.

## 14.11 Exercises

1. **(Conceptual)** Explain why a sliding window with a 2-minute slide over 10-minute windows inflates a naive total by 5x.
2. **(Conceptual)** Why must “allowed lateness” be finite for any stateful stream?
3. **(Hands-on)** Complete the Thursday lab with both state stores; capture the comparison.
4. **(Hands-on)** Measure your source’s lateness distribution (p50/p99/p99.9) and set the watermark from the measurement.
5. **(Hands-on)** Implement the update-mode alert path from the Friday scenario with a keyed MERGE sink.
6. **(Design)** Choose window types for: hourly revenue, 5-minute error-rate alerting, user session duration. Justify each.
7. **(Operations)** Write the alert rule for “dropped-late-event count > 0.1% of volume.”
8. **(Architecture)** When would session windows break the append-mode output contract, and what mode do you use then?

# Kafka, Event Hubs, and Kinesis Integration

## Learning Objectives

- Explain Kafka's log model: topics, partitions, consumer groups, and offset semantics.
- Connect Structured Streaming to Kafka with authentication and secure credential handling.
- Use the Event Hubs Kafka endpoint and the Kinesis source with their platform-specific options.
- Control throughput with `maxOffsetsPerTrigger`, `startingOffsets`, and partition sizing.
- Deserialize JSON and Avro (schema registry) safely under schema evolution.
- Execute a broker migration with replay-based reconciliation.

## Cross-Cloud Equivalence

Kafka is the **lingua franca** of streaming backbones; each cloud's managed bus either speaks Kafka or offers a close analog.

| Capability        | Kafka                             | AWS                            | Azure / GCP / Fabric                                |
|-------------------|-----------------------------------|--------------------------------|-----------------------------------------------------|
| Managed offering  | MSK / Confluent                   | MSK; Kinesis Data Streams      | Event Hubs (Kafka endpoint) / Pub/Sub / Eventstream |
| Scale unit        | Partition                         | Shard (Kinesis)                | Throughput unit / partitions                        |
| Consumer model    | Consumer groups + offsets         | Iterator per shard             | Consumer groups / subscriptions                     |
| Retention         | Configurable per topic            | 24 h–365 d (Kinesis)           | 1–90 d / Pub/Sub retention                          |
| Schema governance | Schema Registry (Avro/Proto/JSON) | Glue Schema Registry           | Azure Schema Registry / Pub/Sub schemas             |
| Spark source      | <code>kafka</code> format         | <code>kinesis</code> connector | Kafka endpoint / Pub/Sub connectors                 |

**Snowflake** and **MongoDB** sit downstream of these buses (Snowpipe Streaming, Kafka Connector; Atlas Stream Processing, change streams as a source).

## 15.1 Week 15 Roadmap and Mental Model

Weeks 13–14 streamed from files and synthetic sources. Production streams come from message buses, and Apache Kafka is the one you will meet most—directly, or through Azure Event Hubs’ Kafka-compatible endpoint, or alongside Amazon Kinesis in AWS estates [2, 6]. Week 15 is the integration chapter: connections, security, throughput control, schema discipline, and replay.

The mental model: **Kafka is a durable, partitioned, replayable log; your stream is a cursor.** Producers append; the log retains; consumers (identified by group) track an offset per partition. Replay, scaling, and exactly-once all follow from that: offsets are the cursor, partitions are the parallelism, retention is your replay window.

### Definition

**Apache Kafka** is a distributed commit log: producers append records to **topics** split into ordered **partitions**; **consumer groups** read partitions with each group tracking its own **offsets**. Delivery is at-least-once by default—consumers choose when to commit progress.

### Key Terms

**Partition**: the unit of ordering and parallelism. **Consumer group**: a named set of consumers sharing a topic’s partitions. **Offset**: a consumer’s position in a partition. **Schema registry**: a versioned contract store for message schemas. **maxOffsetsPerTrigger**: the per-batch volume cap—the backpressure knob.

## 15.2 Monday: The Log Model for Data Engineers

### 15.2.1 Topics, Partitions, Groups

Ordering exists only *within a partition*; parallelism exists only *across partitions*. A topic for 50k events/second with 3 partitions gives you 3 lanes no matter how big the cluster (Chapter 13’s ad-tech case). A consumer group of N readers gets each partition assigned to exactly one member—two groups reading the same topic each see every message (fan-out for free).

### 15.2.2 Offset Semantics

Kafka’s default is at-least-once: read, process, commit. Crash between process and commit → redelivery. Structured Streaming stores offsets in its checkpoint and commits after the sink write—the at-least-once delivery plus the idempotent Delta sink is what composes into exactly-once effect (Chapter 14’s argument, now with the source half explicit).

### Common Pitfall

**Deriving per-event ordering across partitions.** Events for one entity must land in one partition—key your producers by entity ID (`card_id`, `device_id`). Cross-partition ordering does not exist; a watermark cannot repair an ordering you never had.

## 15.3 Tuesday: The Spark Kafka Source, Securely

### 15.3.1 Connection and Deserialization

The Kafka source is a `format("kafka")` stream; values arrive as bytes—you deserialize. JSON with an explicit schema for simple cases; Avro/Protobuf with a schema registry for governed evolution.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.avro.functions import from_avro
3
4 kafka_user = dbutils.secrets.get("kv-lab", "kafka-user")
5 kafka_pass = dbutils.secrets.get("kv-lab", "kafka-password")
6
7 raw = (spark.readStream.format("kafka")
8       .option("kafka.bootstrap.servers", "broker1:9093,broker2:9093")
9       .option("subscribe", "card_transactions")
10      .option("kafka.security.protocol", "SASL_SSL")
11      .option("kafka.sasl.mechanism", "PLAIN")
12      .option("kafka.sasl.jaas.config",
13             f'kafkashaded.org.apache.kafka.common.security.plain.'
14             f'PlainLoginModule required username="{kafka_user}" '
15             f'password="{kafka_pass}";')
16      .option("startingOffsets", "latest")
17      .option("maxOffsetsPerTrigger", 100000)           # backpressure cap
18      .option("failOnDataLoss", "false")               # survive retention gaps
19      .load())
20
21 events = raw.select(
22     F.col("key").cast("string").alias("event_key"),
23     from_avro(F.expr("substring(value, 6)"),          # strip registry magic bytes
24              avro_schema_from_registry).alias("v"),
25     F.col("timestamp").alias("kafka_ts"))
26 .select("event_key", "v.*", "kafka_ts")

```

Listing 15.1: Kafka source with schema-registry Avro and scoped credentials.

### 15.3.2 Event Hubs via the Kafka Endpoint

Azure Event Hubs exposes a Kafka-compatible endpoint: the same source configuration with the Event Hubs connection string as the SASL password and the namespace as the bootstrap server [55]. The migration story is the headline: consumers written against Kafka move to Event Hubs with configuration, not code. (Note the Event Hubs Kafka surface does not implement every Kafka API—verify transactions and admin operations against current docs.)

## 15.4 Wednesday: Kinesis, Throughput, and Replay

### 15.4.1 The Kinesis Source

For AWS estates, the Kinesis connector reads shards with per-shard iterators [1]. The mapping: shard  $\approx$  partition, iterator  $\approx$  offset, enhanced fan-out  $\approx$  dedicated consumer throughput. Rate limits matter—`DescribeStream` and read throttles appear as consumer lag; coalesce small records and size shard count to peak, not average.

### 15.4.2 Replay and Reconciliation

Both buses retain; both let a new consumer (or a reset one) re-read history. The discipline from Chapter 11 applies: replay into a *separate* recovery table with its own checkpoint, reconcile counts against the primary, and only then consider the primary touched. `startingOffsets="earliest"` is the replay switch; `failOnDataLoss` governs behavior when retention has already expired.

**Tip**

Budget replay before you need it: re-reading 7 days of a 50k/s topic at capped offsets takes hours. Measure your replay throughput once, record it in the DR runbook, and set the recovery-time expectation from the measurement.

## 15.5 Thursday Hands-on Project: Local Kafka to Bronze, with Replay

Stand up a local Kafka in Docker, produce synthetic card transactions, consume into Bronze Delta with a capped trigger, then replay from earliest into a recovery table and reconcile.

```
1 # docker-compose: single-broker Kafka (KRaft) on localhost:9092
2 docker compose up -d
3 docker exec kafka kafka-topics --create --topic card_transactions '
4   --partitions 6 --replication-factor 1 --bootstrap-server localhost:9092
```

Listing 15.2: Week 15 lab: local broker and topic.

```
1 # producer.py (faker-seeded; run outside Spark)
2 from confluent_kafka import Producer
3 from faker import Faker
4 import json, uuid, time
5 fake = Faker(); Faker.seed(42)
6 p = Producer({"bootstrap.servers": "localhost:9092"})
7 for i in range(100_000):
8     evt = {"event_id": str(uuid.uuid5(uuid.NAMESPACE_DNS, f"evt-{i}")),
9           "card_id": f"C{fake.random_int(1,5000):05d}",
10          "city": fake.city(), "amount": round(fake.random.uniform(1,500),2),
11          "event_ts": time.time()}
12     p.produce("card_transactions", key=evt["card_id"],
13             value=json.dumps(evt))
14 p.flush()
15
16 # consumer: capped trigger into Bronze
17 stream = (spark.readStream.format("kafka")
18     .option("kafka.bootstrap.servers", "localhost:9092")
19     .option("subscribe", "card_transactions")
20     .option("startingOffsets", "latest")
21     .option("maxOffsetsPerTrigger", 1000) # visible batching
22     .load())
23 parsed = stream.select(F.from_json(F.col("value").cast("string"),
24     "event_id STRING, card_id STRING, city STRING, amount DOUBLE, event_ts
25     DOUBLE"
26     ).alias("v")).select("v.*")
27 q = (parsed.writeStream.format("delta")
28     .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/cards")
29     .trigger(processingTime="10 seconds")
30     .toTable("de_training.bronze.card_txns"))
31
32 # Replay drill: fresh checkpoint + earliest offsets into a RECOVERY table
33 replay = (spark.readStream.format("kafka")
34     .option("kafka.bootstrap.servers", "localhost:9092")
35     .option("subscribe", "card_transactions")
36     .option("startingOffsets", "earliest")
37     .option("maxOffsetsPerTrigger", 10000)
38     .load())
39 # ... same parsing, checkpoint _ckpt/cards_replay, toTable ...card_txns_replay
40 # Reconcile: COUNT(*), COUNT(DISTINCT event_id), SUM(amount) must match.
```

Listing 15.3: Producer (unique IDs) and capped consumer with replay drill.

## Deliverables

- The seeded producer, the capped streaming consumer, and the Bronze table.
- The replay table plus the three-way reconciliation output.
- A note on where credentials live (secret scope) even in the local lab.

## Acceptance Criteria

- 100k produced events all land in Bronze; no duplicates (unique-ID check).
- Replay from `earliest` reconciles exactly against the primary table.
- `maxOffsetsPerTrigger` demonstrably bounds batch size in `lastProgress`.
- No connection strings or credentials appear in committed code.

## Stretch Goals

- Repeat against an Event Hubs namespace using the Kafka endpoint; document the configuration delta.
- Add a schema-registry Avro variant and break it with an incompatible schema change; capture the failure.
- Measure replay throughput (events/second) and record the 7-day replay estimate.

## 15.6 Friday Use Case: On-Prem Kafka to Event Hubs, Zero Downtime

A retailer runs self-managed Kafka on premises and must migrate 40 topics to Azure Event Hubs while Databricks consumers keep running—no downtime, no duplicate billing events.

### 15.6.1 Requirements and Assumptions

Producers cut over gradually over two weeks (by service). Consumers must not see duplicates across the boundary. Some topics have 30-day retention needs; Event Hubs retention caps require planning (or a capture-to-ADLS pattern).

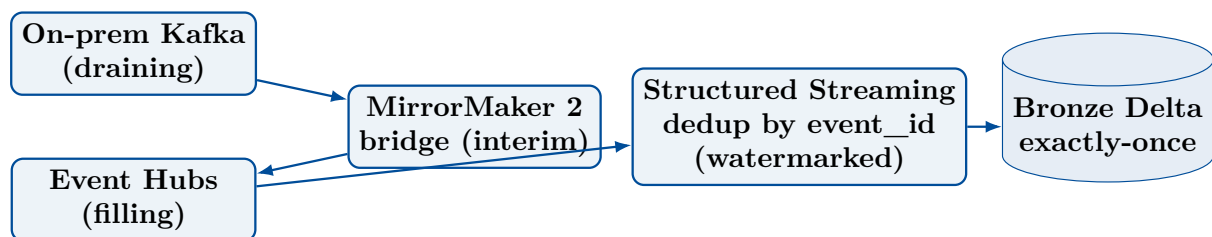


Figure 15.1: Migration topology: mirror the old bus into the new, consume both through one deduplicating stream, retire the bridge.

### 15.6.2 Architecture Decisions

**Mirror, then move.** MirrorMaker 2 replicates on-prem topics into Event Hubs during the transition; producers migrate service by service; the Spark consumers read the Event Hubs

endpoint throughout—configuration changes, not code. The dual-write window is absorbed by the stream’s watermarked `dropDuplicates` on `event_id` (Chapter 14): the same event arriving via both paths is seen once.

**Cutover gates.** Per topic: producer moved → mirror verified empty of new messages for that topic → consumer lag zero → reconciliation query (counts + sums vs. the old cluster’s log-end offsets) → topic retired. Duplicates across the boundary are impossible *by construction* only because dedup keys survived the producer migration—the one invariant the whole plan depends on.

### Common Pitfall

**Changing the key scheme during a bus migration.** The dedup key, partition key, and schema must all survive the move untouched. A “while we’re in there” key change turns a configuration migration into a data-correctness incident spanning two weeks of dual-run ambiguity.

### 15.6.3 Risks and Mitigations

| Risk                                         | Mitigation                                                  |
|----------------------------------------------|-------------------------------------------------------------|
| Migration dual-write duplicates events       | Watermarked dedup on a stable event ID across both buses    |
| Retention difference truncates replay window | Capture-to-storage pattern; replay budget pre-measured      |
| Producer key scheme changes mid-migration    | Key contract frozen during migration; CI check on producers |
| Event Hubs Kafka-surface feature gaps        | Feature checklist verified per workload before commitment   |
| Credential rotation drops consumers          | Vault-backed scopes; post-rotation smoke test automated     |

Table 15.1: Week 15 scenario risks and mitigations.

## 15.7 Design Decision Guide

Bus integration decisions are contracts: schemas, keys, and offsets outlive any single consumer.

| Decision                                | Choose the first when                   | Choose the second when                    |
|-----------------------------------------|-----------------------------------------|-------------------------------------------|
| Kafka vs. Event Hubs vs. Kinesis        | Ecosystem, portability, existing estate | Cloud-native simplicity on Azure / AWS    |
| JSON vs. Avro + registry                | Prototypes, tolerant consumers          | Governed, evolving, multi-consumer topics |
| Keyed vs. unkeyed producers             | Per-entity ordering or dedup matters    | Truly order-independent telemetry         |
| <b>earliest</b> vs. <b>latest</b> start | Backfill/replay                         | Steady-state new consumer                 |
| Cap offsets vs. uncapped                | Production, always                      | Never—uncapped is a lab setting           |
| Mirror vs. dual-write migration         | Consumers stay put during migration     | Producer fleet is small and coordinated   |

Table 15.2: Week 15 decision guide.



The invariant worth memorizing: dedup keys, partition keys, and schema contracts must survive any migration or consumer change. Those three are the stream’s public API.

## 15.8 Operational Guidance for Week 15

### 15.8.1 Performance and Reliability

Consumer lag is the top-line metric: alert on sustained lag, not instantaneous spikes. Rotate bus credentials on the vault’s schedule and smoke-test the stream after each rotation. Register schema changes through the registry with compatibility enforcement; track deserialization error rates so a bad producer deploy is visible in minutes. Measure replay throughput once and record it in the DR runbook.

### 15.8.2 Security and Governance Checklist

- JAAS/connection configuration references secret scopes—never literals.
- TLS enforced (SASL\_SSL); plaintext listeners disabled on managed offerings.
- Producer key contracts (entity ID as key) documented per topic.
- Topics carry an owner, retention, and classification tag.

### 15.8.3 Troubleshooting Playbook

| Symptom                             | Likely cause                      | First action                                                       |
|-------------------------------------|-----------------------------------|--------------------------------------------------------------------|
| Consumer lag grows steadily         | Partition ceiling or slow batches | Check batch duration; scale partitions/parallelism                 |
| Deserialization errors after deploy | Incompatible schema change        | Registry compatibility report; roll back producer                  |
| Duplicates after a replay           | Dedup key changed or missing      | Verify <code>event_id</code> continuity across the producer change |
| Event Hubs throttling               | Throughput units exhausted        | Check TU usage; raise units or smooth producer bursts              |

Table 15.3: Week 15 troubleshooting quick reference.

## 15.9 Interview Q&A

### 15.9.1 Concepts and Trade-offs

1. **Q: Why at-least-once, and where does exactly-once come from?**

A: The bus cannot know when your processing finished safely, so redelivery is always possible. Exactly-once effect is composed: checkpointed offsets + deterministic state + idempotent sink—all three.

2. **Q: What does `maxOffsetsPerTrigger` protect?**

A: Executors from oversized micro-batches (memory, GC, SLA breaches) at peak or on backlog replay—you trade a little latency for bounded, predictable batches.

3. **Q: Kafka or Kinesis?**

A: Kinesis for AWS-native simplicity and per-shard managed scaling; Kafka (MSK/self-managed) for the ecosystem (connectors, schema registry, exactly-once producers) and multi-cloud portability. Most enterprises answer “both, somewhere.”

4. **Q: Why a schema registry for streams?**

A: Schemas become versioned contracts with compatibility rules; a producer breaking change fails at registration instead of corrupting every consumer at 03:00.

5. **Q: How do consumer groups give you free fan-out?**

A: Each group tracks independent offsets, so the fraud group and the billing group each read every message—one write, N independent readers, no copies.

6. **Q: How many partitions should a topic have?**

A: Ceiling of parallelism needed at peak: peak rate divided by per-partition sustainable throughput, times headroom—measured, then rounded up, because partitions only ever get added.

7. **Q: What breaks when a schema registry enforces backward compatibility?**

A: Producers cannot deploy incompatible changes—the failure moves from “consumers corrupt at 03:00” to “producer CI fails at commit time,” which is exactly where you want it.

8. **Q: What is your answer when a producer says “just infer the schema”?**

A: Inference is a convenience for onboarding, not a contract. Production topics get registered schemas with compatibility rules; inference belongs in the sandbox where its surprises are free.

## 15.10 Further Reading

The Spark–Kafka integration guide is the configuration reference [6]; Kafka’s own documentation covers the log model and consumer semantics [2]. Event Hubs’ Kafka endpoint is documented in [55]; Kinesis in [1]. The streaming foundations this chapter builds on are in [7].

## Key Takeaways

- Kafka is a replayable partitioned log; offsets are cursors, partitions are parallelism, groups are fan-out.
- Credentials come from secret scopes; connection strings never appear in code.
- `maxOffsetsPerTrigger` is the backpressure dial; partition count is the parallelism ceiling.
- Schema registries turn message shapes into enforced, versioned contracts.
- Migrations move buses by mirroring and deduplicating—keys and schemas do not change mid-flight.

## 15.11 Exercises

1. **(Conceptual)** Why does ordering exist only within a partition, and what does that force on your producer keys?
2. **(Conceptual)** Explain `failOnDataLoss` and when `false` is the honest choice.
3. **(Hands-on)** Complete the Thursday lab including the replay reconciliation.
4. **(Hands-on)** Add a second consumer group reading the same topic into an independent table; verify both receive everything.

5. **(Hands-on)** Chart consumer lag versus `maxOffsetsPerTrigger` at three settings under a synthetic burst.
6. **(Design)** Write the per-topic cutover checklist for the Friday scenario.
7. **(Design)** Choose partition counts for 1k/s, 50k/s, and 1M/s topics; show your arithmetic.
8. **(Architecture)** When would you keep self-managed Kafka instead of Event Hubs/MSK? Name two technical reasons.

# Real-Time Lakehouse Architectures

**Learning Objectives**

- Design a streaming medallion architecture: Bronze append streams, Silver dedup and joins, Gold incremental aggregates.
- Use `foreachBatch` to apply MERGE-based sinks within exactly-once semantics.
- Implement stream-static and stream-stream joins with correct watermarks on both sides.
- Serve real-time Gold to AI/BI dashboards with explicit freshness indicators.
- Model latency and cost trade-offs between continuous, micro-batch, and triggered processing.
- Reconcile streaming and batch views of the same business truth.

**Cross-Cloud Equivalence**

The real-time lakehouse—streams landing in governed tables that also serve batch—is the converged endpoint every platform is building toward.

| Capability             | Databricks                      |           | AWS                            |            | Azure / GCP / Fabric                            |
|------------------------|---------------------------------|-----------|--------------------------------|------------|-------------------------------------------------|
| Stream-to-table        | Structured → Delta              | Streaming | Flink → Tables                 | Iceberg/S3 | Eventstream → Eventhouse / Dataflow → BigQuery  |
| Upsert sink            | <code>foreachBatch</code> MERGE | +         | Flink upsert writers           |            | BigQuery MERGE / KQL update policies            |
| Real-time serving      | Gold streaming tables + AI/BI   | +         | Kinesis Analytics → QuickSight |            | KQL + Real-Time Dashboards / BigQuery BI Engine |
| Batch+stream one table | Delta (both writers)            |           | Iceberg (Flink batch)          | +          | BigQuery streaming + batch / OneLake            |
| Reconciliation         | Batch recompute vs stream       |           | Lambda/Kappa variants          |            | Same pattern                                    |
| Latency floor          | Seconds (micro-batch)           |           | Sub-second (Flink)             |            | Sub-second (KQL) / seconds (Dataflow)           |

**Snowflake’s** version is Snowpipe Streaming into tables queried by the same engine as batch—the same architectural promise with a fully managed engine and no stream topology to design.

## 16.1 Week 16 Roadmap and Mental Model

Part IV’s previous chapters built streaming *mechanics*. Week 16 assembles them into *architecture*: a lakehouse where streams and batches write the same governed tables, real-time aggregates serve live dashboards, and a batch path reconciles the streaming view against business truth. This is the chapter where the medallion (Part I) and streaming (Part IV) become one system [7, 46].

The mental model: **streaming is a latency tier of the lakehouse, not a separate estate**. Bronze streaming tables capture events; Silver applies the same dedup/quality rules as batch, continuously; Gold maintains incremental aggregates. The batch path is not legacy—it is the reconciliation and reprocessing tier that keeps the stream honest.

### Definition

**A real-time lakehouse** is a medallion architecture whose layers are maintained continuously by streaming pipelines, with batch processing retained for backfill, reconciliation, and heavy reprocessing—one governed storage layer (Delta) serving both latency tiers.

### Key Terms

**foreachBatch**: the escape hatch running arbitrary batch logic (MERGE, fan-out) per micro-batch with checkpointed exactly-once. **Stream-static join**: enriching a stream against a slowly changing table. **Stream-stream join**: joining two streams with watermarks on both. **Latency budget**: the end-to-end freshness requirement decomposed per stage. **Reconciliation**: the batch proof that streaming Gold matches source truth.

## 16.2 Monday: The Streaming Medallion

### 16.2.1 Layer by Layer

**Bronze**: the append stream from Kafka/Auto Loader—exactly-once facts, liquid-clustered for the serving queries. **Silver**: dedup (watermarked), typing, expectations (DLT streams carry them), and stream-static enrichment against dimensions. **Gold**: incremental aggregates—windowed revenue, running counts—served directly to dashboards. The batch pipeline still exists: it recomputes Gold nightly from Bronze as the reconciliation of record.

### 16.2.2 One Table, Two Writers

Delta’s optimistic concurrency lets a stream and a batch job write the same table—if they write disjoint slices (stream owns recent data; batch owns restatements) or coordinate through keys. The practical pattern: streams write live partitions; nightly batch rewrites the same logical window with reconciled values in a `replaceWhere` scoped overwrite. Conflicts are commit-time failures you design away by slice discipline, not luck.

### Common Pitfall

**Stream and batch both upserting the same keys.** Two writers MERGE-ing the same keyspace will serialize against each other, fail commits under contention, and produce ordering-dependent state. Partition the responsibility: stream owns now, batch owns then; or make the batch the only writer and the stream a staging feed.

## 16.3 Tuesday: foreachBatch and Streaming Joins

### 16.3.1 foreachBatch: The Correct Escape Hatch

Streaming DataFrames cannot MERGE directly. `foreachBatch` hands each micro-batch to a function as a *batch* DataFrame—where MERGE, multiple sinks, and side effects are legal—while the checkpoint still guarantees the batch is processed exactly once:

```

1 def upsert_to_silver(batch_df, batch_id):
2     batch_df.createOrReplaceTempView(f"batch_{batch_id}")
3     spark.sql(f"""
4         MERGE INTO de_training.silver.orders_rt AS t
5         USING batch_{batch_id} AS s
6         ON t.order_id = s.order_id
7         WHEN MATCHED AND s.event_ts > t.event_ts THEN UPDATE SET *
8         WHEN NOT MATCHED THEN INSERT *
9     """)
10
11 (stream.writeStream
12     .foreachBatch(upsert_to_silver)
13     .option("checkpointLocation", "/Volumes/de_training/silver/_ckpt/orders_rt")
14     .trigger(processingTime="30 seconds")
15     .start())

```

Listing 16.1: foreachBatch MERGE sink: idempotent upserts from a stream.

### 16.3.2 Streaming Joins

**Stream-static** (events  $\times$  dimension): the static side is snapshotted per batch—broadcast it when small, and know that dimension changes apply to *new* micro-batches only. **Stream-stream**: both sides arrive over time; the join needs watermarks on both sides plus an event-time join constraint (`A.ts BETWEEN B.ts - INTERVAL 5 MINUTES AND B.ts`), which tells the engine when join state can be evicted. Unbounded stream-stream joins without watermarks are unbounded state.

## 16.4 Wednesday: Serving and the Latency/Cost Model

### 16.4.1 Real-Time Gold to Dashboards

Gold streaming tables serve AI/BI dashboards refreshing at decision cadence (Chapter 7): every minute for operations, not every second because the platform allows it. Add a freshness tile—`max(event_ts)` and last-write timestamp—so a stale pipeline is visible on the same dashboard that depends on it.

### 16.4.2 The Latency/Cost Dial

| Mode               | Latency               | Cost profile      | Use when                    |
|--------------------|-----------------------|-------------------|-----------------------------|
| Continuous stream  | Sub-second to seconds | Always-on compute | Measured sub-minute updates |
| Micro-batch (30 s) | ~30–60 s              | High but tunable  | Operational dashboards      |
| Triggered hourly   | Minutes-hours         | Pay per run       | Finance-grade freshness     |
| Nightly batch      | Hours                 | Cheapest          | Reconciliation, history     |

Table 16.1: The latency/cost dial: four settings, four price points.

## 16.5 Thursday Hands-on Project: The Real-Time Orders Pipeline

Build the full streaming medallion for orders: Kafka → Bronze → Silver deduped MERGE → Gold 5-minute revenue → a live AI/BI dashboard with a freshness tile.

```

1 from pyspark.sql import functions as F
2
3 silver = spark.readStream.table("de_training.silver.orders_rt")
4
5 gold5m = (silver
6     .withWatermark("event_ts", "10 minutes")
7     .groupBy(F.window("event_ts", "5 minutes"), "region")
8     .agg(F.sum("amount").alias("revenue"),
9         F.countDistinct("order_id").alias("orders")))
10
11 def publish_gold(batch_df, batch_id):
12     batch_df.createOrReplaceTempView("g")
13     spark.sql("""
14         MERGE INTO de_training.gold.revenue_5m AS t
15         USING g AS s
16         ON t.window_start = s.window.start AND t.region = s.region
17         WHEN MATCHED THEN UPDATE SET *
18         WHEN NOT MATCHED THEN INSERT *
19     """)
20
21 (gold5m.writeStream.foreachBatch(publish_gold)
22     .outputMode("update")
23     .option("checkpointLocation", "/Volumes/de_training/gold/_ckpt/revenue_5m")
24     .trigger(processingTime="1 minute")
25     .start())
26
27 # Dashboard dataset (AI/BI):
28 # SELECT *, current_timestamp() AS refreshed_at
29 # FROM de_training.gold.revenue_5m
30 # ORDER BY window_start DESC LIMIT 288 -- one day of 5-min bars

```

Listing 16.2: Week 16 lab: the streaming Gold aggregation.

### Deliverables

- The three streaming queries (Bronze, Silver MERGE, Gold aggregate) with governed checkpoints.
- The AI/BI dashboard with 5-minute revenue bars and a freshness tile.

- A nightly batch reconciliation query comparing streaming Gold against a Bronze recompute.

### Acceptance Criteria

- A Kafka replay produces zero duplicate Silver rows and zero duplicate Gold updates.
- The dashboard shows data no older than two trigger intervals under normal load.
- The reconciliation query agrees within documented tolerance (late-drop policy stated).
- Cost per day of the continuous pipeline is computed from `system.billing.usage` and compared against a triggered alternative.

### Stretch Goals

- Add a stream-stream join (orders  $\times$  payments, 5-minute constraint) with correct watermarks.
- Convert Gold to a DLT streaming pipeline and compare operational surface with the hand-built version.
- Implement the freshness alert: page when `max(window_start)` falls 3 intervals behind.

## 16.6 Friday Use Case: Ten Thousand Vehicles, One Dispatch Screen

A logistics fleet streams 10k GPS/telemetry events per second from vehicles. Dispatch needs a sub-minute live map; routing optimization retrains nightly on full history; the finance team bills customers from the same events.

### 16.6.1 Requirements and Assumptions

Events carry `vehicle_id`, position, speed, and timestamps. Dispatch decisions expire in 60 seconds. Billing must reconcile to the source of record (the event log) to the event. Overnight, batch recomputes daily Gold from Bronze.

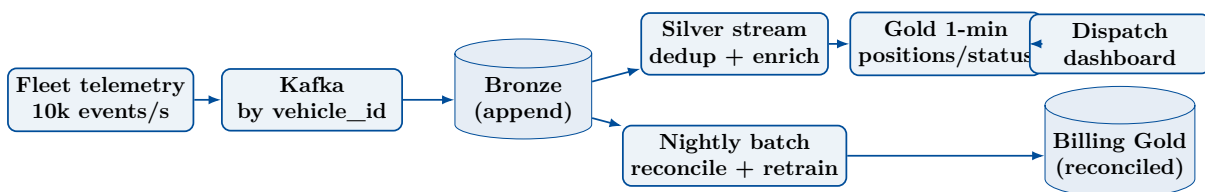


Figure 16.1: Fleet lakehouse: one Bronze log feeding a real-time dispatch tier and a reconciled nightly tier.

### 16.6.2 Architecture Decisions

**Two tiers, one truth.** The dispatch tier optimizes for freshness: 30-second micro-batches, update-mode aggregates into `gold.vehicle_status`, a 1-minute dashboard. The billing tier optimizes for correctness: nightly batch recompute from Bronze with exact dedup and windowing—its Gold is the invoice source. The streaming Gold is explicitly labeled operational; the batch



Gold is financial. They agree because they share Bronze, dedup keys, and window definitions—and a daily diff job proves it.

**Capacity and cost.** Kafka partitioned by `vehicle_id` (ordering per vehicle, parallelism for scale); the dispatch stream is the only always-on compute; everything else is triggered. The cost model (table 16.1) is reviewed monthly against the measured freshness dispatch actually uses.

### Common Pitfall

**Billing from the streaming tier.** Operational aggregates tolerate late-drop and at-least-once edge cases that invoices may not. The batch reconciliation tier is not optional overhead—it is what makes the real-time tier safe to move fast.

## 16.6.3 Risks and Mitigations

| Risk                                             | Mitigation                                                        |
|--------------------------------------------------|-------------------------------------------------------------------|
| Stream/batch divergence normalized               | Daily reconciliation diff trended and alerted, not eyeballed      |
| Dispatch screen drives a wrong hold              | Operational tier labeled; safety actions read the reconciled tier |
| <code>foreachBatch</code> grows slower with load | Per-batch duration alert; merge-condition data skipping verified  |
| Continuous trigger left on after a demo          | Trigger config in code review; weekend cost alert                 |
| Late events rewrite financial windows            | Billing reads only the batch-reconciled Gold tier                 |

Table 16.2: Week 16 scenario risks and mitigations.

## 16.7 Design Decision Guide

Real-time lakehouse decisions are about which tier answers which question.

| Decision                                      | Choose the first when                      | Choose the second when                |
|-----------------------------------------------|--------------------------------------------|---------------------------------------|
| <code>foreachBatch</code> vs. streaming table | Custom upsert logic, multi-sink            | Standard append/aggregate flows       |
| Stream-static vs. pre-joined enrichment       | Dimension changes matter going forward     | Point-in-time correctness is required |
| Continuous vs. triggered                      | Sub-minute mandate with budget             | Minutes-freshness at batch cost       |
| Operational vs. financial Gold tier           | Fast approximate answers                   | Reconciled, invoice-grade numbers     |
| Kappa vs. Lambda (batch shadow)               | Team can own one codebase and accept drift | Financial/audit truth must recompute  |
| Stream-stream vs. stream-static join          | Both sides arrive over time                | One side is reference data            |

Table 16.3: Week 16 decision guide.

The tiering question settles most rows: if the answer drives money or safety, the batch-reconciled tier owns it; if it drives a dashboard someone glances at, the streaming tier does.

## 16.8 Operational Guidance for Week 16

### 16.8.1 Performance and Reliability

The nightly reconciliation between streaming Gold and batch truth is the tier’s license to operate: automate it, trend the diff, and alert on drift beyond tolerance. Review trigger cadence monthly against the freshness the business actually consumes—latency nobody looks at is spend nobody needs. Keep the `foreachBatch` function lean; per-batch overhead (connections, catalog calls) multiplies by trigger frequency.

### 16.8.2 Security and Governance Checklist

- Dashboard publish identity scoped to the Gold serving views.
- Streaming checkpoints and reconciliation tables classified with their source data.
- The operational/financial Gold split documented—consumers know which tier answers which question.
- Freshness alerting covers every real-time dashboard (no silent staleness).

### 16.8.3 Troubleshooting Playbook

| Symptom                                | Likely cause                            | First action                                        |
|----------------------------------------|-----------------------------------------|-----------------------------------------------------|
| Stream and batch disagree              | Late-drop policy or dedup mismatch      | Diff on shared keys; reconcile watermark policies   |
| <code>foreachBatch</code> batches slow | Per-batch MERGE on degraded layout      | Optimize the target; check merge-condition skipping |
| MERGE commit conflicts                 | Batch and stream writing same key space | Enforce slice discipline (stream=now, batch=then)   |
| Dashboard fresh but wrong              | Aggregate logic drift between tiers     | Single-source the window definitions                |

Table 16.4: Week 16 troubleshooting quick reference.

## 16.9 Interview Q&A

### 16.9.1 Concepts and Trade-offs

1. **Q: Why does the stream need a batch shadow at all?**

A: Streams trade exactness for latency: watermarks drop late data, edge cases duplicate, windows close early. The nightly batch from Bronze is the reconciled truth; the stream is the fast approximation that must provably agree with it.

2. **Q: Why `foreachBatch` instead of a native MERGE sink?**

A: Streaming DataFrames do not support MERGE directly; `foreachBatch` exposes each micro-batch as a batch DataFrame—where MERGE is legal—while the checkpoint preserves exactly-once across the hand-off.

3. **Q: Stream-stream join: what makes it safe?**

A: Watermarks on *both* sides plus an event-time interval constraint, so join state is bounded and evictable; without them the join state is unbounded memory.

4. **Q: When is continuous processing justified over 30-second micro-batches?**

A: When the measured business cost of 30 seconds of staleness exceeds the DBU premium—

rare outside market-making, real-time bidding, and safety systems. Most “real-time” requirements are micro-batch requirements.

5. **Q: How do you keep the dashboard honest?**

A: A freshness tile showing `max(event_ts)` and last-write time, plus an alert when lag exceeds the budget—the dashboard that monitors its own pipeline.

6. **Q: Why not query the streaming Silver directly for billing?**

A: Because streams encode lateness and replay policy; billing needs the reconciled batch computation over immutable Bronze. Fast and correct are different tiers by design.

7. **Q: What is the cost question to ask before any continuous trigger?**

A: “What does one minute of staleness cost the business?” If the answer is unmeasurable, the continuous premium is unpayable-by-logic—choose triggered.

8. **Q: What is the migration path from a nightly batch estate to this architecture?**

A: Incremental: Auto Loader replaces file polling, `availableNow` streams replace cron reads, and only consumers with measured freshness needs graduate to micro-batch triggers. The batch tier remains the reconciler throughout.

## 16.10 Further Reading

The Structured Streaming guide covers `foreachBatch` and streaming joins [7]; Delta’s streaming integration is in [46]; Kafka source options in [6]. The medallion rationale this chapter streams into is [50], and the optimization chapter (5) explains why the streaming Bronze needs clustering and predictive optimization.

## Key Takeaways

- The real-time lakehouse is one medallion estate with two latency tiers—not a separate streaming stack.
- `foreachBatch` + MERGE is the exactly-once upsert sink; dedup keys make replays safe.
- Stream-stream joins require watermarks and interval constraints on both sides.
- Latency is a priced dial: continuous, micro-batch, triggered, batch—choose per consumer.
- The batch reconciliation tier is what licenses the streaming tier to move fast.

## 16.11 Exercises

1. **(Conceptual)** Explain why two writers to one Delta table are safe when their slices are disjoint.
2. **(Conceptual)** Why does update output mode pair naturally with `foreachBatch` MERGE?
3. **(Hands-on)** Complete the Thursday lab, including the reconciliation query and its tolerance statement.
4. **(Hands-on)** Add the freshness tile and the lag alert; force a stream stoppage and capture the alert.
5. **(Hands-on)** Run the cost comparison: one day continuous vs. hourly triggered on the same pipeline.

6. **(Design)** Write the watermark and join-constraint spec for a payments-joins-orders stream-stream join.
7. **(Operations)** Draft the incident runbook for “dispatch dashboard 10 minutes stale.”
8. **(Architecture)** Redesign the Friday scenario as Kappa (no batch tier). What breaks? What must you add to compensate?

## 16.12 Milestone 4 Capstone: The Real-Time Fraud Telemetry Platform

### Capstone Project

**Mission.** Close Part IV with a production-shaped streaming system: synthetic card transactions from Kafka into a Bronze stream, deduplicated watermarked Silver, a stateful impossible-travel rule, Gold 5-minute aggregates on a live dashboard, and a replay drill proving exactly-once with zero duplicate alerts.

**Estimated time.** 8–10 hours.

**What you’ll practice.**

- Kafka production with unique event IDs and keyed partitioning.
- Watermarked dedup before rule evaluation—the replay-safety ordering.
- `foreachBatch` MERGE sinks and stateful per-card logic.
- Live dashboards with freshness telemetry and a measured latency budget.

### Scenario

Meridian Bank’s card fraud team currently reviews alerts batched hourly. You are delivering the real-time platform: flag a card used in two cities within 10 minutes, alert within 30 seconds of the second event, and give analysts a live windowed view—with zero tolerance for duplicate alerts under replays.

### Requirements

- Seeded Kafka producer (6 partitions, keyed by `card_id`),  $\geq 100k$  events including planted impossible-travel cases and a replayable ID space.
- Bronze append stream with governed checkpoint; Silver dedup with a 10-minute watermark.
- Impossible-travel detection (stateful per-card or windowed `countDistinct(city)  $\geq 2$` ) feeding a keyed alert sink.
- Gold `revenue_5m`-style aggregates + AI/BI dashboard with freshness tile.
- Replay drill: full topic replay produces zero duplicate alerts and zero duplicate Silver rows.
- Cost report: continuous vs. triggered comparison with the chosen trigger justified.

## Reference Architecture

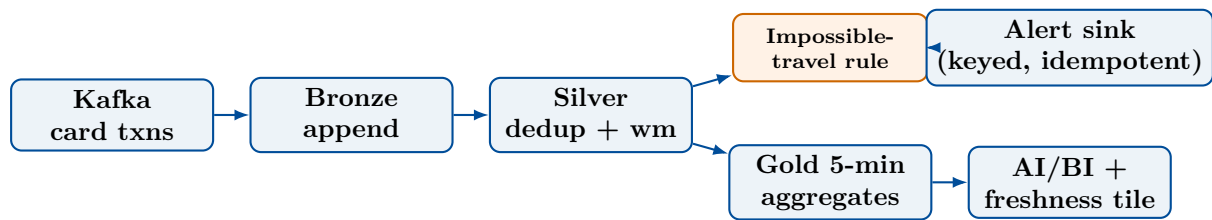


Figure 16.2: Milestone 4 reference architecture: dedup-before-rules fraud telemetry.

## Implementation Steps

1. Producer with `uuid5`-deterministic event IDs; plant the fraud cases.
2. Bronze stream (capped trigger, governed checkpoint).
3. Silver: watermark, `dropDuplicates`, `foreachBatch MERGE`.
4. Rule stream in update mode; alert sink keyed by `event_id`.
5. Gold aggregates + dashboard + freshness tile.
6. Replay drill and cost report; finalize the README per the portfolio guide.

## Deliverables

- All pipeline code, producer, and dashboard export in Git.
- Replay-drill evidence: alert count before and after replay identical.
- Latency measurement (event time to alert) at p50/p95.
- Cost report with trigger decision.

## Acceptance Criteria

- Zero duplicate alerts on full replay; zero duplicate Silver rows.
- The planted fraud case alerts within the latency budget.
- Late events within the watermark are included; beyond it, counted and dropped.
- Credentials in secret scopes; checkpoints in governed volumes.

## Stretch Goals

- Reimplement the rule with `applyInPandasWithState` and compare code clarity and state metrics.
- Add the nightly batch reconciliation tier and the daily diff job.
- Failover drill: new checkpoint from `earliest` into a recovery estate with reconciliation.

## **Part V**

# **Machine Learning and MLOps**

# MLflow and Experiment Tracking

## Learning Objectives

- Explain MLflow’s components: Tracking, Models, Registry, and evaluation.
- Instrument training with autologging plus deliberate manual logging of parameters, metrics, and artifacts.
- Tag runs with dataset snapshots and code versions for reproducibility.
- Register models in Unity Catalog and manage lifecycle with aliases (champion/challenger).
- Load models by alias for inference without code changes.
- Design experiment governance for a regulated environment.

## Cross-Cloud Equivalence

MLflow is the **experiment ledger**; every cloud has a tracking/registry pair, but MLflow is the open standard most of them also integrate.

| Capability       | Databricks                    | AWS                   |             | Azure / GCP / Fabric                               |
|------------------|-------------------------------|-----------------------|-------------|----------------------------------------------------|
| Tracking         | Managed MLflow                | SageMaker Experiments | Experiments | Azure ML runs / Vertex Experiments / Fabric MLflow |
| Registry         | Unity Catalog model registry  | SageMaker Registry    | Model       | Azure ML registry / Vertex Model Registry          |
| Autologging      | sklearn/XGBoost/etc.          | SageMaker (partial)   | Debugger    | Azure ML MLflow integration                        |
| Lifecycle        | Aliases (champion/challenger) | Approval states       |             | Stages / aliases                                   |
| Governance       | UC lineage: data → model      | SageMaker lineage     |             | Purview / Vertex lineage                           |
| Deployment hooks | Model Serving (Ch. 20)        | SageMaker endpoints   |             | Azure ML online endpoints / Vertex endpoints       |

**Snowflake** registries models in its own model registry with Snowpark ML; **MongoDB** has no registry—Atlas pairs with external MLOps tooling, which is often MLflow.

## 17.1 Week 17 Roadmap and Mental Model

Part V connects data engineering to machine learning—not by turning you into a data scientist, but by making you the engineer who makes ML reproducible, governed, and operable. Week 17

starts with the ledger: every training run recorded with its parameters, metrics, artifacts, code, and data version, so that “which model is in production and why is it this one” is a query, not an archaeology project [56, 32].

The mental model: **MLflow is version control for the experiment dimension**. Git versions code; Delta versions data; MLflow versions the *combination*: code + data + parameters → metrics + artifacts. The registry then versions the promotion decision. All four versioned layers together are what “reproducible ML” actually means.

### Definition

**MLflow Tracking** records runs (parameters, metrics, artifacts, code version, environment) against experiments. **MLflow Models** package artifacts with signatures and dependencies. The **Model Registry in Unity Catalog** versions models as governed catalog objects with aliases, permissions, and lineage to the training data.

### Key Terms

**Run**: one execution of training code. **Experiment**: a named collection of runs. **Autologging**: framework integrations that capture parameters/metrics/artifacts without explicit calls. **Model signature**: the input/output schema contract of a model. **Alias**: a mutable pointer (e.g. *champion*) to a registered version—the deployable reference.

## 17.2 Monday: Tracking, Runs, and Artifacts

### 17.2.1 What a Run Records

A run bundles: parameters (hyperparameters, config), metrics (accuracy, AUC, custom), artifacts (the model binary, plots, data profiles), tags (your metadata—dataset snapshot, business owner), the code version (git commit when in a Git folder), and the environment. Autologging fills most of this for common frameworks; your job is the tags and the dataset identity—the parts that make runs comparable and reproducible.

```

1 import mlflow
2 from sklearn.ensemble import RandomForestClassifier
3
4 mlflow.set_experiment("/de_training/churn")
5
6 with mlflow.start_run(run_name="rf-depth-sweep-12"):
7     mlflow.autolog()                                # params, metrics, artifacts
8     mlflow.set_tags({
9         "dataset_snapshot": "de_training.ml.training_v2026_08_31",
10        "git_commit": "a1b2c3d",
11        "feature_set": "customer_v3",
12        "owner": "de-team",
13    })
14    model = RandomForestClassifier(max_depth=12, random_state=42)
15    model.fit(X_train, y_train)
16    mlflow.log_metric("holdout_auc", holdout_auc)
17    mlflow.log_input(mlflow.data.load_delta(
18        table_name="de_training.ml.training_v2026_08_31"), context="training")

```

Listing 17.1: Autologging plus the tags that make runs governable.



## 17.2.2 Comparing Runs Honestly

The experiment UI compares runs—valid comparisons require a *fixed data snapshot* and evaluation harness. Comparing a Tuesday model against a Friday model trained on different data measures data drift, not modeling progress. The tag discipline above is what makes the comparison honest.

### Common Pitfall

**The unlogged winner.** “The best model was that Tuesday run, but we overwrote the notebook.” If a run is not tracked, it does not exist: no artifact, no environment, no defense in an audit. Autolog plus tags is two lines; archaeology is two weeks.

## 17.3 Tuesday: Signatures, Inputs, and Reproducibility

### 17.3.1 Model Signatures and Input Examples

A signature declares the model’s input/output schema; an input example documents a valid request. Both are validated at log time and enforced at serving time (Chapter 20)—they are the compile-time type-check of the ML lifecycle, catching the “production sends strings, training expected floats” class of incident before deployment.

### 17.3.2 Reproducing a Run

Reproducibility = code (git SHA) + data (Delta version or snapshot table) + parameters + environment (logged `conda.yaml/requirements`). With all four logged, `mlflow run` or a re-executed notebook rebuilds the model; Delta time travel pins the training data to the exact version even if the table has moved on. The audit story for a bank: *any* production prediction traces to a reproducible run.

## 17.4 Wednesday: The Unity Catalog Model Registry

### 17.4.1 Versions, Aliases, Governance

Registering a model in UC creates versioned, permissioned catalog objects with lineage back to the training notebook and data [32]. **Aliases** (`champion`, `challenger`) are mutable pointers production code references by name—promotion moves the alias; nothing redeploys.

```

1 import mlflow
2
3 mlflow.set_registry_uri("databricks-uc")
4 result = mlflow.register_model(
5     f"runs:{run.info.run_id}/model",
6     "ml.churn_model")                                # catalog.schema.model
7
8 client = mlflow.MlflowClient()
9 client.set_registered_model_alias(
10     "ml.churn_model", "champion", result.version)
11
12 # Inference code NEVER hardcodes a version:
13 model = mlflow.pyfunc.load_model("models:/ml.churn_model@champion")

```

Listing 17.2: Register, promote by alias, load by alias.

**Definition**

**Alias-based deployment:** production jobs load models: /<name>@champion; promotion is `set_registered_model_alias`; rollback is re-pointing the alias. Version numbers are immutable history; aliases are the operational interface. This one indirection eliminates redeploy-on-promote and makes rollback instantaneous.

## 17.5 Thursday Hands-on Project: Six Runs, One Champion

Train six churn model variants with autologging on a fixed snapshot, tag every run, register the winner, promote it to `champion`, and load it in a fresh notebook by alias.

```

1 import mlflow
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.linear_model import LogisticRegression
4
5 train = spark.table("de_training.ml.training_v2026_08_31").toPandas()
6 X, y = train.drop(columns=["churned", "customer_id"]), train["churned"]
7
8 mlflow.set_experiment("/de_training/churn_week17")
9 for i, depth in enumerate([4, 6, 8, 10, 12]):
10     with mlflow.start_run(run_name=f"rf_d{depth}"):
11         mlflow.autolog()
12         m = RandomForestClassifier(max_depth=depth, random_state=42)
13         m.fit(X, y)
14         mlflow.set_tag("dataset_snapshot", "training_v2026_08_31")
15         mlflow.log_metric("holdout_auc", holdout_auc_of(m))
16 with mlflow.start_run(run_name="logreg_baseline"):
17     mlflow.autolog()
18     m = LogisticRegression(max_iter=1000).fit(X, y)
19     mlflow.log_metric("holdout_auc", holdout_auc_of(m))
20
21 # Winner -> registry -> alias -> clean-notebook load by alias
22 best = mlflow.search_runs(order_by=["metrics.holdout_auc DESC"]).iloc[0]
23 reg = mlflow.register_model(f"runs://{best.run_id}/model", "ml.churn_model")
24 mlflow.MlflowClient().set_registered_model_alias(
25     "ml.churn_model", "champion", reg.version)

```

Listing 17.3: Week 17 lab: the tracked sweep.

### Deliverables

- The experiment with six fully tagged runs and a comparison screenshot or export.
- The registered model with the `champion` alias.
- A fresh notebook loading by alias and scoring a sample—proving registry round-trip.

### Acceptance Criteria

- Every run carries dataset snapshot, git commit, and feature-set tags.
- The comparison table is reproducible: re-running the evaluation yields identical metrics on the fixed snapshot.
- The clean notebook loads `@champion` with no version number anywhere in its code.
- A re-promotion (`champion` → previous version) is demonstrated as rollback.

## Stretch Goals

- Add a model signature and input example; show serving-time validation rejecting a bad payload (preview of Chapter 20).
- Reproduce the champion run end-to-end from its logged environment on a different cluster.
- Write the lineage query: registered model → training notebook → source tables.

## 17.6 Friday Use Case: Experiment Governance for a Bank

A bank’s models approve credit. Regulators require every production model to trace to training data, code, hyperparameters, evaluation evidence, and a named approver; and any prediction to be explainable back to a model version within one business day.

### 17.6.1 Requirements and Assumptions

DoS-style protection: models retrain monthly; approvals are committee decisions recorded outside Databricks; production scoring is batch nightly plus a real-time endpoint.

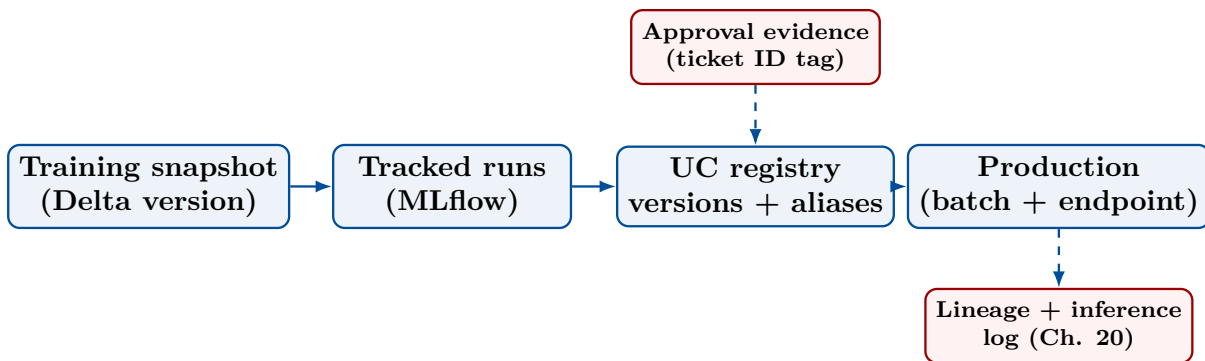


Figure 17.1: Regulated model lifecycle: every link from data to production prediction is recorded and queryable.

### 17.6.2 Architecture Decisions

**The traceability chain.** Snapshot tables are immutable per training cycle (Delta versions pinned); every run logs the snapshot name, commit, and environment; registration requires an `approval_ticket` tag (enforced by a deployment-time check in CI); aliases move only via the release pipeline, which writes the committee decision ID into the model version’s tags. The one-business-day question—“why did the model decline this application?”—is answered by joining the inference log (Chapter 20) to the registry version to the run to the snapshot.

**Separation of duties.** Data scientists propose (experiments, challenger alias); the release pipeline disposes (champion moves only in CI with the approval tag); engineers operate (serving, monitoring). Nobody both trains and promotes.

#### Common Pitfall

**Approvals in chat.** A “LGTM” in a chat channel is not audit evidence. The approval must land *on the artifact*—a tag on the model version written by the pipeline that verified the ticket—or it did not happen.

### 17.6.3 Risks and Mitigations

| Risk                             | Mitigation                                              |
|----------------------------------|---------------------------------------------------------|
| Approval tag forged or by-passed | Alias moves only via the CI pipeline’s principal        |
| Snapshot tables pruned too early | Retention schedule aligned to model-risk policy         |
| Experiment metadata leaks PII    | Tag taxonomy review; metadata scanning in CI            |
| Registry permissions accumulate  | Quarterly registry access review with the access matrix |
| Rollback target itself untested  | Quarterly rollback drill against a staging endpoint     |

Table 17.1: Week 17 scenario risks and mitigations.

## 17.7 Design Decision Guide

Experiment-governance decisions decide whether your best model can be proven to exist.

| Decision                                  | Choose the first when          | Choose the second when                 |
|-------------------------------------------|--------------------------------|----------------------------------------|
| Autologging vs. manual logging            | Framework-standard artifacts   | Custom metrics, tags, dataset identity |
| Alias vs. version in production code      | Always alias                   | Never version-pin deployable code      |
| UC registry vs. workspace registry        | Governance, lineage, sharing   | Legacy constraints only                |
| Fixed snapshot vs. latest data per run    | Any comparable experimentation | Exploratory drift studies only         |
| Signature enforcement vs. lenient serving | Production endpoints           | Prototypes behind no consumers         |
| Promotion via pipeline vs. by hand        | Anything production-visible    | Personal experiments                   |

Table 17.2: Week 17 decision guide.

Reproducibility is the currency: code + data + parameters + environment, all four, every run. A run missing one is a demo; a registry entry with all four is an asset.

## 17.8 Operational Guidance for Week 17

### 17.8.1 Performance and Reliability

Experiment hygiene is a reliability property: name runs, tag them with the dataset snapshot and commit, and prune or archive experiment clutter monthly so the comparison UI stays usable. Registry changes (alias moves) belong in the release pipeline with a log; a hand-moved champion is untraceable. Watch artifact-store growth—large model artifacts on every sweep add up; autolog what you need, not everything loggable.

### 17.8.2 Security and Governance Checklist

- Registry permissions: scientists register versions; only the release pipeline moves aliases.
- Approval tags required for registration to production-visible namespaces.

- Training snapshots retained per the model-risk retention schedule.
- Experiment names and tags carry no PII or secrets (they are metadata, widely visible).

### 17.8.3 Troubleshooting Playbook

| Symptom                            | Likely cause                          | First action                                                    |
|------------------------------------|---------------------------------------|-----------------------------------------------------------------|
| Cannot reproduce an old run        | Missing snapshot tag or environment   | Check tags; time-travel the data; rebuild env from logged spec  |
| Alias points at wrong version      | Hand-moved alias outside the pipeline | Audit alias history; re-point; tighten registry permissions     |
| Model fails to load in serving     | Environment/signature mismatch        | Check the logged environment and the signature validation error |
| Experiment comparison looks random | Runs trained on different snapshots   | Filter by <code>dataset_snapshot</code> tag before comparing    |

Table 17.3: Week 17 troubleshooting quick reference.

## 17.9 Interview Q&A

### 17.9.1 Concepts and Trade-offs

1. **Q: What does autologging not capture that you must add?**  
A: Your context: dataset identity/version, business owner, feature-set version, approval linkage—the metadata that makes runs comparable, governable, and reproducible.
2. **Q: Why register in Unity Catalog rather than the workspace registry?**  
A: Governance: permissions, lineage to training data, cross-workspace sharing, and one namespace with the rest of your governed assets.
3. **Q: Alias versus version in production code?**  
A: Always the alias. Version-pinned code redeploys on every promotion and turns rollback into a release; alias-referenced code promotes and rolls back by moving a pointer.
4. **Q: Reproduce a nine-month-old run. What do you need?**  
A: Git SHA, dataset snapshot (or Delta version), parameters, and environment—all four logged. Missing any one, you are approximating, not reproducing.
5. **Q: Two runs, same code, different AUC. First suspect?**  
A: The data. Check dataset versions/snapshots before suspecting the model—this is why the snapshot tag is mandatory.
6. **Q: How do you compare two candidate models fairly?**  
A: Same snapshot, same evaluation harness, logged metrics—anything else compares data vintages, not models.
7. **Q: What is the registry’s role in an incident?**  
A: Instant answers: which version served, trained on what, approved by whom, and how to roll back—the alias re-point measured in seconds, with evidence.
8. **Q: How many experiments should one project have?**  
A: One per question being investigated, with runs named for hypotheses. A single `Default` experiment with 400 runs is a landfill, not a lab notebook.

## 17.10 Further Reading

The MLflow documentation covers tracking, models, signatures, and evaluation [56]; MLflow on Databricks adds the UC registry and workspace integration [32]. Feature Store (Chapter 18) and Model Serving (Chapter 20) complete the lifecycle [27, 33].

## Key Takeaways

- MLflow versions the experiment dimension: code + data + parameters → metrics + artifacts.
- Autolog captures the framework; your tags capture the context that makes runs governable.
- Compare runs only on fixed snapshots—otherwise you are measuring data drift.
- Aliases are the deployment interface: promote and roll back by moving a pointer, never by redeploying code.
- In regulated settings, approval evidence lives on the artifact, enforced by the pipeline.

## 17.11 Exercises

1. **(Conceptual)** List the four things required to reproduce a training run and the MLflow mechanism that records each.
2. **(Conceptual)** Why is a model signature a data-engineering artifact as much as an ML one?
3. **(Hands-on)** Complete the Thursday lab including the rollback demonstration.
4. **(Hands-on)** Break a run’s reproducibility deliberately (no snapshot tag) and document what you can no longer prove.
5. **(Hands-on)** Write the deployment-time check that refuses registration without an `approval_ticket` tag.
6. **(Design)** Design the tag taxonomy (required keys, formats, owners) for a 10-team ML organization.
7. **(Operations)** Write the audit query: “all production predictions last month, by model version, traced to training snapshot.”
8. **(Architecture)** Where does the registry’s governance intersect Chapter 8’s access matrix? Design the grants for the three personas.

# Feature Engineering and the Feature Store

**Learning Objectives**

- Define label leakage and point-in-time correctness, and detect them in a pipeline.
- Build governed feature tables in Unity Catalog with as-of timestamps.
- Assemble training sets with timestamp-aware feature lookups.
- Serve the same features online for inference without skew.
- Trace feature-to-model lineage and monitor feature freshness.
- Audit a leaking model and redesign its feature pipeline.

**Cross-Cloud Equivalence**

The feature store is the **contract between training and serving**; every serious ML platform implements one.

| Capability         | Databricks                                         | AWS                             | Azure / GCP / Fabric                          |
|--------------------|----------------------------------------------------|---------------------------------|-----------------------------------------------|
| Feature registry   | Feature Engineering in UC                          | SageMaker Feature Store         | Azure ML FS / Vertex FS / Fabric feature sets |
| Offline store      | Delta tables (UC)                                  | S3 + Glue catalog               | ADLS / BigQuery / OneLake                     |
| Online store       | Feature serving endpoints                          | ElastiCache-backed online store | Cosmos DB / Bigtable serving                  |
| Point-in-time join | <code>create_training_set</code> timestamp lookups | Event-time joins                | As-of joins                                   |
| Lineage            | Auto: features → model in registry                 | SageMaker lineage               | Purview / Vertex lineage                      |
| Skew defense       | Same table feeds train + serve                     | Same store, dual interfaces     | Same principle                                |

**Snowflake** has its own Feature Store over Snowflake tables; the pattern—one governed definition feeding both training and inference—is identical.

## 18.1 Week 18 Roadmap and Mental Model

Chapter 17 versioned the experiment; Week 18 versions the *inputs*. The Feature Store’s promise to the data engineer: features are data products—defined once as code, computed by pipelines you own, stored as governed Delta tables, joined into training sets with time correctness, and

served to models through the same definitions [27].

The mental model: **a feature is a function of history, evaluated as of a moment.** `customer_90d_spend` means “spend in the 90 days *before the event being predicted*.” Train with tomorrow’s data in that window and the model learns the future—spectacular validation metrics, useless production model. Point-in-time correctness is the discipline of evaluating every feature strictly before its label’s timestamp.

#### Definition

**Label leakage** is the inclusion of information in training features that would not be available at prediction time. **Point-in-time correctness** means each training row’s features are computed only from data available before that row’s label timestamp. **Training-serving skew** is computing features differently in training versus inference.

#### Key Terms

**Feature table:** a governed Delta table of (entity keys, timestamp, feature values). **Feature lookup:** a declared join from label rows to feature tables. **Timestamp lookup:** a lookup constrained to feature rows  $\leq$  the label’s timestamp. **Online store:** low-latency feature serving for inference. **As-of date:** the reference timestamp for a feature computation.

## 18.2 Monday: Leakage and Point-in-Time Discipline

### 18.2.1 Recognizing Leakage

Leakage wears disguises: the `days_to_refund` column (refunds happen *after* the prediction moment), the aggregate computed over the full table instead of up-to-as-of, the “last known address” join that returns tomorrow’s value. The test for any feature: *could I compute this exact value at prediction time, given only what exists then?* If the answer needs a time machine, it leaks.

### 18.2.2 As-of Computation

Features are computed per as-of date: for labels at time  $t$ , features aggregate history  $[t - w, t)$ . This is a pipeline-design constraint, not a notebook convenience—the feature job takes `as_of_date` as a parameter (Chapter 3’s parameterization pays off) and never reads past it.

#### Common Pitfall

**The full-history aggregate.** `SUM(amount) OVER (...)` without a timestamp bound uses future rows for early labels. It validates beautifully—the leakage is in the validation too. The tell: offline metrics that production cannot reproduce within tolerance.

## 18.3 Tuesday: Feature Tables and Training Sets

### 18.3.1 Feature Engineering in Unity Catalog

A feature table is a Delta table with entity keys and (for time-series features) a timestamp, registered so that training and serving can look it up [27]. Writing one is a normal pipeline with an as-of parameter:

```
1 from pyspark.sql import functions as F
2
```



```

3 as_of = dbutils.widgets.get("as_of_date")      # e.g. 2026-08-31
4 orders = (spark.table("de_training.silver.orders")
5           .where(F.col("order_ts") < F.lit(as_of))) # never past as_of
6
7 features = (orders.groupBy("customer_id")
8             .agg(F.count("*").alias("orders_90d"),
9                F.sum("total_amount").alias("spend_90d"))
10            .withColumn("as_of_date", F.lit(as_of)))
11
12 (features.write.format("delta").mode("overwrite")
13  .option("replaceWhere", f"as_of_date = '{as_of}'") # idempotent per date
14  .saveAsTable("de_training.ml.customer_features"))

```

Listing 18.1: A point-in-time-safe feature job.

### 18.3.2 Training Sets with Timestamp Lookups

`FeatureEngineeringClient.create_training_set` joins label rows to feature tables *as of each label's timestamp*—the point-in-time join done correctly by the framework, with the resulting training set linked to the model at registration:

```

1 from databricks.feature_engineering import FeatureEngineeringClient,
   FeatureLookup
2
3 fe = FeatureEngineeringClient()
4 training_set = fe.create_training_set(
5     df=spark.table("de_training.ml.churn_labels"), # customer_id + event_ts +
6     label
7     feature_lookups=[
8         FeatureLookup("de_training.ml.customer_features",
9                       lookup_key="customer_id",
10                      timestamp_lookup_key="as_of_date"),
11     ],
12     label="churned",
13     exclude_columns=["customer_id", "event_ts"])
14 df = training_set.load_df()
15 # At log_model time: fe.log_model(..., training_set=training_set)
16 # -> the registry records exactly which feature tables built this model.

```

Listing 18.2: A training set with point-in-time-correct lookups.

## 18.4 Wednesday: Online Serving and Skew Prevention

### 18.4.1 The Same Features, Online

At inference, the model needs the same feature values fast. Feature serving exposes the latest feature rows through a low-latency endpoint (or you precompute into an online store); models logged with the feature store know their lookups and fetch features by entity key at scoring time. The anti-skew property is the point: *one definition, two access patterns*—training reads history as-of; serving reads latest.

### 18.4.2 Freshness and Monitoring

Features decay: a job that fails silently serves stale values that look valid. Every feature table carries a freshness contract (`max(as_of_date)` within SLA of now); the Workflow from Chapter 12 gates model scoring on feature freshness, and Lakehouse Monitoring (Chapter 20) watches feature drift over time.

**Tip**

Name feature tables with the entity and window—`customer_90d_features`, not `features_v2`—and document the window, source tables, and owner in the table comment. Six months later, the name is the documentation someone debugging skew will actually read.

## 18.5 Thursday Hands-on Project: Point-in-Time Churn Features

Build `customer_features` and `order_features` with as-of timestamps, assemble a point-in-time training set, train and register a model with feature lineage, and verify serving consistency.

```

1 # 1. Feature jobs run for three historical as_of dates (idempotent overwrites).
2 for as_of in ["2026-06-30", "2026-07-31", "2026-08-31"]:
3     build_customer_features(as_of)    # the lst:ch18-features pattern
4
5 # 2. Training set with timestamp lookups (lst:ch18-training-set).
6
7 # 3. Deliberate-leakage control: build a leaky feature table
8 #     (full-history aggregate, no as_of bound) and compare AUCs:
9 #     leaky: 0.97 (suspicious) vs point-in-time: 0.83 (believable).
10
11 # 4. Register with lineage; load by alias; score via feature lookups:
12 import mlflow
13 model = mlflow.pyfunc.load_model("models:/ml.churn_model@champion")
14 # scoring with feature lookups fetches features by key at serve time,
15 # from the SAME tables training used -> no skew by construction.

```

Listing 18.3: Week 18 lab: the leakage hunt.

### Deliverables

- Two feature tables with as-of partitions and documented windows.
- The training set code with timestamp lookups; three historical rebuilds identical on rerun.
- The leakage control experiment with both AUCs and the explanation.
- Registry evidence of feature-table lineage on the model version.

### Acceptance Criteria

- No feature reads past its label’s timestamp—proven by rebuilding one label’s features with a mocked “prediction moment.”
- Feature jobs are idempotent per as-of date (`replaceWhere`).
- The leaky-feature experiment is documented as the negative control.
- Scoring loads features via declared lookups, not hand-written joins.

### Stretch Goals

- Add an on-demand feature (computed from request-time fields) and explain when on-demand is legitimate.
- Wire the freshness gate into the scoring Workflow from Chapter 12.

- Publish the feature table to an online store and measure lookup latency.

## 18.6 Friday Use Case: The Churn Model That Lied

A churn model shows 0.97 AUC in training and 0.61 in production. The data science team blames “distribution shift.” You are asked to audit the feature pipeline.

### 18.6.1 Requirements and Assumptions

Features were built in a notebook from Gold tables; training used a 12-month extract; production scores nightly via a separate SQL query “equivalent” to the notebook logic. The 0.97/0.61 gap appeared at launch and has persisted.

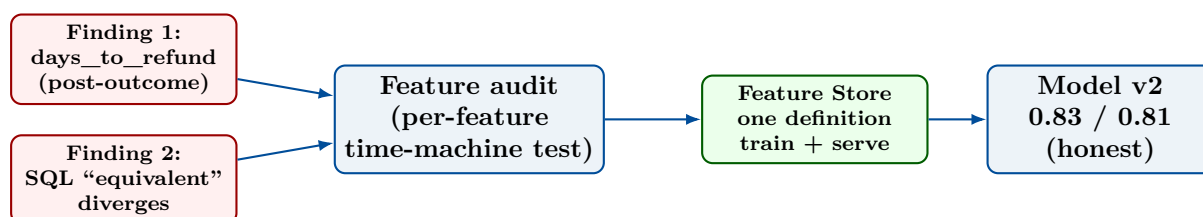


Figure 18.1: The audit: find the leak, find the skew, replace both with one governed feature definition.

### 18.6.2 Architecture Decisions

**The two findings.** (1) *Leakage*: `days_to_refund` and a full-history `lifetime_spend` both encode post-outcome information—the 0.97 was partly the model reading the future. (2) *Skew*: production’s “equivalent” SQL rounds timestamps differently and misses a null-handling branch, so even honest features arrive differently at serving.

**The fix.** Both die the same death: features become governed tables computed by one parameterized job with as-of timestamps; training uses timestamp lookups; serving uses the same tables through the feature store. New metrics: 0.83 offline, 0.81 online—lower, honest, and *stable*. The audit’s lasting artifact is the per-feature time-machine test added to the feature-review checklist.

#### Common Pitfall

**Blaming drift before auditing leakage.** “Distribution shift” is the comforting explanation; leakage is the common one. The gap signature tells: leakage shows as a large, persistent train/production gap from day one; drift degrades a correct model gradually.

### 18.6.3 Risks and Mitigations

| Risk                                   | Mitigation                                                       |
|----------------------------------------|------------------------------------------------------------------|
| New feature leaks quietly              | Time-machine test is a review checklist item, enforced in CI     |
| Feature window drifts from intent      | Window definitions versioned; quarterly review vs. model metrics |
| Serving reads a stale feature table    | Freshness gate blocks scoring; alert pages the feature owner     |
| Training set rebuilt differently later | <code>create_training_set</code> spec versioned with the model   |
| Lineage gap for a hand-joined model    | Registration requires feature lookups, not ad-hoc frames         |

Table 18.1: Week 18 scenario risks and mitigations.

## 18.7 Design Decision Guide

Feature decisions are time decisions; every row below is a variant of “as of when?”

| Decision                               | Choose the first when              | Choose the second when           |
|----------------------------------------|------------------------------------|----------------------------------|
| Feature store vs. ad-hoc joins         | Any model that reaches production  | Throwaway exploration            |
| Timestamp lookup vs. latest join       | Training sets                      | Serving-time enrichment          |
| Precomputed vs. on-demand features     | History-derived values             | Request-time-only inputs         |
| SCD2-aware features vs. current values | Historical attribute truth matters | Current-state-only models        |
| Batch refresh vs. streaming features   | Hourly/daily freshness suffices    | Measured sub-minute feature SLAs |
| Online store vs. serving endpoint      | Ultra-low-latency lookups at scale | Standard request-time fetches    |

Table 18.2: Week 18 decision guide.

The time-machine test settles arguments: if you could not compute the value at prediction time, it is leakage regardless of how good the validation score looks.

## 18.8 Operational Guidance for Week 18

### 18.8.1 Performance and Reliability

Feature tables are pipelines with SLAs: freshness gates in the scoring workflow, a daily `max(as_of_date)` check per table, and an owner per table. Review feature windows quarterly against model performance—stale window definitions are silent quality drag. Keep the feature-to-model lineage report current; it is the impact analysis for every feature change.

### 18.8.2 Security and Governance Checklist

- Feature tables containing PII carry classification tags and masked serving views.
- Feature job parameters (`as_of_date`) logged per run for auditability.

- New features pass the time-machine test in review before registration.
- Online-serving endpoints inherit the feature tables' access model.

### 18.8.3 Troubleshooting Playbook

| Symptom                         | Likely cause                    | First action                                                       |
|---------------------------------|---------------------------------|--------------------------------------------------------------------|
| Train/serve metric gap          | Leakage or skew                 | Run the per-feature time-machine test; diff train vs. serve values |
| Scores silently stale           | Feature job failing upstream    | Freshness gate should catch it—verify the gate is wired            |
| Training set row count off      | Lookups missing keys/timestamps | Check join coverage and timestamp granularity                      |
| Feature rebuild changes history | Non-idempotent feature job      | Add <code>replaceWhere</code> per as-of date; rerun and diff       |

Table 18.3: Week 18 troubleshooting quick reference.

## 18.9 Interview Q&A

### 18.9.1 Concepts and Trade-offs

1. **Q: Define point-in-time correctness and its violation cost.**  
A: Every feature for a label at time  $t$  computed only from data before  $t$ . Violate it and the model trains on the future: inflated offline metrics, useless production behavior, and a debugging odyssey.
2. **Q: How does a feature store prevent training-serving skew?**  
A: One governed definition feeds both paths: training reads the same tables through timestamp lookups that serving reads through latest-value lookups. Skew requires two definitions; the store has one.
3. **Q: When is an on-demand (request-time) feature legitimate?**  
A: When its inputs genuinely exist only at request time—current cart contents, device of this request. The rule holds: request-time inputs are fine; future-relative-to-label data is not.
4. **Q: What lineage should a registered model carry?**  
A: The training code (commit), the training set definition, and the feature tables with their versions—so a production answer traces back to every input.
5. **Q: A feature job failed silently for three days. What do users see?**  
A: Stale-but-plausible predictions—the dangerous failure. Defense: freshness gates in the scoring workflow and drift monitoring on feature distributions.
6. **Q: Why not let data scientists hand-join features for training?**  
A: Because the hand-join is the skew and leakage factory: a different join at serving, or a join without the as-of bound, invalidates the model silently. The training-set API makes the correct join the easy one.
7. **Q: How do you detect feature drift before model quality drops?**  
A: Monitor the feature tables' distributions (Lakehouse Monitoring) against the training baseline—feature drift is the leading indicator; quality decay is the lagging one.

**8. Q: When does a feature graduate from a notebook to the store?**

A: The moment a second model wants it, or the first model approaches production—whichever comes first. The store is the sharing and correctness layer; both triggers arrive earlier than intuition suggests.

## 18.10 Further Reading

Feature Engineering in Unity Catalog is the product reference [27]; MLflow ties feature lineage into the registry [32, 56]. The Delta write patterns that keep feature jobs idempotent are in [46]; monitoring arrives in Chapter 20 [29].

## Key Takeaways

- A feature is a function of history evaluated as of a moment—parameterize the moment.
- Leakage inflates offline metrics; the train/production gap is the tell; audit before blaming drift.
- Timestamp lookups make point-in-time correctness the framework’s job, not your discipline’s.
- One feature definition feeding training and serving is the structural end of skew.
- Freshness is a contract: gate scoring on it, monitor drift on it.

## 18.11 Exercises

1. **(Conceptual)** Classify as safe or leaking: `orders_last_30d`, `refund_amount_next_7d`, `account_age_at_event`, `final_dispute_outcome`.
2. **(Conceptual)** Why does the leaky model also inflate validation metrics?
3. **(Hands-on)** Complete the Thursday lab including the leakage control.
4. **(Hands-on)** Add the freshness gate to a scoring workflow; force it to fail and capture the behavior.
5. **(Hands-on)** Write the registry-lineage query from a model version to its feature tables.
6. **(Design)** Spec the feature-review checklist (the time-machine test and friends) for new features.
7. **(Operations)** Write the incident runbook for “feature table 2 days stale.”
8. **(Architecture)** When would you replicate features to an external online store instead of feature serving? Quantify the latency case.

# Mosaic AI, Vector Search, and Generative AI (RAG)

**Learning Objectives**

- Explain the GenAI building blocks on Databricks: Foundation Model APIs, embeddings, and serving endpoints.
- Build a chunking pipeline from documents into a governed Delta table.
- Create and query Mosaic AI Vector Search indexes, choosing managed versus Delta-sync.
- Assemble a retrieval-augmented generation (RAG) chain with prompt templates and metadata filters.
- Evaluate RAG quality with MLflow metrics instead of eyeballing.
- Design a governed, cost-bounded GenAI application.

**Cross-Cloud Equivalence**

RAG is the **dominant GenAI production pattern**; every cloud packages the same four parts—embed, index, retrieve, generate.

| Capability        | Databricks                                           | AWS                                   | Azure / GCP / Fabric                            |
|-------------------|------------------------------------------------------|---------------------------------------|-------------------------------------------------|
| Foundation models | Foundation Model APIs (pay-per-token or provisioned) | Bedrock                               | Azure OpenAI / Vertex AI / Fabric AI            |
| Embeddings        | Hosted embedding endpoints                           | Bedrock Titan Embeddings              | Azure OpenAI embeddings / Vertex embeddings     |
| Vector index      | Mosaic AI Vector Search                              | OpenSearch / Kendra / pgvector on RDS | AI Search / Vertex Vector Search / OneLake + AI |
| RAG orchestration | LangChain/LlamaIndex + MLflow                        | Bedrock Bases Knowledge               | Azure AI RAG pattern / Vertex RAG Engine        |
| Evaluation        | MLflow evaluate (LLM judges)                         | Bedrock evaluations                   | Azure AI eval / Vertex eval                     |
| Governance        | UC over chunks + audit of endpoints                  | IAM + Bedrock guardrails              | Entra + content filters / Vertex safety         |

**MongoDB Atlas Vector Search** and **Snowflake Cortex** implement the same pattern against their own data planes—the architecture you learn here ports everywhere.

## 19.1 Week 19 Roadmap and Mental Model

Weeks 17–18 productionized classical ML. Week 19 adds the GenAI workload your data platform will be asked to serve: retrieval-augmented generation over the organization’s documents. The data engineering content is substantial: documents are ingested, chunked, embedded, indexed, freshness-managed, and governed—a pipeline problem before it is a prompt problem [35, 48].

The mental model: **RAG is a search problem feeding a generation step**. The model’s quality is bounded by retrieval quality; retrieval quality is bounded by chunking and embedding choices; everything is bounded by the freshness and governance of the source Delta table. Engineers who treat the vector index as a derived, rebuildable projection of governed data ship systems they can debug; those who treat it as the source of truth ship mysteries.

### Definition

**Retrieval-augmented generation (RAG)** answers questions by (1) embedding the question, (2) retrieving the most similar document chunks from a vector index, (3) assembling a prompt with that context, and (4) generating an answer from a foundation model. **Mosaic AI Vector Search** is Databricks’ managed vector index over Delta tables.

### Key Terms

**Embedding**: a dense vector encoding of text. **Chunk**: a document slice sized for retrieval. **Managed index**: embeddings computed and maintained by Vector Search. **Delta-sync index**: index synced from a Delta table (your pipeline computes embeddings or uses a managed embedding model). **Foundation Model API**: pay-per-token hosted model endpoints. **MLflow evaluation**: metric-based LLM output scoring, including LLM-as-judge.

## 19.2 Monday: Foundation Models and the Serving Surface

### 19.2.1 The Building Blocks

Mosaic AI exposes hosted models (Llama-family, and partner models) behind **Foundation Model APIs**: pay-per-token for variable load, provisioned throughput for latency/throughput guarantees. Embedding models run the same way. Everything is an endpoint—which means authentication, rate limits, cost per call, and inference logging are the integration surface, and Chapter 20’s serving discipline applies to LLMs exactly as to sklearn [33].

### 19.2.2 Cost and Latency Arithmetic

GenAI economics are per-token: a RAG answer costs embedding tokens + retrieved context tokens + generated tokens. The cost ceiling from the Friday scenario is an engineering constraint: chunk count per answer ( $k$ ), context length, and model choice are the dials. Record token usage per request from day one; “the LLM bill” is otherwise the least explainable line item you will ever defend.

### Common Pitfall

**Provisioned throughput for a demo**. Pay-per-token scales to zero; provisioned throughput bills around the clock. Match the endpoint mode to the load shape: provisioned when measured p95 latency demands it under real traffic—not when a demo felt slow once.



## 19.3 Tuesday: Vector Search

### 19.3.1 From Documents to an Index

The pipeline: documents → chunks (with source metadata) → embeddings → index. On Databricks, chunks live in a Delta table (governed, versioned, lineage-tracked); a **Delta-sync index** keeps the index synchronized with that table—updates flow automatically [48].

```

1 from pyspark.sql import functions as F
2
3 docs = spark.read.text("/Volumes/de_training/bronze/docs/", wholertext=True)
4
5 def chunk(text, size=800, overlap=100):
6     return [text[i:i+size] for i in range(0, len(text), size - overlap)]
7
8 chunks = (docs
9     .withColumn("path", F.col("path"))
10    .withColumn("chunk_arr", F.udf(chunk, "array<string>")(F.col("value")))
11    .select("path", F.posexplode("chunk_arr").alias("chunk_id", "chunk_text"))
12    .withColumn("id", F.md5(F.concat("path", F.col("chunk_id").cast("string"))))
13    )
14 (chunks.write.format("delta").mode("overwrite")
15     .saveAsTable("de_training.rag.doc_chunks")) # enable Change Data Feed
16
17 # Vector Search endpoint + Delta-sync index (UI, SDK, or API):
18 #   source: de_training.rag.doc_chunks, primary key: id,
19 #   embedding source column: chunk_text (managed embedding model),
20 #   metadata columns: path -> filters like path LIKE '%runbook%'

```

Listing 19.1: Chunking into Delta and creating a Delta-sync index.

### 19.3.2 Querying with Filters

Similarity search returns top- $k$  chunks; **metadata filters** scope retrieval (e.g., only current-runbook chunks, only the caller's department). Filters are where governance enters retrieval: the filter is the row-level security of GenAI.

```

1 from databricks.vector_search.client import VectorSearchClient
2
3 vs = VectorSearchClient()
4 index = vs.get_index(index_name="de_training.rag.doc_chunks_idx")
5 results = index.similarity_search(
6     query_text="how do I repair a failed pipeline run?",
7     columns=["chunk_text", "path"],
8     filters={"path LIKE": "%runbook%"},
9     num_results=5)

```

Listing 19.2: Similarity search with a governance filter.

## 19.4 Wednesday: The RAG Chain and Evaluation

### 19.4.1 Assembling the Chain

Retrieve (top- $k$  with filters) → assemble prompt (instructions + context + question) → generate via a Foundation Model endpoint → log everything. LangChain or plain Python both work; the discipline is in logging: question, retrieved chunk IDs, prompt, answer, tokens, latency—per request.

```

1 import mlflow
2
3 def answer(question: str) -> dict:
4     hits = index.similarity_search(query_text=question, num_results=5,
5                                   columns=["chunk_text", "path"])
6     context = "\n\n".join(r[0] for r in hits["result"]["data_array"])
7     prompt = (f"Answer using ONLY this context; cite sources.\n\n"
8              f"CONTEXT:\n{context}\n\nQUESTION: {question}")
9     resp = client.serving_endpoints.query( # Foundation Model endpoint
10      name="databricks-meta-llama-3-3-70b-instruct",
11      messages=[{"role": "user", "content": prompt}])
12     out = {"question": question, "answer": resp.choices[0].message.content,
13           "sources": [r[1] for r in hits["result"]["data_array"]]}
14     mlflow.log_table(out, "rag_responses.json") # evidence, per eval run
15     return out

```

Listing 19.3: A minimal, fully-logged RAG chain.

### 19.4.2 Evaluation as a Pipeline Stage

Eyeballing three answers is a demo; a logged metric set is engineering. `mlflow.evaluate` with LLM-judged metrics (faithfulness, relevance) over a fixed question set turns prompt/index changes into measurable deltas—and gates them.

#### Common Pitfall

**No golden set, no progress.** Without a fixed evaluation set (10–100 questions with known-good sources), every prompt tweak is vibes. Build the golden set first; it is the unit test suite of the whole RAG system.

## 19.5 Thursday Hands-on Project: The Program’s Own RAG Assistant

Build a RAG assistant over this program’s documentation: chunk, index (Delta-sync), retrieve, generate, and evaluate 10 golden questions with MLflow.

```

1 import mlflow
2 import pandas as pd
3
4 golden = pd.DataFrame([
5     {"question": "How do I repair a failed workflow run?",
6      "expected_source": "week12_runbook.md"},
7     {"question": "What retention must VACUUM respect?",
8      "expected_source": "week02_delta.md"},
9     # ... 10 total, with known-good source documents
10 ])
11
12 results = []
13 for _, row in golden.iterrows():
14     out = answer(row.question)
15     results.append(**row.to_dict(), **out)
16
17 eval_df = pd.DataFrame(results)
18 with mlflow.start_run(run_name="rag_eval_v1"):
19     metrics = mlflow.evaluate(
20         data=eval_df, predictions="answer",
21         model_type="question-answering") # LLM-judged quality metrics
22     mlflow.log_metric("source_hit_rate",
23         (eval_df.apply(lambda r: r.expected_source in str(r.sources), axis=1)

```

24

`.mean()))`

Listing 19.4: Week 19 lab: evaluation harness.

## Deliverables

- The chunking pipeline writing `rag.doc_chunks` with source metadata.
- A Delta-sync Vector Search index and the retrieval function with filters.
- The evaluation run: 10 golden questions, logged metrics, one diagnosed failure.

## Acceptance Criteria

- Every answer carries its source list; the source-hit rate is computed and logged.
- One retrieval failure is fixed by chunking or metadata-filter changes—with before/after metrics.
- A document update flows to the index via Delta-sync without manual re-indexing.
- Token usage per question is logged; the per-answer cost is computed.

## Stretch Goals

- Compare chunk sizes (400/800/1200) on the golden set; report the retrieval-quality trade-off.
- Add a PII redaction step before embedding and prove a planted PII string never reaches the index.
- Deploy the chain behind a serving endpoint with an inference table (preview of Chapter 20).

## 19.6 Friday Use Case: The Support Copilot with a Cost Ceiling

A SaaS company wants a support copilot over 2M help-center articles: answers within 3 seconds, sources cited, PII never exposed, monthly cost capped at a fixed budget.

### 19.6.1 Requirements and Assumptions

Articles update hourly; support agents are the users; wrong answers with confident tone are worse than “I don’t know”; the cost ceiling is hard.

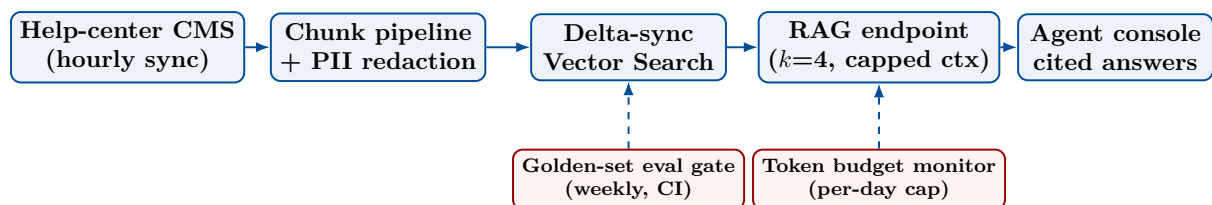


Figure 19.1: Governed copilot: redaction before indexing, evaluation gating changes, token budgets bounding cost.

19.6.2 Architecture Decisions

**Freshness and safety by construction.** The hourly CMS sync lands in the chunks table; Delta-sync keeps the index fresh; PII redaction runs *before* embedding so PII can never be retrieved. Answers must cite sources; retrieval-failure (low similarity scores) returns a scripted “not in the help center” rather than a confabulated answer.

**The cost ceiling as an engineering constraint.**  $k = 4$  chunks, context trimmed to a token cap, the smaller foundation model for drafts with the larger one reserved for low-confidence escalations, and a daily token budget monitor fed by the inference table. The golden-set eval runs weekly in CI (Chapter 23): a prompt or index change that regresses faithfulness does not ship.

Common Pitfall

Letting the model answer without retrieval evidence. If no chunk clears the similarity threshold, the correct output is abstention. A model answering from parametric knowledge in a support context invents plausible policy—the most expensive sentence your platform will ever publish.

19.6.3 Risks and Mitigations

| Risk                              | Mitigation                                                 |
|-----------------------------------|------------------------------------------------------------|
| PII reaches the vector index      | Redaction before embedding; planted-PII eval test          |
| Stale index serves retired policy | Delta-sync lag alert; abstention on old-document citations |
| Token budget breached mid-month   | Per-day caps with degradation to the smaller model         |
| Prompt change regresses quality   | Golden-set gate in CI blocks the deploy                    |
| Confabulated answers reach agents | Similarity-threshold abstention with scripted fallback     |

Table 19.1: Week 19 scenario risks and mitigations.

19.7 Design Decision Guide

GenAI decisions are measured against the golden set and the token budget—both first.

| Decision                       | Choose the first when                 | Choose the second when                |
|--------------------------------|---------------------------------------|---------------------------------------|
| Small vs. large chunks         | Precise fact lookup                   | Context-heavy synthesis questions     |
| Delta-sync vs. managed index   | Chunks live in Delta and change       | External pipelines produce embeddings |
| Small $k$ vs. large $k$        | Cost/precision focus                  | Recall-critical questions             |
| Pay-per-token vs. provisioned  | Variable or demo traffic              | Measured p95 latency mandates         |
| RAG vs. fine-tuning            | Knowledge changes, citations required | Style/format/behavior adaptation      |
| Abstain vs. best-effort answer | Accuracy-critical domains             | Brainstorming aids                    |

Table 19.2: Week 19 decision guide.

Two invariants: the chunks table is the governed source of truth, and the eval gate decides what ships. Everything else—models, chunkers, prompts—is interchangeable behind those two.

## 19.8 Operational Guidance for Week 19

### 19.8.1 Performance and Reliability

Index freshness is the RAG system’s uptime: monitor Delta-sync lag and alert when the index trails the chunks table. Run the golden-set evaluation on every prompt/model/index change—in CI once Week 23 lands—and block regressions. Track token usage per request with a daily budget alert; the cost curve is the first place a misbehaving loop or oversized context shows up.

### 19.8.2 Security and Governance Checklist

- PII redaction runs before embedding; a planted-PII test in the eval set proves it.
- Metadata filters enforce the caller’s data scope at retrieval time.
- Serving endpoints ACL’d; inference logging enabled for audit.
- Source documents’ UC classification governs what may enter the chunks table.

### 19.8.3 Troubleshooting Playbook

| Symptom                      | Likely cause                        | First action                                            |
|------------------------------|-------------------------------------|---------------------------------------------------------|
| Answers cite wrong documents | Chunking or metadata-filter gap     | Inspect retrieved chunks for golden questions; retune   |
| Index stale after doc update | Change Data Feed off / sync failure | Verify CDF on the chunks table; check index sync status |
| Token cost spiked            | Context too large or retry loop     | Inspect per-request token logs; cap context and retries |
| Confident wrong answers      | Retrieval abstention missing        | Add similarity threshold with scripted fallback         |

Table 19.3: Week 19 troubleshooting quick reference.

## 19.9 Interview Q&A

### 19.9.1 Concepts and Trade-offs

- Q: Why keep chunks in Delta if the vector index is what serves queries?**  
A: Delta is the governed, versioned, rebuildable source of truth; the index is a derived projection. When embeddings models change or a bad sync corrupts the index, you rebuild from the table—and lineage/audit cover the content.
- Q: Managed versus Delta-sync index?**  
A: Delta-sync when your chunks live in Delta and freshness matters—the index follows the table automatically. Managed (direct vector writes) when a non-Databricks pipeline produces embeddings.
- Q: How does chunk size affect quality?**  
A: Small chunks retrieve precisely but lose context; large chunks carry context but dilute similarity. The golden set answers it empirically—there is no universal size.

4. **Q: How do you evaluate a RAG system?**

A: Fixed golden set; retrieval metrics (source-hit rate) and generation metrics (LLM-judged faithfulness/relevance) logged per run; changes gated on the metrics in CI.

5. **Q: Where does governance enter a GenAI pipeline?**

A: At every stage: UC grants on the chunks table, PII redaction before embedding, meta-data filters as retrieval-time row security, endpoint access control, and inference logging for audit.

6. **Q: Why not fine-tune instead of RAG for the copilot?**

A: Fine-tuning changes behavior, not knowledge: it cannot cite, it goes stale with the content, and it costs orders of magnitude more per update. RAG keeps knowledge in governed data where freshness and revocation exist.

7. **Q: What is the RAG system's rollback story?**

A: The chunks table is versioned Delta; the index is rebuildable from it; prompts and models are versioned artifacts behind the eval gate. Every layer rolls back independently.

8. **Q: How do you onboard a new document corpus into the assistant?**

A: Land it in the governed volume, classify it, chunk it through the same pipeline, and let Delta-sync update the index—new content is a data operation, not a model operation.

## 19.10 Further Reading

The Mosaic AI documentation covers the GenAI surface [35]; Vector Search specifics are in [48]; model serving in [33]; MLflow evaluation in [56]. The Unity Catalog governance of the chunks table reuses Chapter 8 [51].

## Key Takeaways

- RAG is a search problem feeding a generation step—retrieval quality bounds everything.
- Chunks live in governed Delta; the vector index is a derived, rebuildable projection.
- The golden evaluation set is the unit-test suite of the whole system; build it first.
- GenAI cost is per-token arithmetic:  $k$ , context length, and model choice are the dials; log usage per request.
- Abstention beats confabulation: no retrieval evidence, no answer.

## 19.11 Exercises

1. **(Conceptual)** Explain why embedding must happen *after* PII redaction, not before with filtered retrieval.
2. **(Conceptual)** What does a Delta-sync index guarantee about freshness, and what does it not guarantee?
3. **(Hands-on)** Complete the Thursday lab including the diagnosed retrieval failure.
4. **(Hands-on)** Run the chunk-size comparison and report the retrieval-quality/cost trade-off.
5. **(Hands-on)** Add abstention on low similarity scores; add a golden question that requires it.

6. **(Design)** Design the two-model cost strategy (draft vs. escalate) with the routing rule.
7. **(Operations)** Write the daily token-budget alert and the action it triggers.
8. **(Architecture)** When would you fine-tune instead of RAG? Give a workload where each wins.

# Model Serving, Monitoring, and MLOps

## Learning Objectives

- Deploy registered models to serverless serving endpoints with governed access.
- Implement idempotent batch scoring with alias-loaded models.
- Capture inference tables and monitor input drift and prediction quality.
- Build the retraining loop with metric thresholds and human-gated promotion.
- Execute rollback by alias re-pointing and prove it.
- Design an auditable MLOps control plane for a regulated operator.

## Cross-Cloud Equivalence

Model serving and monitoring close the MLOps loop; each platform pairs an endpoint service with a drift monitor.

| Capability        | Databricks                     | AWS                       | Azure / GCP / Fabric                           |
|-------------------|--------------------------------|---------------------------|------------------------------------------------|
| Real-time serving | Model Serving (serverless)     | SageMaker endpoints       | Azure ML online / Vertex endpoints             |
| Batch scoring     | Spark UDF from registry + jobs | SageMaker Batch Transform | Batch endpoints / Vertex batch / notebook jobs |
| Request logging   | Inference tables (Delta)       | Endpoint data capture     | Application Insights / request logging         |
| Drift monitoring  | Lakehouse Monitoring           | SageMaker Model Monitor   | Azure ML monitoring / Vertex monitoring        |
| Promotion         | Alias + CI gate                | Registry approval states  | Registered model stages / aliases              |
| Rollback          | Alias re-point                 | Endpoint config rollback  | Traffic shift / alias                          |

The Databricks distinction: inference logs are *Delta tables in your catalog*—monitoring is a SQL/BI problem over data you already govern, not a separate observability silo.

## 20.1 Week 20 Roadmap and Mental Model

A registered model is inventory; Week 20 makes it a *service*. Serving exposes the champion to applications; batch scoring applies it at pipeline scale; monitoring watches what production does to it; and the promotion loop—gated, evidenced, reversible—decides when the challenger becomes the champion [33, 29].

The mental model: **production ML is a control loop, not a deployment**. Sense (inference



tables, drift metrics), decide (thresholds, human approval), act (retrain, promote by alias), verify (post-promotion metrics, rollback readiness). Each stage is a pipeline you already know how to build; MLOps is wiring them with evidence.

### Definition

**Model Serving** exposes registered models as REST endpoints (serverless, scaling to zero, or provisioned throughput). **Inference tables** capture endpoint requests and responses into Delta. **Lakehouse Monitoring** computes profile and drift metrics over any Delta table—including inference logs and feature tables.

### Key Terms

**Serverless endpoint**: scale-to-zero model serving. **Provisioned throughput**: reserved capacity for latency guarantees. **Inference table**: the Delta log of requests/responses behind an endpoint. **Drift**: distribution change in inputs or predictions versus the training baseline. **Champion/challenger**: alias pattern for gated promotion.

## 20.2 Monday: Model Serving

### 20.2.1 Serverless Endpoints

Deploying is a registry-to-endpoint operation: pick the model version (by alias), the compute size, scale-to-zero, and—always—the inference table. Access is governed: endpoint-level permissions decide who can query, manage, or even see the endpoint [33].

```

1 from databricks.sdk import WorkspaceClient
2 from databricks.sdk.service.serving import (EndpointCoreConfigInput,
3     ServedEntityInput, AutoCaptureConfigInput)
4
5 w = WorkspaceClient()
6 w.serving_endpoints.create(
7     name="churn-champion",
8     config=EndpointCoreConfigInput(
9         served_entities=[ServedEntityInput(
10             entity_name="ml.churn_model",
11             entity_version=None,           # resolve via alias below
12             workload_size="Small",
13             scale_to_zero_enabled=True)],
14         auto_capture_config=AutoCaptureConfigInput(
15             catalog_name="de_training", schema_name="ml",
16             table_name_prefix="churn_inference", enabled=True)),
17     # In the UI/API you can bind the served entity to the @champion alias,
18     # so promotion changes what the endpoint serves without redeploying it.
19 )

```

Listing 20.1: Creating a governed serverless endpoint with inference logging.

### 20.2.2 Endpoint Hygiene

Scale-to-zero for intermittent traffic; provisioned throughput when measured p95 demands it. Every endpoint gets: inference logging on, an owner tag, and a queryable health dashboard (latency, error rate, request volume)—the same operational surface you would demand of any production service.

**Common Pitfall**

**The unlogged endpoint.** An endpoint without an inference table is unmonitorable by construction: no drift detection, no audit trail, no answer to “what did the model say to this customer?” Inference capture is not an option; it is the price of admission.

## 20.3 Tuesday: Batch Scoring at Scale

### 20.3.1 The Alias-Loaded UDF

Batch scoring loads the champion as a Spark UDF and writes results idempotently—the Chapter 3 patterns applied to predictions:

```

1 import mlflow
2 from pyspark.sql import functions as F
3
4 predict = mlflow.pyfunc.spark_udf(
5     spark, "models:/ml.churn_model@champion", result_type="double")
6
7 as_of = dbutils.widgets.get("as_of_date")
8 features = (spark.table("de_training.ml.customer_features")
9             .where(F.col("as_of_date") == as_of))
10
11 scored = features.withColumn("churn_score", predict(F.struct(*features.columns)
12 ))
13 (scored.write.format("delta").mode("overwrite")
14  .option("replaceWhere", f"as_of_date = '{as_of}'") # rerun-safe
15  .saveAsTable("de_training.ml.churn_scores"))

```

Listing 20.2: Idempotent batch scoring from the governed feature table.

### 20.3.2 Scoring Discipline

Score from the governed feature tables (Chapter 18), never from ad-hoc joins—that is training-serving skew by design. The scoring job is a Workflow task gated on feature freshness; its output is itself a governed table with the model version recorded per partition, so “scored with which model” is always answerable.

## 20.4 Wednesday: Monitoring and the Retraining Loop

### 20.4.1 Lakehouse Monitoring

Create a monitor over the inference table (or the scores table): it computes profile metrics and drift versus a baseline (the training snapshot), refreshing on a schedule and writing metrics tables you can dashboard and alert on [29]. Input drift (feature distributions shifting), prediction drift (score distribution shifting), and—when labels arrive—quality decay are three different alarms with three different responses.

### 20.4.2 The Gated Loop

Drift alarm → retraining job (on fresh snapshots) → challenger registered → evaluation gate (beats champion on the fixed eval set *and* passes fairness/quality checks) → **human approval** → alias moves. Never auto-promote: retraining on drifted data proves the new model fits the new data, not that it serves the business better.

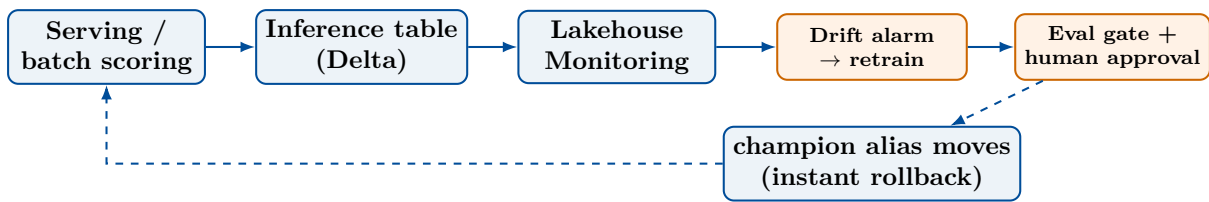


Figure 20.1: The MLOps control loop: sense, decide (with a human gate), act by alias, stay reversible.

## 20.5 Thursday Hands-on Project: Serve, Score, Drift, Gate

Deploy the Week 17 champion, run idempotent batch scoring, inject skewed inputs, and prove the monitor alarms while retraining fires—but promotion waits for a human.

```

1 # 1. Endpoint + inference table (1st:ch20-endpoint); send 100 requests.
2 # 2. Batch score with replaceWhere idempotency (1st:ch20-scoring);
3 #   rerun the same as_of date -> identical row count, no duplicates.
4
5 # 3. Create the monitor over the inference table (UI or API):
6 from databricks.sdk.service.catalog import MonitorInfoSec, MonitorMetric
7 # (shown schematically; the UI flow: Catalog > inference table > Quality)
8 #   baseline = training snapshot; schedule = daily; output = ml.monitors
9
10 # 4. Inject drift: score a batch where spend_90d is shifted 10x.
11 # 5. Read the drift metrics table -> input_drift alert fires.
12 # 6. Retraining job runs; challenger registers; eval gate runs;
13 #   WITHOUT the human approval tag, the alias does NOT move:
14 from databricks.sdk import WorkspaceClient
15 w = WorkspaceClient()
16 champ = w.model_registry.get_model_alias("ml.churn_model", "champion")
17 assert champ.version == "3" # still v3: the gate held
  
```

Listing 20.3: Week 20 lab: drift drill and the promotion gate.

### Deliverables

- The serving endpoint with inference table; latency evidence from a 100-request run.
- The idempotent scoring job with rerun proof.
- The drift-drill evidence: metric rows, the alarm, the held promotion.

### Acceptance Criteria

- Scoring rerun on the same `as_of_date` changes zero rows.
- The injected skew is detected by the monitor (metric evidence, not vibes).
- Retraining fires on the alarm; promotion demonstrably requires the approval step.
- Rollback is executed (alias back to the previous version) and verified with a live request.

### Stretch Goals

- Add prediction-drift and (with delayed labels) quality-decay monitors; document the three alarm responses.
- Wire the whole loop as a Workflow with the approval as a manual task.

- Canary the challenger: 10% of traffic, compare live metrics, then promote.

## 20.6 Friday Use Case: The Lender’s MLOps Control Plane

A consumer lender must show its regulator, for every model in production: drift evidence over time, approval records for every promotion, and a demonstrated rollback capability. Quarterly exams sample these.

### 20.6.1 Requirements and Assumptions

Twelve models in production; batch scoring nightly; two real-time endpoints. Regulators sample quarterly and expect evidence within one business day.

### 20.6.2 Architecture Decisions

**Evidence by default.** Every endpoint has an inference table; every inference table and feature table has a monitor; every monitor’s metrics land in governed tables; every promotion is a CI step that requires the approval tag (Chapter 17) and writes the decision record. The quarterly exam is a SQL query pack over system tables, registry tags, and monitor metrics—assembled continuously, not in a panic.

**The rollback drill.** Quarterly, in staging and annually in production (off-peak, guarded): move the alias back, verify with a scored request, move it forward, file the timings. Rollback that has never been executed is a hypothesis; the drill is the evidence the regulator actually asks for.

#### Common Pitfall

**Monitoring the model but not the features.** A model can be stable while its inputs rot—a broken upstream job serving stale features is invisible to prediction-drift monitors that only watch outputs. Monitor the feature tables (freshness + distribution) alongside the inference table; Chapter 18’s freshness gate is part of the same control plane.

### 20.6.3 Risks and Mitigations

| Risk                                       | Mitigation                                             |
|--------------------------------------------|--------------------------------------------------------|
| Drift alarm noise mutes the channel        | Monthly precision review; seasonal baselines           |
| Stale features masquerade as model failure | Feature monitors run beside prediction monitors        |
| Regulator request exceeds retention        | Inference/metrics retention aligned to exam schedule   |
| Rollback drill never executed              | Quarterly drill with timing recorded as an SLA metric  |
| Champion overwritten without evidence      | Alias writes restricted to the gated pipeline identity |

Table 20.1: Week 20 scenario risks and mitigations.

## 20.7 Design Decision Guide

Serving decisions close the loop; reversibility is the design goal.

| Decision                      | Choose the first when          | Choose the second when                  |
|-------------------------------|--------------------------------|-----------------------------------------|
| Endpoint vs. batch scoring    | Interactive applications       | Bulk periodic decisions                 |
| Scale-to-zero vs. provisioned | Intermittent traffic           | Measured latency-critical load          |
| Gated vs. automatic promotion | Anything business-critical     | Internal experiments only               |
| Monitor inputs vs. outputs    | Both—inputs catch pipeline rot | Outputs catch behavior shift            |
| Canary vs. big-bang release   | Traffic can split safely       | Small blast radius and instant rollback |
| Alias rollback vs. re-deploy  | Always alias                   | Never redeploy to roll back             |

Table 20.2: Week 20 decision guide.

The control loop only works end to end: inference tables without monitors are logs nobody reads; monitors without a gated promotion path are alarms nobody can act on.

## 20.8 Operational Guidance for Week 20

### 20.8.1 Performance and Reliability

Treat endpoints as services: latency/error-rate dashboards, scale-to-zero reviewed against actual traffic, and a promotion log (who approved, when, which metrics). Review monitor alarm precision monthly—a drift alarm that always fires gets muted, and a muted alarm is worse than none. Schedule the rollback drill; record its duration as a first-class SLA metric.

### 20.8.2 Security and Governance Checklist

- Endpoint query permissions scoped to calling applications’ identities.
- Inference tables classified—they contain request payloads, possibly PII.
- Promotion requires the approval tag enforced by CI, not by convention.
- Scores tables record the model version per partition for traceability.

### 20.8.3 Troubleshooting Playbook

| Symptom                       | Likely cause                            | First action                                            |
|-------------------------------|-----------------------------------------|---------------------------------------------------------|
| First request slow after idle | Scale-to-zero cold start                | Expected; provisioned concurrency if p95 mandates it    |
| Drift alarm fires weekly      | Threshold tighter than real seasonality | Rebaseline against weekly seasonality; widen bands      |
| Scoring rerun duplicated rows | Non-idempotent write                    | Convert to <code>replaceAll/ keyed MERGE</code>         |
| “Rollback” changed nothing    | Alias not what serving resolves         | Verify the served entity binds the alias, not a version |

Table 20.3: Week 20 troubleshooting quick reference.

## 20.9 Interview Q&A

### 20.9.1 Concepts and Trade-offs

1. **Q: Serverless endpoint versus batch scoring—how do you choose?**

A: By consumption pattern: interactive applications need endpoints (latency, scale-to-zero economics); bulk periodic decisions need batch scoring through Spark (throughput economics, pipeline integration). Many models need both—from the same registry version.

2. **Q: What does the inference table buy you?**

A: The raw material for everything after deployment: drift monitoring, audit trails, quality measurement when labels arrive, and dispute answers—all as governed Delta data.

3. **Q: Why must promotion stay human-gated?**

A: Automated retraining proves fit to recent data, not business acceptability—fairness, stability, and threshold effects need judgment. Automate everything up to the gate; gate the alias move.

4. **Q: How fast is rollback, and why?**

A: As fast as an alias re-point: serving resolves the alias, so rollback is a registry operation, not a deployment. The drill exists because untested rollbacks fail when needed.

5. **Q: Your champion drifts weekly. Retrain weekly?**

A: First ask why inputs drift that fast (feature bug? seasonality? genuine behavior change?). Retraining cadence follows the drift mechanism, not the alarm frequency—an alarm you always answer with retraining is a cron job with extra steps.

6. **Q: What changes when a model serves both batch and real-time?**

A: Nothing about the model—the same registry version backs both paths. What changes is your testing: the batch and endpoint outputs must be reconciled, because serving-mode divergence is its own drift class.

7. **Q: Which monitor would you build first with one week to launch?**

A: Input drift on the top features plus the freshness gate. Prediction drift without label latency teaches little; input drift catches the pipeline failures that actually happen first.

8. **Q: What does “the model is in production” mean, concretely?**

A: An alias resolves to a version with logged lineage; an endpoint or job serves it; inference is captured; monitors watch it; a gate controls its promotion; a drill proves its rollback. Six artifacts, all queryable.

## 20.10 Further Reading

Model Serving [33] and Lakehouse Monitoring [29] are the product references; the registry mechanics come from Chapter 17 [32, 56]; the feature freshness contract from Chapter 18 [27]. The Workflow wiring reuses Chapter 12 [25].

## Key Takeaways

- Production ML is a control loop: inference tables sense, monitors measure, gates decide, aliases act.
- Every endpoint gets inference logging, an owner, and a health dashboard—no exceptions.
- Batch scoring is alias-loaded, feature-table-fed, and partition-idempotent.

- Retraining is triggered by drift; promotion is gated by humans; rollback is drilled.
- In regulated settings, the evidence pack is continuous queries over governed tables, not a quarterly scramble.

## 20.11 Exercises

1. **(Conceptual)** Why does “the model said so” fail an audit, and what artifact answers it correctly?
2. **(Conceptual)** Distinguish input drift, prediction drift, and quality decay; give a business cause of each.
3. **(Hands-on)** Complete the Thursday lab, including the rollback verification.
4. **(Hands-on)** Build the three-alarm dashboard (input drift, prediction drift, endpoint health) over the metrics tables.
5. **(Hands-on)** Write the eval-gate notebook: challenger beats champion on the fixed set *and* approval tag present, else no alias move.
6. **(Design)** Design the quarterly evidence pack for the Friday scenario as a list of queries and artifacts.
7. **(Operations)** Write the rollback runbook with timings and verification steps.
8. **(Architecture)** Where would the RAG endpoint from Chapter 19 plug into this control plane? What monitors differ for LLM outputs?

## 20.12 Milestone 5 Capstone: The End-to-End MLOps Pipeline

### Capstone Project

**Mission.** Close Part V by operating the full ML lifecycle: leakage-safe feature tables, tracked and tagged training, a UC-registered champion, a RAG assistant over the platform’s own runbooks, serverless serving with inference capture, idempotent batch scoring, drift monitoring, and a human-gated retraining loop.

**Estimated time.** 8–10 hours.

**What you’ll practice.**

- Point-in-time feature engineering with registry lineage.
- Alias-based deployment and rollback.
- RAG with a golden-set evaluation gate.
- Monitoring-driven retraining with held promotion.

### Scenario

Contoso’s retention team gets a churn score list every morning from a notebook nobody owns, on features nobody can reproduce. You are replacing it with the governed MLOps platform—and adding the runbook RAG assistant the on-call team keeps asking for.

### Requirements

- Feature tables with as-of discipline; training set via timestamp lookups; the leakage negative control documented.

- Six tracked runs; champion registered and alias-served; rollback demonstrated.
- RAG assistant with a 10-question golden set and logged evaluation metrics.
- Serverless endpoint with inference table; batch scoring with `replaceWhere` idempotency.
- Drift monitor with an injected-skew alarm; retraining fires; promotion waits for approval.
- README with architecture diagram, DBU/token cost estimate, and the failure story.

## Reference Architecture

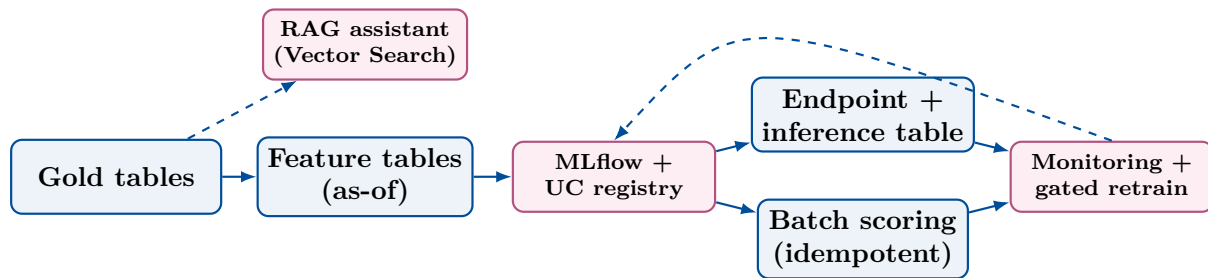


Figure 20.2: Milestone 5 reference architecture: governed MLOps loop with the RAG branch.

## Implementation Steps

1. Feature jobs + training set + lineage (Chapter 18).
2. Tracked sweep, registry, alias, rollback drill (Chapter 17).
3. Endpoint, scoring job, monitors, drift drill, gated promotion (Chapter 20).
4. RAG pipeline with golden-set evaluation (Chapter 19).
5. Cost report (DBUs + tokens) and README; present the failure story.

## Deliverables

- All code in Git; the architecture diagram; the runbook.
- Evidence pack: lineage query, drift alarm, held promotion, rollback log, eval metrics.
- Cost report with at least two documented optimizations.

## Acceptance Criteria

- Point-in-time correctness is proven, not asserted; the leaky control is documented.
- A rerun of batch scoring changes zero rows; a replay of the endpoint load changes nothing downstream.
- Drift demonstrably triggers retraining and demonstrably does not promote without approval.
- The RAG assistant abstains correctly on an out-of-corpus question.



**Stretch Goals**

- Canary deployment of the challenger with live metric comparison.
- Add the feature-freshness gate to the scoring workflow and demonstrate a held score.
- Wire the weekly golden-set RAG eval into CI (preview of Chapter 23).

## Part VI

# Security, Production, and Certification

# Security, IAM, SCIM, and Network Isolation

## Learning Objectives

- Design identity architecture: account console versus workspace, SSO, and SCIM provisioning.
- Operate service principals and OAuth machine-to-machine credentials for automation.
- Manage secrets with Databricks-backed and Key Vault-backed secret scopes.
- Apply network controls: private connectivity, IP access lists, and storage firewalls.
- Enforce personal access token policy and audit authentication events.
- Design time-boxed contractor access and production job identity.

## Cross-Cloud Equivalence

Identity and network isolation are **cloud-native disciplines** with a Databricks control-plane twist.

| Capability         | Databricks                                      | AWS                         | Azure / GCP                            |
|--------------------|-------------------------------------------------|-----------------------------|----------------------------------------|
| Workforce identity | SSO + SCIM from IdP                             | IAM Identity Center         | Entra ID / Cloud Identity              |
| Machine identity   | Service principal + OAuth M2M                   | IAM role                    | Managed identity / service account     |
| Long-lived secret  | PAT (policy-controlled)                         | Access keys                 | Client secrets / SA keys               |
| Secrets store      | Secret scopes (DB- or vault-backed)             | Secrets Manager             | Key Vault / Secret Manager             |
| Private link       | Private connectivity to control plane + storage | PrivateLink + VPC endpoints | Private Link / Private Service Connect |
| Perimeter control  | IP access lists                                 | Security groups, SCPs       | NSGs, firewall rules / VPC SC          |
| Audit              | system.access.audit + auth logs                 | CloudTrail                  | Activity Log / Cloud Audit Logs        |

**Snowflake** mirrors this with SSO/SCIM, network policies, and Tri-Secret/CMK options; **MongoDB Atlas** with federated auth, private endpoints, and IP access lists—the same checklist, different console.

## 21.1 Week 21 Roadmap and Mental Model

Part VI hardens everything Parts I–V built. Week 21 covers the perimeter and the principals: who (and what) can reach the platform, and what they can do once inside. The recurring failure this week prevents: production pipelines running as a person, secrets in notebooks, and a network posture that assumes the SaaS control plane is “handled” [39].

The mental model: **identity first, network second, secrets always**. Identity decides who acts (users via SSO, automation via service principals); network decides from where (private connectivity, IP lists); secrets decide with what credentials (scoped, vaulted, rotated). Auditors and attackers both start from these three lists.

### Definition

A **service principal** is a non-human identity for automation, granted least-privilege access and authenticated via OAuth M2M or a managed token. **SCIM** is the provisioning protocol that syncs users and groups from your identity provider into the Databricks account, automating joiner/mover/leaver hygiene.

### Key Terms

**SSO**: single sign-on via SAML/OIDC from the IdP. **Secret scope**: a named, ACL-controlled secrets namespace; Databricks-backed or backed by Azure Key Vault / AWS Secrets Manager. **IP access list**: allow/deny network ranges for workspace access. **Private connectivity**: private endpoints between your network, the control plane, and storage. **PAT policy**: workspace rules on token creation, lifetime, and scope.

## 21.2 Monday: Identity Architecture

### 21.2.1 Account versus Workspace

The **account console** governs identity (users, groups, service principals), metastores, and budgets across workspaces; the **workspace** governs objects and grants within it. SSO binds the account to your IdP (Entra ID, Okta, Google); users then reach workspaces by entitlement. The discipline: identity changes happen at the account, from the IdP—never as one-off local users.

### 21.2.2 SCIM Provisioning

SCIM keeps Databricks group membership a projection of IdP groups: a new analyst appears in **analysts** on day one; a leaver loses access at HR offboarding, not at the quarterly review [37]. Design the IdP group tree first (business unit × role × environment), then map it—the Chapter 8 access matrix becomes self-maintaining.

### Common Pitfall

**Local users beside SCIM**. One hand-created user bypasses the whole leaver process. Policy: account-level user creation is IdP-only; audits diff workspace membership against SCIM-synced groups monthly.

## 21.3 Tuesday: Service Principals, OAuth, and Secrets

### 21.3.1 Machine Identity

Production jobs, CI pipelines, and the RAG endpoint all authenticate as service principals: least-privilege grants (exactly the Chapter 8 matrix), OAuth M2M credentials with short-lived tokens, and an owner recorded per principal. The rule is absolute: **no production workload runs as a person**—people leave, rotate, and get phished; principals are scoped, owned, and auditable.

### 21.3.2 Secret Scopes

Notebooks read credentials at runtime via `dbutils.secrets.get(scope, key)`; the value never reaches code, logs (Databricks redacts scope-read values in output), or Git [31]. Databricks-backed scopes are simple; Key Vault/AWS-backed scopes put rotation and policy in your cloud vault—choose the vault-backed scope when rotation SLAs and cross-service sharing matter.

```
1 # The secret lives in the vault-backed scope; the notebook reads a handle.
2 kafka_password = dbutils.secrets.get(scope="kv-prod", key="kafka-password")
3 # Databricks redacts this value if printed; it never exists in source control.
```

Listing 21.1: The only acceptable way a notebook touches a credential.

## 21.4 Wednesday: Network Isolation

### 21.4.1 Private Connectivity and Perimeter Controls

The network posture has three surfaces [39]: (1) **user-to-control-plane**—IP access lists and conditional access at the IdP; (2) **control-plane-to-data-plane**—private connectivity so orchestration traffic avoids the public internet; (3) **cluster-to-storage/data-sources**—private endpoints and storage firewall rules so data-plane traffic stays on the cloud backbone. Classic compute runs in your VNet/VPC (or a Databricks-managed one), so your existing egress controls apply to clusters.

### 21.4.2 What IP Lists Do Not Do

An IP access list is a perimeter, not authentication: it stops the coffee-shop login, not the phished token replayed from the office VPN. Defense in depth means the list plus SSO with MFA plus short-lived tokens plus audit alerts—each layer catching what the others miss.

| Control              | Stops                         | Does not stop                            |
|----------------------|-------------------------------|------------------------------------------|
| SSO + MFA            | Password-only compromise      | Token theft after login                  |
| SCIM                 | Orphaned leaver access        | Privilege creep within synced groups     |
| IP access lists      | Off-network access            | Stolen credentials from allowed networks |
| Secret scopes        | Secrets in code/logs          | Over-broad scope ACLs                    |
| Private connectivity | Public-internet data exposure | Misconfigured endpoints/DNS              |

Table 21.1: Each control has a known blind spot; the posture is the combination.

## 21.5 Thursday Hands-on Project: Harden a Lab Workspace

Provision a service principal, scope its grants, move a notebook off a hardcoded PAT, add an IP access list, and prove the blocked path is blocked.

```

1 # 1. Service principal with a least-privilege grant set
2 databricks service-principals create --display-name sp-etl-lab
3 #    -> grant per Chapter 8: MODIFY on bronze/silver/gold, nothing else
4
5 # 2. Secret scope + secret
6 databricks secrets create-scope kv-lab
7 databricks secrets put-secret kv-lab kafka-password --string-value '<from-vault
8   >'
9
10 # 3. Convert the notebook: delete the PAT literal, replace with
11 #    dbutils.secrets.get("kv-lab", "kafka-password") (see lst:ch21-secrets)
12
13 # 4. IP access list: allow the office range only
14 databricks ip-access-lists create --label office --list-type ALLOW '
15   --ip-addresses '["203.0.113.0/24"]'
16
17 # 5. Negative test: from an off-list network (phone hotspot),
18 #    databricks current-user me -> must be denied; capture the 403.

```

Listing 21.2: Week 21 lab: principal, scope, and perimeter.

### Deliverables

- The service principal with its grant list (and the one-line justification per grant).
- The converted notebook running without any literal credential.
- The IP access list with the denied-request evidence.

### Acceptance Criteria

- The principal cannot read outside its schemas (negative test captured).
- A repository secret scan comes back clean; the old PAT is revoked.
- The blocked-network test fails closed (deny), with the audit event found in system tables.
- The lab runs as the service principal end to end, proving person-independence.

### Stretch Goals

- Migrate the scope to Key Vault-backed and document the rotation procedure.
- Set the workspace PAT policy (max lifetime, admin-only creation) and show a violating token request denied.
- Write the monthly SCIM-vs-workspace membership diff script.

## 21.6 Friday Use Case: Contractors, Service Principals, and No Public Internet

A platform team must onboard 15 contractors for a 90-day engagement, run all production jobs as non-human identities, and move all data-plane traffic off the public internet—without slowing the two dozen engineers already onboard.

### 21.6.1 Requirements and Assumptions

Contractors need Silver read and Gold read/write in one domain, nothing else, auto-expiring at day 90. Production spans 40 jobs. The security team requires private connectivity evidence for the next audit.

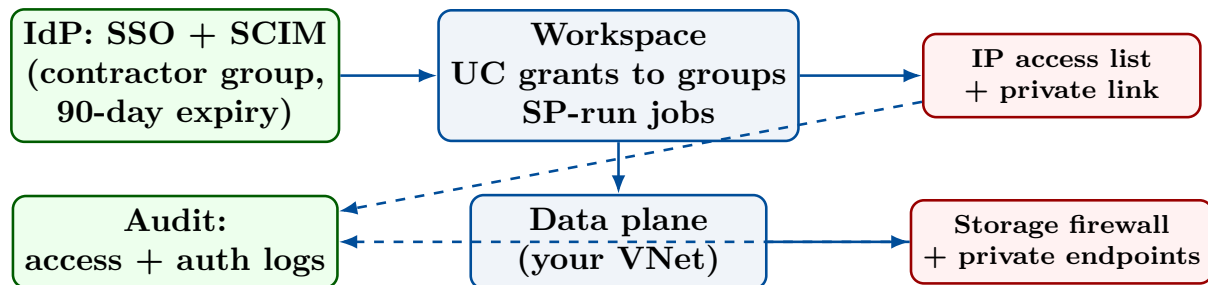


Figure 21.1: Zero-public-internet posture: IdP-driven identity, group-scoped grants, private data paths, continuous audit.

### 21.6.2 Architecture Decisions

**Contractor lifecycle as IdP configuration.** A `contractors-engagement-7` IdP group with a 90-day membership expiry; SCIM syncs it; UC grants target the group at the domain’s schemas. Offboarding is the expiry firing—evidenced by the membership diff script—not a ticket someone remembers.

**Private data plane.** Private endpoints for storage, storage firewall rules denying public access, private connectivity for control-plane traffic, and IP access lists on the workspace. The audit evidence is the negative test (public path denied) plus the endpoint inventory—asserted configurations are not evidence.

#### Common Pitfall

**Granting contractors to shared groups.** “Just add them to `data-engineers` for now” defeats expiry (the group outlives the engagement) and scope (the group sees all domains). One group per engagement with per-domain grants is five minutes of IdP work and zero cleanup risk.

### 21.6.3 Risks and Mitigations

| Risk                                      | Mitigation                                                     |
|-------------------------------------------|----------------------------------------------------------------|
| Contractor access outlives the engagement | IdP group expiry; membership diff evidences the exit           |
| Service-principal secret leaks            | Vault rotation runbook; audit baselines catch volume anomalies |
| Private path bypassed by a new route      | Quarterly negative tests; config drift detection               |
| SCIM outage strands new joiners           | Documented manual-override with 48-hour auto-expiry            |
| IP list blocks a legitimate office move   | Change window with staged allow-list; break-glass procedure    |

Table 21.2: Week 21 scenario risks and mitigations.

## 21.7 Design Decision Guide

Security decisions layer; none of these controls is sufficient alone.

| Decision                                  | Choose the first when                | Choose the second when            |
|-------------------------------------------|--------------------------------------|-----------------------------------|
| Service principal vs. PAT                 | Any automated workload               | Short-lived personal CLI use only |
| OAuth M2M vs. long-lived token            | CI/CD and scheduled jobs             | Legacy tooling that cannot OAuth  |
| Vault-backed vs. Databricks-backed scope  | Rotation SLAs, cross-service secrets | Platform-local simplicity         |
| IP access list vs. private connectivity   | User perimeter filtering             | Data-plane traffic privacy        |
| SCIM groups vs. manual membership         | Any shared workspace                 | Never manual at scale             |
| Group expiry vs. ticket-based offboarding | Time-boxed engagements               | Permanent staff via IdP lifecycle |

Table 21.3: Week 21 decision guide.

The layering table from Wednesday (table 21.1) is the check: for each row here, ask which blind spot remains and which other control covers it.

## 21.8 Operational Guidance for Week 21

### 21.8.1 Performance and Reliability

Security operations run on cadence: monthly SCIM-membership diffs, quarterly access reviews with negative tests, and immediate alerts on authentication anomalies (new-country logins, PAT creation bursts). Keep the break-glass path drilled and documented—a lockout discovered during an incident is a design flaw, not bad luck. Token and secret rotation is scheduled work with a runbook, verified by a post-rotation smoke test.

### 21.8.2 Security and Governance Checklist

- Quarterly diff of workspace membership against SCIM-synced groups.
- PAT inventory with expiries; policy caps enforced at the workspace.
- Service principal secrets rotated on schedule with owner sign-off.
- IP access lists and private endpoints re-validated after any network change.



### 21.8.3 Troubleshooting Playbook

| Symptom                                 | Likely cause                            | First action                                        |
|-----------------------------------------|-----------------------------------------|-----------------------------------------------------|
| SCIM group not appearing                | Provisioning scope/filter misconfigured | Check the IdP provisioning log first                |
| <code>dbutils.secrets.get</code> denied | Principal missing scope ACL             | Grant READ on the scope to the run-as identity      |
| Legitimate user locked out              | IP access list too narrow               | Break-glass admin; add the range; review the change |
| CI fails auth after rotation            | SP secret rotated without CI update     | Update the vault entry CI reads; never the code     |

Table 21.4: Week 21 troubleshooting quick reference.

## 21.9 Interview Q&A

### 21.9.1 Concepts and Trade-offs

- Q: Why must production jobs run as service principals?**  
 A: Person-run jobs break when the person leaves, inherit the person’s full access (not least-privilege), and blur accountability. Principals are scoped, owned, rotated, and auditable.
- Q: What does SCIM actually automate?**  
 A: Joiner/mover/leaver synchronization: group membership in Databricks tracks the IdP, so access appears and disappears with HR state—no orphaned accounts, no provisioning tickets.
- Q: Databricks-backed versus vault-backed secret scope?**  
 A: Vault-backed when rotation SLAs, cross-service sharing, or existing vault governance matter; Databricks-backed for simplicity when platform-local secrets suffice.
- Q: Is an IP access list sufficient network security?**  
 A: No—it is a perimeter filter. Stolen credentials from an allowed network pass it. Layer it with SSO/MFA, short-lived tokens, private connectivity, and audit alerts (table 21.1).
- Q: How do you prove least privilege for a service principal?**  
 A: The grant list with per-grant justification, negative tests for excluded access, and an audit sample showing actual use matches granted scope.
- Q: A manager asks for “temporary admin” for a contractor. Response?**  
 A: Time-boxed group membership for the needed scope, expiring automatically—admin is never the temporary answer to a scoped need.
- Q: How do you security-test your own platform?**  
 A: Negative tests as code: principals attempt what they must not do, on a schedule, with failures logged as pass/fail evidence. Security that is only reviewed, not tested, is a rumor.
- Q: Which single control gives the most risk reduction per hour?**  
 A: Running production workloads as least-privilege service principals. It caps blast radius, survives personnel change, and makes every automated action attributable.

### 21.10 Further Reading

The Databricks security and compliance documentation is the reference [39]; SCIM provisioning is covered in [37]; secrets in [31]; audit via system tables in [34]. The identity-to-grants mapping

builds directly on Chapter 8 [51].

## Key Takeaways

- Identity at the account, from the IdP: SSO for people, SCIM for lifecycle, service principals for machines.
- No production workload runs as a person; no secret lives in code—scopes and OAuth M2M make both avoidable.
- Every network control has a known blind spot; the posture is the layered combination plus audit.
- Time-boxed access is IdP configuration (group expiry), not a calendar reminder.
- Security evidence is negative tests and audit queries, not asserted configuration.

## 21.11 Exercises

1. **(Conceptual)** Map a joiner/mover/leaver event to the exact SCIM/UC changes each should cause.
2. **(Conceptual)** Why is PAT policy a governance control rather than a convenience setting?
3. **(Hands-on)** Complete the Thursday lab, including the fail-closed network test.
4. **(Hands-on)** Convert a second notebook to OAuth M2M via environment variables; document the credential lifetime change.
5. **(Hands-on)** Run the membership diff and the authentication audit query for your lab workspace.
6. **(Design)** Write the contractor group design (naming, grants, expiry) for the Friday scenario.
7. **(Operations)** Draft the incident runbook for “service principal secret leaked.”
8. **(Architecture)** What changes in this design on serverless-only compute, where there is no customer VNet? What do you gain and lose?

# Unity Catalog Deep Dive: Lineage, Audit, and Delta Sharing

**Learning Objectives**

- Capture and query table- and column-level lineage for impact analysis and debugging.
- Build an audit practice on `system.access.audit`: sensitive-access alerts and compliance reporting.
- Share live data with Delta Sharing—open protocol and Databricks-to-Databricks—with revocation tested.
- Apply data classification (tags) and use classification-driven policy.
- Trace a Gold metric to its Bronze sources and enumerate every downstream consumer.
- Design consortium data sharing with auditability and revocation.

**Cross-Cloud Equivalence**

Lineage, audit, and sharing are the **accountability layer**; each platform implements them at different depths.

| Capability                | Databricks (UC)                  | AWS                                 | Azure / GCP / Fabric                                          |
|---------------------------|----------------------------------|-------------------------------------|---------------------------------------------------------------|
| Lineage                   | Automatic table + column lineage | Glue lineage (partial) / SageMaker  | Purview / Dataplex / Fabric lineage                           |
| Audit log                 | <code>system.access.audit</code> | CloudTrail + Lake Formation         | Activity Log / Cloud Audit Logs                               |
| Secure sharing            | Delta Sharing (open protocol)    | Redshift sharing / S3 Access Grants | Azure Data Share / BigQuery Analytics Hub / OneLake shortcuts |
| Cross-org compute         | Clean rooms                      | Clean Rooms (AWS)                   | Azure confidential clean rooms                                |
| Classification            | Tags + discovery (PII)           | Macie                               | Purview scans / DLP                                           |
| Recipient without account | Yes (open protocol)              | No (needs AWS entity)               | Varies / BigQuery subscriber                                  |

**Snowflake** shares via listings and reader accounts (its own clean-room story); Delta Sharing’s distinction is the *open* protocol—recipients consume with pandas, Spark, or Power BI without any Databricks account.

## 22.1 Week 22 Roadmap and Mental Model

Chapter 8 built the grants; Week 22 builds the *evidence*: lineage that shows how data flows, audit that shows who touched it, and sharing that extends the governance boundary past your own organization. This is the week the platform starts answering its own hard questions [13, 34].

The mental model: **governance is a query workload**. Lineage is system tables you join; audit is an event log you alert on; sharing is a grant with a network protocol. Treating them as data—queryable, testable, alertable—is what separates a governed platform from a documented one.

### Definition

**Delta Sharing** is an open protocol for sharing live tables and volumes: the provider publishes a **share**; recipients query it through short-lived, pre-signed access without holding storage credentials or copying data. Revocation ends access centrally. **Column-level lineage** records which source columns produced which target columns across notebooks, jobs, and dashboards.

### Key Terms

**Share**: a named set of shared objects. **Recipient**: the consuming principal (Databricks or open-protocol). **Clean room**: a governed multi-party compute environment. **Impact analysis**: enumerating downstream consumers of an object before changing it. **Classification tag**: a UC tag marking sensitivity (e.g. pii=true) that policy and audit reference.

## 22.2 Monday: Lineage

### 22.2.1 What Gets Captured

UC captures lineage automatically for reads/writes through Databricks compute: table-to-table, column-to-column, and into notebooks, jobs, dashboards, and registered models. The queries that change how you work: *upstream* (what produced `gold.fact_orders.total_amount`?—the debugging question) and *downstream* (what breaks if I change `silver.orders.order_ts`?—the impact question) [13].

```

1 -- Everything downstream of silver.orders.order_ts:
2 SELECT DISTINCT target_table_full_name, target_column_name
3 FROM system.access.column_lineage
4 WHERE source_table_full_name = 'de_training.silver.orders'
5     AND source_column_name = 'order_ts';
6
7 -- And the full provenance of a Gold metric:
8 SELECT DISTINCT source_table_full_name, source_column_name
9 FROM system.access.column_lineage
10 WHERE target_table_full_name = 'de_training.gold.fact_orders'
11     AND target_column_name = 'total_amount';

```

Listing 22.1: Column-level impact analysis before a breaking change.

### Tip

Run the downstream query from a pre-merge check in CI (Chapter 23): a PR that changes a column with downstream consumers gets an automatic comment listing them. Impact analysis that requires remembering to run it will not be run.

## 22.3 Tuesday: The Audit Practice

### 22.3.1 system.access.audit as an Alerting Source

The audit table records authentication, grants, reads, writes, and administrative actions with user, object, and timestamp [34]. The practice, in ascending maturity: (1) ad-hoc answers (“who read `dim_customer` last Tuesday?”); (2) scheduled reports (weekly access review per PII-tagged table); (3) alerts (any PII read by a non-approved group pages compliance); (4) baselines (anomaly: this service principal suddenly reads 40x its normal volume).

```

1 SELECT event_time, user_identity.email AS actor, request_params
2 FROM system.access.audit
3 WHERE action_name IN ('getTable', 'commandText')
4     AND request_params.full_name_arg = 'de_training.gold.dim_customer'
5     AND user_identity.email NOT IN
6         (SELECT member FROM de_training.governance.approved_pii_readers)
7     AND event_date >= current_date() - 1;
8 -- Scheduled every 15 minutes; non-empty result pages the steward.
```

Listing 22.2: The PII access alert query.

### 22.3.2 Classification-Driven Policy

Tag sensitive objects (`pii=true`, `domain=finance`) so policy references the classification, not the table list: the alert above generalizes to “any table tagged `pii`,” and new tables inherit scrutiny the day they are tagged. Classification without policy is decoration; policy without classification is a spreadsheet that rots.

## 22.4 Wednesday: Delta Sharing and Clean Rooms

### 22.4.1 Sharing Without Copies

The provider creates a share, adds tables (or partitions of them), and adds recipients. Databricks-to-Databricks recipients see the share in their own catalog; open-protocol recipients get a credential file and query via pandas/Spark/Power BI connectors [47]. The provider’s storage credentials never cross the boundary; access is short-lived and revocable; every read is auditable on the provider side.

```

1 CREATE SHARE supply_chain_orders;
2 ALTER SHARE supply_chain_orders
3     ADD TABLE de_training.gold.fact_orders_shared;
4 GRANT SELECT ON SHARE supply_chain_orders TO RECIPIENT acme_logistics;
5
6 -- The revocation drill (the part everyone skips):
7 REVOKE SELECT ON SHARE supply_chain_orders FROM RECIPIENT acme_logistics;
8 -- Verify: the recipient’s next read fails. Capture both sides as evidence.
```

Listing 22.3: Create, populate, grant, and—critically—revoke a share.

### 22.4.2 Clean Rooms

When parties must compute *together* (match lists, joint aggregates) without either seeing the other’s raw rows, a clean room provides the governed, audited meeting point—notebooks run against both parties’ shared assets with outputs constrained by the room’s rules.

**Common Pitfall**

**Sharing the whole schema.** Shares should contain the curated, agreed artifacts—not the schema they came from. One over-broad `ADD TABLE` is a data-sharing incident waiting for its audit. Share narrow, document the agreement, and test revocation before the first consumer connects.

## 22.5 Thursday Hands-on Project: Trace, Alert, Share, Revoke

Build the lineage trace for a Gold column, wire the PII audit alert, and execute a full share-revoke cycle with an external recipient.

```

1 # Recipient environment (separate machine/env, no Databricks account):
2 import delta_sharing
3 import pandas as pd
4
5 profile = "acme_profile.share" # the credential file you were sent
6 client = delta_sharing.SharingClient(profile)
7 print([t.name for t in client.list_all_tables()])
8
9 df = delta_sharing.load_as_pandas(
10     f"{profile}#supply_chain_orders.de_training.fact_orders_shared")
11 print(len(df))
12
13 # After revocation, the same call raises an authorization error.
14 # Capture the before/after outputs: that pair IS the revocation evidence.

```

Listing 22.4: Week 22 lab: the recipient-side check (open protocol).

### Deliverables

- The column-lineage trace from `gold.fact_orders.total_amount` to Bronze, plus the downstream consumer list.
- The PII alert query scheduled, with one planted violation captured.
- The share lifecycle evidence: grant, recipient read, revoke, recipient denial.

### Acceptance Criteria

- The lineage report includes at least one dashboard and one model as downstream consumers.
- The alert fires on the planted read and stays quiet on approved reads (both evidenced).
- Revocation demonstrably ends the recipient's access.
- The shared object set is the curated view—not the base table—with the agreement documented.

### Stretch Goals

- Add the CI pre-merge impact check using the column-lineage query.
- Classify all Gold tables with sensitivity tags and generalize the alert to tag-driven coverage.
- Share a volume (files) instead of a table and document the difference in recipient experience.

## 22.6 Friday Use Case: The Supply-Chain Consortium

Five companies exchange curated datasets—forecasts, inventory positions, shipment events—with three requirements: no cross-company credentials, complete auditability, and provable revocation when a member leaves.

### 22.6.1 Requirements and Assumptions

Members run mixed platforms (two Databricks, one Snowflake, one pure-Python analytics shop, one Power BI shop). Data agreements are bilateral; leaving the consortium is contractual with a 30-day wind-down.

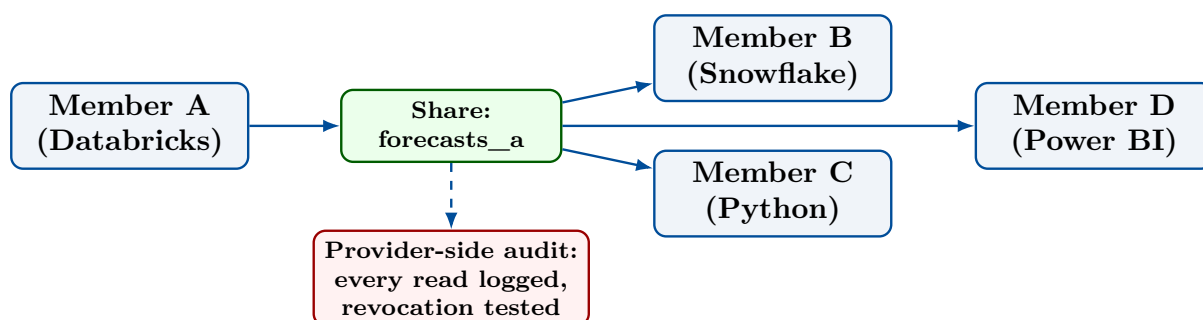


Figure 22.1: Consortium sharing: open-protocol shares serve every member platform; the provider keeps credentials, audit, and the revocation switch.

### 22.6.2 Architecture Decisions

**Open protocol as the equalizer.** Because Delta Sharing is an open protocol, Member B consumes via its Delta Sharing connector, Member C via the Python client, Member D via Power BI—no member needs a Databricks account or the provider’s storage credentials. Each bilateral agreement is a separate share (never a shared share), so obligations, schemas, and revocation are per-agreement by construction.

**The exit drill.** Wind-down is: revoke the member’s share grants (access ends immediately, evidenced), retain the audit history per the agreement, and archive the share definition. The quarterly consortium review includes a live revocation test on a canary share—the same drill from the lab, run where it matters.

#### Common Pitfall

**Emailing extracts “because sharing is hard to set up.”** An emailed CSV is un-auditable, unrevocable, and instantly stale—precisely the three requirements. The share setup is one afternoon; the first leaked extract is a contract breach.

### 22.6.3 Risks and Mitigations

| Risk                                    | Mitigation                                                     |
|-----------------------------------------|----------------------------------------------------------------|
| Share contents drift past the agreement | Share objects are curated views; schema changes reviewed       |
| Revocation assumed but untested         | Quarterly revocation drill on a canary share                   |
| Lineage trusted where coverage ends     | Coverage report lists objects written outside UC-aware compute |
| Audit alerts ignored after false alarms | Precision reviewed monthly; alerts route to a rotation         |
| Member disputes a read claim            | Provider-side audit rows are the shared evidence standard      |

Table 22.1: Week 22 scenario risks and mitigations.

## 22.7 Design Decision Guide

Sharing and audit decisions define who can prove what, afterward.

| Decision                                   | Choose the first when                  | Choose the second when           |
|--------------------------------------------|----------------------------------------|----------------------------------|
| Databricks-to-Databricks vs. open protocol | Both sides run Databricks              | Any non-Databricks recipient     |
| Share vs. extract/copy                     | Live, revocable, auditable access      | Never for governed data          |
| Clean room vs. bilateral shares            | Joint computation without raw exposure | One-directional curated delivery |
| Tag-driven vs. table-list policy           | Estates that grow                      | Tiny fixed estates               |
| Audit alert vs. scheduled report           | Actionable, rare events                | Trend and compliance review      |
| Per-agreement share vs. one big share      | Always per agreement                   | Never bundle obligations         |

Table 22.2: Week 22 decision guide.

The discipline that makes this week stick: every shared artifact can answer, on demand, four questions—who consumes it, under which agreement, when was revocation last tested, and what did they read last quarter.

## 22.8 Operational Guidance for Week 22

### 22.8.1 Performance and Reliability

Lineage and audit pipelines are platform services with their own SLOs: lineage freshness (time from write to visible edge), audit-query latency, and alert precision. Tune the PII alert quarterly—false positives train people to ignore the real event. Inventory shares monthly: every share has an owner, an agreement reference, and a last-revocation-test date.

### 22.8.2 Security and Governance Checklist

- Shares contain curated artifacts only; scope reviewed per agreement.



- Revocation tests scheduled quarterly per share, evidenced on both sides.
- Classification-tag coverage tracked as a metric (percent of tables tagged).
- Audit-alert recipients are a group with an on-call rotation, not a person.

### 22.8.3 Troubleshooting Playbook

| Symptom                     | Likely cause                                | First action                                             |
|-----------------------------|---------------------------------------------|----------------------------------------------------------|
| Lineage gap for a table     | Written outside Databricks compute          | Register the external writer; document the gap           |
| PII alert floods            | Over-broad table set or rule                | Tune to tag-driven scope; baseline normal access         |
| Recipient cannot read share | Credential file expired / share not granted | Re-issue the activation; verify GRANT on the share       |
| Audit query slow            | Scanning months of events unpartitioned     | Filter by event_date first; aggregate to a summary table |

Table 22.3: Week 22 troubleshooting quick reference.

## 22.9 Interview Q&A

### 22.9.1 Concepts and Trade-offs

1. **Q: What does column-level lineage change operationally?**  
A: Impact analysis becomes a query: every downstream consumer of a column is enumerable before you change it, and every metric is traceable to sources after something looks wrong. Debugging and change review both accelerate.
2. **Q: How would you alert on PII access?**  
A: Scheduled query over `system.access.audit` joined to the classification tags and the approved-reader list; non-approved access pages the steward. Tag-driven, so new PII tables are covered on classification day.
3. **Q: Why does Delta Sharing not share storage credentials?**  
A: Recipients get short-lived, pre-signed access per request through the sharing server; the provider’s storage account is never exposed, and revocation is a provider-side grant change with immediate effect.
4. **Q: Open-protocol versus Databricks-to-Databricks sharing?**  
A: D2D is zero-credential and fully governed on both sides (UC end to end); open protocol reaches any platform with a credential file. Use D2D when both sides are Databricks; open protocol for everyone else.
5. **Q: What makes a consortium exit provable?**  
A: The revocation drill: grant, read, revoke, failed read—all four logged. “We believe access ended” is not evidence; the recipient’s denied request is.
6. **Q: How does lineage change schema-change review?**  
A: The review starts from the consumer list, not the table: every downstream table, dashboard, and model is enumerated, so “is this safe” has a concrete answer.
7. **Q: What is the clean-room decision rule?**  
A: When parties must compute jointly without exposing raw rows—match rates, joint aggregates. If one party could just receive a curated table, a share suffices; clean rooms are for mutual privacy.

8. **Q: How do you handle a data subject erasure request in a lakehouse?**

A: Targeted deletes through the layers with deletion vectors, a provenance query to find every copy, and VACUUM timing understood—erasure is a pipeline with evidence, not a one-off script.

## 22.10 Further Reading

Lineage is documented in [13], system-table audit in [34], and Delta Sharing in [47]. The grant model these build on is Chapter 8 [51]; the identity layer underneath is Chapter 21 [39].

## Key Takeaways

- Lineage and audit are system tables: governance questions are queries, and policy is a scheduled job.
- Impact analysis belongs in CI; a column change should enumerate its consumers before merge.
- Classification tags make policy cover new data on day one; untagged policy rots.
- Delta Sharing extends governance past your organization—with no copies, no shared credentials, and provable revocation.
- Test revocation before the first consumer connects; drill it quarterly after.

## 22.11 Exercises

1. **(Conceptual)** Why is lineage capture “free” in UC but historically a separate product elsewhere?
2. **(Conceptual)** What is the difference between auditing access and auditing usage patterns, and which catches exfiltration?
3. **(Hands-on)** Complete the Thursday lab, including the open-protocol recipient test.
4. **(Hands-on)** Build the weekly PII-access review report and the per-domain access summary.
5. **(Hands-on)** Write the CI pre-merge lineage check for one repository.
6. **(Design)** Design the share taxonomy (naming, per-agreement isolation, schema contracts) for the Friday scenario.
7. **(Operations)** Write the member-exit runbook with the evidence checklist.
8. **(Architecture)** When would a clean room beat bilateral shares? Give a concrete consortium computation that requires it.

# CI/CD with Databricks Asset Bundles, Terraform, and Cost Optimization

**Learning Objectives**

- Define jobs, pipelines, and compute as code with Databricks Asset Bundles.
- Structure bundle targets (dev/staging/prod) with variables and mode constraints.
- Provision workspace resources with the Terraform Databricks provider and remote state.
- Build a GitHub Actions pipeline: PR validation, staging gate, prod promotion.
- Produce per-domain chargeback from `system.billing.usage` and act on it.
- Design a delivery and FinOps operating model for a multi-team platform.

**Cross-Cloud Equivalence**

Asset Bundles are Databricks’ **native IaC**; Terraform is the cross-cloud constant; the CI/CD pattern is identical everywhere.

| Capability             | Databricks                                    | AWS                  | Azure / GCP / Fabric                                |
|------------------------|-----------------------------------------------|----------------------|-----------------------------------------------------|
| Native IaC             | Asset Bundles ( <code>databricks.yml</code> ) | CloudFormation / CDK | Bicep / Deployment Manager / Fabric Git + Terraform |
| Cross-platform IaC     | Terraform provider                            | Terraform            | Terraform                                           |
| Validate-before-deploy | <code>bundle validate</code>                  | cfn-lint / CDK synth | <code>terraform plan</code> / what-if               |
| CI auth                | Service principal + OAuth M2M                 | OIDC → IAM role      | OIDC → workload identity                            |
| State                  | Terraform remote state                        | S3 + DynamoDB lock   | Storage account / GCS backend                       |
| Cost telemetry         | <code>system.billing.usage</code>             | CUR 2.0              | Cost Management / billing export / Capacity Metrics |

**Snowflake**’s story is Terraform-first plus database-change tools (schemachange, dbt); the lesson that transfers everywhere: *the merge is the deploy, and the pipeline is the only writer to prod.*

## 23.1 Week 23 Roadmap and Mental Model

Twenty-two weeks of platform skills converge on the delivery question: how does a change travel from a laptop to production *safely*? Week 23’s answer is infrastructure as code (Asset Bundles for Databricks resources, Terraform for the workspace perimeter) behind a CI/CD pipeline with a staging gate—plus the FinOps loop that keeps the whole estate affordable [42, 24].

The mental model: **production is a build artifact**. Nobody clicks in prod; a merge to `main` triggers validation, staging deployment, tests, and—only then—prod deployment. The bundle is the unit of deployment; the target is the environment; the pipeline is the policy.

### Definition

A **Databricks Asset Bundle** is a versioned declaration (`databricks.yml` plus resource files) of jobs, pipelines, and other workspace objects, deployed to named **targets** (`dev/staging/prod`) with variable substitution, deployment state tracking, and `validate/deploy/run` lifecycle commands.

### Key Terms

**Target:** a named deployment environment with its own workspace, variables, and mode. **Mode:** `development` (prefixed, paused schedules) or `production` (unpaused, locked). **Variable:** a bundle input substituted per target. **Remote state:** Terraform’s shared, locked state backend. **Chargeback:** attributing platform cost to domains from billing system tables.

## 23.2 Monday: Asset Bundles

### 23.2.1 Anatomy

A bundle declares resources (jobs, pipelines, dashboards, models) and targets; `databricks bundle validate` checks the declaration against the workspace schema before anything deploys; `bundle deploy` syncs code and applies resources; `bundle run` executes a resource in a target [42].

```

1 bundle:
2   name: governed-platform
3
4 variables:
5   catalog: {default: de_training}
6
7 targets:
8   dev:
9     mode: development      # prefixed names, paused schedules
10    default: true
11    workspace: {host: https://dev-ws.azuredatabricks.net}
12   staging:
13     mode: production
14     workspace: {host: https://stg-ws.azuredatabricks.net}
15     variables: {catalog: de_staging}
16   prod:
17     mode: production
18     workspace: {host: https://prd-ws.azuredatabricks.net}
19     variables: {catalog: de_prod}
20     run_as: {service_principal_name: sp-etl-prod}
21
22 resources:
23   pipelines:

```

```

24   medallion:
25     name: medallion-${bundle.target}
26     catalog: ${var.catalog}
27     libraries:
28       - notebook: {path: ../src/pipelines/medallion.py}
29   jobs:
30     nightly_elt:
31       name: nightly_elt-${bundle.target}
32       tasks:
33         - task_key: pipeline
34           pipeline_task: {pipeline_id: ${resources.pipelines.medallion.id}}
35         - task_key: quality_gate
36           depends_on: [{task_key: pipeline}]
37           notebook_task: {notebook_path: ../src/quality_gate}

```

Listing 23.1: The course bundle: one job, one pipeline, three targets.

### 23.2.2 Targets and Modes

Development mode prefixes resource names and pauses schedules so ten engineers' dev deployments coexist without collisions; production mode unpins schedules and enforces `run_as`. The discipline that prevents the classic incident: *prod is deployed only by CI*, from the pinned merge commit, after staging passes.

#### Common Pitfall

**Deploying prod from a laptop.** Local prod deploys bypass the staging gate, the audit trail, and the pinned commit. Remove human deploy rights to prod targets; CI's service principal holds them. The exception path (break-glass deploy) is documented, credentialed separately, and alarmed.

## 23.3 Tuesday: Terraform for the Workspace Perimeter

### 23.3.1 What Terraform Owns versus What Bundles Own

The split this book uses: **Terraform** provisions the slow-moving perimeter—workspaces, service principals, groups, metastore bindings, cluster policies, secret scopes; **bundles** deploy the fast-moving application—jobs, pipelines, dashboards. Both are code; both go through the same review; neither is clicked in production [24].

```

1 terraform {
2   backend "azurerm" {                                # remote, locked state
3     resource_group_name = "rg-de-terraform"
4     storage_account_name = "stdetfstate"
5     container_name      = "tfstate"
6     key                  = "databricks-prod.tfstate"
7   }
8   required_providers {
9     databricks = { source = "databricks/databricks" }
10  }
11 }
12
13 provider "databricks" { host = var.workspace_host }
14
15 resource "databricks_service_principal" "etl_prod" {
16   display_name = "sp-etl-prod"
17 }
18
19 resource "databricks_cluster_policy" "training" {

```

```

20 name          = "de-training-policy"
21 definition = jsonencode({
22   autotermination_minutes = { type = "fixed", value = 30 }
23   num_workers             = { type = "range", maxValue = 4 }
24 })
25 }
26
27 resource "databricks_grants" "gold_readers" {
28   schema = "de_prod.gold"
29   grant { principal = "analysts"
30         privileges = ["USE_SCHEMA", "SELECT"] }
31 }

```

Listing 23.2: Terraform perimeter for the prod target.

### 23.3.2 State Discipline

Terraform state is the record of what exists: remote (never a laptop file), locked (no concurrent applies), and access-controlled (it describes your security estate). A lost or forked state file is how “Terraform wants to delete the metastore binding” happens.

## 23.4 Wednesday: The Pipeline and the FinOps Loop

### 23.4.1 GitHub Actions Shape

```

1 name: platform-cicd
2 on:
3   pull_request: {branches: [main]}
4   push: {branches: [main]}
5 env:
6   DATABRICKS_CLIENT_ID: ${vars.SP_CLIENT_ID}
7   DATABRICKS_CLIENT_SECRET: ${secrets.SP_CLIENT_SECRET}
8 jobs:
9   validate:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v4
13      - uses: databricks/setup-cli@main
14      - run: databricks bundle validate -t staging
15   deploy_staging:
16    needs: validate
17    if: github.ref == 'refs/heads/main'
18    runs-on: ubuntu-latest
19    steps:
20      - uses: actions/checkout@v4
21      - uses: databricks/setup-cli@main
22      - run: |
23          databricks bundle deploy -t staging
24          databricks bundle run nightly_elt -t staging
25          python tests/smoke_staging.py           # contract + row-count checks
26   deploy_prod:
27    needs: deploy_staging
28    environment: production                       # requires reviewer approval
29    runs-on: ubuntu-latest
30    steps:
31      - uses: actions/checkout@v4
32      - uses: databricks/setup-cli@main
33      - run: databricks bundle deploy -t prod

```

Listing 23.3: CI/CD: validate on PR, deploy to staging on merge, gate prod.

### 23.4.2 FinOps from System Tables

Weekly cost practice: `system.billing.usage` by SKU and tag, per-domain chargeback, and the two-optimization rule (every monthly review implements at least two cost changes: auto-termination enforcement, serverless conversion, trigger-cadence rightsizing, storage lifecycle) [34].

```

1 SELECT custom_tags.domain,
2        sku_name,
3        ROUND(SUM(usage_quantity), 1) AS dbus,
4        ROUND(SUM(usage_quantity * list_price), 0) AS est_usd
5 FROM system.billing.usage
6 JOIN system.billing.list_prices USING (sku_name, cloud, price_start_time)
7 WHERE usage_date >= date_trunc('month', current_date()) - INTERVAL 1 MONTH
8 GROUP BY custom_tags.domain, sku_name
9 ORDER BY est_usd DESC;
```

Listing 23.4: Per-domain monthly chargeback.

## 23.5 Thursday Hands-on Project: Bundle, Pipeline, and the Cost Report

Convert the Week 12 job into a bundle with dev/prod targets, drive it from GitHub Actions through a staging gate, provision the perimeter with Terraform, and ship the weekly DBU report.

```

1 # 1. Scaffold and validate
2 databricks bundle init # or author databricks.yml per lst:ch23-bundle
3 databricks bundle validate -t dev
4 databricks bundle deploy -t dev
5 databricks bundle run nightly_elt -t dev
6
7 # 2. Terraform the perimeter (lst:ch23-terraform)
8 terraform init; terraform plan -out=tfplan; terraform apply tfplan
9
10 # 3. Wire .github/workflows/platform-cicd.yml (lst:ch23-actions);
11 # PR -> validate; merge -> staging deploy + smoke; approval -> prod.
12
13 # 4. The Friday habit, automated:
14 databricks bundle run weekly_cost_report -t prod
```

Listing 23.5: Week 23 lab: from hand-built job to gated deployment.

### Deliverables

- The bundle with dev/staging/prod targets and variables; validation output.
- The Terraform configuration with remote, locked state.
- The Actions run history: PR validation, staging gate, approved prod deploy.
- The chargeback report with two implemented optimizations.

### Acceptance Criteria

- A clean clone deploys the platform to dev with two commands (`terraform apply`, `bundle deploy`).
- A deliberately broken bundle fails `validate` in the PR and never reaches staging.

- Prod deploys only via the approved environment; the audit trail shows it.
- The cost report attributes  $\geq 95\%$  of DBUs to a domain tag.

### Stretch Goals

- Add the lineage impact check from Chapter 22 as a PR comment step.
- Add bundle tests (notebook unit tests) that run in the staging job.
- Implement the break-glass prod deploy path with its own alarm.

## 23.6 Friday Use Case: Thirty Pipelines, Three Environments, Monthly Accountability

A platform team ships 30 pipelines for five business domains across dev/staging/prod, with a per-domain monthly budget and a release-train cadence. Design the delivery and FinOps operating model.

### 23.6.1 Requirements and Assumptions

Domains ship weekly on the train; urgent fixes ride a documented hotfix path. Finance expects per-domain monthly variance explanations. Two production incidents last quarter were untracked manual changes.

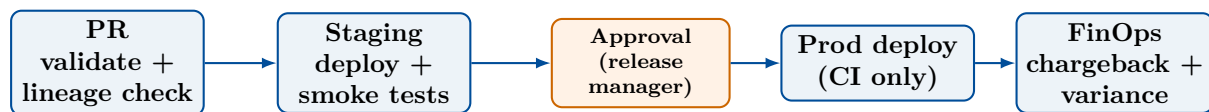


Figure 23.1: The release train: every change rides the same validated, gated track; cost closes the loop monthly.

### 23.6.2 Architecture Decisions

**One track, no exceptions.** Every resource change rides PR → staging → approval → prod; the hotfix path is the same track with an expedited approval, so the audit trail survives the emergency. The two manual-change incidents become impossible by removing human prod-deploy rights and alarming on drift (a scheduled job compares deployed state to the bundle registry).

**Cost as a domain conversation.** Chargeback lands monthly per domain from listing 23.4; variance  $>15\%$  requires a note; the two-optimization rule keeps the estate’s unit cost falling even as volume grows. Domains see their DBU dashboard—cost awareness without central nagging.

#### Common Pitfall

**The untracked manual fix.** Every “quick fix in the UI” forks production from the code. Detect it (drift job), reverse it (the bundle redeploys over it), and fix the loop time that tempted it—if the train takes two hours, hotfixes will always look faster, and they always cost more than they save.



### 23.6.3 Risks and Mitigations

| Risk                                   | Mitigation                                                     |
|----------------------------------------|----------------------------------------------------------------|
| Manual prod change under pressure      | Humans lack prod deploy rights; drift job alarms on divergence |
| Staging no longer predicts prod        | Scheduled staging refresh; parity checks in the pipeline       |
| Terraform state corrupted or locked    | Remote versioned backend; documented force-unlock procedure    |
| Chargeback tags missing on new compute | Tag enforcement in cluster policy, not in documentation        |
| Approval becomes rubber-stamping       | Two-reviewer rule for schema or grant changes                  |

Table 23.1: Week 23 scenario risks and mitigations.

## 23.7 Design Decision Guide

Delivery decisions are policy made executable; optimize for the audit trail surviving the emergency.

| Decision                            | Choose the first when                   | Choose the second when                       |
|-------------------------------------|-----------------------------------------|----------------------------------------------|
| Bundles vs. Terraform               | Application resources (jobs, pipelines) | Perimeter (workspaces, principals, policies) |
| Staging gate vs. direct prod        | Anything user-facing or scheduled       | Dev-only sandboxes                           |
| Service principal in CI vs. PAT     | Always the principal                    | Never a personal token in CI                 |
| Hotfix via pipeline vs. manual fix  | Always the pipeline, expedited          | Never manual—drift is the incident           |
| Chargeback vs. show-back            | Budgets have owners                     | Early-stage awareness building               |
| Weekly train vs. continuous release | Team coordination matters               | Mature trunk-based teams                     |

Table 23.2: Week 23 decision guide.

The two-command redeploy (Terraform apply, bundle deploy) is the health metric of the whole delivery system: when it stops being true, the estate is drifting from its code.

## 23.8 Operational Guidance for Week 23

### 23.8.1 Performance and Reliability

Measure the delivery system itself: deploy frequency, PR-to-prod lead time, validation failure rate, and staging-to-prod divergence time. A staging environment that drifts from prod (data volume, configuration) stops predicting prod—refresh it on a schedule. The drift-detection job (deployed state versus bundle registry) runs daily; any diff is either an untracked change or a failed deploy, and both are incidents-lite.

### 23.8.2 Security and Governance Checklist

- CI service principal holds deploy rights; humans hold approval rights—separate duties.

- Terraform state backend access-controlled and versioned.
- Bundle variables containing environment secrets resolved from CI secret stores.
- Prod targets locked to pipeline deployment; break-glass path alarmed.

### 23.8.3 Troubleshooting Playbook

| Symptom                               | Likely cause                           | First action                                           |
|---------------------------------------|----------------------------------------|--------------------------------------------------------|
| Validate passes, deploy fails         | Permission gap on the target workspace | Deploy as the CI principal; compare grants per target  |
| Staging green, prod red               | Environment/config drift               | Diff target variables and workspace configuration      |
| Terraform state locked                | Aborted apply left the lock            | Force-unlock by lock ID after confirming no live apply |
| Bundle variable missing in one target | Variable declared without default      | Add per-target overrides or a safe default             |

Table 23.3: Week 23 troubleshooting quick reference.

## 23.9 Interview Q&A

### 23.9.1 Concepts and Trade-offs

1. **Q: Asset Bundles versus Terraform—what goes where?**  
A: Terraform for the slow perimeter (workspaces, principals, policies, metastores); bundles for the fast application layer (jobs, pipelines, dashboards) with per-target variables and modes. Both in Git, both through the same review.
2. **Q: What does bundle validate actually catch?**  
A: Schema errors, missing variables, bad resource references, and permission problems—before any cloud change. It is the compile step for your platform.
3. **Q: Why remote, locked Terraform state?**  
A: State is the record of what exists. Local state forks under team/CI use; a forked state is how plans propose recreating—or deleting—your production estate.
4. **Q: How do you keep humans out of prod without slowing hotfixes?**  
A: The same pipeline with an expedited approval gate: the hotfix is a PR with a faster review SLA, not a different process. Audit and staging evidence survive the emergency.
5. **Q: Where does chargeback data come from, and what breaks it?**  
A: `system.billing.usage` plus tags (cluster policy-enforced, bundle-injected). Untagged compute breaks attribution—which is why tag enforcement lives in policy, not in a wiki.
6. **Q: What is the first signal your delivery process is failing?**  
A: Drift between deployed state and the bundle registry—caught by the drift job. Manual changes are not a people problem; they are a missing-guardrail problem.
7. **Q: How do you attribute a cost spike to a change?**  
A: Join the billing spike window to the deployment history: bundle deploys and Terraform applies are logged, so the cost curve diffs against the release curve.
8. **Q: How do you convince a team to stop deploying by hand?**  
A: Make the pipeline faster than the manual path and prove it once: a two-command

redeploy and a five-minute PR-to-staging loop. Teams abandon manual deploys when the governed path is also the easiest one.

## 23.10 Further Reading

Asset Bundles are documented in [42]; the Terraform provider in [24]; billing system tables in [34]. The jobs being deployed come from Chapter 12 [25]; the service principals CI runs as come from Chapter 21 [39].

## Key Takeaways

- Production is a build artifact: bundles + Terraform in Git, deployed by CI, gated by staging.
- Targets and modes make dev collisions and prod schedules structurally impossible to confuse.
- Terraform state is remote, locked, and access-controlled—it describes your security estate.
- Hotfixes ride the same track faster, never a different track.
- FinOps is a monthly loop with per-domain chargeback and two implemented optimizations—enforced by tags in policy.

## 23.11 Exercises

1. **(Conceptual)** Why does development mode prefix resource names, and what breaks without it?
2. **(Conceptual)** Explain the two-command clean-clone redeploy and what each command owns.
3. **(Hands-on)** Complete the Thursday lab, including the deliberately-broken PR.
4. **(Hands-on)** Add a drift-detection job comparing deployed jobs against the bundle registry.
5. **(Hands-on)** Build the per-domain DBU dashboard and write this month’s variance note.
6. **(Design)** Write the hotfix runbook (SLA, approvers, evidence) for the Friday scenario.
7. **(Operations)** Draft the incident review template for “prod changed outside the pipeline.”
8. **(Architecture)** When would you introduce dbt into this delivery model, and which bundle resources would it displace?

# Final Capstone and Certification Preparation

Learning Objectives

- Integrate all six milestones into one production-grade, code-deployed platform.
- Execute the clean-environment redeploy drill and the replay drill as final evidence.
- Map the Databricks certification domains to this book’s chapters and labs.
- Run two timed mock exams and remediate weak domains by rebuilding labs.
- Present the platform as a 15-minute architecture defense with numbers.
- Package the six capstones into a portfolio that survives an interviewer’s probing.

Cross-Cloud Equivalence

Certification week looks different per platform, but the **readiness method** is identical: domains mapped to labs, mocks under time, remediation by rebuilding.

| Element          | Databricks                                             | AWS                                | Azure / GCP / Fabric                         |
|------------------|--------------------------------------------------------|------------------------------------|----------------------------------------------|
| Target exam      | Data Engineer Associate / Professional                 | Data Engineer Associate            | DP-700 / Professional Data Engineer / DP-600 |
| Heavy domains    | Platform, Delta Lake, Spark ETL, pipelines, governance | Storage, ingestion, transform, ops | Fabric workloads / GCP services / Power BI   |
| Lab weight       | High (build fluency)                                   | High                               | High                                         |
| Mock standard    | Two mocks, 80%+, no domain <70%                        | Same                               | Same                                         |
| Capstone defense | Architecture review with cost numbers                  | Same                               | Same                                         |

The portability argument of the whole series: if you can defend the Databricks capstone *and* sketch its AWS or Fabric equivalent from the cross-cloud boxes, you have learned data engineering, not one console.

## 24.1 Week 24 Roadmap and Mental Model

Week 24 is integration and examination. The capstone reassembles the program—lakehouse, pipelines, streaming, ML, governance, delivery—into one platform deployed from one repository. The certification work converts six months of building into exam readiness: domains mapped to labs, two timed mocks, remediation by rebuilding rather than re-reading [15].

The mental model: **you are ready when the platform redeploys without you.** The clean-clone drill—Terraform apply, bundle deploy, pipelines green, dashboards live, monitors armed—is the complete answer to “do you understand this system?” Everything else this week is measurement and polish.

#### Definition

**Certification readiness** (this program’s definition): two consecutive timed mock exams at 80%+ with no domain below 70%, plus a capstone that redeploys from a clean clone and survives a 15-minute architecture defense with cost, latency, and volume numbers from memory.

#### Key Terms

**Domain:** a scored exam section (Databricks platform, Delta Lake, ETL with Spark/DLT, production pipelines, governance). **Mock exam:** a timed, closed-book practice run scored by domain. **Remediation:** rebuilding the lab that exercises a weak domain. **Clean-clone drill:** full redeploy from the repository into a fresh workspace. **Architecture defense:** the narrated trade-off review of the final platform.

## 24.2 Monday: The Integrated Architecture Review

### 24.2.1 One Map

Draw the whole platform on one page before touching any exam material: ingestion (Auto Loader, Kafka), medallion storage (Delta, liquid clustered), pipelines (DLT + Workflows), serving (SQL warehouse, AI/BI, Genie), ML (features, registry, endpoints, monitors), governance (UC, lineage, audit, sharing), and delivery (bundles, Terraform, Actions). For every box: one sentence of purpose, one number (volume, latency, or cost), one known limitation. That page is your capstone diagram, your exam warm-up, and your interview artifact.

### 24.2.2 The Weak-Link Audit

Walk the map asking “what breaks this?” per component: checkpoint deletion, skewed key, stale feature job, expired token, drifted schema, runaway DBU spend. Each answer should be a mechanism you built (alert, gate, runbook)—any answer of “I’d notice” is a gap to close before the defense.

## 24.3 Tuesday: Certification Domains

### 24.3.1 Mapping Domains to Chapters

The Databricks Data Engineer exams weight: the platform and tooling, Delta Lake, ETL with Spark (including DLT and Auto Loader), production pipelines and orchestration, and governance with Unity Catalog [15]. Map each domain to its chapters (1/4, 2/5, 9–11, 12–16, 8/21/22) and score your confidence per domain from mock evidence, not feeling.

| Exam domain                   | Chapters  | Rebuild these labs if weak                                  |
|-------------------------------|-----------|-------------------------------------------------------------|
| Databricks platform & compute | 0, 1, 6   | Cluster policies, warehouse sizing lab                      |
| Delta Lake                    | 2, 5      | Enforcement/evolution/RESTORE drill; optimization benchmark |
| ETL with Spark & DLT          | 4, 9, 10  | Skew remediation; expectations poison drill                 |
| Incremental ingestion & CDC   | 11        | Auto Loader drift drill; SCD2 flow                          |
| Production pipelines          | 12, 23    | Repair-run drill; bundle CI/CD lab                          |
| Streaming                     | 13–16     | Restart proof; watermark/dedup lab                          |
| Governance & security         | 8, 21, 22 | Masking + negative tests; share revocation                  |
| ML awareness                  | 17–20     | Registry alias flow; drift drill                            |

Table 24.1: Exam domains mapped to chapters and their remedial labs.

## 24.4 Wednesday: Mock Exam Discipline

### 24.4.1 How to Take the Mocks

Timed, closed-book, no notes, scored by domain. The diagnostic value is in the *misses*: classify each as knowledge gap (never learned it), reasoning error (misread the scenario), or time management. Knowledge gaps remediate by rebuilding the lab in table 24.1; reasoning errors by re-reading two scenario questions slowly; time issues by question budgeting. Reviewing answer keys without rebuilding the lab converts a mock into trivia.

### 24.4.2 The Two-Pass Standard

Readiness is two consecutive 80%+ with no domain under 70% [15]. One good mock is variance; two is signal. Book the exam when the standard is met—and keep the lab-rebuild habit; it is also how senior engineers stay honest.

#### Common Pitfall

**Studying features instead of failure modes.** Scenario questions reward knowing what breaks: which operation needs a new checkpoint, when expectations fail a pipeline, what VACUUM destroys, why a shared job cluster corrupts attribution. The labs taught these by breaking things—the exam tests exactly that.

## 24.5 Thursday: Final Capstone Build

See section 24.12 for the full specification. Today is the build day: the clean-clone drill, the replay drill, the evidence pack assembly.

## 24.6 Friday Use Case: The Architecture Defense

Present the platform in 15 minutes as if to a staff engineer who has never seen it.

### 24.6.1 The Structure That Scores

Problem and scale first (“60 GB nightly plus 5k events/second, 200 analysts”), then the diagram, then two design decisions with trade-offs narrated (liquid clustering versus partitioning; triggered versus continuous), one failure story (what broke, how detected, what changed), and the closing numbers (DBU cost per month, p95 latency, rows per day). The examiner’s follow-ups will target your stated trade-offs—choose ones you can defend two levels deep.

#### Common Pitfall

**The feature tour.** Listing services is not architecture. Every box must earn its place with a requirement it serves; every requirement must land in a box. If a box has no requirement, remove it; if a requirement has no box, that is the gap the defense exists to find.

### 24.6.2 Risks and Mitigations

| Risk                                         | Mitigation                                                 |
|----------------------------------------------|------------------------------------------------------------|
| Capstone works only with owner’s credentials | Clean-clone drill runs as the CI service principal         |
| Evidence pack rots after exam week           | Drills and evidence refresh quarterly, owned by the team   |
| Mock-exam cramming without retention         | Readiness standard requires consecutive passes weeks apart |
| Architecture defense reveals an unknown box  | One-page map review with a peer before the presentation    |
| Portfolio setup steps drift from reality     | A stranger follows the README cold before publishing       |

Table 24.2: Week 24 scenario risks and mitigations.

## 24.7 Design Decision Guide

Certification and capstone decisions are about evidence: what you can show beats what you can say.

| Decision                           | Choose the first when          | Choose the second when                  |
|------------------------------------|--------------------------------|-----------------------------------------|
| Lab rebuild vs. re-reading notes   | Any domain below 70% on a mock | Reviewing already-strong domains        |
| Deep defense vs. feature tour      | The 15-minute review           | Never the tour                          |
| Associate vs. Professional exam    | First certification, breadth   | Proven production experience, depth     |
| Scope cut vs. schedule slip        | Capstone weeks                 | — (protect evidence quality)            |
| Book exam at standard vs. wait     | Two consecutive mocks          | 80%+ One good mock—variance, not signal |
| Public portfolio vs. private repos | Job hunting                    | Employer constraints                    |

Table 24.3: Week 24 decision guide.

The week’s real deliverable is a habit: evidence over assertion, drills over documents, rebuilds over re-reads. The certification is the receipt; the platform is the proof.

## 24.8 Operational Guidance for Week 24

### 24.8.1 Performance and Reliability

The evidence pack is a living system: mock scores logged per domain, clean-clone drill timings trended per run, and the runbook re-executed (not re-read) quarterly. A clean-clone drill that takes longer each quarter is accumulating hand-rolled state—treat drill-duration regression like test-duration regression. Keep the defense recording current; the platform changes and the 15-minute story must change with it.

### 24.8.2 Security and Governance Checklist

- Evidence packs (lineage, audit, cost) stored in the governed catalog, access-controlled.
- Clean-clone drill runs in an isolated workspace with no production credentials.
- Mock-exam artifacts contain no real platform data or secrets.
- The final platform’s rollback and DR runbooks are attached to the release.

### 24.8.3 Troubleshooting Playbook

| Symptom                 | Likely cause                                 | First action                                          |
|-------------------------|----------------------------------------------|-------------------------------------------------------|
| Mock score plateaus     | Reviewing answers instead of rebuilding labs | Rebuild the weakest domain’s lab from scratch         |
| Clean-clone drill fails | Hand-rolled state not in code                | Add the missing resource to bundle/Terraform; rerun   |
| Defense runs over time  | Feature tour instead of decisions            | Cut to two trade-offs; rehearse the 90-second version |
| Drill passes but slowly | Cold-start and quota friction                | Pre-warm quotas; document the critical path timings   |

Table 24.4: Week 24 troubleshooting quick reference.

## 24.9 Interview Q&A

### 24.9.1 Capstone-Grade Questions

1. **Q: Walk me through your platform end to end.**  
A: Practice the 90-second version: sources → ingestion → medallion → serving/ML → governance → delivery, with one number per arrow. If it takes five minutes, you do not yet own it.
2. **Q: What would you rebuild differently at 10x scale?**  
A: Have the real answer: usually partition/parallelism ceilings, the orchestration fan-in, and the cost model. “Nothing” fails the question.
3. **Q: Your most expensive mistake in this build?**  
A: The untested rollback, the unmonitored endpoint, the leaked token—name one, quantify it, show the mechanism that now prevents it.
4. **Q: How does your platform prove data correctness to a skeptic?**  
A: Reconciliation arithmetic per transition, expectation metrics from the event log, replay drills, and the audit trail—evidence, not assurances.



**5. Q: Why Databricks for this, over a point-solution stack?**

A: One governed storage and catalog under batch, streaming, SQL, and ML—the alternative is four governance models and four copies. Know the honest counter-costs (DBU economics, Spark operational depth) too.

## 24.10 Further Reading

The certification page lists current exams, domains, and preparation paths [15]; Databricks Academy self-paced courses align with the domains. Revisit the weakest domain’s chapter Further Reading: platform [19], Delta [8, 53], Spark [4, 10], pipelines [26, 25], governance [51], ML [32, 35].

## Key Takeaways

- Readiness = two consecutive 80%+ mocks with no domain under 70% + a clean-clone redeploy.
- Remediate by rebuilding labs, not re-reading notes.
- The one-page map with a purpose, a number, and a limitation per box is the defense and the study guide.
- The defense scores trade-off reasoning: two decisions defended deeply beat ten features listed.
- The portfolio README—problem, diagram, setup, evidence, cost, lessons—is the interview artifact.

## 24.11 Exercises

1. **(Conceptual)** Write your 90-second platform walkthrough; time it; cut it to 90 seconds.
2. **(Conceptual)** For each table 24.1 domain, name its three most failure-mode-flavored facts.
3. **(Hands-on)** Run both mocks under exam conditions; log scores by domain.
4. **(Hands-on)** Rebuild the lab for your weakest domain from scratch, in a fresh schema, without your notes.
5. **(Hands-on)** Execute the clean-clone drill in a fresh workspace; record timings per step.
6. **(Design)** Write the “10x scale” change list for your capstone with one justification each.
7. **(Operations)** Record the 15-minute defense; review it against the structure in Friday’s section.
8. **(Portfolio)** Publish all six capstone repos with the standard README structure; have a peer follow one setup cold.

## 24.12 Milestone 6 Capstone: The Governed Data Platform

### Capstone Project

**Mission.** Deliver the complete program as one platform: medallion lakehouse with optimized Delta storage, DLT pipelines with expectations, Auto Loader + Kafka streaming, an MLflow-managed model served and monitored, Unity Catalog governance with lineage/audit/sharing—all defined as code, promoted dev→staging→prod by CI, with cost and security evidence attached.

**Estimated time.** 12–16 hours across the week.

**What you'll practice.**

- Extracting five months of hand-built artifacts into bundle/Terraform code.
- The clean-clone redeploy drill as the definitive correctness proof.
- Governance, cost, and reliability evidence packs.
- The certification-style architecture defense.

### Scenario

Your capstone platform from Milestones 1–5 works—in your workspace, with your credentials, by your hands. Meridian Retail's platform team now requires it to be *operable by strangers*: deployable from the repo, governed by default, costed per domain, and defensible in an architecture review. This milestone closes that gap.

### Requirements

- One bundle (jobs, pipelines, dashboards, serving endpoint, monitors) with dev/staging/prod targets; Terraform for the perimeter.
- Clean-environment redeploy: `terraform apply + databricks bundle deploy -t prod` rebuilds the platform from the repo alone.
- Batch and streaming paths reconciled in Gold; the replay drill producing zero duplicates.
- Governance evidence: access matrix implemented, lineage report, PII audit alert, one Delta Sharing share with revocation test.
- CI/CD with PR validation, staging gate, approved prod promotion, and a documented rollback.
- Cost report per domain with two implemented optimizations; security review checklist complete.
- Two mock exams meeting the readiness standard; the recorded 15-minute defense.

## Reference Architecture

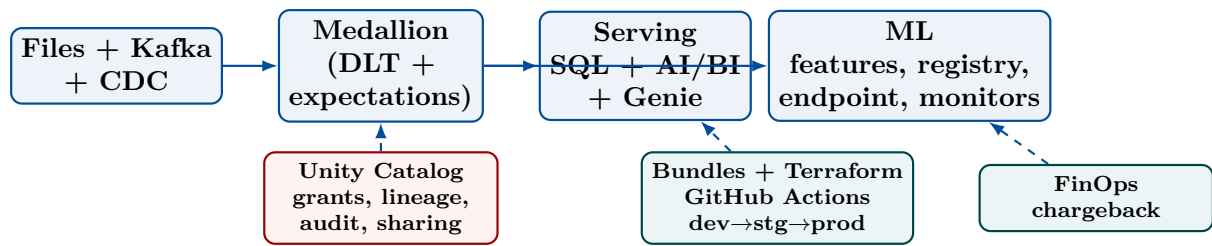


Figure 24.1: Milestone 6: the complete platform—one governed estate, deployed as code, measured continuously.

## Implementation Steps

1. Inventory every hand-built artifact from Milestones 1–5; express each as bundle YAML or Terraform.
2. Define targets with variables; wire the Actions pipeline with the staging gate.
3. Run the clean-clone drill in a fresh workspace; fix everything the drill finds.
4. Assemble the evidence packs: governance, reliability (drills), cost, security.
5. Both mocks, the remediation rebuilds, and the recorded defense.

## Deliverables

- The repository: bundles, Terraform, pipelines, tests, README per the portfolio guide.
- The drill logs: clean-clone timings, replay-drill counts, revocation evidence.
- The cost and security packs; the mock scores; the defense recording.

## Acceptance Criteria

- A teammate (or you, in a fresh workspace) redeploys the platform from the README alone.
- No human holds prod deploy rights; the pipeline audit trail is complete.
- Every acceptance criterion from Milestones 1–5 still passes against the code-deployed estate.
- The readiness standard (two mocks, 80%+, no domain <70%) is met.

## Stretch Goals

- Add the drift-detection job proving prod matches the bundle registry.
- Publish one Gold dataset via Delta Sharing to a second (free) workspace with the revocation drill.
- Write the one-page “operating this platform” runbook a stranger could follow oncall.

*“Consistency is the architecture of mastery.”*

You have built the platform. Now go defend it.

# Environment Setup and Sample Data

This appendix is the standing setup reference for every lab in the book. Complete it once before Chapter 0 and return whenever a lab assumes a tool, a connection, or a dataset that your machine or workspace does not yet have. Everything here targets Windows with PowerShell, matching the environment used throughout the program.

## A.1 Accounts and Workspace

- **Trial or Community Edition.** Create a free trial workspace from the Databricks portal [20], or use Databricks Community Edition for the no-cost path. The trial credits cover every lab in the book; they expire, so they are unsuitable for anything with an SLA. If your employer provides a workspace, request a dedicated catalog for labs.
- **Unity Catalog metastore.** Most trial workspaces are Unity Catalog-enabled. Verify you can create a catalog before Chapter 2; otherwise fall back to the `hive_metastore` for the early chapters and revisit governance labs when a metastore is available [51].
- **Cloud account guardrails.** A classic (non-serverless) workspace bills cloud VMs to your cloud account: set a cloud budget alert before Week 1's first cluster.

## A.2 Local Tooling

Install once, in this order:

1. **Python 3.x** — used by the SDK scripts, the Week 15 Kafka producer, and synthetic data generation. Verify with `python -version`.
2. **Git** — required from Chapter 0 onward; Week 23 depends on it. Verify with `git -version`.
3. **VS Code** — with the Python extension and the Databricks extension; the editor for bundle work and local development.
4. **Databricks CLI v0.200+** — the command plane for workspace, jobs, bundles, and secrets operations (Appendix B).
5. **Python packages** — `databricks-sdk` (Chapter 1), `databricks-cli`, `faker` (Chapters 13, 15, 24), `mlflow` and `scikit-learn` (Chapters 17–20), `confluent-kafka` (Chapter 15).
6. **Docker Desktop** (optional) — the cleanest way to run the local Kafka broker for the Chapter 15 lab.

## 7. Terraform (Chapter 23) — for the infrastructure-as-code labs.

```
1 python --version
2 git --version
3 databricks --version
4 databricks auth login --host https://<your-workspace>.azuredatabricks.net
5 databricks current-user me
6 pip install databricks-sdk faker mlflow scikit-learn confluent-kafka
```

Listing A.1: One-time tool verification (PowerShell).

## A.3 Repository and Credential Conventions

Clone or create the course repository with the standard layout—`data/`, `src/`, `tests/`, `bundles/`, `docs/`—and observe three credential rules from day one:

- Secrets live in environment variables (`DATABRICKS_HOST`, `DATABRICKS_TOKEN`) or Databricks secret scopes—never in notebooks, bundle YAML, or Git.
- Prefer OAuth (`databricks auth login`) over personal access tokens; where a token is unavoidable, set an expiry and store it outside the repository.
- Enable secret scanning on the course repository before Week 23’s CI/CD makes every committed file a deployment artifact.
- Commit generator scripts, never generated data; the datasets below are reproducible by design.

## A.4 Workspace Naming Conventions

Labs reference objects by predictable names: catalog `de_training` with schemas `bronze`, `silver`, `gold`; volume `de_training.bronze.landing`; clusters `de-weekNN-cluster`; jobs `de-weekNN-<purpose>`; bundle targets `dev`, `staging`, `prod`. Consistency matters because runbooks, alerts, and the acceptance checklists all reference these names.

## A.5 Sample Datasets

The book’s labs and capstones standardize on three data sources:

| Dataset                     | Used in                             | Notes                                                                                                                                              |
|-----------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| NYC Taxi (TLC trip records) | Parts I–II (Chapters 2, 4–6)        | Available in-workspace under <code>/databricks-datasets/nyctaxi/</code> and as <code>samples.nyctaxi</code> ; 1–5M rows suffice for capstones [58] |
| Olist Brazilian e-commerce  | Parts I and III (Chapters 3, 10–12) | ~100k orders across 9 related CSVs: orders, items, customers, products, payments, reviews [57]                                                     |
| Synthetic (faker)           | Parts IV–VI (Chapters 13–16, 21–24) | Card transactions, IoT telemetry, PII-style CRM rows; seed generators for reproducibility [54]                                                     |

Table A.1: Standard datasets and where they are used.

Volume guidance: Part I medallion work at 1–10M rows; the Part II optimization benchmark at ~50M rows; Part III re-ingests the nine Olist tables daily; Part IV streams at 5–20k events

per minute during demos; Part V scores ~100k rows per batch.

```

1 from faker import Faker
2 import csv
3
4 fake = Faker()
5 Faker.seed(42) # reproducible
6
7 with open("data/synthetic_transactions.csv", "w", newline="") as f:
8     w = csv.writer(f)
9     w.writerow(["event_id", "card_id", "city", "amount", "event_ts"])
10    for i in range(100_000):
11        w.writerow([f"E{i:07d}", f"C{fake.random_int(1, 5000):05d}",
12                    fake.city(), round(fake.random.uniform(1, 500), 2),
13                    fake.date_time_this_year().isoformat()]

```

Listing A.2: Seeded synthetic transaction generator (commit the script, not the data).

## A.6 Troubleshooting Checklist

- **CLI cannot authenticate:** re-run `databricks auth login`; confirm the host has no trailing slash and the profile name matches `DEFAULT` or `-p <profile>`.
- **Cluster start fails on quota:** request vCPU quota in the cloud account, or switch the lab to serverless compute where available.
- **Table not found from SQL:** confirm the three-level name (`catalog.schema.table`) and that your principal has `USE CATALOG/SCHEMA` and `SELECT` (Chapter 8).
- **Auto Loader finds no files:** the checkpoint remembers processed files; drop new files *after* the checkpoint was created, or use a fresh checkpoint for a deliberate re-ingest (Chapter 11).
- **Bill grew overnight:** check `system.billing.usage` by SKU; an unterminated all-purpose cluster is the usual cause (Chapter 1's cluster policy prevents this).

### Tip

When a lab fails, reproduce the failure at the smallest possible scope—one row, one file, one minute of telemetry—before investigating the estate. Every chapter's validation checklist is written to support exactly that reduction.

## Appendix B

# CLI, SDK, and API Quick Reference

This appendix collects the commands, endpoints, and idioms the book’s labs use repeatedly, in one lookup-friendly place. It is a reference, not a tutorial: each section points back to the chapter that teaches the concept.

## B.1 Databricks CLI Essentials

```
1 databricks auth login --host https://<workspace>.azuredatabricks.net # OAuth (
    Ch. 0)
2 databricks auth profiles # list configured profiles
3 databricks current-user me # confirm identity
4
5 # Workspace and clusters (Ch. 1)
6 databricks workspace list /
7 databricks clusters list --output json
8 databricks clusters create --json @cluster.json
9
10 # Jobs and pipelines (Ch. 12)
11 databricks jobs list
12 databricks jobs run-now <job-id>
13 databricks jobs submit --json @run.json
14 databricks runs repair --run-id <run-id> # repair failed tasks only
15
16 # Unity Catalog (Ch. 8, 22)
17 databricks catalogs list
18 databricks schemas create bronze --catalog de_training
19 databricks volumes create de_training bronze landing MANAGED
20 databricks grants get schema de_training.gold
21
22 # Secrets (Ch. 21)
23 databricks secrets create-scope kv-lab
24 databricks secrets put-secret kv-lab kafka-password --string-value '<from-vault
    >'
25
26 # Asset Bundles (Ch. 23)
27 databricks bundle validate -t dev
28 databricks bundle deploy -t staging
29 databricks bundle run nightly_elt -t prod
```

Listing B.1: Databricks CLI commands used across the labs.

## B.2 Databricks SDK for Python

The SDK mirrors the REST API and powers automation scripts throughout the book (Chapters 1, 12, 23):

```

1 from databricks.sdk import WorkspaceClient
2
3 w = WorkspaceClient() # reads DATABRICKS_HOST / profile from env
4
5 me = w.current_user.me()
6 print(me.user_name)
7
8 # Submit a one-off job run (Ch. 1 smoke test)
9 from databricks.sdk.service.jobs import SubmitTask, NotebookTask
10 run = w.jobs.submit(run_name="ch0-smoke", tasks=[
11     SubmitTask(task_key="smoke",
12               notebook_task=NotebookTask(notebook_path="/Repos/<you>/course/
13               src/smoke"),
14               new_cluster={"spark_version": "15.4.x-scala2.12",
15                           "num_workers": 1,
16                           "node_type_id": "Standard_DS3_v2"})
17 ])
18 # List catalog tables (Ch. 8)
19 for t in w.tables.list(catalog_name="de_training", schema_name="gold"):
20     print(t.full_name)

```

Listing B.2: SDK essentials.

## B.3 REST API

The REST API is the operations plane beneath Week 23's CI/CD. Pattern: acquire a token (OAuth preferred), call the API, poll long-running operations.

```

1 # List jobs
2 curl -H "Authorization: Bearer *****" '
3     https://<workspace>.azuredatabricks.net/api/2.1/jobs/list
4
5 # Trigger a job run
6 curl -X POST -H "Authorization: Bearer *****" '
7     -H "Content-Type: application/json" '
8     -d '{"job_id": 12345}' '
9     https://<workspace>.azuredatabricks.net/api/2.1/jobs/run-now
10
11 # Execute SQL on a warehouse
12 curl -X POST -H "Authorization: Bearer *****" '
13     -H "Content-Type: application/json" '
14     -d '{"statement": "SELECT COUNT(*) FROM de_training.gold.fact_orders",
15         "warehouse_id": "<id>"}' '
16     https://<workspace>.azuredatabricks.net/api/2.0/sql/statements

```

Listing B.3: REST API essentials (token redacted).

### Common Pitfall

The REST API authenticates as *someone*: use a service principal with least-privilege grants, an OAuth or vaulted secret, and a rotation date. Tokens pasted into scripts reproduce Chapter 21's token-in-notebook failure at the platform layer.



## B.4 Spark SQL and Delta Quick Reference

```

1  -- Three-level namespace (Ch. 1, 8)
2  SELECT * FROM de_training.silver.orders LIMIT 10;
3
4  -- History and time travel (Ch. 2)
5  DESCRIBE HISTORY de_training.silver.orders;
6  SELECT COUNT(*) FROM de_training.silver.orders VERSION AS OF 12;
7  RESTORE TABLE de_training.silver.orders TO VERSION AS OF 11;
8
9  -- Maintenance (Ch. 5)
10 OPTIMIZE de_training.silver.orders ZORDER BY (customer_id, order_date);
11 ALTER TABLE de_training.gold.fact_orders CLUSTER BY (order_date, region);
12 VACUUM de_training.silver.orders RETAIN 168 HOURS;
13
14 -- Governance (Ch. 8, 22)
15 GRANT SELECT ON SCHEMA de_training.gold TO 'analysts';
16 CREATE OR REPLACE VIEW de_training.gold.v_customer_masked AS
17 SELECT customer_id,
18         CASE WHEN is_member('pii_readers') THEN email
19              ELSE sha2(email, 256) END AS email
20 FROM de_training.gold.dim_customer;
21
22 -- Cost introspection (Ch. 6, 23)
23 SELECT sku_name, SUM(usage_quantity) AS dbus
24 FROM system.billing.usage
25 WHERE usage_date >= current_date() - 30
26 GROUP BY sku_name ORDER BY dbus DESC;

```

Listing B.4: SQL idioms used across Chapters 2–8.

## B.5 PySpark and Streaming Quick Reference

```

1  # Read a managed table; stamp lineage columns (Ch. 3)
2  df = (spark.read.table("de_training.bronze.orders")
3        .withColumn("_ingested_at", F.current_timestamp()))
4
5  # Idempotent partition-scoped write (Ch. 9, 12, 20)
6  (df.write.format("delta").mode("overwrite")
7   .option("replaceWhere", "as_of_date = '2026-07-31'")
8   .saveAsTable("de_training.gold.features_daily"))
9
10 # Schema evolution (Ch. 2, 11): default enforcement rejects; opt in
    deliberately
11 (df.write.format("delta").mode("append")
12  .option("mergeSchema", "true")
13  .saveAsTable("de_training.silver.orders"))
14
15 # Auto Loader (Ch. 11)
16 stream = (spark.readStream.format("cloudFiles")
17           .option("cloudFiles.format", "csv")
18           .option("cloudFiles.schemaLocation",
19                 "/Volumes/de_training/bronze/_schemas/orders")
20           .load("/Volumes/de_training/bronze/landing/orders"))
21
22 # Checkpointed streaming sink (Ch. 13)
23 (stream.writeStream.format("delta")
24  .option("checkpointLocation", "/Volumes/de_training/bronze/_ckpt/orders")
25  .trigger(availableNow=True)
26  .toTable("de_training.bronze.orders_stream"))
27
28 # Watermark + dedup (Ch. 14)

```

```

29 deduped = (df.withWatermark("event_ts", "10 minutes")
30             .dropDuplicates(["event_id"]))
31
32 # Registered model as UDF (Ch. 17, 20)
33 predict = mlflow.pyfunc.spark_udf(
34     spark, "models:/ml.churn_model@champion", result_type="double")

```

Listing B.5: PySpark/Streaming idioms used throughout.

## B.6 Asset Bundle Skeleton

```

1 bundle:
2   name: governed-platform
3
4 variables:
5   catalog:
6     default: de_training
7
8 targets:
9   dev:
10    mode: development
11    default: true
12    workspace:
13      host: https://<dev-workspace>.azuredatabricks.net
14   prod:
15    mode: production
16    workspace:
17      host: https://<prod-workspace>.azuredatabricks.net
18
19 resources:
20   jobs:
21     nightly_elt:
22       name: nightly-elt-${bundle.target}
23       tasks:
24         - task_key: pipeline
25           pipeline_task:
26             pipeline_id: ${resources.pipelines.medallion.id}
27   pipelines:
28     medallion:
29       name: medallion-${bundle.target}
30       catalog: ${var.catalog}
31       libraries:
32         - notebook:
33           path: ../src/medallion_pipeline.py

```

Listing B.6: Minimal databricks.yml with dev/prod targets (Ch. 23).

### Tip

Keep this appendix open during capstones. Every idiom here appears in at least one chapter's validation checklist; when an acceptance criterion says “idempotent” or “dedup,” these are the exact patterns it refers to.

# Lab Solutions and Acceptance Guidance

This appendix provides reference solutions for the weekly Thursday labs and the six milestone capstones. Each entry gives the approach, the critical steps, and the evidence that satisfies the chapter’s acceptance criteria. Use it to unblock yourself after a genuine attempt—not instead of one: the labs are the curriculum, and the Databricks Data Engineer certification measures exactly the judgment they build [15].

### Note

A solution counts only when the chapter’s acceptance criteria are met with evidence: row counts, screenshots, history entries, drill outputs. “It ran once on my cluster” is the starting point of each lab, never the end.

## C.1 Part I: Lakehouse Foundations (Weeks 1–4)

### C.1.1 Week 1: Workspace, CLI, and Smoke Test

**Approach.** Create the trial workspace, authenticate the CLI with OAuth, create a policy-guarded cluster, and submit the smoke-test notebook via the SDK. **Critical evidence.** `databricks current-user me` output; the cluster showing a 30-minute auto-termination *enforced by policy*; the job run visible in the UI. **Common failure.** A personal access token pasted into a script—the lab exists to learn OAuth profiles, not to leak a secret.

### C.1.2 Week 2: Schema Enforcement, Evolution, and RESTORE

**Approach.** Create Delta tables from CSVs, force a schema mismatch, evolve deliberately, delete a partition, and recover via RESTORE. **Critical evidence.** DESCRIBE HISTORY showing the failed append (rejected), the `mergeSchema` evolution, the delete, and the restore; recovered row count matching the pre-delete baseline. **Common failure.** Running VACUUM before the recovery step, destroying the versions the lab exists to demonstrate.

### C.1.3 Week 3: Medallion Transformations

**Approach.** Bronze landing as-is with ingestion metadata, Silver typed and deduplicated, Gold star schema. **Critical evidence.** Row-count reconciliation Bronze → Silver (accepted + rejected = input), and Gold aggregates matching a direct source computation. **Common failure.** Silent row drops between layers with no quarantine accounting.

### C.1.4 Week 4: Spark Skew Remediation

**Approach.** Ingest 10M taxi rows, aggregate, read the Spark UI stage summary, fix skew with salting or `repartition`. **Critical evidence.** Before/after task-duration distributions and total stage time. **Common failure.** “Fixing” the job by caching everything and concluding from a warm-cache second run.

### C.1.5 Milestone 1 Capstone

The reference flow: volume-landed CSVs → append-only Bronze → parameterized Silver notebooks → Gold star schema, with the corruption/`RESTORE` drill and reconciliation notebook as the differentiating artifacts. Reviewers check the idempotent Silver `MERGE` and the history evidence first.

## C.2 Part II: Warehousing and Analytics (Weeks 5–8)

### C.2.1 Week 5: OPTIMIZE versus Liquid Clustering

**Approach.** Build the small-file table, baseline a selective query, compact with `ZORDER BY`, convert a copy to liquid clustering, re-benchmark. **Critical evidence.** The results table with file counts and timings for all three variants. **Common failure.** Benchmarking before the data is cached consistently—first-run cold reads pollute the comparison.

### C.2.2 Week 6: Warehouse and Materialized View

**Approach.** Serverless warehouse on Gold, materialized view for daily revenue, query-profile capture, auto-stop configured. **Critical evidence.** The before/after query profiles and a `system.billing.usage` cost line for the warehouse. **Common failure.** Leaving the warehouse at max-clusters with no auto-stop—the cost section of the report then writes itself, unhappily.

### C.2.3 Week 7: Dashboard and Genie Space

**Approach.** Parameterized AI/BI dashboard over certified Gold views; Genie space with instructions and example queries. **Critical evidence.** The captured wrong Genie answer and the instruction (or certified measure) that fixed it. **Common failure.** Pointing Genie at Bronze—fluent, confident, wrong SQL.

### C.2.4 Week 8: Grants and Masking View

**Approach.** Groups, schema-scoped `GRANTS`, the dynamic masking view, validated as a restricted principal. **Critical evidence.** The same query returning unmasked values as owner and masked values as the analyst test principal. **Common failure.** Testing grants as the object owner—where every check passes trivially.

### C.2.5 Milestone 2 Capstone

The reference flow: liquid-clustered Gold → serverless warehouse → AI/BI dashboard + instructed Genie space, behind group grants and the masking view, with `system.access.audit` evidence. Reviewers check the benchmark table, the access matrix implemented (not described), and the audit query, in that order.

## C.3 Part III: Batch ETL and Orchestration (Weeks 9–12)

### C.3.1 Week 9: PySpark Tuning Evidence Table

**Approach.** Fix shuffle partitions, broadcast the small side, salt the skewed key, replace the Python UDF with a pandas UDF. **Critical evidence.** Four independent improvements each with isolated before/after metrics. **Common failure.** Changing four things at once and being unable to attribute the gain.

### C.3.2 Week 10: DLT Expectations

**Approach.** Rebuild the medallion flow as DLT with warn/drop/fail expectations; inject a poison record. **Critical evidence.** The event log query showing the dropped record counted, and Gold untouched by the failed-expectation run. **Common failure.** Expectations that log but are never wired to drop or fail—aspirational quality control.

### C.3.3 Week 11: Auto Loader and CDC

**Approach.** Triggered Auto Loader from the volume, mid-stream schema change, rescued row, AUTO CDC SCD2 on customers. **Critical evidence.** The schema-evolved batch ingested without code edits; the corrupted row visible in `_rescued_data`; the changed customer showing two SCD rows with contiguous validity. **Common failure.** Deleting the checkpoint to “re-run” and double-ingesting the volume.

### C.3.4 Week 12: Orchestrated Workflow

**Approach.** Ingest → DLT → quality gate → conditional alert, parameterized by date, with a forced failure and repair run. **Critical evidence.** The repair-run page re-executing only failed tasks; one ledger row per run outcome. **Common failure.** The quality gate existing but not blocking Gold publish.

### C.3.5 Milestone 3 Capstone

The reference flow: Auto Loader Bronze → DLT Silver/Gold with expectations → gated publish under one Workflow. The decisive artifacts are the repair-run evidence and same-day idempotence (rerun changes zero Gold counts).

## C.4 Part IV: Streaming and Real-Time (Weeks 13–16)

### C.4.1 Week 13: Checkpointed Stream Restart

**Approach.** Rate/file source → Delta sink with checkpoint; kill and restart mid-stream. **Critical evidence.** Post-restart counts proving no loss and no duplication. **Common failure.** Pointing two queries at one checkpoint—offsets corrupt silently.

### C.4.2 Week 14: Watermark and Dedup

**Approach.** 10-minute watermark, `dropDuplicates` on `event_id`, injected late/duplicate events, RocksDB comparison. **Critical evidence.** Late events within the watermark included, beyond it excluded; replay producing zero duplicates; state-store memory comparison. **Common failure.** Dedup without a watermark—state grows until the executor dies.

### C.4.3 Week 15: Kafka Replay Reconciliation

**Approach.** Local Kafka producer with unique IDs, capped offsets per trigger, replay from **earliest** into a recovery table. **Critical evidence.** Recovery-table counts reconciled against the primary table. **Common failure.** The producer connection string committed to source.

### C.4.4 Week 16: Real-Time Medallion

**Approach.** Kafka → Bronze → Silver `foreachBatch` MERGE → Gold tumbling aggregates → dashboard. **Critical evidence.** The dashboard refreshing at decision cadence with a freshness indicator; the daily batch reconciliation agreeing with the stream. **Common failure.** Refresh cadence set to platform maximum rather than decision cadence.

### C.4.5 Milestone 4 Capstone

The reference flow: Kafka → Bronze → deduped watermarked Silver → impossible-travel rule → Gold aggregates → AI/BI, with the replay drill producing zero duplicate alerts. Reviewers check the dedup-before-rule ordering and the exactly-once evidence.

## C.5 Part V: ML and MLOps (Weeks 17–20)

### C.5.1 Week 17: Autologged Champion

**Approach.** Six autologged runs on a fixed snapshot, registry round-trip by alias. **Critical evidence.** Every run tagged with dataset snapshot and commit; the champion loading from `models:/...@champion` in a fresh notebook. **Common failure.** Comparing runs across different data snapshots and calling the difference model quality.

### C.5.2 Week 18: Point-in-Time Training Set

**Approach.** Feature tables with as-of timestamps, `create_training_set` with timestamp lookups, lineage recorded. **Critical evidence.** Rebuilds for three historical as-of dates, identical on rerun. **Common failure.** A windowed feature computed past its as-of date—leakage by construction.

### C.5.3 Week 19: RAG with Evaluation

**Approach.** Chunks in Delta, Delta-sync Vector Search index, retrieval + Foundation Model assembly, MLflow evaluation on 10 questions. **Critical evidence.** The logged quality metrics and one retrieval failure diagnosed and fixed via chunking or metadata filters. **Common failure.** Evaluating by eyeballing three answers and declaring victory.

### C.5.4 Week 20: Serving and Drift Gate

**Approach.** Serverless endpoint with inference table, idempotent batch scoring, injected skew, gated retraining. **Critical evidence.** The drift alert firing; the retrain run completing while promotion stays blocked. **Common failure.** Scoring against ad hoc features instead of the governed feature tables—training-serving skew by design.

### C.5.5 Milestone 5 Capstone

The reference flow: leakage-safe features → tracked training → UC registry → gated serving → monitoring, plus the RAG branch with evaluated answers. The decisive artifacts are the point-in-time proof and the human-gated promotion evidence.

## C.6 Part VI: Security, Production, and Certification (Weeks 21–24)

### C.6.1 Week 21: Service Principal and Secret Scope

**Approach.** Least-privilege service principal, Key Vault-backed scope, notebook converted to `dbutils.secrets.get`, IP access list verified. **Critical evidence.** The blocked connection denied; the notebook running without any literal credential. **Common failure.** The service principal granted account admin “temporarily.”

### C.6.2 Week 22: Lineage, Audit, and Sharing

**Approach.** Column-lineage trace, audit alert on PII reads, Delta Sharing share with revocation test. **Critical evidence.** The revoked recipient’s failed read; the audit alert firing on a planted non-approved read. **Common failure.** Sharing the whole schema when one table was the agreement.

### C.6.3 Week 23: Bundle and Terraform Deployment

**Approach.** Bundle with dev/staging/prod targets, GitHub Actions with service-principal auth, Terraform workspace resources, DBU cost report. **Critical evidence.** The clean-environment redeploy; the cost report with two implemented optimizations. **Common failure.** Terraform state in a local file—the second apply from CI corrupts it.

### C.6.4 Week 24: Mock Exams and Remediation

**Approach.** Two timed, closed-book mocks scored by domain; misses classified; weak domains remediated by rebuilding labs. **Critical evidence.** Two consecutive 80%+ mocks with no domain below 70%. **Common failure.** Reviewing answer keys instead of rebuilding labs.

### C.6.5 Milestone 6 Capstone

The reference flow: one bundle defining the platform across three targets, Terraform-provisioned workspace resources, governance evidence (access matrix, lineage, audit alert, share revocation), and the cost report. The decisive artifacts are the clean-environment redeploy and the replay drill—the two things that cannot be faked in a screenshot.

#### Tip

For portfolio use, each capstone README should lead with the problem statement and architecture diagram, include clean-environment setup, screenshots of the pipeline run, dashboard, and alert, the DBU cost estimate, and a lessons-learned section naming what broke. Interviewers probe trade-offs: be ready to defend DLT versus plain Spark, Z-ORDER versus liquid clustering, and continuous versus triggered streaming at ten times the scale.

# Production-Readiness and Governance Checklists

This appendix distills the book’s production notes and lab acceptance criteria into checkbox-style checklists, one per platform domain. Use them as pre-merge review gates, go-live criteria, or audit preparation: an unchecked box is either work remaining or a documented, accepted risk. Each item names the chapter that teaches it.

### Note

A checklist item is satisfied by *evidence*, not assertion: a query result, a history entry, a drill log, a passing negative test. If the evidence cannot be produced on demand, the box is not checked—however confident the team feels.

## D.1 Workspace and Unity Catalog (Chapters 0, 1, 8)

- ☐ Unity Catalog metastore attached and verified; no production assets left in `hive_metastore`.
- ☐ One catalog per environment (dev/staging/prod); environment separation by catalog, not by convention.
- ☐ Medallion schemas (`bronze/silver/gold`) named and governed per catalog; naming conventions documented and followed.
- ☐ Managed tables are the default; every external table carries a stated reason.
- ☐ Grants assigned to groups at the highest level that expresses intent (schema for “analysts read Gold”); individual and table-level grants are documented exceptions.
- ☐ Row filters and column masks applied on the table for declarative policy; dynamic views reserved for compound or shared masking logic.
- ☐ Every grant validated by a negative test executed as a restricted principal (never as the object owner).
- ☐ Cluster policies enforce auto-termination ceilings, node restrictions, and mandatory cost tags; unattributed compute is structurally impossible.
- ☐ Production code lives in Git folders; workspace folders hold exploration only.
- ☐ Storage credentials scoped to single containers, rotation scheduled, no wildcard storage rights.



## D.2 Delta Lake and Storage (Chapters 2, 3, 5)

- ☐ Every transition between layers is idempotent (keyed `MERGE` or `replaceWhere`); a same-day rerun changes zero Gold counts.
- ☐ Reconciliation arithmetic per run: `input = accepted + quarantined`, asserted in the pipeline and logged to a run ledger.
- ☐ Quarantine tables have a named steward and a review cadence.
- ☐ `VACUUM` retention exceeds the recovery objective and the longest query duration; retention changes are change-controlled.
- ☐ `RESTORE` drill executed and evidenced in `DESCRIBE HISTORY` before go-live.
- ☐ Schema evolution is deliberate: `mergeSchema` only where drift is tolerated; breaking changes halt the pipeline for review.
- ☐ Layout decisions (liquid clustering keys, partitioning, `Z-ORDER`) made from measured query patterns, with a benchmark table of file counts and timings as evidence.
- ☐ Deletion vectors understood: readers pay a merge cost until the next `OPTIMIZE`; compaction scheduled accordingly.
- ☐ Small-file posture monitored; `OPTIMIZE` (or predictive optimization) scheduled in bands that do not fight active writers.

## D.3 SQL Warehouses and BI (Chapters 6, 7)

- ☐ Warehouse tier chosen from requirements (serverless for BI by default); auto-stop configured; max clusters sized for the known burst window.
- ☐ Certified views are the single contract for metrics; dashboards and Genie spaces point at them, never at Bronze.
- ☐ Materialized views used for stable derived shapes; versioned business logic lives in tested, scheduled pipelines.
- ☐ Query profiles captured for the critical dashboards; dataset SQL reads MVs/certified views, not raw fact scans.
- ☐ Published dashboards run as a service identity whose data access equals what subscribers may see.
- ☐ Genie spaces scoped per domain with instructions, example queries, and a measured first-try accuracy on a fixed question set.
- ☐ Cost attribution live: `system.billing.usage` joined to warehouse and user identity, reviewed monthly.

## D.4 DLT and Batch Pipelines (Chapters 9–12)

- ☐ Every expectation has a deliberate severity: warn, drop (with quarantine visibility), or fail (protecting downstream consumers).
- ☐ The DLT event log is queried, alerted on, and retained as quality evidence—not just viewed in the UI.

- ☐ Full-refresh requirements known per dataset before deployment.
- ☐ Auto Loader checkpoints governed (one per stream, durable location); rescued data column monitored and stewarded.
- ☐ CDC flows carry a **SEQUENCE BY** guard whose column is unique and monotonic per key.
- ☐ Ingestion contracts with producers specify schema/compatibility rules, key and sequence semantics, arrival SLA, and the malformed-data policy.
- ☐ Tuning evidence table exists: isolated before/after metrics per change, no confounded experiments.
- ☐ Every job task is idempotent, so repair runs re-execute only the failed subtree safely.
- ☐ Quality gates actually block Gold publish; conditional alerts wired and tested with a forced failure.
- ☐ Job parameters and task values used for their distinct purposes; no dates hardcoded into task code.
- ☐ SLA alerting fires on trajectory (elapsed + remaining-p50), not only on breach.

## D.5 Streaming and Real-Time (Chapters 13–16)

- ☐ Exactly-once argued end to end: checkpointed offsets, deterministic state, idempotent sink—all three, per pipeline.
- ☐ One checkpoint per query; incompatible changes deploy with a new checkpoint, a replay, and a reconciliation before cutover.
- ☐ Restart drill executed mid-stream: post-restart counts prove no loss and no duplication.
- ☐ Every stateful operator has a watermark; watermark sized from a measured lateness histogram (p99 plus margin), never guessed.
- ☐ RocksDB state store enabled when state size exceeds comfortable executor memory, with before/after latency measured.
- ☐ `maxOffsetsPerTrigger` (or equivalent) caps micro-batch size; partition count sized for peak rate with headroom.
- ☐ Message schemas registered with compatibility rules; inference confined to the sandbox.
- ☐ Connection strings and broker credentials come from secret scopes; none appear in code or source control.
- ☐ Input rate versus processing rate graphed per stream; freshness indicator on every real-time dashboard.
- ☐ A nightly batch tier reconciles the streaming tier; replay drills produce zero duplicates.

## D.6 ML and Mosaic AI (Chapters 17–20)

- ☐ Every training run carries git SHA, dataset snapshot/version, parameters, and environment; nine-month-old runs are reproducible.
- ☐ Models registered in Unity Catalog; production code resolves aliases (`@champion`), never pinned versions.

- ☐ Feature tables are the single definition feeding training (timestamp lookups) and serving; point-in-time rebuilds verified for historical as-of dates.
- ☐ Leakage audit performed before any drift investigation: offline/online gap checked against as-of correctness.
- ☐ Every endpoint has inference logging, an owner, and a health dashboard.
- ☐ Promotion is human-gated at the alias move; retraining is drift-triggered, never alarm-triggered.
- ☐ Rollback is an alias re-point, drilled on a schedule.
- ☐ RAG systems keep chunks in governed Delta with the index as a rebuildable projection; a golden evaluation set gates every retrieval or prompt change.
- ☐ Batch and endpoint scoring paths reconciled against the same registry version.

## D.7 Security and Networking (Chapters 21, 22)

- ☐ SSO binds the account to the IdP; SCIM syncs group membership; no one-off local users.
- ☐ No production workload runs as a person: least-privilege service principals with OAuth M2M, per-grant justification, and audit sampling.
- ☐ PAT creation governed by workspace policy (lifetime and scope limits).
- ☐ All secrets in secret scopes (vault-backed where rotation SLAs apply); notebooks call `dbutils.secrets.get`—zero literal credentials.
- ☐ Network posture layered: private connectivity, IP access lists, SSO/MFA, short-lived tokens—with each control’s blind spot documented.
- ☐ Time-boxed access implemented as expiring IdP group membership, never “temporary admin.”
- ☐ Negative tests run as code on a schedule: principals attempt what they must not do; results logged as evidence.
- ☐ PII classified with tags at table creation; a scheduled audit query pages the steward on non-approved reads.
- ☐ Delta Sharing shares scoped to the agreed tables; the revocation drill (grant, read, revoke, failed read) executed and logged quarterly.
- ☐ Erasure requests handled as a pipeline: targeted deletes, provenance query for every copy, `VACUUM` timing understood.

## D.8 CI/CD and FinOps (Chapters 23, 24)

- ☐ Production is a build artifact: bundles plus Terraform in Git, deployed by CI, gated by staging. Nobody clicks in prod.
- ☐ `bundle validate` runs on every pull request; targets and modes make dev collisions and prod schedules structurally distinct.
- ☐ Terraform state is remote, locked, and access-controlled.
- ☐ CI authenticates as a service principal (OAuth M2M / OIDC), never with a personal token.

- ☐ Hotfixes ride the same pipeline with an expedited approval gate—never a different process.
- ☐ A drift job compares deployed state against the bundle registry; manual changes are caught, not debated.
- ☐ The clean-clone drill (`terraform apply + bundle deploy -t prod` into a fresh workspace) succeeds and is timed.
- ☐ Chargeback runs monthly from `system.billing.usage` plus policy-enforced tags; the review ends with two implemented optimizations.
- ☐ Every repo README leads with problem, diagram, setup, evidence, cost, and lessons learned.

**Tip**

Run these checklists as a standing review: pick one domain per month, produce the evidence for every box, and file the gaps as tickets. The first pass always finds unchecked boxes; the discipline is that the second pass finds fewer.

# Certification Exam Mapping and Study Guide

This appendix maps the Databricks Certified Data Engineer exams—Associate and Professional—onto the book’s chapters, then gives the study strategy and self-assessment checklist that Chapter 24 operationalizes. Exam blueprints are revised periodically; always confirm the current domain list and weightings in the official exam guides before booking [15, 16, 17].

## E.1 The Two Exams at a Glance

- **Data Engineer Associate.** Foundational, multiple-choice: the Databricks platform, Delta Lake, ETL with Spark SQL and PySpark, incremental ingestion, production pipelines (jobs, DLT), and governance with Unity Catalog. Reachable at the end of Part III of this book (Week 12), with Parts I–II as prerequisite fluency [16].
- **Data Engineer Professional.** The same surface at production depth: scenario questions that reward failure-mode knowledge—checkpoint semantics, expectation behavior, concurrency, recovery, and performance trade-offs. Expect it to assume everything in Parts IV–VI: streaming, delivery, security, and cost discipline [17].

## E.2 Domain-to-Chapter Mapping

The book’s curriculum was built around the exam domains; table E.1 consolidates the mapping from Chapter 24 (its table 24.1) with the remedial lab for each domain. Score each domain from mock-exam evidence, not feeling.

| Exam domain                          | Chapters  | Rebuild these labs if weak                                  |
|--------------------------------------|-----------|-------------------------------------------------------------|
| Databricks platform & compute        | 0, 1, 6   | Cluster policies; warehouse sizing lab                      |
| Delta Lake                           | 2, 5      | Enforcement/evolution/RESTORE drill; optimization benchmark |
| Medallion design                     | 3         | Quarantine and reconciliation lab                           |
| ETL with Spark & DLT                 | 4, 9, 10  | Skew remediation; expectations poison drill                 |
| Incremental ingestion & CDC          | 11        | Auto Loader drift drill; SCD2 flow                          |
| Production pipelines & orchestration | 12, 23    | Repair-run drill; bundle CI/CD lab                          |
| Streaming                            | 13–16     | Restart proof; watermark/dedup lab                          |
| Governance & security                | 8, 21, 22 | Masking + negative tests; share revocation                  |
| ML awareness                         | 17–20     | Registry alias flow; drift drill                            |

Table E.1: Exam domains mapped to chapters and remedial labs (extends Chapter 24’s mapping).

### E.2.1 Where the Exams Probe

The scenario questions concentrate on exactly the judgment calls the labs teach by breaking things:

- **Recovery semantics:** what VACUUM destroys, when RESTORE beats re-running, what a new checkpoint implies (Chapters 2, 11, 13).
- **Quality behavior:** when `expect_or_drop` versus `expect_or_fail` is correct, and what each does to downstream datasets (Chapter 10).
- **Concurrency and idempotency:** optimistic MERGE conflicts, safe repair runs, replay without duplicates (Chapters 2, 12, 14, 15).
- **Cost and layout:** job versus all-purpose clusters, liquid clustering versus partitioning, triggered versus continuous streaming (Chapters 1, 5, 13–16).
- **Governance mechanics:** grant scope, masking enforcement, lineage queries, sharing revocation (Chapters 8, 21, 22).

## E.3 Study Strategy

1. **Map confidence to domains.** Score every domain in table E.1 from evidence: the last lab you rebuilt unaided and your mock results, per domain.
2. **Remediate by rebuilding, not re-reading.** A weak domain means rebuild that domain’s lab from scratch, in a fresh schema, without notes. Reviewing answer keys without rebuilding converts a mock into trivia.
3. **Take mocks under exam conditions.** Timed, closed-book, no notes, scored by domain. Classify every miss as knowledge gap (rebuild the lab), reasoning error (re-read two scenario questions slowly), or time management (budget per question).
4. **Apply the two-pass standard.** Book the exam when you hold two consecutive 80%+ mocks with no domain below 70%. One good mock is variance; two is signal.

5. **Study failure modes, not feature lists.** For each domain, name its three most failure-mode-flavored facts before exam day.

## E.4 Self-Assessment Checklist

Before booking, you should be able to answer “yes, with evidence” to each line:

### Platform and Storage

- ☐ I can explain the control/data plane split and choose between all-purpose, job, and serverless compute for a given workload (Chapters 0, 1).
- ☐ I can walk a Delta commit end to end and resolve a **MERGE** conflict (Chapter 2).
- ☐ I have executed a corruption-and-RESTORE drill and can prove row counts before and after (Chapter 2).
- ☐ I can defend a layout choice—liquid clustering, partitioning, Z-ORDER—with a benchmark table (Chapter 5).

### Pipelines and Processing

- ☐ I can read **explain** and the Spark UI and fix skew with two different mechanisms (Chapters 4, 9).
- ☐ I can build a DLT pipeline with warn/drop/fail expectations and query its event log (Chapter 10).
- ☐ I can ingest with Auto Loader through a mid-stream schema change and apply CDC with an SCD2 sequence guard (Chapter 11).
- ☐ I have run a Workflow repair and can state what makes it safe (Chapter 12).

### Streaming

- ☐ I can argue exactly-once end to end for a pipeline I built (Chapters 13–15).
- ☐ I can size a watermark from a lateness histogram and defend the drop horizon (Chapter 14).
- ☐ I have reconciled a replayed Kafka window against the primary table (Chapter 15).

### Governance, Delivery, and Readiness

- ☐ I can prove least privilege with a grant matrix and a negative test (Chapters 8, 21).
- ☐ I can trace a metric to its sources with lineage and audit queries (Chapter 22).
- ☐ I can deploy a bundle across dev/staging/prod with a CI pipeline and roll it back (Chapter 23).
- ☐ I hold two consecutive 80%+ mocks with no domain below 70% (Chapter 24).
- ☐ I can present my capstone in 15 minutes with one number per box and two trade-offs defended two levels deep (Chapter 24).

**Common Pitfall**

**Booking on the first good mock.** A single 85% is variance; the exam fee buys a second data point you could have gotten for free. Meet the two-pass standard, and keep the lab-rebuild habit afterward—it is also how senior engineers stay honest.



# Interview Question Bank

This appendix consolidates the Interview Q&A sections of Chapters 1–24 into a single bank, organized by domain and tagged by difficulty. Answers are abbreviated to the core argument; the chapter reference holds the full explanation, diagrams, and labs. Use it for spaced repetition, mock interviews, and capstone-defense rehearsal.

### Note

**Difficulty tags.** [F] foundational—expected of any candidate; [I] intermediate—separates practitioners from readers; [A] advanced—staff-level trade-off and failure-mode reasoning. Answer aloud in under ninety seconds; if you cannot, rebuild the referenced chapter’s lab, not just its prose.

## F.1 Platform and Workspace (Chapter 1)

- [F] **Why does Databricks separate the control plane from the data plane?** (Ch. 1) The SaaS control plane orchestrates while data and compute stay in the customer’s cloud account—data residency, network isolation, existing cloud commitments.
- [F] **All-purpose or job cluster for a nightly pipeline?** (Ch. 1) Job cluster: created per run, cheaper, workload-exact, destroyed after—no idle spend or shared state.
- [I] **What problem do cluster policies actually solve?** (Ch. 1) They make standards enforceable: budgets, node restrictions, and required tags become properties of every cluster, not documentation.
- [I] **How would you give 200 users compute without 200 bill shocks?** (Ch. 1) Policy-bound clusters per team, serverless where possible, job clusters for schedules, weekly per-tag chargeback from `system.billing.usage`.
- [A] **What would make you split into a second workspace?** (Ch. 1) Only a hard trust or environment boundary; performance and organization are handled by catalogs and policies.

## F.2 Delta Lake and Medallion Design (Chapters 2, 3, 5)

- [F] **How can a file store be ACID?** (Ch. 2) Files are immutable; the atomic unit is the single-file commit to the transaction log—readers see a version only after its commit exists.

- **[I] Two jobs MERGE into the same table simultaneously—what happens?** (Ch. 2) Optimistic concurrency: one commits first; the other re-reads and reconciles (disjoint keys) or retries/fails on conflict.
- **[I] RESTORE versus re-running the pipeline to fix a bad load?** (Ch. 2) RESTORE for table-level mistakes (instant, auditable); re-run when the bug is in the transformation logic itself.
- **[I] Why not set VACUUM retention to 1 hour?** (Ch. 2) It deletes the files time travel and long-running readers need; retention must exceed the recovery objective and the longest query.
- **[A] Does Delta’s ACID protect against a logically wrong write?** (Ch. 2) No—ACID guarantees atomicity and durability, not correctness. Correctness comes from constraints, expectations, and reconciliation.
- **[F] Why keep Bronze forever when storage costs money?** (Ch. 3) Bronze is the only answer to “what did the source actually say” and the only input for rebuilding after logic changes.
- **[I] How do you prove a transition lost no rows?** (Ch. 3) Reconciliation arithmetic per run: input = accepted + quarantined, asserted and logged to a run ledger.
- **[I] MERGE or replaceWhere for Silver?** (Ch. 3) MERGE for keyed upserts; `replaceWhere` for full-partition restatements. Both idempotent; never mixed per table.
- **[A] Why does the medallion make streaming easier later?** (Ch. 3) Each layer already has idempotent, keyed writes—exactly what streaming sinks require. Batch becomes streaming by changing triggers, not contracts.
- **[F] Why do small files hurt even at the same total bytes?** (Ch. 5) Per-file costs (listing, footers, scheduling) scale with file count; metadata work dominates and the driver becomes the bottleneck.
- **[I] Z-ORDER versus liquid clustering?** (Ch. 5) Both co-locate rows for skipping; liquid keeps keys as changeable metadata maintained incrementally. New tables: liquid; legacy: Z-ORDER until migrated.
- **[A] How would you convince a skeptic that liquid clustering helped?** (Ch. 5) Same query, same data, controlled medians across cold and warm runs, plus the file-statistics story. Feelings are not benchmarks.

### F.3 Spark Execution and Performance (Chapters 4, 9)

- **[F] Walk through what happens on `df.write.saveAsTable()`.** (Ch. 4) Catalyst optimizes the logical plan, the physical plan splits into stages at shuffles, one task per partition runs on executors, and the final commit publishes the table version.
- **[I] Why is the shuffle the usual suspect?** (Ch. 4) It is the only all-to-all data movement—network, serialization, spill—plus a barrier. Most big-job pathologies are shuffle pathologies.
- **[I] What does AQE give you, and when do you override it?** (Ch. 4) Runtime coalescing, broadcast conversion, and skew splitting from actual statistics; override on known-skewed keys or a provably fittable broadcast it declined.

- **[I] When do you cache?** (Ch. 4) When one expensive DataFrame feeds multiple actions. Never for single-scan pipelines; always unpersist on shared clusters.
- **[A] How do you size a cluster for a brand-new job?** (Ch. 4) Estimate shuffle volume, partition for ~100–200 MB per task, memory for the largest task plus spill headroom—then let three runs of UI metrics correct the estimate.
- **[I] When is salting better than AQE skew handling?** (Ch. 9) Extreme, known skew (one dominant key): salting spreads it by construction; AQE splits only after the plan exists.
- **[F] Why are pandas UDFs faster than Python UDFs?** (Ch. 9) Arrow columnar batches cross the JVM/Python boundary once per batch, not once per row.
- **[A] When is a job “tuned enough”?** (Ch. 9) When the next measured improvement costs more engineering time than the compute it saves, given the SLA margin—tuning has an economic stopping rule.

## F.4 SQL Warehousing and BI (Chapters 6, 7)

- **[F] Why is Databricks SQL different from a BI tool on a Spark cluster?** (Ch. 6) Photon execution, warehouse-level caching and scaling, governance integration, per-query profiles—a serving layer, not a general cluster with a SQL cell.
- **[I] Materialized view or scheduled Gold table?** (Ch. 6) MV for stable derived shapes with freshness tolerance; scheduled Gold when the logic is versioned business logic needing tests and review.
- **[I] A dashboard is slow only at 9 a.m. Diagnose.** (Ch. 6) Queueing at the concurrency bound: raise max clusters for the burst, move the dashboard to an MV, confirm serverless for seconds-fast scale-up.
- **[I] What does the query profile tell you that duration does not?** (Ch. 6) Where the time went: which operator scanned what and spilled what. Duration says slow; the profile says why.
- **[I] How do you stop Genie from hallucinating metrics?** (Ch. 7) Scope to certified tables, encode definitions as instructions, provide example queries, and measure first-try accuracy on a fixed question set.
- **[A] Who should a published dashboard run as?** (Ch. 7) A service identity whose access equals what subscribers may see; publishing as a privileged user silently escalates every subscriber’s access.

## F.5 Governance and Security (Chapters 8, 21, 22)

- **[F] Managed or external table—how do you decide?** (Ch. 8) Managed by default; external only when a non-Databricks writer owns the files or the location is mandated.
- **[I] Where do you grant—catalog, schema, or table?** (Ch. 8) The highest level matching intent: schema for “analysts read Gold,” table only for true exceptions. High-level grants are reviewable; a thousand table grants are not.
- **[I] Row filters/column masks versus dynamic views?** (Ch. 8) Filters and masks live on the table and apply to every consumer—prefer them; dynamic views for compound or shared logic.

- **[A] How do you prove least privilege to an auditor?** (Ch. 8) The grant matrix, a negative test per principal class, and `system.access.audit` samples—implemented, tested, logged.
- **[A] Why test grants as a restricted principal rather than reviewing GRANTS?** (Ch. 8) Grants compose (ownership, inheritance, nesting) in ways reviewers misread; only the negative test survives composition.
- **[F] Why must production jobs run as service principals?** (Ch. 21) Person-run jobs break when the person leaves, inherit excessive access, and blur accountability; principals are scoped, owned, rotated, auditable.
- **[I] Is an IP access list sufficient network security?** (Ch. 21) No—it is a perimeter filter; stolen credentials from an allowed network pass. Layer with SSO/MFA, short-lived tokens, private connectivity, and audit alerts.
- **[A] Which single control gives the most risk reduction per hour?** (Ch. 21) Running production workloads as least-privilege service principals: caps blast radius, survives personnel change, makes actions attributable.
- **[I] What does column-level lineage change operationally?** (Ch. 22) Impact analysis becomes a query—every downstream consumer enumerable before a change, every metric traceable after an anomaly.
- **[A] How do you handle a data-subject erasure request in a lakehouse?** (Ch. 22) Targeted deletes through the layers with deletion vectors, a provenance query for every copy, VACUUM timing understood—erasure is a pipeline with evidence.

## F.6 Ingestion, Pipelines, and Orchestration (Chapters 10–12)

- **[F] What does DLT do that a notebook chain does not?** (Ch. 10) Dependency inference, execution-mode management, checkpointing, retries, per-dataset quality metrics, an event log, and UC lineage—the operational layer you would otherwise hand-build.
- **[I] Streaming table versus materialized view?** (Ch. 10) A streaming table appends each source record exactly once (facts); a materialized view maintains a query result (derived state). “What arrived” versus “what is true.”
- **[I] expect\_or\_drop versus expect\_or\_fail?** (Ch. 10) Drop when the row is expendable and the pipeline must continue (with quarantine visibility); fail when bad data invalidates the whole batch.
- **[I] How does Auto Loader achieve exactly-once?** (Ch. 11) The checkpoint durably records processed files per micro-batch; restarts resume from it, and idempotent Delta sink commits complete the contract.
- **[A] What does SEQUENCE BY protect against?** (Ch. 11) Out-of-order and replayed CDC events: without it, an older update overwrites a newer one and silently regresses table state.
- **[I] What belongs in the ingestion contract with an external producer?** (Ch. 11) Schema and compatibility rules, key and sequence semantics, arrival SLA, and the malformed-data policy.
- **[I] What makes a repair run safe?** (Ch. 12) Task idempotency plus true dependency modeling: repair reuses successful outputs and re-runs only the failed subtree.

- **[A] How do you alert on trajectory, not breach?** (Ch. 12) Track per-task duration baselines; compare elapsed + remaining-p50 against the SLA and page early. Breach alerts tell you what you already lost.

## F.7 Streaming and Real-Time (Chapters 13–16)

- **[F] Explain micro-batch execution to a backend engineer.** (Ch. 13) The stream is an infinite table; each trigger runs a small incremental batch over rows since the last committed offset, writes transactionally, and commits offsets to the checkpoint.
- **[I] What exactly does the checkpoint make safe?** (Ch. 13) Restart position, stateful-operator state, and query metadata. With an idempotent sink: exactly-once; without it: at-least-once with duplicates.
- **[I] When is availableNow the right trigger?** (Ch. 13) Whenever freshness is minutes-to-hours: streaming sources at batch economics—process the backlog, stop, pay nothing until next schedule.
- **[F] What does a watermark actually do?** (Ch. 14) Defines the lateness horizon after which events drop, and authorizes eviction of older state—unbounded memory becomes a bounded budget.
- **[I] Why is dropDuplicates dangerous without a watermark?** (Ch. 14) Deduplication remembers every key ever seen; without eviction, state grows until the executors die.
- **[A] Argue exactly-once for a pipeline, end to end.** (Ch. 14) Checkpointed offsets, deterministically derived and checkpointed state, idempotent sink (keyed MERGE or Delta replace): a crash replays a batch into a sink that ignores replays.
- **[A] A stakeholder wants “no late data dropped, ever.” Response?** (Ch. 14) Then state is unbounded and windows never close; the honest design is a long watermark plus a batch reconciliation path for the long tail.
- **[I] Why a schema registry for streams?** (Ch. 15) Schemas become versioned contracts with compatibility rules; a breaking producer change fails at registration instead of corrupting every consumer at 03:00.
- **[I] How many partitions should a topic have?** (Ch. 15) Peak rate divided by per-partition sustainable throughput, times headroom—measured, then rounded up, because partitions only ever get added.
- **[A] Why foreachBatch instead of a native MERGE sink?** (Ch. 16) Streaming DataFrames do not support MERGE directly; `foreachBatch` exposes each micro-batch as a batch DataFrame while the checkpoint preserves exactly-once across the hand-off.
- **[A] When is continuous processing justified over 30-second micro-batches?** (Ch. 16) When the measured business cost of 30 seconds of staleness exceeds the premium—rare. Most “real-time” requirements are micro-batch requirements.

## F.8 ML and MLOps (Chapters 17–20)

- **[F] What does autologging not capture that you must add?** (Ch. 17) Your context: dataset identity/version, business owner, feature-set version, approval linkage—the metadata that makes runs governable.

- **[I] Alias versus version in production code?** (Ch. 17) Always the alias: promote and roll back by moving a pointer, never by redeploying version-pinned code.
- **[I] Reproduce a nine-month-old run. What do you need?** (Ch. 17) Git SHA, dataset snapshot (or Delta version), parameters, and environment—all four logged.
- **[I] Define point-in-time correctness and its violation cost.** (Ch. 18) Every feature for a label at time  $t$  computed only from data before  $t$ ; violate it and the model trains on the future—inflated offline metrics, useless production behavior.
- **[A] How does a feature store prevent training-serving skew?** (Ch. 18) One governed definition feeds both paths: timestamp lookups for training, latest-value lookups for serving. Skew requires two definitions; the store has one.
- **[A] A feature job failed silently for three days. What do users see?** (Ch. 18) Stale-but-plausible predictions—the dangerous failure. Defense: freshness gates in scoring and drift monitoring on feature distributions.
- **[I] Why keep chunks in Delta if the vector index serves queries?** (Ch. 19) Delta is the governed, versioned, rebuildable source of truth; the index is a derived projection you rebuild when embeddings change or sync corrupts.
- **[I] How do you evaluate a RAG system?** (Ch. 19) A fixed golden set; retrieval metrics and LLM-judged generation metrics logged per run; changes gated on the metrics in CI.
- **[I] Serverless endpoint versus batch scoring—how do you choose?** (Ch. 20) By consumption pattern: interactive apps need endpoints; bulk periodic decisions need batch scoring. Many models need both, from the same registry version.
- **[A] Why must promotion stay human-gated?** (Ch. 20) Automated retraining proves fit to recent data, not business acceptability—fairness, stability, and threshold effects need judgment. Automate up to the gate; gate the alias move.

## F.9 Delivery and CI/CD (Chapter 23)

- **[I] Asset Bundles versus Terraform—what goes where?** (Ch. 23) Terraform for the slow perimeter (workspaces, principals, metastores); bundles for the fast application layer (jobs, pipelines, dashboards). Both in Git, both through review.
- **[F] What does bundle validate actually catch?** (Ch. 23) Schema errors, missing variables, bad references, and permission problems—before any cloud change. It is the compile step for your platform.
- **[I] Why remote, locked Terraform state?** (Ch. 23) State records what exists; local state forks under team/CI use, and a forked state proposes recreating—or deleting—your production estate.
- **[A] How do you keep humans out of prod without slowing hotfixes?** (Ch. 23) The same pipeline with an expedited approval gate: the hotfix is a PR with a faster review SLA, not a different process.
- **[I] How do you attribute a cost spike to a change?** (Ch. 23) Join the billing window to deployment history—bundle deploys and Terraform applies are logged, so the cost curve diffs against the release curve.

## F.10 Capstone and Architecture Defense (Chapter 24)

- **[F] Walk me through your platform end to end.** (Ch. 24) Ninety seconds: sources → ingestion → medallion → serving/ML → governance → delivery, one number per arrow. Five minutes means you do not yet own it.
- **[A] What would you rebuild differently at 10x scale?** (Ch. 24) Have the real answer: partition/parallelism ceilings, orchestration fan-in, the cost model. “Nothing” fails the question.
- **[A] Your most expensive mistake in this build?** (Ch. 24) Name one—the untested rollback, the unmonitored endpoint, the leaked token—quantify it, and show the mechanism that now prevents it.
- **[I] How does your platform prove data correctness to a skeptic?** (Ch. 24) Reconciliation arithmetic per transition, expectation metrics from the event log, replay drills, and the audit trail—evidence, not assurances.
- **[A] Why Databricks over a point-solution stack?** (Ch. 24) One governed storage and catalog under batch, streaming, SQL, and ML—versus four governance models and four copies. Know the honest counter-costs too (DBU economics, Spark operational depth).

### Tip

Mock-interview protocol: draw three questions at random across domains, answer aloud in ninety seconds each, and have the interviewer probe one level deeper than your answer. Any question you survive only by reciting—rather than reasoning from a lab you built—goes on the rebuild list from Appendix E.

# Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the seven platforms studied in this book series: Databricks, Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, open table formats, elastic compute, declarative pipelines, streaming, and disciplined governance—recur everywhere.

## Note

This appendix consolidates the equivalence tables from every chapter’s cross-cloud callout—Chapters 1 through 24—into a single Rosetta Stone for moving fluently between platforms. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

## G.1 Platform Foundations and Compute

Databricks’ Chapter 1 topic—workspaces, clusters, and the control/data plane split—maps to each cloud’s way of bundling compute and administration.

| Concept             | Databricks                        | AWS                            | Azure / GCP                       |
|---------------------|-----------------------------------|--------------------------------|-----------------------------------|
| Administrative unit | Workspace (+ account console)     | Account / IAM Identity Center  | Entra tenant / GCP project        |
| Interactive compute | All-purpose cluster               | EMR cluster / Glue interactive | Synapse Spark pool / Dataproc     |
| Job compute         | Job cluster (per run)             | Glue job / EMR Serverless      | Synapse job / Dataproc Serverless |
| Compute guardrails  | Cluster policies + pools          | IAM + service quotas           | Policies / quotas                 |
| Billing unit        | DBU + cloud VM cost               | DPU / node-hours               | vCore-hours / slot-hours          |
| CLI / SDK           | Databricks CLI, databricks-sdk-py | AWS CLI, Boto3                 | az CLI / gcloud                   |

**Microsoft Fabric** replaces per-cluster decisions with a shared capacity pool (F-SKU CUs); **Snowflake** replaces clusters with virtual warehouses; **MongoDB Atlas** abstracts compute as cluster tiers.

## G.2 Table Storage and the Lakehouse

Databricks’ Chapter 2 topic—Delta Lake—is the open table format at the center of the lakehouse pattern on every cloud.



| Capability        | Databricks                 | AWS / Azure / GCP                        | Snowflake / Fabric                  |
|-------------------|----------------------------|------------------------------------------|-------------------------------------|
| Table format      | Delta Lake                 | Delta / Iceberg / Hudi on object storage | Micro-partitions / Delta on OneLake |
| ACID on objects   | Transaction log            | Iceberg/Hudi catalogs; S3 Tables         | Native / Delta transaction log      |
| Time travel       | VERSION AS OF              | Iceberg snapshots                        | Time Travel / Delta versions        |
| Compaction        | OPTIMIZE                   | Iceberg <code>rewrite_data_files</code>  | Automatic / OPTIMIZE VORDER         |
| Clustering        | Liquid clustering, Z-ORDER | Iceberg sort order, Athena partition     | Clustering keys / V-Order + ZORDER  |
| Row-level deletes | Deletion vectors           | Iceberg position/m equality deletes      | Native / deletion vectors           |

**MongoDB** is the instructive contrast: a node-local document engine (WiredTiger) rather than object-storage tables.

### G.3 The Medallion Pattern

Databricks' Chapter 3 topic—Bronze/Silver/Gold—is a portable design pattern rather than a product.

| Layer              | Databricks           |        |           | Data lake (AWS/Azure/GCP)        | Snowflake / Fabric                |
|--------------------|----------------------|--------|-----------|----------------------------------|-----------------------------------|
| Bronze (raw)       | <b>bronze</b> schema | Delta  | tables    | <b>raw/</b> prefix (S3/ADLS/GCS) | RAW schema / Bronze Lakehouse     |
| Silver (validated) | <b>silver</b> schema | Delta  | tables    | <b>validated/</b> prefix         | STAGING schema / Silver Lakehouse |
| Gold (curated)     | <b>gold</b> schema   | schema | star      | <b>curated/</b> prefix           | ANALYTICS schema / Gold Lakehouse |
| Governance         | Unity Catalog        | grants | per layer | Lake Formation / RBAC per zone   | Role grants / workspace roles     |

### G.4 Managed Spark and Execution Engines

Databricks' Chapter 4 topic—managed Apache Spark—is the reference implementation of managed Spark; the engine is the same everywhere, but the operational wrapper differs.

| Capability            | Databricks                         |              | AWS                          | Azure / GCP                            |
|-----------------------|------------------------------------|--------------|------------------------------|----------------------------------------|
| Managed Spark         | Databricks (Photon)                | Runtime      | Glue / EMR / EMR Serverless  | Synapse Spark / Fabric / Dataproc      |
| Cluster manager       | Built-in (no YARN to run)          |              | YARN / EMR managed           | Synapse / Dataproc managed             |
| GPU/vectorized engine | Photon                             |              | EMR runtime (partial)        | Fabric native engine                   |
| Autoscaling           | Autoscaling clusters               | + serverless | EMR managed scaling          | Autoscale pools / Dataproc autoscaling |
| UI and diagnostics    | Spark UI + Ganglia + query profile |              | Spark UI on EMR              | Spark UI / Fabric monitoring           |
| Default table format  | Delta                              |              | Iceberg/Hudi/Delta by config | Delta (Fabric) / Big-Query tables      |

**Snowflake** plays this role with Snowpark (DataFrame API on its engine); **MongoDB** integrates via the Spark Connector. Skills from Chapter 4 transfer to every managed-Spark offering almost unchanged.

## G.5 SQL Warehousing and BI

Chapters 5–7 cover optimization, SQL warehouses, and AI/BI; every platform fields a SQL serving layer and a BI surface.

| Capability         | Databricks                             | AWS                              | Azure / GCP / Fabric                                |
|--------------------|----------------------------------------|----------------------------------|-----------------------------------------------------|
| SQL compute        | SQL warehouse (serverless/pro/classic) | Redshift Serverless / Athena     | Synapse / BigQuery slots / Fabric Warehouse         |
| Execution engine   | Photon                                 | Redshift engine / Athena (Trino) | Synapse / Dremel (BigQuery) / Fabric engine         |
| Result reuse       | Result cache + materialized views      | Result cache / Athena reuse      | Result-set cache / BI Engine / Direct Lake          |
| BI surface         | AI/BI dashboards + Genie               | QuickSight (+ Q)                 | Power BI / Looker / Power BI in Fabric              |
| NL analytics       | Genie spaces                           | Amazon Q in QuickSight           | Copilot / Gemini in Looker / Fabric Copilot         |
| Cost introspection | <code>system.billing.usage</code>      | CUR + CloudWatch                 | Cost Management / billing export / Capacity Metrics |

## G.6 Governance and Catalogs

Chapters 8 and 22 cover Unity Catalog; each hyperscaler answers governance differently.

| Capability           | Databricks                               | AWS                                 | Azure / GCP                            |
|----------------------|------------------------------------------|-------------------------------------|----------------------------------------|
| Catalog + governance | Unity Catalog                            | Glue Catalog + Lake Formation       | Purview + OneLake / Dataplex + BigLake |
| Namespace            | metastore → catalog → schema → table     | account → database → table          | tenant / project → dataset → table     |
| Row/column security  | Dynamic views, row filters, column masks | LF cell filters                     | RLS + OPA / policy tags                |
| Lineage              | UC lineage (table + column)              | Glue lineage / SageMaker            | Purview / Dataplex lineage             |
| Audit                | <code>system.access.audit</code>         | CloudTrail + LF audit               | Activity logs / audit logs             |
| Secure sharing       | Delta Sharing (open protocol)            | S3 Access Grants / Redshift sharing | Azure Data Share / Analytics Hub       |

**Snowflake** governs via account-level role hierarchies and shares via Secure Data Sharing; **MongoDB** governs at the cluster/collection level with Atlas roles and auditing.

## G.7 Declarative Pipelines, Ingestion, and Orchestration

Chapters 9–12 cover PySpark tuning, DLT, Auto Loader, and Workflows.

| Capability              | Databricks                       | AWS                           | Azure / GCP / Fabric                           |
|-------------------------|----------------------------------|-------------------------------|------------------------------------------------|
| Declarative pipelines   | Delta Live Tables (expectations) | Glue Data Quality + workflows | Fabric pipelines + DQ gates / Dataform         |
| Incremental file ingest | Auto Loader (cloudFiles)         | S3 events + Glue bookmarks    | Event Grid + ADF / Storage transfer + Dataflow |
| CDC application         | MERGE / AUTO CDC                 | DMS CDC to S3 + Glue          | ADF CDC / Datasstream + BigQuery MERGE         |
| Orchestration           | Workflows (jobs + tasks)         | Step Functions / MWAA         | ADF / Fabric pipelines / Cloud Composer        |
| Pipeline as code        | Asset Bundles + Terraform        | CDK / Terraform               | Bicep / Terraform / Fabric Terraform provider  |

## G.8 Streaming

Chapters 13–16 cover Structured Streaming and messaging integration.

| Capability        | Databricks                            | AWS                              | Azure / GCP / Fabric                             |
|-------------------|---------------------------------------|----------------------------------|--------------------------------------------------|
| Stream processing | Structured Streaming                  | Flink (Managed Service) / Lambda | Stream Analytics / Dataflow / Fabric Eventstream |
| Message bus       | Kafka (external), Event Hubs, Kinesis | Kinesis / MSK                    | Event Hubs / Pub/Sub / Eventstream               |
| Exactly-once      | Checkpoint + idempotent sink          | Flink checkpoints + 2PC sinks    | Dataflow exactly-once / at-least-once + dedup    |
| State backend     | RocksDB state store                   | RocksDB (Flink)                  | Managed state / in-memory                        |
| Real-time serving | Gold streaming tables + AI/BI         | Kinesis Analytics → QuickSight   | KQL DB + Real-Time Dashboards                    |

## G.9 Machine Learning and MLOps

Chapters 17–20 cover MLflow, the Feature Store, Mosaic AI, and serving.

| Capability          | Databricks                    | AWS                         |            | Azure / GCP / Fabric                          |
|---------------------|-------------------------------|-----------------------------|------------|-----------------------------------------------|
| Experiment tracking | MLflow (managed)              | SageMaker Experiments       | Experiment | Azure ML / Vertex AI / Fabric MLflow          |
| Model registry      | Unity Catalog registry        | SageMaker Registry          | Model      | Azure ML registry / Vertex registry           |
| Feature store       | Feature Engineering in UC     | SageMaker Store             | Feature    | Azure ML FS / Vertex FS / Fabric feature sets |
| Vector search       | Mosaic AI Vector Search       | OpenSearch / Kendra vectors |            | AI Search / Vertex Vector Search / Fabric AI  |
| GenAI platform      | Mosaic AI (Foundation Models) | Bedrock                     |            | Azure OpenAI / Vertex AI / Fabric AI skills   |
| Serving             | Model Serving endpoints       | SageMaker endpoints         |            | Azure ML online / Vertex endpoints            |
| Monitoring          | Lakehouse Monitoring          | SageMaker Monitor           | Model      | Azure ML monitoring / Vertex monitoring       |

**Snowflake** answers this layer with Snowpark ML, the Feature Store, and Cortex; **MongoDB** with Atlas Vector Search and stream processing.

## G.10 Security, Identity, and Networking

Databricks' Chapter 21 topic—identity, secrets, and network isolation—is a cloud-native discipline with a Databricks control-plane twist.

| Capability         | Databricks                                      | AWS                         |  | Azure / GCP                            |
|--------------------|-------------------------------------------------|-----------------------------|--|----------------------------------------|
| Workforce identity | SSO + SCIM from IdP                             | IAM Identity Center         |  | Entra ID / Cloud Identity              |
| Machine identity   | Service principal + OAuth M2M                   | IAM role                    |  | Managed identity / service account     |
| Long-lived secret  | PAT (policy-controlled)                         | Access keys                 |  | Client secrets / SA keys               |
| Secrets store      | Secret scopes (DB- or vault-backed)             | Secrets Manager             |  | Key Vault / Secret Manager             |
| Private link       | Private connectivity to control plane + storage | PrivateLink + VPC endpoints |  | Private Link / Private Service Connect |
| Perimeter control  | IP access lists                                 | Security groups, SCPs       |  | NSGs, firewall rules / VPC SC          |
| Audit              | <code>system.access.audit</code> + auth logs    | CloudTrail                  |  | Activity Log / Cloud Audit Logs        |

**Snowflake** mirrors this with SSO/SCIM, network policies, and Tri-Secret/CMK options; **MongoDB Atlas** with federated auth, private endpoints, and IP access lists—the same checklist, different console.

## G.11 Infrastructure as Code and CI/CD

Databricks' Chapter 23 topic—Asset Bundles, Terraform, and the delivery pipeline—has a native answer per platform, while Terraform is the cross-cloud constant.

| Capability             | Databricks                                       | AWS                                            | Azure / GCP / Fabric                                        |
|------------------------|--------------------------------------------------|------------------------------------------------|-------------------------------------------------------------|
| Native IaC             | Asset Bundles<br>( <code>databricks.yml</code> ) | CloudFormation /<br>CDK                        | Bicep / Deployment<br>Manager / Fabric Git<br>+ Terraform   |
| Cross-platform IaC     | Terraform provider                               | Terraform                                      | Terraform                                                   |
| Validate-before-deploy | <code>bundle validate</code>                     | <code>cfn-lint</code> / <code>CDK synth</code> | <code>terraform plan</code> /<br><code>what-if</code>       |
| CI auth                | Service principal +<br>OAuth M2M                 | OIDC → IAM role                                | OIDC → workload<br>identity                                 |
| State                  | Terraform remote state                           | S3 + DynamoDB lock                             | Storage account /<br>GCS backend                            |
| Cost telemetry         | <code>system.billing.usage</code>                | CUR 2.0                                        | Cost Management /<br>billing export / Capac-<br>ity Metrics |

**Snowflake**’s story is Terraform-first plus database-change tools (schemachange, dbt); the lesson that transfers everywhere: *the merge is the deploy, and the pipeline is the only writer to prod.*

## G.12 Certification and Readiness

Databricks’ Chapter 24 topic—certification readiness—looks different per platform, but the readiness method is identical: domains mapped to labs, mocks under time, remediation by rebuilding.

| Element          | Databricks                                                   | AWS                                   | Azure / GCP / Fabric                                |
|------------------|--------------------------------------------------------------|---------------------------------------|-----------------------------------------------------|
| Target exam      | Data Engineer Asso-<br>ciate / Professional                  | Data Engineer Asso-<br>ciate          | DP-700 / Professional<br>Data Engineer / DP-<br>600 |
| Heavy domains    | Platform, Delta Lake,<br>Spark ETL, pipelines,<br>governance | Storage, ingestion,<br>transform, ops | Fabric workloads /<br>GCP services / Power<br>BI    |
| Lab weight       | High (build fluency)                                         | High                                  | High                                                |
| Mock standard    | Two mocks, 80%+, no<br>domain <70%                           | Same                                  | Same                                                |
| Capstone defense | Architecture review<br>with cost numbers                     | Same                                  | Same                                                |

The portability argument of the whole series: if you can defend the Databricks capstone *and* sketch its AWS or Fabric equivalent from the cross-cloud boxes, you have learned data engineering, not one console.

### Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on Databricks, immediately locate its equivalent here and predict how the other platforms would express the same idea. That habit turns seven separate vocabularies into one mental model.

# Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

**Access tier** A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

**ACID transaction** Atomicity, consistency, isolation, and durability guarantees for table writes; Delta Lake adds ACID semantics to Parquet files via its transaction log.

**AlloyDB** Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

**Apache Spark** A distributed data-processing engine; Databricks is its original commercial steward, with counterparts in AWS Glue/EMR, Azure Synapse/Fabric, and GCP Dataproc.

**Asset Bundle** Databricks’ infrastructure-as-code format (`databricks.yml`) defining jobs, pipelines, and deployment targets for CI/CD.

**Auto Loader** Databricks’ incremental file-ingestion source (`cloudFiles`) with checkpointed exactly-once semantics, schema inference/evolution, and rescued-data handling.

**Aurora** Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

**Availability SLA** The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

**Bigtable** Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

**Bronze / Silver / Gold** The medallion layers: raw append-only capture, validated and conformed tables, and curated dimensional or aggregate tables for consumption.

**Bucket (storage)** The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*; Unity Catalog governs file access via volumes.

**Change Data Capture (CDC)** Streaming inserts, updates, and deletes from source transaction logs to targets; on Databricks applied with `MERGE INTO` or `DLT AUTO CDC`.

**Checkpoint** The state directory that makes Structured Streaming and Auto Loader exactly-once across restarts; also a compacted Parquet snapshot of the Delta transaction log.

**Clustering key** A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries; compare Delta liquid clustering.

- CMEK / CMK** Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.
- Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).
- DBU** Databricks Unit, the compute billing unit consumed per second of cluster or serverless runtime; total cost is DBU rate plus underlying cloud VM/infrastructure charges.
- Delta Lake** The open table format (created by Databricks) adding ACID transactions, time travel, schema enforcement, and deletion vectors to Parquet files in cloud object storage.
- Delta Live Tables (DLT)** Databricks’ declarative pipeline framework with streaming tables, materialized views, expectations, and an event log; successor concept: Lakeflow Declarative Pipelines.
- Delta Sharing** An open protocol for sharing live Delta tables to any recipient without copying data or granting storage credentials.
- Deletion vector** A Delta Lake row-level delete marker that avoids rewriting entire Parquet files for DELETE, UPDATE, and MERGE operations.
- DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.
- Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.99999999%) for cloud object storage.
- Expectation** A DLT data-quality rule with warn (`expect`), drop (`expect_or_drop`), or fail (`expect_or_fail`) semantics.
- Feature Store** A governed repository of feature tables providing point-in-time-correct training sets and online serving; on Databricks, Feature Engineering in Unity Catalog.
- Genie** Databricks AI/BI’s natural-language analytics interface, generating SQL over curated datasets guided by instructions and example queries.
- Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; Databricks’ founding architecture, also realized as Microsoft Fabric OneLake and AWS Lake Formation-governed lakes.
- Liquid clustering** A Delta Lake layout technique (`CLUSTER BY`) that replaces fixed partition columns and Z-ORDER with clustering keys that can change without rewriting the table.
- Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Databricks SQL, Snowflake, Redshift, BigQuery).
- Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.
- Metastore** The Unity Catalog top-level container binding catalogs, credentials, and audit for a cloud region; the successor to the per-workspace Hive metastore.
- Micro-partition** Snowflake’s internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.

- MLflow** The open-source ML lifecycle platform (tracking, models, registry, evaluation); Databricks manages it and integrates the registry with Unity Catalog.
- Mosaic AI** Databricks' AI suite: Foundation Model APIs, Vector Search, model serving, and evaluation tooling for GenAI and classic ML.
- OPTIMIZE** The Delta Lake command that compacts small files, optionally applying ZORDER BY multi-dimensional clustering.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Photon** Databricks' vectorized C++ execution engine accelerating SQL and DataFrame workloads on clusters and SQL warehouses.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window; realized as Delta Lake time travel/RESTORE, Snowflake Time Travel, and RDS PITR.
- Serverless SQL warehouse** Databricks SQL compute that starts instantly, autoscales, and bills per second, removing cluster management for BI workloads.
- Service principal** A non-human identity for automated jobs, pipelines, and CI/CD, granted least-privilege access; the Databricks counterpart to an AWS IAM role or Azure managed identity.
- Small-file problem** The degradation caused by thousands of tiny data files: slow listings, metadata overhead, and poor scan efficiency; remediated by compaction (OPTIMIZE).
- SQL warehouse** Databricks' SQL-optimized compute (serverless, pro, or classic) powering Databricks SQL queries, AI/BI dashboards, and Genie.
- Streaming table** A DLT dataset backed by a Structured Streaming query, ingesting each source record exactly once; contrast with a DLT materialized view.
- Structured Streaming** Spark's streaming engine, treating a stream as an unbounded table processed in micro-batches with checkpointed exactly-once semantics.
- Time travel** Querying a Delta table as of a prior version or timestamp (VERSION AS OF / TIMESTAMP AS OF); Snowflake's analog is Time Travel.
- Unity Catalog** Databricks' unified governance layer: metastore, catalogs, schemas, tables, volumes, functions, and models, with lineage, audit, and sharing.
- VACUUM** The Delta Lake command that deletes data files no longer referenced by the transaction log's retention horizon (default 7 days), permanently ending time travel past that point.
- Vector Search** Mosaic AI's managed vector index over Delta tables, powering semantic retrieval for retrieval-augmented generation (RAG) applications.
- Volume** A Unity Catalog object governing access to non-tabular files in cloud storage.
- Watermark** A Structured Streaming event-time threshold bounding how late events are accepted into windows and when state is evicted.
- WiredTiger** MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.
- Workflow** A Databricks job composed of a DAG of tasks (notebooks, pipelines, SQL, dbt) with scheduling, retries, repair runs, and alerting.



**Z-ordering** A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and largely superseded by liquid clustering for new tables.

# Bibliography

- [1] Amazon Web Services. Amazon kinesis data streams. <https://docs.aws.amazon.com/kinesis/>, 2026. AWS documentation.
- [2] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Open-source documentation.
- [3] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2026. Open-source documentation.
- [4] Apache Software Foundation. Apache spark documentation. <https://spark.apache.org/docs/latest/>, 2026. Open-source documentation.
- [5] Apache Software Foundation. Spark sql performance tuning. <https://spark.apache.org/docs/latest/sql-performance-tuning.html>, 2026. Open-source documentation.
- [6] Apache Software Foundation. Structured streaming + kafka integration guide. <https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html>, 2026. Open-source documentation.
- [7] Apache Software Foundation. Structured streaming programming guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>, 2026. Open-source documentation.
- [8] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. Delta lake: High-performance acid table storage over cloud object stores. In *Proceedings of the VLDB Endowment*, volume 13, pages 3411–3424, 2020.
- [9] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [10] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O’Reilly Media, 2018.
- [11] Databricks. Ai/bi dashboards. <https://docs.databricks.com/dashboards/>, 2026. Databricks documentation.
- [12] Databricks. The auto cdc apis: Simplify change data capture with delta live tables. <https://docs.databricks.com/delta-live-tables/cdc.html>, 2026. Databricks documentation.
- [13] Databricks. Capture and view data lineage. <https://docs.databricks.com/data-governance/unity-catalog/data-lineage.html>, 2026. Databricks documentation.
- [14] Databricks. Compute configuration and policies. <https://docs.databricks.com/compute/>, 2026. Databricks documentation.

- [15] Databricks. Databricks certification: Data engineer associate and professional. <https://www.databricks.com/learn/certification>, 2026. Databricks certification program.
- [16] Databricks. Databricks certified data engineer associate: Exam guide. <https://www.databricks.com/learn/certification/data-engineer-associate>, 2026. Official exam guide: domains, weightings, and objectives.
- [17] Databricks. Databricks certified data engineer professional: Exam guide. <https://www.databricks.com/learn/certification/data-engineer-professional>, 2026. Official exam guide: domains, weightings, and objectives.
- [18] Databricks. Databricks cli. <https://docs.databricks.com/dev-tools/cli/>, 2026. Databricks documentation.
- [19] Databricks. Databricks documentation. <https://docs.databricks.com/>, 2026. Platform documentation.
- [20] Databricks. Databricks free trial and community edition. <https://www.databricks.com/try-databricks>, 2026. Databricks documentation.
- [21] Databricks. Databricks git folders (repos). <https://docs.databricks.com/repos/>, 2026. Databricks documentation.
- [22] Databricks. Databricks sdk for python. <https://docs.databricks.com/dev-tools/sdk-python.html>, 2026. Databricks documentation.
- [23] Databricks. Databricks sql. <https://docs.databricks.com/sql/>, 2026. Databricks documentation.
- [24] Databricks. Databricks terraform provider. <https://registry.terraform.io/providers/databricks/databricks/latest/docs>, 2026. Terraform Registry documentation.
- [25] Databricks. Databricks workflows (jobs). <https://docs.databricks.com/workflows/>, 2026. Databricks documentation.
- [26] Databricks. Delta live tables (lakeflow declarative pipelines). <https://docs.databricks.com/delta-live-tables/>, 2026. Databricks documentation.
- [27] Databricks. Feature engineering in unity catalog. <https://docs.databricks.com/machine-learning/feature-store/>, 2026. Databricks documentation.
- [28] Databricks. Jobs api reference. <https://docs.databricks.com/api/workspace/jobs>, 2026. Databricks API reference.
- [29] Databricks. Lakehouse monitoring. <https://docs.databricks.com/lakehouse-monitoring/>, 2026. Databricks documentation.
- [30] Databricks. Manage data quality with pipeline expectations. <https://docs.databricks.com/delta-live-tables/expectations.html>, 2026. Databricks documentation.
- [31] Databricks. Manage secrets with secret scopes. <https://docs.databricks.com/security/secrets/>, 2026. Databricks documentation.
- [32] Databricks. Mlflow on databricks. <https://docs.databricks.com/mlflow/>, 2026. Databricks documentation.
- [33] Databricks. Model serving with mosaic ai. <https://docs.databricks.com/machine-learning/model-serving/>, 2026. Databricks documentation.

- [34] Databricks. Monitor usage with system tables. <https://docs.databricks.com/admin/system-tables/>, 2026. Databricks documentation.
- [35] Databricks. Mosaic ai. <https://docs.databricks.com/machine-learning/>, 2026. Databricks documentation.
- [36] Databricks. Optimize performance with file management. <https://docs.databricks.com/delta/optimize.html>, 2026. Databricks documentation.
- [37] Databricks. Provision users and groups with scim. <https://docs.databricks.com/admin/users-groups/scim/>, 2026. Databricks documentation.
- [38] Databricks. Rocksdb state store. <https://docs.databricks.com/structured-streaming/rocksdb-state-store.html>, 2026. Databricks documentation.
- [39] Databricks. Security and compliance on databricks. <https://docs.databricks.com/security/>, 2026. Databricks documentation.
- [40] Databricks. Sql warehouses. <https://docs.databricks.com/compute/sql-warehouse/>, 2026. Databricks documentation.
- [41] Databricks. Use liquid clustering for delta tables. <https://docs.databricks.com/delta/clustering.html>, 2026. Databricks documentation.
- [42] Databricks. What are databricks asset bundles? <https://docs.databricks.com/dev-tools/bundles/>, 2026. Databricks documentation.
- [43] Databricks. What are deletion vectors? <https://docs.databricks.com/delta/deletion-vectors.html>, 2026. Databricks documentation.
- [44] Databricks. What is an ai/bi genie space? <https://docs.databricks.com/genie/>, 2026. Databricks documentation.
- [45] Databricks. What is auto loader? <https://docs.databricks.com/ingestion/auto-loader/>, 2026. Databricks documentation.
- [46] Databricks. What is delta lake on databricks? <https://docs.databricks.com/delta/>, 2026. Databricks documentation.
- [47] Databricks. What is delta sharing? <https://docs.databricks.com/delta-sharing/>, 2026. Databricks documentation.
- [48] Databricks. What is mosaic ai vector search? <https://docs.databricks.com/generative-ai/vector-search.html>, 2026. Databricks documentation.
- [49] Databricks. What is photon? <https://docs.databricks.com/compute/photon.html>, 2026. Databricks documentation.
- [50] Databricks. What is the medallion lakehouse architecture? <https://www.databricks.com/glossary/medallion-architecture>, 2026. Architecture documentation.
- [51] Databricks. What is unity catalog? <https://docs.databricks.com/data-governance/unity-catalog/>, 2026. Databricks documentation.
- [52] Databricks. Workspace administration and navigation. <https://docs.databricks.com/workspace/>, 2026. Databricks documentation.
- [53] Delta Lake Project. Delta lake documentation. <https://docs.delta.io/>, 2026. Open-source documentation.

- [54] Faker Contributors. Faker: Synthetic data generator for python. <https://github.com/joke2k/faker>, 2026. Open-source library.
- [55] Microsoft. Azure event hubs for apache kafka. <https://learn.microsoft.com/azure/event-hubs/event-hubs-for-kafka-ecosystem-overview>, 2026. Microsoft Learn documentation.
- [56] MLflow Project. Mlflow documentation. <https://mlflow.org/docs/latest/>, 2026. Open-source documentation.
- [57] Olist. Brazilian e-commerce public dataset by olist. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>, 2026. Public dataset on Kaggle.
- [58] Registry of Open Data on AWS. Nyc taxi and limousine trip records. <https://registry.opendata.aws/nyc-tlc-trip-records-pds/>, 2026. Public dataset, also mirrored under [/databricks-datasets/](https://databricks-datasets/).
- [59] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

# Index

- acceptance criteria, 235
- ACID, 21
- ad-tech, 127
- AI/BI, 68
- Apache Kafka, 139
- Apache Spark, 39
  - equivalence, 257
- AQE, 87
- architecture decision, 100
- architecture defense, 222
- architecture review, 220
- Asset Bundles, 211
  - equivalence, 260
  - lab, 215
  - quick reference, 234
- audit, 163
- auditability, 26
- AUTO CDC, 104
  - lab, 107
- Auto Loader, 104
  - capstone, 120
  - lab, 107
- auto-termination, 1
- autologging, 159
- availableNow, 123
- Avro, 139
  
- backpressure, 139
- batch scoring, 184
- BI
  - equivalence, 258
- broadcast join, 87
  - lab, 90
- Bronze layer, 30
  
- caching, 39
- capacity planning, 64
- capstone
  - solutions, 235
- capstone project
  - final, 220
  - Part I, 47
  - Part II, 84
  - Part III, 120
  - Part IV, 156
  - Part V, 191
  - Part VI, 226
- case studies, 7
- catalog, 1, 76
- Catalyst, 39
- CDC, 104
- certification, 71, 220
  - Associate versus Professional, 245
  - domain mapping, 245
  - equivalence, 261
  - exam mapping, 245
- certified dataset, 68
- champion challenger, 159
- checklists, 240
- checkpoint, 104, 123
  - lab, 126
- chunking, 175
- CI/CD, 211
  - checklist, 243
  - equivalence, 260
  - lab, 215
- clean rooms, 203
- clean-clone drill, 226
- cloudFiles, 104
- cluster policies, 11
  - lab, 14
- column lineage, 203
- column masking, 76
  - lab, 79
- Community Edition, 1
- community resources, 7
- compute, 11
  - equivalence, 256
- consumer group, 139
- continuous pipeline, 96
- continuous vs triggered, 148
- contractor access, 198
- control plane, 11
- cost ceiling, 179
- cost optimization, 211

- credentials
  - conventions, 229
- cron, 112
- cross-cloud equivalence, 256
- DAG, 39
- dashboard
  - lab, 71
- dashboards, 68
- data classification, 203
- data consortium, 207
- Data Engineer Associate, 220, 245
- Data Engineer Professional, 220, 245
- data plane, 11
- data quality, 30, 96
- data skipping, 51
- Databricks CLI, 1, 11, 228, 231
  - essentials, 231
- Databricks platform, 11
- Databricks SQL, 60
- Databricks trial, 228
  - setup, 228
- Databricks workspace, 1
- databricks-sdk-py, 1, 232
- databricks.yml, 211
- DBU, 1
- dbutils, 11
- decision guide
  - Chapter 0, 5
  - Week 1, 17
  - Week 10, 101
  - Week 11, 109
  - Week 12, 117
  - Week 13, 128
  - Week 14, 136
  - Week 15, 144
  - Week 16, 153
  - Week 17, 164
  - Week 18, 172
  - Week 19, 180
  - Week 2, 27
  - Week 20, 188
  - Week 21, 200
  - Week 22, 208
  - Week 23, 217
  - Week 24, 223
  - Week 3, 35
  - Week 4, 44
  - Week 5, 56
  - Week 6, 65
  - Week 7, 73
  - Week 8, 81
  - Week 9, 92
- declarative pipelines, 96
- deletion vectors, 51
- Delta Lake, 21
  - checklist, 241
  - equivalence, 256
  - optimization, 51
  - quick reference, 233
- Delta Live Tables, 96
  - checklist, 241
- Delta Sharing, 203
  - design, 207
  - lab, 206
- Delta-sync index, 175
- dev staging prod, 211
- DLT
  - adoption, 100
- drift
  - lab, 187
- drift detection, 184
- driver, 39
- dropDuplicates, 131
- dynamic view, 76
- embeddings, 175
- environment isolation, 16
- Event Hubs, 139
  - migration, 143
- event log, 96
- event time, 131
- exactly-once, 123, 131
  - lab, 126
- executor, 39
- expectations, 96
  - lab, 99
- experiment tracking, 159
- external location, 76
- faker, 229
- feature engineering, 167
- feature lookup, 167
- Feature Store, 167
  - lab, 170
- feature table, 167
- finance, 71
- financial services, 26
- FinOps, 216
  - checklist, 243
- for-each task, 112
- foreachBatch, 148
- Foundation Model APIs, 175
- fraud detection, 135

- capstone, 156
- free trial, 1
- Ganglia, 87
- GenAI governance, 175
- Genie, 68
  - capstone, 84
  - lab, 71
- Git
  - installation, 228
- Git folders, 11
- GitHub Actions, 211
- glossary, 262
- Gold layer, 30
- governance
  - checklists, 240
  - design, 80
  - equivalence, 258
- GRANT, 76
  - lab, 79
- hands-on lab, 14, 24, 33, 41, 54, 63, 71, 79, 90, 99, 107, 114, 126, 134, 142, 151, 162, 170, 178, 187, 198, 206, 215
- healthcare, 80
- HIPAA, 80
- IAM, 195
- idempotency, 30
- identity
  - equivalence, 260
- if/else task, 112
- impact analysis, 203
- incremental ingestion, 104
- inference table, 184
- infrastructure as code, 211
  - equivalence, 260
- ingestion design, 108
- interview questions, 249
  - architecture defense, 255
  - CI/CD, 254
  - Delta Lake, 249
  - governance, 251
  - MLOps, 253
  - pipelines, 252
  - platform, 249
  - security, 251
  - Spark, 250
  - SQL and BI, 251
  - streaming, 253
- IoT, 152
- IP access list, 195
- jobs, 112
- Kafka
  - lab, 142
- Kinesis, 139
- lab solutions, 235
- label leakage, 167
- lakehouse
  - equivalence, 256
- Lakehouse Monitoring, 184
- late data, 131
- latency budget, 148
- lateness, 135
- lazy evaluation, 39
- leakage
  - audit, 171
- least privilege, 195
- lineage
  - lab, 206
- liquid clustering, 51
  - lab, 54
- logistics, 152
- materialized view, 60
  - DLT, 96
- maxOffsetsPerTrigger, 139
- medallion architecture, 30
  - capstone, 47
  - equivalence, 257
  - lab, 33
- MERGE INTO, 21
- metastore, 76
- micro-batch, 123
- migration, 143
- Milestone 1 capstone, 47
- Milestone 2 capstone, 84
- Milestone 3 capstone, 120
- Milestone 4 capstone, 156
- Milestone 5 capstone, 191
- Milestone 6 capstone, 226
- MLflow, 159
  - equivalence, 259
  - lab, 162
- MLflow evaluation, 175
- MLOps, 184
  - capstone, 191
  - checklist, 242
  - equivalence, 259
  - regulated, 188
- mock exam, 220
  - strategy, 246



- model alias, 159
- model debugging, 171
- model governance, 163
- model promotion, 184
- model registry, 159
  - lab, 162
- model risk management, 188
- Model Serving, 184
- model serving
  - lab, 187
- Mosaic AI, 175
  - checklist, 242
- multi-cloud, 256
- multi-team, 16
- naming conventions, 1
- natural-language analytics, 68
- networking
  - checklist, 243
  - equivalence, 260
- notebooks, 11
- OAuth, 1
- offsets, 139
- online store, 167
- operating model, 216
- operational guidance
  - Chapter 0, 6
  - Week 1, 18
  - Week 10, 101
  - Week 11, 109
  - Week 12, 117
  - Week 13, 128
  - Week 14, 136
  - Week 15, 145
  - Week 16, 154
  - Week 17, 164
  - Week 18, 172
  - Week 19, 181
  - Week 2, 27
  - Week 20, 189
  - Week 21, 200
  - Week 22, 208
  - Week 23, 217
  - Week 24, 224
  - Week 3, 35
  - Week 4, 44
  - Week 5, 57
  - Week 6, 65
  - Week 7, 73
  - Week 8, 81
  - Week 9, 93
- OPTIMIZE, 51
  - lab, 54
- orchestration, 112
  - checklist, 241
  - design, 116
  - equivalence, 258
- output mode, 123
- pandas UDF, 87
- Parquet, 21
- partition, 39
- payments, 108
- performance engagement, 91
- performance tuning, 43, 87
- personal access token, 1
- Photon, 60
  - equivalence, 257
- pipelines
  - equivalence, 258
- point-in-time
  - lab, 170
- point-in-time correctness, 167
- portfolio projects, 7
- predictive optimization, 51
- presentation, 222
- private connectivity, 195
- production readiness, 240
- PySpark, 87
  - quick reference, 233
  - reference, 231
- quarantine, 30
  - lab, 33
- query profile, 60
  - lab, 63
- question bank, 249
- quick reference, 231
- RAG, 175
  - lab, 178
  - production, 179
- real-time lakehouse, 148
  - lab, 151
- regression, 43
- regulation, 163
- release train, 216
- repair run, 112
  - lab, 114
- replay
  - lab, 142
- reprocessing, 34
- reproducibility, 159

- rescued data, 104
- REST API, 231
  - Databricks, 232
- restatement, 34
- RESTORE, 21
  - capstone, 47
  - lab, 24
- result cache, 60
- retail, 34
- retention, 26
- retry, 112
- RocksDB
  - lab, 134
- RocksDB state store, 131
- rollback, 184
- row-level security, 76
- run artifacts, 159
- salting, 87
  - lab, 90
- sample data, 228
  - NYC Taxi, 229
  - Olist, 229
- SCD Type 2, 104
- schema, 76
- schema enforcement, 21
- schema evolution, 21
  - lab, 24
- schema inference, 104
- schema registry, 139
- SCIM, 195
- secret scope, 195
  - lab, 198
- security, 195
  - checklist, 243
  - equivalence, 260
- self-assessment, 247
- self-service, 71
- self-service BI, 68
- semantic model, 68
- serverless endpoint, 184
- serverless SQL, 60
- service mapping, 256
- service principal, 195
  - lab, 198
- session window, 131
- setup, 228
- share revocation, 203
- shuffle, 39
- shuffle partitions, 87
- Silver layer, 30
- single sign-on, 195
- skew
  - lab, 41
- SLA, 91, 116
- small-file problem, 51
  - remediation, 55
- smoke test, 14
- sort-merge join, 87
- Spark UI, 39
  - lab, 41
- SQL
  - quick reference, 233
  - reference, 231
- SQL warehouse, 60
  - checklist, 241
  - equivalence, 258
  - lab, 63
- star schema, 30
- stateful streaming, 131
- stream-static join, 148
- stream-stream join, 148
- streaming
  - architecture, 152
  - checklist, 242
  - compaction, 55
  - design, 127, 135
  - equivalence, 259
- streaming medallion, 148
- streaming sink, 123
- streaming source, 123
- streaming table, 96
- Structured Streaming, 123
  - quick reference, 233
- study plan, 220
- study strategy, 246
- support copilot, 179
- system tables, 60
- system.access.audit, 203
- system.billing.usage, 211
- task values, 112
- tasks, 112
- Terraform, 211
- time travel, 21
- tools
  - installation, 228
- training-serving skew, 167
- transaction log, 21
- trigger, 123
- triggered pipeline, 96
- troubleshooting
  - setup, 230

- Unity Catalog, [76](#)
  - capstone, [84](#)
  - checklist, [240](#)
  - equivalence, [258](#)
  - lineage, [203](#)
  - naming, [30](#)
- VACUUM, [21](#), [51](#)
- Vector Search, [175](#)
  - lab, [178](#)
- volume, [76](#)
- VS Code, [228](#)
- watermark, [131](#)
  - lab, [134](#)
- windows, [131](#)
- Workflows, [112](#)
  - capstone, [120](#)
  - lab, [114](#)
- workload isolation, [64](#)
- workspace, [11](#)
  - equivalence, [256](#)
  - naming, [229](#)
- workspace conventions, [228](#)
- workspace design, [16](#)
- Z-ORDER, [51](#)
- zero downtime, [143](#)
- zero trust, [198](#)